INDUSTRY STARTER PACKS · THE ARCHITECTURE WORKBOOK

Insurance

Running the ADM in an insurer — a step-by-step workbook for the repository you have been handed

The companion volume. Not something to read — something to work through. Every phase of the ADM as numbered steps: what you produce, which sheet of the repository it goes in, which columns, who signs it, how you know it is done, and the way it usually goes wrong. Written against a real populated metamodel and a real comparison tool, not against the standard in the abstract.

Use it with the learner guide. South African lens. October 2026.

First edition · October 2026

Thotnr Consulting

Contents

BEFORE YOU START

Chapter 1 · How to use this workbook 3

PRELIMINARY PHASE

Chapter 2 · Standing the practice up 12

PHASE A

Chapter 3 · Architecture Vision 25

PHASE B

Chapter 4 · Business Architecture 35

PHASE C

Chapter 5 · Data Architecture 50

Chapter 6 · Application Architecture 63

PHASE D

Chapter 7 · Technology Architecture 79

PHASE E

Chapter 8 · Opportunities and Solutions 93

PHASE F

Chapter 9 · Migration Planning 109

PHASE G

Chapter 10 · Implementation Governance 122

PHASE H

Chapter 11 · Architecture Change Management 135

CONTINUOUS

Chapter 12 · Requirements Management 147

APPENDICES

Appendix A · The sheet-by-sheet reference 158

Appendix B · The report catalogue — what each RPT sheet answers 170

Appendix C · Workshop packs, checklists and the ninety-day calendar 176

Appendix D · The repository in ArchiMate 4 185

Appendix E · Where to look in volume 1 197

Appendix F · The forms 205

Appendix G · The principles and standards starter set 247

Appendix H · Running the maturity assessment 266

CHAPTER 1

How to use this workbook

Three files, one invented insurer, and the versioning discipline that decides whether any of it survives the year.

Three files, one invented insurer, and a discipline that most architecture repositories never acquire. This chapter is the orientation: what you have, how it fits together, and the handful of conventions that make the other fourteen chapters usable.

> **MERIDIAN IS INVENTED**
>
> Meridian Assurance Group does not exist. The `00 Index` sheet of the repository says so in its own words: the group is fictional, and while the regulatory drivers, market conditions and technology lifecycle dates reflect the South African market as at 2026, the numbers against Meridian's name are illustrative [1]. Treat every figure in this volume the way you would treat a worked example in a textbook — as a shape to copy, not a benchmark to quote. Cape Mutual, which appears in volume 1, is a separate invention. Do not mix their data.

The three files, and what each one is for

File	What it is	What it is not
`Meridian_TOGAF_Metamodel.xlsx`	The repository. 162 sheets: one per object type, one per relationship type, and the reports that run off them	Not a document. Nothing in it is prose to be read front to back
`Meridian_Architecture_Comparison_Tool.xlsx`	A twelve-sheet instrument that compares two extracts of the repository and sorts the differences into gaps, governed drift and ungoverned change	Not a gap catalogue. It produces candidates; a person classifies them
`Meridian_ArchiMate4_Views.pptx`	Worked views — the notation applied to this content, at the level of detail a board will actually look at	Not a model. The repository is the model; these are pictures of parts of it

The repository is the centre of gravity. `RPT01 Dashboard` counts it for you: 66 object types holding 1,727 object records, 56 relationship types holding 2,077 relationship instances, which is 1.20 links per record [2]. By domain: Motivation 175 records across 8 types, Business 732 across 12, Data 323 across 11, Application 60 across 3, Technology 95 across 8, Realization 225 across 13. Thirty-two report sheets sit on top, every figure a formula.

> **THE INDEX HEADER IS STALE, AND THIS MATTERS**
>
> `00 Index` describes the repository as "33 object types, 36 relationship types, 13 reports". `RPT01 Dashboard` counts 66 and 56, and the workbook holds thirty-two `RPT` sheets [1][2]. The header was written when the repository was half its present size and was never updated. Where a hand-typed description and a formula disagree, the formula wins. You will meet this pattern repeatedly: narrative text in a workbook rots, calculated cells do not. It is the first argument for the versioning discipline described later in this chapter.

The comparison tool matters more than its size suggests. It takes three inputs, not two: the baseline extract into `10 Input A Baseline`, the target extract into `11 Input B Target`, and — the part people leave out — the change log into `12 Input Change Log`. All three come from the repository's own export sheets: `05 Export Attributes`, `06 Export Records` and `07 Export Change Log`. Paste values only; the tool's own guidance explains why external links are not used [3].

With the third input in place, a difference between two extracts resolves into one of four things rather than one. `32 Diff Attributes` carries the verdict in its `Classification` column, and `60 Summary` counts them. In the worked example shipped with the tool — the 2026-09-30 Preliminary baseline against the repository at v1.23 — the split is: 1,264 attributes the same, 31 governed drift, 1 governed drift with further change, 5 **ungoverned change**, 2,739 attributes added in B, 138 records removed in B [3].

Classification	What it means	What you do
Same	No difference on this attribute	Nothing
Governed drift	The baseline moved and an approved change request explains it	Check whether the target is now stale
Ungoverned change	The value moved and nothing explains it	Raise a change request. This is a governance defect, not a gap
Added in B / Record removed in B	Present on one side only	The usual shape of a genuine design delta

The five ungoverned changes are the most valuable thing in the file. That is the tool's claim about itself and it is correct: a difference nobody can account for is evidence that somebody edited the repository without raising a change request, and that is a breach of the repository's own governance principle `GP-01` [3].

One honest caution before you trust the shipped example. `00 How to Use` says the example compares the Preliminary baseline against "the repository as at 2027-01-20", while `01 Config` labels snapshot B "Repository as at 2027-11-20, v1.23" and dates it 2027-11-20. The configuration sheet is the one the formulas read. The prose is ten months out of date — the same rot, in a different file.

Running the comparison once, properly

Six actions, forty minutes, and worth doing before you change anything.

1. Open the repository and copy `06 Export Records`, values only, into `10R Records A` of the tool — or take the baseline copy you saved in `04 Baselines` if you are comparing against a real baseline.
2. Copy `05 Export Attributes` into `10 Input A Baseline` and the later extract into `11 Input B Target`.
3. Copy `07 Export Change Log` into `12 Input Change Log`. This is the input people leave out, and leaving it out turns every governed change into a false gap.
4. Set `01 Config`: Snapshot A label and date, Snapshot B label and date, and `Comparison mode`. The mode changes the meaning of the answer — Baseline vs Target makes unexplained differences gap candidates, Snapshot vs Snapshot makes them ungoverned changes.
5. Read `60 Summary` before anything else, and read the classification split before the totals.
6. Filter `32 Diff Attributes` on `Classification` and work the differences into `50 Review`, which is the only sheet you type in. Classify each as Gap, Drift, Variance or Accepted, with a `Rationale` and an `Owner`.

Only rows classified Gap reach `70 Gap Candidates`, which is shaped to paste into `O29 Gap`. In the shipped example, eight differences have been reviewed and 2,906 have not — which is the honest state of every comparison anybody has ever run. The tool's own guidance makes the point: a difference is not a finding until a person has classified it and taken it [3].

Relationship sheets, and what a lookup error means

Every `R` sheet has the same shape: `Source (XXX)`, `Source Name`, `Target (YYY)`, `Target Name`, one or two columns of relationship properties, then `Key`, then the seven spine columns. The two name columns resolve by formula from the object sheets, which makes them a free integrity check — a lookup error in `Source Name` or `Target Name` means the ID in the adjacent column does not exist. `Key` concatenates the two IDs with a pipe (`ASM-01|WP-01`), and it is what the comparison tool matches on. Do not edit a `Key` by hand; do not sort a relationship sheet in a way that separates a `Key` from its row.

The fourth thing on your desk

Three files and a book. *Insurance — a starter pack for enterprise architects* is volume 1. The division of labour is strict.

Volume 1 is the content. The lines of business, the money, the operating model, twelve L1 processes in BPMN, twenty data entities, twenty-five conceptual applications, vendors, regulation, and how to draw all of it in ArchiMate 4. Read it front to back, once; it answers *what is true about insurance*.

Volume 2 is the method. It never re-teaches insurance. Which sheet, which columns, in what order, signed by whom, and the report that proves it. It answers *what do I do on Tuesday*.

So read volume 1 once, before the Preliminary Phase, then keep it open at the chapter the phase names. Every chapter from 2 onwards opens with a *Before you start this phase* box; steps needing a judgement volume 1 has already worked through carry a **Look it up.** line.

One warning, repeated wherever it bites. Volume 1's catalogues are numbered for volume 1 — `P-01` … `P-12` processes, `C-01` … `C-20` capabilities, `R-01` … `R-33` roles, `D-01` … `D-20` data entities, `A-01` … `A-25` conceptual applications. Meridian's repository is numbered for Meridian: 22 `BPR-` rows, 556 capability IDs on the four-level `C01.02.03` pattern, 27 `ROL-` rows, 24 `DEN-` rows, 26 `APP-` rows. Same industry, different grain, different quantities, built independently. **Map by name, never by number.** `D-05` is not `DEN-05`, `C-07` is not `C07`, and twenty-five conceptual applications against twenty-six `APP-` rows is a coincidence. Every step that loads one of these catalogues says so again.

Workbook chapter	Phase	Volume 1 chapters to have read
1 How to use this workbook	Before you start	1
2 Standing the practice up	Preliminary	1, 6, 15, 18
3 Architecture Vision	A	4, 5, 15, 17
4 Business Architecture	B	5, 7, 8, 13, 19
5 Data Architecture	C	9, 10, 15, 19
6 Application Architecture	C	11, 12, 13, 19
7 Technology Architecture	D	12, 13, 14, 15
8 Opportunities and Solutions	E	4, 11, 13, 14, 17
9 Migration Planning	F	12, 14, 17, 18
10 Implementation Governance	G	15, 16
11 Architecture Change Management	H	2, 14, 15
12 Requirements Management	Continuous	10, 15, 16
13 Appendix A, the sheet-by-sheet reference	Reference	16, 20
14 Appendix B, the report catalogue	Reference	6, 13
15 Appendix C, workshop packs and the calendar	Reference	5, 18
16 Appendix D, the repository in ArchiMate 4	Reference	16, 17

That column is the minimum, not a reading list.

The notation: ArchiMate 4, and the one exception

Every diagram in this volume is ArchiMate 4. So is `Meridian_ArchiMate4_Views.pptx`, the third file you were handed — worked views in the same notation, at the level of detail a board will look at [6]. Volume 1, chapter 16 (Modelling insurance in ArchiMate) teaches the notation itself: the elements, the relationships, and the views an insurer needs. This section says only how the notation lands on the repository, which is the part volume 1 cannot tell you, because it does not know which sheets you have.

The repository is a TOGAF metamodel, not an ArchiMate model. Its sheets are object types and relationship types. ArchiMate is how you draw them. The mapping below is fixed: a row in a given sheet becomes the same element every time, so that two architects drawing the same corner of the repository produce the same picture [7].

Sheet	ArchiMate 4 element	Sheet	ArchiMate 4 element
`001 Driver`	Driver	`012 Location`	Location
`002 Goal`	Goal	`013 Business Capability`	Capability
`003 Objective`	Outcome	`014 Value Stream`	Value Stream
`004 Measure`	Assessment	`015 Business Service`	Service
`005 Principle`	Principle	`016 Business Process`	Process
`006 Requirement`	Requirement	`017 Business Function`	Function
`009 Organization Unit`	Business Actor	`018 Product`	Product
`010 Actor`	Business Actor	`019 Contract`	Business Object
`011 Role`	Role	Stakeholders	Stakeholder

Sheet	ArchiMate 4 element	Sheet	ArchiMate 4 element
`020 Data Entity`	Business Object	`028 Platform Service`	Service
`021 Logical Data Component`	Data Object	`029 Gap`	Assessment
`022 Physical Data Component`	Artifact	`030 Work Package`	Work Package
`023 Application Component`	Application Component	`031 Deliverable`	Deliverable
`024 Information System Service`	Service	`032 Standard`	Requirement
`025 Technology Service`	Service	`053 Transition Architecture`	Plateau
`026 Logical Technology Component`	Node	`055 Building Block`	Grouping
`027 Physical Technology Component`	Node, Device or System Software		

Four notes on that mapping, each of which catches somebody.

Four sheets map to `Service`. `015 Business Service`, `024 Information System Service`, `025 Technology Service` and `028 Platform Service` are all the one ArchiMate `Service` element. The layer is carried by what realises the service, not by the element itself. Draw the realiser and the layer is unambiguous.

`027 Physical Technology Component` takes whichever element fits the thing. A server is a `Device`, a database engine is `System Software`, everything else is a `Node`. The sheet does not record which, so you decide at the point of drawing and stay consistent across views.

Gaps and measures are both `Assessment`. `029 Gap` and `004 Measure` are statements about the state of affairs, which is exactly what `Assessment` means. A gap is not a `Requirement`; the requirement is the thing that closes it.

A stakeholder is the `Stakeholder` element, not a `Business Actor`. The same `011 Role` row is drawn as a `Role` in a business view and a `Stakeholder` in a motivation view. Both are right, and the view decides which.

Relationships follow ArchiMate's rules rather than the relationship sheet's name. An application **realises** a service. A role is **assigned to** a process. A process **accesses** a business object. A service **serves** a process or an actor. A capability is **composed of** its sub-capabilities. A work package **realises** a deliverable or a plateau. A driver **influences** a goal, and a principle **influences** a requirement. Where a relationship sheet joins two rows in a way ArchiMate will not allow, the sheet has found you a modelling error, not a notation problem.

> **THE EXCEPTION: LEVEL 2 PROCESS MODELS ARE BPMN 2.0**
>
> ArchiMate does not model swimlanes, and it has no way to show a hand-off from one lane to another. An L2 insurance process is mostly hand-offs — underwriter to reinsurer, assessor to loss adjuster, adjuster back to the claims handler — so drawing one in ArchiMate loses the thing worth drawing. Volume 1, chapter 8 (The process model) publishes the eleven L2 processes in BPMN 2.0 and this volume keeps to that. Everything at L1 and above stays in ArchiMate: `016 Business Process` as a `Process`, joined to its value stream, its owning role and the objects it touches.

The ADM as this workbook runs it

phase	the sheets it fills	proves it
Preliminary	005 009 057 058 001-004 031	RPT12 13
A Vision	007 008 010 011 014 006 031	RPT14 31
B Business	013 015-019 035 R09-R16	RPT03 07 23
C Data	020-022 036-044 R17-R19	RPT04 05 15
C Application	023 024 045 R20-R24 R37	RPT02 06 20
D Technology	025-028 032 046-050	RPT10 25 26
E Solutions	029 051 054 055 R28 R48	RPT09 24 28
F Migration	030 052 053 056 066 R47	RPT27 29
G Governance	033 059 060 061 062 R33	RPT30 31
H Change	02 03 04 061 066	RPT13

Requirements Management runs throughout: 006, R04, R05, R32, R60, RPT08, RPT32.

Figure 1.1 The ADM as this volume runs it: each phase, the sheets it fills, and the reports that show whether it landed. Preliminary and Phase A are chapters 2 and 3.

Fifteen chapters, one phase each where the ADM has a phase, plus three appendices. The phase order is the standard one and this volume does not argue with it [4]. What this volume adds is the mapping: which sheets each phase fills, and which report tells you the phase is finished.

Ch	Phase	Primary sheets	The report that closes it
2	Preliminary	`O05 Principle`, `O09 Organization Unit`, `O57 Governance Body`, `O58 Governance Gate`, `O01 – O04`, `O31 Deliverable`	`RPT12 Integrity Check`, `RPT13 Version Control`
3	A — Vision	`O07 Constraint`, `O08 Assumption`, `O10 Actor`, `O11 Role`, `O14 Value Stream`, `O06 Requirement`	`RPT14 Deliverable Register`
4	B — Business	`O13 Business Capability`, `O15 – O19`, `O35 Business Building Block`	`RPT03 Capability Heatmap`, `RPT07 Capability Coverage`
5	C — Data	`O20 Data Entity`, `O21`, `O22`, `O36 – O44`	`RPT15 Catalogue Completeness`, `RPT17 Master Data`
6	C — Application	`O23 Application Component`, `O24`, `O45`	`RPT02 Application Portfolio`, `RPT06 Interface Register`
7	D — Technology	`O25 – O28`, `O32 Standard`, `O46 – O50`	`RPT10 Standards Conformance`, `RPT26 Resilience Test Status`
8	E — Opportunities	`O29 Gap`, `O51 Solution Building Block`, `O54 Architecture Alternative`, `O55 Building Block`	`RPT09 Gaps and Roadmap`, `RPT24 Building Block Coverage`
9	F — Migration	`O30 Work Package`, `O52`, `O53 Transition Architecture`, `O56 Project`	`RPT27 Programme Sequencing`, `RPT29 Transition Readiness`
10	G — Governance	`O33 Architecture Contract`, `O59 Compliance Assessment`, `O60`, `O61`, `O62`	`RPT30 Compliance Position`, `RPT31 Governance Activity`
11	H — Change	`02 Change Log`, `03 Change Requests`, `04 Baselines`, `O66 Architecture Increment`	`RPT13 Version Control`
12	Requirements	`O06 Requirement`, `R04`, `R05`, `R32`, `R60`	`RPT08 Traceability`, `RPT32 Requirement Traceability`

Two traps in that table, both of which have caught people already. The gap report is `RPT09 Gaps and Roadmap`; `RPT07` is `Capability Coverage`, which is a different question. And there are two sheets named "Building Block Coverage": `RPT19` covers **data** building blocks, `RPT24` covers all four domains with a `Funded` column. Name the number when you quote one.

How a step is laid out

From chapter 2 onward the volume is numbered steps, roughly 150 of them. Every one has the same eight parts, in the same order. Learn the shape once.

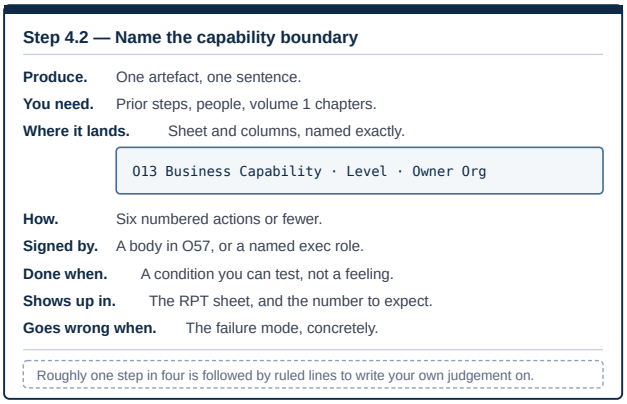

Figure 1.2 The step template. Eight parts, always in this order. You will meet it about 150 times.

- **Produce.** The artefact, in one sentence. If you cannot say what the step produces, the step is a meeting.
- **You need.** Inputs: earlier steps, the people whose time you must book, the volume 1 chapter that supplies the content.
- **Where it lands.** The sheet, named exactly, and the columns to fill, with what goes in each non-obvious one. This is the part the volume exists for.
- **How.** Six numbered actions or fewer. If something needs more than six, it is two steps.
- **Signed by.** A body from `O57 Governance Body` or a named executive role. Architecture that nobody signed is a draft forever.
- **Done when.** A condition you can test. "Stakeholders are aligned" is not a condition. "Every `Tier 1` capability has a named `Owner Org`" is.
- **Shows up in.** The report sheet where the work becomes visible, and what the number should look like if the step went well. This is how you check yourself without asking anybody.
- **Goes wrong when.** The real failure mode, stated concretely enough to recognise in your own repository.

Roughly one step in four is followed by ruled lines. Those are for the judgements only you can make: what is in scope, which stakeholder actually decides, the one sentence that fixes a capability boundary. Write in the book. A judgement you did not write down is a judgement you will make

differently next month.

The spine: version control, and why repositories rot

Most architecture repositories die the same death. Content goes in during a programme, nobody records who changed what, and eighteen months later no one can tell which records are current. The sheets are still there. They are just no longer evidence of anything.

This repository is built to resist that, and the mechanism is four sheets plus seven columns. Drawn in ArchiMate 4, it is one process with three passive objects and a plateau.

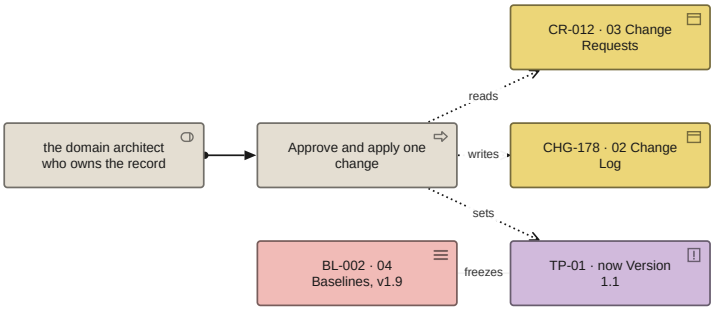

Figure 1.3 One approved change, in ArchiMate 4. The domain architect's change process reads the change request, writes the log entry and sets the record's version — which is why the version is derived and never typed. The baseline freezes the record at a named version to compare against.

> **`01 Version Control — the rules, and the current state`**
>
> The seven rules are stated on the sheet itself: records carry version metadata; that metadata is **derived from the change log, never typed**; every change starts as a change request; amendment is by replacement, with previous and new value both kept; a baseline is a named, dated, approved snapshot; object records are changed by the domain architect who owns them and relationship records by the architect owning the source object; the change log is never truncated, because it is the evidence base for Joint Standard 1 of 2023 and for principle `GP-01` [5].
>
> The lower block of the same sheet reports the repository's current state: workbook version 1.23, state as at 2027-11-20, 63 change requests raised, 59 approved, 4 open, 172 logged changes since baseline, 4 changes awaiting approval, 2 baselines taken.

Every object and relationship sheet carries the same seven columns at the right-hand end: `Version`, `Record Status`, `In Repository Since`, `Last Changed`, `Changed By`, `Change Ref`, `Approved By`. They are the spine. Read them this way:

Column	What it tells you	What a bad value looks like
`Version`	`1.0` is the baseline load; the minor number increments once per approved change, so `1.1` means one change since baseline	A version above `1.0` with an empty `Change Ref` — somebody typed over the sheet
`Record Status`	`Baseline`, `Approved`, or `Draft - change pending` while a change request is undecided	`Draft - change pending` on a record whose change request was decided months ago
`In Repository Since`	When the record first appeared. On records added after baseline this is the date of the adding change request	A date earlier than the baseline on a record the baseline does not contain
`Last Changed`	Date of the most recent approved change	Equal to `In Repository Since` on a record that has visibly moved
`Changed By`	`Repository load` for baseline records, otherwise the role that made the change	A person's name where a role belongs. Roles outlive people
`Change Ref`	The join back to `03 Change Requests`. This is the audit trail	Empty on anything that is not `1.0`
`Approved By`	`Architecture Board` for principle-engaging changes, `Chief Enterprise Architect` otherwise	`** pending **`, which means the record is live but unapproved

That last value is worth dwelling on. `GAP-05`, `GAP-10` and `WP-15` all read `Approved By` = `** pending **` with `Record Status` = `Draft - change pending`, against change requests `CR-011` and `CR-014` which are still "Under review" and "Proposed". The repository is telling you, in a column, that three of its records are being quoted in roadmaps while nobody has agreed them. `RPT13 Version Control` has a `Pending approval` column that counts exactly this, per sheet.

Baselines are the other half. `04 Baselines` holds three records: `BL-001` Preliminary Phase Baseline, 2026-09-30, workbook v1.0, approved by the Chief Enterprise Architect, now `Superseded`, with 172 logged changes since; `BL-002` Governance Establishment Baseline, 2026-12-16, v1.9, approved by the Architecture Board, `Current`, 130 changes since; and `BL-003` Phase A Entry Baseline, `Planned` for 2027-03-31, to be cut on approval of the Architecture Vision.

> **THE PLANNED BASELINE THAT NEVER HAPPENED**
>
> `BL-003` is dated 2027-03-31 and still reads `Planned`, while the repository's own state date is 2027-11-20. Eight months late, with 88 changes accumulated against it. Nothing in the workbook flags this — no formula checks a planned baseline against today. It is the single most useful thing to look for when you inherit somebody else's repository: a baseline that was planned, never cut, and quietly stopped being a reference point.

Three rules follow from all of this, and they are the price of admission to every later chapter:

1. **Never edit a cell without a change request.** Add the row to `03 Change Requests` first, with `Type` (Addition, Correction, Reassessment, Restructure, Extension, Reinstate), `Scope`, `Raised By` and `Raised`. Then log the change in `02 Change Log` with `Previous Value` and `New Value`. The record's version follows.
2. **Never renumber.** IDs are referenced from relationship sheets by text match. Renumbering `DEN-04` breaks every row in `R17 BPR-DEN` and `R21 APP-DEN` that mentions it, and the break shows up as a lookup error in a `Source Name` or `Target Name` column.
3. **Cut a baseline before you start a phase and after you finish it.** Gap analysis compares a moving repository against a fixed point. Without the fixed point you are comparing today against yesterday's memory.

The ID conventions already in use

Every object sheet puts the ID in column A and the name in column B, and `00 Index` warns in plain terms not to reorder those two columns, because every lookup in the workbook depends on their position [1]. The prefixes are already set. Do not invent new ones.

Prefix	Sheet	Count	Prefix	Sheet	Count
`DRV-`	`001 Driver`	15	`GAP-`	`029 Gap`	18
`GOL-`	`002 Goal`	6	`WP-`	`030 Work Package`	16
`OBJ-`	`003 Objective`	15	`DLV-`	`031 Deliverable`	20
`MSR-`	`004 Measure`	18	`STD-`	`032 Standard`	16
`BP- DP- AP- TP- SP- GP- IP-`	`005 Principle`	31	`ACO-`	`033 Architecture Contract`	7
`REQ- ARS-`	`006 Requirement`	76	`ABB-`	`035 Business Building Block`	14
`CON-`	`007 Constraint`	8	`DER- ATT- DS- MIG- STM-`	`036` – `040`	221
`ASM-`	`008 Assumption`	6	`SEC- MDE- MDR- DBB-`	`041` – `044`	56
`ORG-`	`009 Organization Unit`	16	`REF-`	`047 Reference Model Entry`	34
`ACT-`	`010 Actor`	20	`ENV- RES- LNK-`	`048` – `050`	26
`ROL-`	`011 Role`	27	`SBB-`	`051 Solution Building Block`	29
`LOC-`	`012 Location`	10	`IF- TA-`	`052`, `053`	29
`C01.02.03`	`013 Business Capability`	556	`PRJ-`	`056 Project`	15
`VST-`	`014 Value Stream`	7	`GB-`	`057 Governance Body`	5
`BSV- BPR- FUN-`	`015` – `017`	54	`G0` – `G3`	`058 Governance Gate`	4
`PRD- CTR-`	`018`, `019`	28	`CA- DEC- CR-G`	`059` – `061`	25
`DEN- LDC- PDC-`	`020` – `022`	54	`RSK- CO- EP- INC-`	`062` – `066`	31
`APP- ISS-`	`023`, `024`	46			
`TSV- LTC- PTC- PSV-`	`025` – `028`	55			

Six irregularities in that list are not mistakes you should tidy. They are load-bearing, and you need to know them before you query anything.

- **`013 Business Capability` does not use `CAP-`.** The sheet is headed "Business Capability (CAP)" and every relationship sheet labels its column `Target (CAP)`, but the IDs are dotted and unprefixed: `C01`, `C01.02`, `C01.02.03`, `C01.02.03.01`. The dots carry the level, which is why `R38 CAP-CAP` can hold 544 decomposition links without a separate hierarchy table. The original load did use `CAP-01` … `CAP-26`; `CR-016` replaced that 26-row set with a four-level, 556-row map.
- **The old `CAP-` IDs still appear in the change log.** `CHG-015` and `CHG-019` record changes to `CAP-26` and `CAP-25`, which no longer exist in `013`. The log is not wrong — those changes really happened to records that have since been replaced — but a key-matched lookup from `12 Input Change Log` will never resolve them. Sheet `50 Review` of the comparison tool carries the same ghost, classifying a maturity change on `CAP-15`.
- **`DLV-` means two different registers.** `031 Deliverable` holds 20 `DLV-` records and `034 Architecture Deliverable` holds 20 more, with the same ID space and different content. `DLV-05` is the Business Model Canvas in `031` and the EA Operating Model in `034`; `DLV-10` is the Request for Architecture Work in `031` and the Business Model Canvas in `034`. Always say which sheet.
- **`APP-` means two different things.** `023 Application Component` numbers applications `APP-01` … `APP-26`. `045 Application Building Block` also numbers `APP-`. So does the consolidated `055 Building Block`, which reuses `ABB-`, `DBB-`, `APP-` and `TEC-` from the four domain building-

block sheets. `SEC-` is shared between `041 Security Control` and `064 Security Policy` in the same way.

- **`054 Architecture Alternative` uses bare letters** `A` to `E`. Readable in a board paper, awkward in a lookup, and a near-collision with the capability prefix.
- **`006 Requirement` carries two populations.** Twenty `REQ-` records with `Source` = "Phase A strategic", split evenly functional and non-functional; and 56 `ARS-` records with `Type` = "Architecture" and `Source` = "Derived in Phase C" or "Derived in Phase D". The strategic requirements are stated in Phase A; the architecture requirements are derived later. One sheet, two lifecycles.

What "done" means for a phase

Not "the document is written". Three conditions, all testable, and the last one is the one people skip.

1. **The deliverable exists and is registered.** `031 Deliverable` has a row with `Status` = Complete, a `Repository Location` and an `Approved By`. `RPT14 Deliverable Register` shows it with a non-draft `Status` and a `Depends on` count that makes sense.
2. **The gate was held.** `058 Governance Gate` names the gate, `059 Compliance Assessment` holds a verdict per acceptance criterion with `Evidence`, and `060 Governance Decision` records what the body decided. A phase with no decision record did not finish; it stopped.
3. **The repository is clean enough to hand over.** `RPT12 Integrity Check` shows no newly-isolated object type; `RPT13 Version Control` shows `Pending approval` at zero for the sheets you touched; and a baseline has been cut in `04 Baselines` with a `Workbook Version` and an `Approved By`.

Meridian passes the first condition on eight deliverables and fails the second on nearly everything: `059 Compliance Assessment` holds thirteen verdicts and `CA-01` through `CA-06` are all `Non-compliant`. That is not a flaw in the sample data. It is what a real repository looks like fourteen months into a capability that was stood up properly and governed badly.

The three ways people wreck this

Modelling everything. The symptom is a repository with more records than relationships. `037 Data Attribute` holds 161 rows and zero links. So does `052 Implementation Factor` with 26, `049 Resilience Tier` with 12, `050 Network Link` with 10. `RPT12 Integrity Check` grades 19 of the 66 object types `ISOLATED` and another 7 `Thinly linked`. An isolated catalogue costs real effort to build and answers no question, because every question an architect is asked is a question about a relationship. If you are about to populate a sheet, name the relationship sheet that will join it to something else. If there is not one, do not populate it yet.

Modelling nothing. The opposite failure, and more common. The architect reads the ADM, agrees with it, produces slides, and the repository stays at its baseline load. `RPT13 Version Control` catches this precisely: it grades a type `Unchanged since baseline` when the change log holds nothing against it. In Meridian, `001 Driver`, `002 Goal`, `003 Objective`, `004 Measure`, `007 Constraint` and `009 Organization Unit` are all `Unchanged since baseline` fourteen months in. The motivation spine was loaded once and never revisited, which means nobody has tested whether the goals still hold.

Modelling the org chart. The most expensive of the three, because it looks like work. `009 Organization Unit` holds 16 records and `R10 ORG-CAP` holds 134 ownership links — and the temptation is to model the hierarchy down to team level and call it business architecture. It is not. `034 Architecture Deliverable` records the honest version: `DLV-14 Organisation Chart`, 99 positions across four layers, owned by the Group HR Director, `Draft`. That is an HR artefact the architecture consumes, not an architecture artefact. Capabilities are what the enterprise can do; organisation units are who currently does it; the whole point of keeping them in separate sheets joined by `R10 ORG-CAP` is that the second changes without the first. Reorganise an insurer and the capability map should not move.

Adapting this to an insurer that is not Meridian

Most of what you have been handed is reusable. Some of it is Meridian's and must go.

Keep as shipped	Re-derive for your insurer	Delete
The sheet structure, column sets and ID prefixes	`009`, `012`, `018`, `019`, `023` — your organisation, sites, products, contracts, applications	Nothing. Mark superseded records rather than deleting rows
The four versioning sheets and the seven spine columns	`001` – `004`: your drivers, goals, objectives and measures	
`005 Principle` as a starting set of 31 to argue with	`013` levels 3 and 4, and every maturity score	
`057`, `058` — the bodies and gates, renamed to your forums	`029 Gap`, `030 Work Package` and everything downstream	

The order matters. Do `009 Organization Unit` and `012 Location` first, because `Owner Org` and `Location` columns across a dozen other sheets reference them. Then `018 Product`, because products drive the whole business layer. Then the motivation sheets. Leave capabilities until you have value streams, for the reason chapter 3 gives.

Two insurer-specific adjustments you will need almost immediately. First, licence class: volume 1, chapter 15 (Regulation, risk and what the regulator inspects) argues that the Insurance Act licence class and sub-class are attributes of every product, derivable for every contract in force, and not a reference-data footnote. `018 Product` has `Line of Business`, `Policies In Force`, `GWP (Rm)`, `Config or Code` and `Status` — and nowhere to put a licence class. If you are standing this up for a real insurer, raise a change request against the metamodel and add the column before you load

products. Second, legal entity: `O09 Organization Unit` has a `Type` column whose values are Group, Line of Business, Group Function and Subsidiary. A composite group with several licensed entities has no place to record which entity holds which licence. Meridian sidesteps this by running two federated subsidiaries, `ORG-15` Namibia and `ORG-16` Botswana, with `CON-08` recording that they are federated rather than centrally governed. Your group may not be so convenient.

READ-ONLY FOR A WEEK

Before you change a cell, work through `RPT01 Dashboard`, `RPT12 Integrity Check` and `RPT13 Version Control`, and run the comparison tool once against its shipped example. Three sheets and one comparison tell you how big the repository is, where it is unlinked, what has moved since baseline and what moved without permission. If you inherit a repository rather than starting one, those four readings are your handover.

How to use this volume day to day

Open it at the step you are on. The part heading tells you the phase, the step number is stable, and chapters 13 to 15 are reference: a sheet-by-sheet description of the repository, a catalogue of what each `RPT` sheet answers, and the workshop packs, checklists and ninety-day calendar. Volume 1 is the content library — when a step says "volume 1 appendix A already drafts these", go and start from there, then argue with it. Your insurer is not Cape Mutual and it is certainly not Meridian.

THE CHAPTER IN TEN LINES

- Three files: the 162-sheet repository, the twelve-sheet comparison tool, the worked views. The repository is the model.
- RPT01 Dashboard counts 66 object types, 1,727 records, 56 relationship types, 2,077 links. The 00 Index header saying 33 and 36 is stale.
- The comparison tool needs three inputs. Without the change log, every difference looks like a gap.
- Ungoverned change is the most valuable finding the tool produces, and the hardest to argue with.
- Every step has the same eight parts: Produce, You need, Where it lands, How, Signed by, Done when, Shows up in, Goes wrong when.
- Four sheets and seven columns are the spine: 01 Version Control, 02 Change Log, 03 Change Requests, 04 Baselines.
- Version is derived from the change log, never typed. An empty Change Ref above v1.0 means somebody edited the sheet.
- ID prefixes are already set. O13 uses dotted C01.02.03, not CAP-; DLV- and APP- each mean two different sheets.
- A phase is done when the deliverable is registered, the gate was held with evidence, and a baseline has been cut.
- The three failure modes: modelling everything, modelling nothing, and modelling the org chart.

SOURCES FOR THIS CHAPTER

1. Meridian_TOGAF_Metamodel.xlsx, sheet 00 Index
2. Meridian_TOGAF_Metamodel.xlsx, sheet RPT01 Dashboard
3. Meridian_Architecture_Comparison_Tool.xlsx, sheets 00 How to Use, 01 Config, 32 Diff Attributes and 60 Summary
4. The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method https://pubs.opengroup.org/architecture/togaf10-doc/adm/
5. Meridian_TOGAF_Metamodel.xlsx, sheets 01 Version Control, 02 Change Log, 03 Change Requests and 04 Baselines
6. Meridian_ArchiMate4_Views.pptx — the worked views supplied with the repository
7. The Open Group, ArchiMate Specification — elements, relationships and viewpoints https://pubs.opengroup.org/architecture/archimate3-doc/

CHAPTER 2

Standing the practice up

The phase that decides who may say no, to what, on what evidence — and the one most often done as a slide deck.

The Preliminary Phase has a bad reputation because it is usually done as a slide deck. Done properly it is the phase that decides whether anything else in this volume will work: who may say no, to what, on what evidence. Fourteen steps. Three or four weeks if the executive sponsorship is real, six months if it is not.

Meridian Assurance Group, the insurer in the repository, is invented; its numbers are illustrative. Its Preliminary Phase is useful precisely because it went half right. The repository was built, the principles were loaded, the bodies were written down — and `057 Governance Body` still records that the Architecture Board "is constituted on paper and has never refused anything", while `WP-01 Establish Architecture Governance` reads `Delivery Status` = "Past end date, not evidenced" [1][2]. Work through the steps in order and you will see where that divergence enters.

> **BEFORE STEP 2.1**
>
> Spend a day on `RPT01 Dashboard`, `RPT12 Integrity Check` and `RPT13 Version Control`, and run the comparison tool once against its shipped example. Chapter 1 explains why. If you are inheriting a repository rather than starting one, that reading is your handover and it will change what you do in step 2.3.

> **BEFORE YOU START THIS PHASE**
>
> Read these first, and keep volume 1 open at them.
>
> - **Volume 1, chapter 1, *The seven things an architect must understand.*** The industry in one sitting.
> - **Volume 1, chapter 15, *The Prudential Authority: licence classes are a data attribute.*** Licence class as a product attribute.
> - **Volume 1, chapter 6, *Reading the ratios.*** The three numbers an insurer steers by.
> - **Volume 1, chapter 18, *The ninety-day plan.*** What a first quarter should produce.

The shape of the phase

Steps	What they settle
2.1 – 2.3	Boundary, baseline, and how much of the metamodel you will actually use
2.4 – 2.5	Principles: adopted, edited, retired, and bound to requirements
2.6 – 2.8	Who decides: bodies, gates, and the architecture function itself
2.9 – 2.11	Where you are starting from, and what the enterprise is trying to do
2.12 – 2.14	The year plan, the health check, and closing the phase against a baseline

Step 2.1 — Fix the enterprise boundary and confirm the licences in scope

Produce. A one-page scope statement naming every legal entity, licence class and organisation unit inside the architecture boundary — and, explicitly, the ones outside it.

You need. The group structure from the company secretary, the licence schedule, and volume 1, chapter 15 (Regulation, risk and what the regulator inspects), which sets out why the Insurance Act 18 of 2017 licence class and sub-class are product attributes rather than reference-data footnotes [3][4]. Volume 1, chapter 2 (The lines of business) tells you what each class means in practice.

Look it up. Volume 1, chapter 2, *The table to keep*: one row per line of business, with its regime.

Where it lands. `009 Organization Unit`, one row per unit:

- `ID` — the next `ORG-` number. Never reuse a retired one.
- `Type` — Meridian's shipped values are Group, Line of Business, Group Function and Subsidiary. There is no "Legal Entity" value; see the warning below.
- `Parent` — an `ORG-` ID. Exactly one record, the group, has an empty `Parent`.
- `Head` — a role title, never a person's name. `MD Life & Funeral`, not a surname. Roles outlive people and this column is referenced from `005 Principle Owner` and `003 Objective Owner`.
- `Headcount`, `Location` — `Location` is an `012 Location` ID, so load `012` first or leave it and come back.

How.

1. List the licensed entities from the licence schedule, with class and sub-class against each.
2. Decide the boundary: which entities this ADM cycle covers. Write down the exclusions with a reason.
3. Load `O12 Location` — Meridian has ten records, from `LOC-01` Sandton Head Office to `LOC-10` Gaborone, each with a `Criticality` tier.
4. Load `009 Organization Unit` top-down: group, then lines of business, then group functions, then subsidiaries.
5. Stop at the level where a unit owns a capability. Meridian stops at sixteen records; the deepest is `ORG-14 Enterprise Architecture`, parented to `ORG-10 Group Technology`.
6. Raise a change request in `03 Change Requests` for anything the metamodel cannot hold — starting with licence class.

Signed by. The Group Chief Executive, or the Executive Committee (`GB-04` in `057 Governance Body`). Nobody else can rule an entity out of scope.

Done when. Every `009` record has a `Parent` except one; every licensed entity either appears or appears on the written exclusion list; every `Head` value is a role title; and the boundary statement has a date and a signature.

Shows up in. `RPT12 Integrity Check`, row `009 Organization Unit`. Meridian's reads 16 records, 180 appearances as source, 11.25 links per record, `Linked` — the most heavily linked object type in the repository, because `R10 ORG-CAP` alone holds 134 ownership rows. Your row should read `Linked` within the first month, not `ISOLATED`.

Goes wrong when. You model to team level, and the structure is obsolete before Phase B. Or `Head` holds a person's name, and six months later the column is a graveyard. Or an offshore licensed entity is quietly omitted, and `CON-04` data residency under POPIA turns up in Phase D as a surprise instead of a constraint.

> **THE REPOSITORY HAS NOWHERE TO RECORD A LICENCE CLASS**
>
> `018 Product` carries `Line of Business`, `Policies In Force`, `GWP (Rm)`, `Config or Code` and `Status`. No licence class, no sub-class. For Meridian this is survivable; for a real composite group it is not, because a product that drifts outside its sub-class is unlicensed business. Raise a change request against the metamodel (`Type` = Extension, `Scope` = Product) and add two columns before you load a single product. `CR-017` in Meridian's register is the precedent for an extension of this kind — it added an object type and two relationship sheets, assessed in six days.

BEFORE YOU DRAW THE BOUNDARY

Which licensed entity are you most tempted to leave out, and what turns up in Phase D if you do?

Which role — not which person — can rule an entity out of scope, and has that role agreed to be asked?

At what level does a unit stop owning a capability in your group, and what does that make your deepest `009` record?

Working table — entities in and out of the boundary

Work the boundary here, then transcribe the in-scope rows into `009 Organization Unit` and the exclusions into the scope statement. Licence class has no column anywhere in the repository, so it stays on paper until the extension change request lands. Signed by the Group Chief Executive or `GB-04`.

Legal entity	Licence class / sub-class	In or out	Reason	ORG- ID
Example, delete me	Life, Risk	In	Writes the funeral book	ORG-03

Step 2.2 — Take the first baseline

Produce. A named, dated, approved snapshot of the repository as you received it — before you change anything.

You need. Step 2.1 complete only if you have already loaded data. If you inherited a populated repository, do this step first, ahead of 2.1.

Where it lands. `04 Baselines`, one row: `Baseline` (the next `BL-` number), `Name`, `Date`, `Workbook Version` from `01 Version Control`, `Approved By`, `Status` (Current, Superseded or Planned), `Note` saying what the baseline is for, and `Changes since`, which is derived — do not type it.

How.

1. Save a read-only copy of the workbook with the baseline ID in the filename.
2. Add the `04 Baselines` row, `Status` = Current, and set the previous current row to Superseded.
3. Copy `05 Export Attributes` and `06 Export Records` into the comparison tool's `10 Input A Baseline`, values only.
4. Set `01 Config` in the comparison tool: Snapshot A label and date to match the baseline exactly.
5. Record the approval in `O60 Governance Decision` if a body approved it.

Signed by. The Chief Enterprise Architect for the first baseline, the Architecture Board for every one after. Meridian did exactly this: `BL-001` approved by the Chief Enterprise Architect, `BL-002` by the Architecture Board.

Done when. Exactly one row in `04 Baselines` reads `Status` = Current, and the comparison tool's Snapshot A label matches its `Name` character for character.

Shows up in. `01 Version Control`, lower block, "Baselines taken". Meridian reads 2, against three rows in `04 Baselines` — because `BL-003` is still `Planned`, eight months after its date.

Goes wrong when. You cut the baseline after loading your own data, so you can never show what you inherited. Or two rows read `Current`, and every subsequent gap analysis quietly compares against the wrong one.

Step 2.3 — Tailor the metamodel, and write down what you are not doing

Produce. A three-column decision: of the 66 object sheets, which you populate in year one, which wait for year two, and which stay empty — with a reason against each in the third group.

You need. Step 2.2. The `RPT12 Integrity Check` reading from before step 2.1. Your honest estimate of architect-days available, which for a fourteen-person function like Meridian's `ORG-14` is less than you think.

Where it lands. Two places. The decision itself goes into `03 Change Requests` as a row with `Type` = Restructure, `Scope` listing the object types, and `Decision` carrying the three lists. The deliverable it produces is `DLV-07 Tailored Architecture Framework` in `O31 Deliverable` — Meridian's reads `In progress`, due 2026-12, still open.

How.

1. Start from the question each sheet answers, not from the sheet. If you cannot name the question, the sheet is year two.
2. For each candidate sheet, name the relationship sheet that will join it to something else. No relationship, no population.
3. Fix year one at around sixteen object sheets. Meridian's effective year-one set is `O01` – `O14`, `O20`, `O23`, `O29` – `O31`, `O57`, `O58`.
4. Put the heavy catalogues in year two or later: `O37 Data Attribute` (161 rows in Meridian, zero links), `O47 Reference Model Entry`, `O52 Implementation Factor`.
5. Write the reason for every sheet you are leaving empty. "Not needed yet" is a reason; "we never got to it" is what you are preventing.
6. Register the decision and date it.

Signed by. The Chief Enterprise Architect, noted to the Architecture Board. This is a scoping decision, not an architecture decision.

Done when. Every one of the 66 object sheets appears on exactly one of the three lists, and the lists are inside the repository rather than in somebody's notebook.

Shows up in. `RPT12 Integrity Check`. A sheet you deliberately left empty reads zero records. A sheet you populated and never linked reads `ISOLATED` — nineteen of Meridian's 66 types do, including all four building-block sheets and every data-governance sheet from `O41` to `O43`.

Goes wrong when. Nobody records the decision, so a successor reads the empty sheets as neglect and fills them. That is how a repository acquires 161 data attributes nothing references.

THE TAILORING DECISION

Which sheet are you about to populate because it is easy, rather than because you can name the question it answers?

For each sheet you are leaving empty in year one, what is the reason a successor will accept three years from now?

How many architect-days does year one actually have, and which sheet comes off the list when that number halves?

Step 2.4 — Adopt, edit or retire the 31 principles you were given

Produce. A principles catalogue where every record has been actively adopted, edited or retired, with a change reference — not a fresh set written from nothing.

You need. Step 2.3. Two hours each from the CIO, CDO, CRO, CISO and Chief Compliance Officer. Volume 1, chapter 15 for the conduct and prudential obligations the principles have to carry.

Look it up. Volume 1, chapter 15, *Treating Customers Fairly: six outcomes, six design consequences* — what the catalogue must carry.

Where it lands. `O05 Principle`, 31 shipped records:

Column	What goes in it
`ID`	Keep the prefix. `BP-` business, `DP-` data, `AP-` application, `TP-` technology, `SP-` security and risk, `GP-` governance, `IP-` information
`Category`	One of Business, Data, Application, Technology, Security & Risk, Governance. Note `IP-03` sits in Security & Risk, not Data
`Tier`	1, 2 or 3. Tier is the conflict-resolution rule: under `GP-03`, where two principles conflict the higher tier prevails
`Statement (condensed)`	One sentence of consequence. Not the full statement, which lives in `DLV-02`
`Owner`	An executive role from `O09 Head`, or a body from `O57`
`Status`	Draft, or your adopted equivalent. See the warning

How.

1. Read all 31 in one sitting. It takes forty minutes.
2. Apply the motherhood test (below) to each. Mark it keep, edit or retire.
3. For an edit, change `Statement (condensed)` and nothing else first; re-tier only if the conflict rule needs it.
4. Retire by setting `Status`, never by deleting the row. The row is referenced from `R05 PRN-REQ` and `R34 PRN-GAP`.
5. Raise one change request covering the whole pass, as `CR-035` did for the five `IP-` principles.
6. Only then consider writing a new principle. You will usually find you do not need one.

Signed by. The Architecture Board (`GB-01`), which "approves dispensations with an expiry" and therefore has to own the rules it dispenses from.

Done when. Every one of the 31 records carries a `Change Ref` from this pass, and the count of `Tier` 1 principles is small enough that an executive can recall them. Meridian has nineteen of its thirty-one at Tier 1, which is too many for `GP-03` to resolve anything: if almost everything is Tier 1, tier cannot break a tie.

Shows up in. `RPT11 Principle Impact`: `Requirements governed`, `Gaps engaging it`, `Direct violations` and an `Assessment` of Applied, VIOLATED or Not yet applied. After a good pass, no principle reads `Not yet applied`. Meridian's `BP-04 Business Continuity` and `DP-01 Data is a Shared Enterprise Asset` both do — zero requirements, zero gaps, zero violations.

Goes wrong when. You write 40 new principles from nothing. They will be unobjectionable, unordered and unused. The second failure is subtler: you adopt all 31 unchanged, which means you have not tested a single one against your own insurer.

THE MOTHERHOOD TEST

Three questions, and a principle must pass all three.

1. **Would any sane executive argue the opposite?** If not, it is a motherhood statement. "Data is a Shared Enterprise Asset" fails on its own; what rescues `DP-01` is the clause that follows — data belongs to the enterprise, not to the system that captured it — because somebody does argue the opposite, every time a line of business refuses to release a file.
2. **Does it tell you what to do when two good things conflict?** `TP-01 Technology Independence` was re-tiered from 1 to 3 under `CR-012`, with the Board's reasoning recorded as independence yielding to other concerns. That is a principle doing work: it now loses, predictably, to something else.
3. **Can you name a decision it would have changed in the last year?** If you cannot, either the principle is decoration or nobody consulted it. `RPT11 Principle Impact` answers this mechanically — `Direct violations` greater than zero and `Requirements governed` at zero means a principle that is being broken and governs nothing.

EVERY PRINCIPLE READS STATUS = DRAFT

All 31 records in `O05 Principle` carry `Status` = Draft. Six of them also carry `Record Status` = Approved, a `Change Ref` and `Approved By` = Architecture Board — approved by the repository's own governance, and still labelled Draft in the business column. The `Status` column was populated at load and never maintained. Make correcting it your first change request: it is thirty-one cells, it takes ten minutes, and the derived `Record Status` column gives you the evidence for which value each one should hold.

TESTING THE 31 PRINCIPLES YOU INHERITED

The 31-row pass itself transcribes onto the Preliminary Phase form for `O05 Principle` in appendix F — keep, edit or retire, with the change reference. This is the thinking that fills it.

Pick three principles. For each, name the decision in the last year it would have changed.

Which Tier 1 principles would you demote, and once you have, what loses to what?

Which principle does your insurer break routinely, and who benefits from it being broken?

Step 2.5 — Bind every principle to at least one requirement

Produce. A relationship set that makes each principle testable: this principle governs these requirements.

You need. Step 2.4. The 20 strategic requirements in `O06 Requirement` with `Source` = "Phase A strategic", or, if you are running Preliminary before Phase A, the five or six obligations you already know about.

Where it lands. `R05 PRN-REQ`: `Source (PRN)`, `Target (REQ)`, `Relationship`, `Note`, and `Key` in the form `BP-01|REQ-20`. The `Source Name` and `Target Name` columns resolve by formula — a lookup error there means the ID does not exist.

How.

1. Take the Tier 1 principles first. They are the ones an executive will be asked to defend.
2. For each, find the requirement it constrains. If none exists, the requirement is missing, not the link.
3. Write the `Relationship` value as a verb that survives scrutiny: governs, constrains, permits.
4. Add the corresponding `R34 PRN-GAP` rows when a gap engages a principle. Meridian holds 22, across 18 principles.
5. Leave a principle unbound rather than inventing a requirement for it. The empty cell is the finding.

Signed by. The Chief Enterprise Architect. The Board sees the result in `RPT11`, not the row.

Done when. No Tier 1 principle has zero rows in `R05 PRN-REQ`.

Shows up in. `RPT11 Principle Impact`, `Requirements governed`. Also `RPT08 Traceability`, which carries a `Governing Principles` column per requirement and a `Chain` column that shows whether the thread runs driver to objective to requirement to application to work package. Meridian's `R05 PRN-REQ` holds 23 links from 18 of its 31 principles, concentrated in `BP-` with eight. Thirteen principles govern nothing.

Goes wrong when. The architecture function writes the links alone, in an afternoon. They will be defensible and nobody will have agreed them. Do it in the same workshop as step 2.4, while the owners are in the room.

Step 2.6 — Constitute the governance bodies

Produce. Five or fewer bodies, each with a mandate stated as what it may decide and — the column that matters — what it may not.

You need. Step 2.1 for the executive structure. An existing committee calendar, because a new forum nobody attends is worse than an old one you borrow.

Where it lands. `057 Governance Body` : `ID` , `Body` , `Cadence` , `Membership` , `May Decide` , `May Not Decide` , `Standing` .

How.

1. Start from the five Meridian ships and rename them to your forums: `GB-01` Architecture Board, `GB-02` Investment Committee, `GB-03` Data Governance Council, `GB-04` Executive Committee, `GB-05` Architecture Function.
2. Fill `May Not Decide` before `May Decide` . It is harder and it is where the argument is. Meridian's Architecture Board cannot approve funding, cannot change a work package budget, and cannot waive a regulatory control objective.
3. Put the separation in writing: the body that funds (`GB-02`) does not judge compliance, and the body that judges compliance does not fund.
4. Name the escalation point for a deadlock, and say what it means when it is used. Meridian's `GB-04` entry is blunt: if the Executive Committee decides architecture, the Board has failed.
5. Fill `Standing` honestly. It is a status column, not a design column.
6. Book the first four meetings before the sheet is signed.

Signed by. The Group Executive Committee. A governance body cannot constitute itself, and `GB-05` records the reason: the architecture function "cannot grant its own dispensations or approve its own work".

Done when. Every body has a non-empty `May Not Decide` , a `Cadence` , and four dated meetings in a diary.

Shows up in. `RPT31 Governance Activity` , which lists decisions, change requests and risks by owner. A healthy board produces decisions at a steady rate. Meridian's shows seven decisions and two outstanding, with no escalations at all — which, against thirteen non-compliant acceptance criteria, is itself the finding.

Goes wrong when. `Standing` is written as an aspiration. Read Meridian's: the Architecture Board "has never refused anything"; the Data Governance Council "has never reached quorum", with ten requirements and R96.4 million gated on it; the Executive Committee has "never been used, because nothing has escalated". Three true sentences that an annual report would never contain, and the most useful column in the sheet.

WHAT YOUR BOARD MAY NOT DECIDE

Write the three things your architecture board may not decide, in the words you would use on the executive who wants it to.

Which body funds and which judges compliance — and what happens the first time one person chairs both?

Where does a deadlock go, and what will its first use say about the board?

Step 2.7 — Define the gates, and insist on G1 and G3

Produce. Four gates, each with a test, a holder and a stated reason for existing.

You need. Step 2.6. The delivery lifecycle your programmes actually run, whatever it is called locally, because a gate has to attach to a milestone that exists.

Where it lands. `058 Governance Gate` : `ID` , `Name` , `When` , `What It Tests` , `Why It Exists` , `Held By` — which must name a body in `057` .

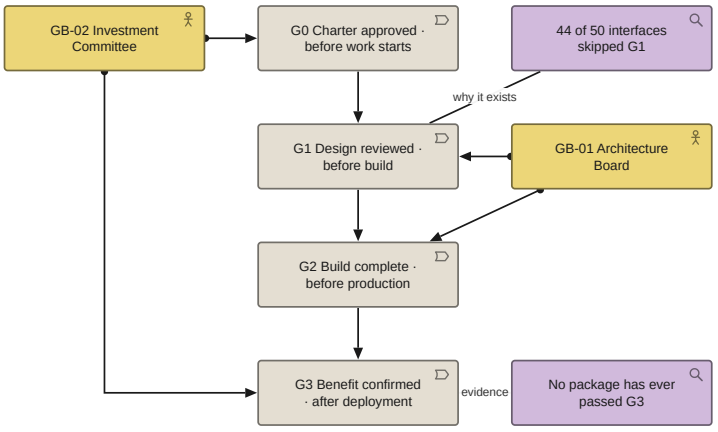

Figure 2.1 The four gates of `058 Governance Gate` in ArchiMate 4. A gate is an `Event` — a point the work either passes or does not — triggering the next; the body that holds it is a `Business Actor` assigned to it. Where the assessing behaviour itself has to be modelled it is a separate `Process`, not the gate. The two assessments are the findings: `G1` and `G3` are the gates that go missing, and the repository records what that cost. Meridian Assurance Group is invented.

How.

1. Adopt the four as shipped unless you have a reason: `G0` charter approved, before work starts; `G1` design reviewed, before build; `G2` build complete, before production; `G3` in production and benefit tracked, after deployment.
2. Phrase `What It Tests` as something that can fail. `G0` tests that acceptance criteria are inherited from the building blocks rather than written fresh — a charter with its own criteria drifts from the architecture within a quarter, which is `DEC-G02` in Meridian's decision log.
3. Assign `Held By` across two bodies, not one. Funding gates to the Investment Committee, design and build gates to the Architecture Board.
4. Define what a dispensation is and that it must carry an expiry date. `CR-G01` in `061 Governance Change Request` is the worked example: point-to-point file transfer permitted in `PRJ-04` until June 2029, lapsing when `PRJ-03` delivers.
5. Agree the consequence of a failed gate in advance. Who stops the work, and who tells the sponsor.

Signed by. The Group Executive Committee, on the Architecture Board's recommendation.

Done when. All four gates have a `Held By` naming a constituted body, and at least one real piece of work has been through `G0`.

Shows up in. `059 Compliance Assessment` — every row carries a `Gate` and a `Verdict` with `Evidence` — and in `RPT30 Compliance Position`, which pairs the verdict with the architecture contract status.

Goes wrong when. G1 and G3 are the two that go missing, every time, and for different reasons. **G1** is the only gate that can say no before the money is spent, which makes it the gate delivery leads resist and the one with no natural milestone to hang on; Meridian's consequence is in the sheet — 44 of 50 recorded interfaces reached production with no design review. **G3** goes missing because by the time benefit is measurable the team has dispersed and nobody volunteers to be measured; Meridian's `058` entry says it plainly: no package in the estate has ever passed `G3`, because it did not exist. A phase that ends at `G2` has delivered software and proved nothing.

THE TWO GATES THAT GO MISSING

Which existing milestone does `G1` hang on, and whose time is lost when it fails?

Who stops the work when a gate fails, and who tells the sponsor? Name both.

At `G3`, who is still in post when the benefit becomes measurable, and what are they measured on?

Step 2.8 — Write the organisational model for the architecture function

Produce. A description of the architecture capability itself: scope, posts, authority levels, and the vacancies you are carrying.

You need. Steps 2.1 and 2.6. Your actual headcount and the budget line it sits on.

Where it lands. Three sheets. `009 Organization Unit` for the function — Meridian's `ORG-14 Enterprise Architecture`, headcount 14, parented to `ORG-10 Group Technology`. `011 Role` for the posts, with `Domain` and `Authority Level`: `ROL-12 Solution Architect` is Advisory, `ROL-13 Domain Architect` is Mandated, `ROL-14 Architecture Board Member` is Approval. And the deliverable register, where

this artefact has two homes — `DLV-09 Organizational Model for EA` in `O31 Deliverable` and `DLV-15 Organization Model for Enterprise Architecture` in `O34 Architecture Deliverable`, with `DLV-05 EA Operating Model` in `O34` covering the same ground again.

How.

1. Write the authority levels first: who is advisory, who is mandated, who approves. The words matter more than the boxes.
2. Map every post to a `ROL-` record. A post with no role record cannot be referenced from `R08 ACT-ROL`.
3. State the vacancies. Meridian's `GB-05` entry names one: the Data Domain Architect post is vacant, and 23 of 24 data entities are owned by a role with no actor behind it.
4. Say where architects sit in a delivery programme and what they may not be asked to do.
5. Register the deliverable in one sheet and cross-reference the other. Do not maintain both.

Signed by. The Chief Information Officer and the Architecture Board. If the function reports elsewhere, say so in `O09 Parent` rather than in prose.

Done when. Every architect post has a `ROL-` record with an `Authority Level`, and every vacancy is named in writing.

Shows up in. `RPT14 Deliverable Register` — Meridian's `DLV-05 EA Operating Model` reads version 0.1, `Draft`, with four dependencies, fourteen months after the phase started.

Goes wrong when. The model describes the team you wish you had. The comparison tool catches this in its shipped example: `ROL-12 Authority Level` moved with no change request, classified by a reviewer as Variance with the note that recorded authority does not match practice and that the still-draft organisational model will settle it. It has not settled it. It is version 0.1.

Step 2.9 — Run the architecture maturity assessment

Produce. A scored baseline of the architecture capability, with evidence per element, and a target.

You need. Steps 2.6 to 2.8. Two days. Nine elements of assessment and somebody other than you to score them.

Where it lands. `DLV-04 Architecture Maturity Assessment` — the one ID that means the same thing in both registers, `Artifact` / Preliminary / Complete in `O31 Deliverable`, and `Capability assessment` / `Baselined` / version 1.0 in `O34 Architecture Deliverable`, where the description records an ACMM assessment across nine elements with a weighted mean. The score itself lands in the motivation sheets: `OBJ-12 Architecture maturity`, `Baseline` 2.0, `Target` 3.0, `Target Date` 2028, owner Chief Enterprise Architect; and `MSR-13 ACMM weighted mean`, `Unit` score 0-5, `Frequency` Annual, `Source System` Manual.

How.

1. Score the nine elements against evidence in the repository, not against opinion. "Architecture process" is not a 3 if `O59 Compliance Assessment` is empty.
2. Have the scores challenged by someone outside the function. A self-assessed maturity score is worth nothing to a board.
3. Write the weighted mean into `O04 Measure` `Baseline` and set a target you can defend as achievable.
4. Convert every element scoring 0 or 1 into a gap record in `O29 Gap` with `Domain` = Realization.
5. Raise one change request carrying both the score and the gaps, and let it move the severities.

Signed by. The Architecture Board, with the score reported to the Executive Committee.

Done when. Nine element scores exist with evidence, the weighted mean is in `O04 Measure`, and every element below 2 has a gap record.

Shows up in. `O29 Gap` and the change log together. Meridian's `CR-004` is the template: raised by the Chief Enterprise Architect, approved in five days, and it raised `GAP-13 No architecture repository` and `GAP-14 No architecture governance forum` from High to Critical, moved `CAP-26` current maturity from 1 to 2, and approved `WP-01`. One assessment, four logged changes, all traceable.

Goes wrong when. The assessment is run, filed and never referenced. Its purpose is to move severities and release a work package. If no gap severity changed as a result, you scored yourself for nothing.

Working table — the nine elements, scored against evidence

One row per element. `Evidence` is a sheet and a reading, not an opinion; anything below 2 becomes a row in `029 Gap` with `Domain` = Realization. Signed by the Architecture Board, and the weighted mean goes to `MSR-13 ACMM weighted mean`.

Element	Score 0–5	Evidence (sheet and reading)	Challenged by	Gap ID
Example, delete me	1	`059` empty, no verdicts	CIO	GAP-14

Step 2.10 — Load the drivers

Produce. The complete set of forces acting on the enterprise, each owned by an executive.

You need. The regulatory calendar from volume 1, chapter 15. The strategy document. The CFO's cost position. Volume 1, chapter 6 (How insurers make money) for the commercial drivers.

Look it up. Volume 1, chapter 15, *A calendar worth carrying*, and chapter 6, *Three consequences you cannot design around*.

Where it lands. `O01 Driver` : `Category` (Meridian uses Regulatory, Market, Environmental, Technology, Financial, Organisational), `Origin` (External or Internal), `Horizon` (a year range, "In force", "Ongoing", "Phased", or a date), `Owner`, `Status`.

How.

1. Take the regulatory drivers from the calendar first. Meridian holds eight of fifteen: `DRV-01` COFI Bill, `DRV-02` Joint Standard 1 of 2023, `DRV-03` Joint Standard 2 of 2024, `DRV-04` PA Guidance Notice G3/2025 on climate disclosure, `DRV-05` the FSCA and Prudential Authority report on artificial intelligence, `DRV-06` IFRS 17, `DRV-07` POPIA, `DRV-08` FICA and FATF remediation.
2. Add the market drivers the lines of business will recognise: claims inflation, digital competitors, affordability.
3. Add the internal drivers, which are the ones nobody writes down and which explain the estate: `DRV-13` legacy platform concentration, `DRV-14` cost of run at 71% of the technology budget, `DRV-15` change fatigue after two failed core replacements.
4. Give every driver an executive `Owner` from `O09 Head`.
5. Link each to at least one goal in `R01 DRV-GOL`, with `Strength`.

Signed by. The Group Executive Committee, as part of the strategy conversation rather than as an architecture artefact.

Done when. Every driver has an owner and at least one row in `R01 DRV-GOL`, and the internal drivers are present.

Shows up in. `RPT12 Integrity Check`, row `O01 Driver`: Meridian's reads 15 records, 19 as source, 0 as target, 1.27 links per record, `Linked`. Drivers are never a target — nothing in a well-formed repository points at a driver.

Goes wrong when. Only the regulatory drivers get loaded, because they are documented elsewhere and easy. A driver set with no internal entries cannot explain why the architecture looks the way it does, and every roadmap built on it will read as a compliance programme.

Step 2.11 — Lay the motivation spine: goals, objectives and measures

Produce. A four-hop chain from each driver to a number somebody reports monthly. Draw it as an ArchiMate 4 motivation view: `Driver`, `Goal`, `Outcome` and `Assessment`, joined by influence and realisation, with the `Stakeholder` who owns the driver beside it.

You need. Step 2.10. The CFO's plan and the board scorecard. Volume 1, chapter 6 for which numbers an insurer actually steers by.

Look it up. Volume 1, chapter 6, *Reading the ratios*. A measure resolving to none of the three is not what the board steers by.

Figure 2.2 The motivation spine in ArchiMate 4. A driver influences a goal, an outcome realises the goal, and a measure assesses the outcome — `R01`, `R02` and `R03`. Walk any measure back to a driver in three hops, or the chain is broken and `RPT08 Traceability` will show it.

Where it lands. Three object sheets and three relationship sheets.

Sheet	The columns that decide whether it is useful
`002 Goal`	`Pillar` groups goals for a board paper; `Owner`; `Target Date`. Meridian has six goals on six pillars — Customer, Efficiency, Conduct, Growth, Resilience, Capability
`003 Objective`	`Goal`, `Baseline`, `Target`, `Target Date`, `Owner`. An objective without a `Baseline` is a wish
`004 Measure`	`Objective`, `Unit`, `Baseline`, `Current`, `Target`, `Frequency`, `Source System`. `Source System` is an `APP-` ID or the word Manual
`R01 DRV-GOL`	`Strength` — how hard the driver pushes
`R02 GOL-OBJ`	`Contribution` — what this objective contributes to the goal
`R03 OBJ-MSR`	`Type` and `Reporting` — leading or lagging, and where it is reported

How.

1. Write six goals or fewer, each on a pillar an executive already uses.
2. For each goal, write objectives with a `Baseline` you can evidence today. Meridian's `OBJ-03` reads 37.4% to under 32% by 2030; `OBJ-05` reads 71% to 60% by 2029.
3. For each objective, name one or two measures and the system that produces them.
4. Fill `R01`, `R02` and `R03` in the same sitting. The chain is the point; the catalogues are not.
5. Flag every measure whose `Source System` is Manual. Meridian has four of eighteen, including `MSR-13` — the maturity score from step 2.9.

Signed by. The Group Executive Committee for goals, the named executive owner for each objective.

Done when. No objective has an empty `Baseline` or `Target`; every objective has at least one measure; and you can pick any measure at random and walk it back to a driver in three hops.

Shows up in. `RPT08 Traceability`, columns `Source Driver`, `Objective`, `Goals served` and `Chain`. And `RPT12 Integrity Check`: `004 Measure` should read 0 as source and one link per record as target.

Goes wrong when. The spine is loaded once and never revisited. All four of Meridian's motivation sheets read `Unchanged since baseline` in `RPT13 Version Control` fourteen months in — the goals have not been tested against the drivers since the day they were typed, and `MSR-01` still reads a current value of 0 against a target of 100.

THE NUMBER THIS PHASE IS JUDGED ON

Which single number will this phase be judged on, who reports it, and out of which `Source System`?

Pick a measure at random and write the three hops back to its driver. Where does the chain break?

Which measures read `Manual`, and what happens to each one when the person who compiles it leaves?

Step 2.12 — Build the deliverable register and plan the year with it

Produce. A dated plan of architecture work for twelve months, expressed as deliverables with owners, not as a Gantt chart.

You need. Steps 2.3 and 2.8. The programme calendar, so architecture deliverables land before the decisions that need them.

Look it up. Volume 1, chapter 18, *The ninety-day plan*, dates the first quarter row by row.

Where it lands. `O31 Deliverable` for the work products: `Type` (Deliverable, Artifact, Supporting reference), `ADM Phase`, `Work Package`, `Owner`, `Due` as a year-month, `Status`, `Repository Location`. And `O34 Architecture Deliverable` for the governance view of the same things: `Artifact Type`, `ADM Phase`, `Formats`, `Deliverable Version`, `Lifecycle Status`, `Owner Org`, `Approver`.

How.

1. Decide which of the two registers is authoritative for your insurer, and record the decision. They overlap, they use the same `DLV-` ID space for different records, and maintaining both by hand guarantees divergence.
2. Normalise `ADM Phase`. `O31` uses "Preliminary", "A", "Preliminary / A", "B, C, D"; `O34` uses "Preliminary", "Phase A", "Governance". Pick one vocabulary and write it into the tailoring decision from step 2.3.
3. Sort by `Due` and read the first four quarters. That is your year.
4. Set `Owner` to a role and `Approver` to a body from `O57`.
5. Fill the dependencies in `R40 DLV-DLV` with `Nature`, and the organisational responsibilities in `R39 ORG-DLV` with `Role`.
6. Re-read the plan against capacity. Meridian's fourteen-person function carries twenty deliverables across nine phases.

Signed by. The Architecture Board, as the plan it will hold you to.

Done when. Every Preliminary and Phase A deliverable has an owner, an approver and a `Due` inside twelve months, and nothing is due in a month when the owner is unavailable.

Shows up in. `RPT14 Deliverable Register`, which adds `Depends on` and `Depended on by` counts per deliverable. A deliverable depended on by three others and still `Draft` is your critical path.

Goes wrong when. The register becomes a wish list. Ten of Meridian's twenty `O31` rows read `Not started`, and six of those are past their `Due` date against a repository state of 2027-11-20 — including `DLV-10 Request for Architecture Work`, which was due in 2026-11 and is the artefact that authorises Phase A at all.

Step 2.13 — Run the first integrity check, and learn to read it weekly

Produce. Three numbers you can quote from memory about your own repository's health: isolated object types, records pending approval, and the worst break in the data catalogue chain.

You need. Everything above. Twenty minutes now, twenty minutes weekly thereafter.

Where it lands. Nowhere. `RPT12 Integrity Check`, `RPT13 Version Control` and `RPT15 Catalogue Completeness` are report sheets — every cell is a formula over the object and relationship sheets. You do not type into them. What lands is a change request for each defect they expose.

How.

1. Read `RPT12 Integrity Check` and write down the count of `ISOLATED` types. Decide, for each, whether it is deliberate under step 2.3 or neglect.
2. Read `RPT13 Version Control` and write down the total in `Pending approval`. Chase every one.
3. Read `RPT15 Catalogue Completeness` and write down the lowest `Links` score and what `Where it breaks` says about it.
4. Raise one change request per genuine defect, not one for all of them. `CR-015` is the precedent: `RPT12` flagged two entity types with no relationships, and the change request closed them in five days.
5. Put the three numbers on the Architecture Board's standing agenda.

Signed by. Nobody. This is the architect's own instrument, and `GB-05` records that the architecture function maintains the repository and raises change requests as part of its standing mandate.

Done when. You can state the three numbers without opening the file, and every defect either has a change request or a recorded reason for being left alone.

Shows up in. Itself, next week. A healthy trend is isolated types falling as relationship sheets fill, and `Pending approval` returning to zero within a fortnight of each Board meeting.

Goes wrong when. These sheets are read once, at the end of a phase, as a formality. By then the defects are six months old and the person who created them has moved on. Read them weekly and each one costs ten minutes to fix.

RPT12, RPT13 and RPT15 — what to read and what the number should be

`RPT12 Integrity Check` — `Records`, `As source`, `As target`, `Total links`, `Links per record`, `Status`. The `Status` column grades each object type `Linked`, `Thinly linked` or `ISOLATED`. Meridian: 40 linked, 7 thinly linked, 19 isolated. Watch for a type that moves from `Linked` to `Thinly linked` as records are added faster than relationships.

`RPT13 Version Control` — `Changes logged`, `Records changed`, `% changed`, `Latest change`, `Pending approval`, `Assessment`. Two readings matter. `Pending approval` above zero means live records nobody has approved. `Assessment` = "Unchanged since baseline" on a sheet you are actively working means your changes are not in the log, which means they did not happen.

`RPT15 Catalogue Completeness` — a six-link chain per information concept: `Named`, `Defined`, `Placed`, `Stored`, `Sourced`, `Consumed`, with `Links` as a score out of six and `Where it breaks` naming the first empty cell. Meridian's `DEN-01 Party`, `DEN-02 Customer` and `DEN-03 Contact Detail` all read 5/6, breaking at "Contested source" — three entities whose authoritative source is recorded as CONTESTED rather than named. That is the honest state of a customer master nobody owns.

Step 2.14 — Close the phase against a baseline

Produce. A decision record, a cut baseline, and a gap set that tells the truth about what Preliminary did not fix.

You need. Steps 2.1 to 2.13. One Architecture Board meeting with a decision on the agenda, not a status update.

Where it lands. `O60 Governance Decision` (`Date`, `Decision`, `Body`, `Status`, `Why`, `Affects`), `04 Baselines` for the snapshot, `O29 Gap` for what remains, and `O31 Deliverable` statuses moved to Complete.

How.

1. Put the decision to the Board as a question it can refuse. Meridian's `DEC-G01` is the shape: adopt four gates, with the reasoning recorded as 44 interfaces having reached production without a design gate.
2. Record every Preliminary gap that is still open, with severity. `GAP-13` and `GAP-14` both stayed Critical through this phase.
3. Cut the baseline — Meridian's `BL-002 Governance Establishment Baseline`, 2026-12-16, workbook v1.9, approved by the Architecture Board, noted as the reference point for Phase A gap analysis.
4. Set the comparison tool's Snapshot A to the new baseline, and keep the old extract.
5. Move `O31 Deliverable` rows to Complete only where `Repository Location` and `Approved By` are both filled.
6. Check `RPT13 Pending approval` reads zero for every sheet you touched.

Signed by. The Architecture Board, with the Executive Committee noting it.

Done when. `04 Baselines` has exactly one `Current` row dated this month; `RPT13 Version Control` shows no pending approvals on the sheets you touched; and every open Preliminary gap has a severity and a work package.

Shows up in. `RPT13 Version Control` and `RPT31 Governance Activity` together. A closed Preliminary Phase looks like a baseline, a decision record with more than one entry, and at least one gap raised by the phase itself rather than inherited.

Goes wrong when. You declare the phase complete while the `Standing` column in `O57 Governance Body` still says the bodies have never met. That is Meridian's exact position: `WP-01 Establish Architecture Governance` was approved, funded at R6.4 million, and reads `Delivery Status` = "Past end date, not evidenced", with `CA-01` recording the acceptance criterion "a design can be refused and the refusal recorded" as `Non-compliant` because `GAP-13` and `GAP-14` are both still open. The deliverables exist. The capability does not.

Before you move to Phase A

THE PRELIMINARY PHASE — A REVIEWER'S QUESTIONS

What has the Architecture Board refused, and where is the refusal recorded? If the answer is nothing, `G1` does not exist yet, whatever `058 Governance Gate` says.

Which bodies in `057 Governance Body` have met with quorum, and what does `Standing` say about the rest?

What did this phase raise rather than inherit — which gap, which change request, which severity moved?

Which sheet reads "Unchanged since baseline" in `RPT13 Version Control` that you believe you have been working on?

THE CHAPTER IN TEN LINES

- Cut a baseline in 04 Baselines before you change a cell. If you inherited the repository, do it first.
- Tailor the metamodel explicitly: of 66 object sheets, name the year-one set and write a reason against every sheet you leave empty.
- Adopt, edit or retire the 31 shipped principles. Do not write new ones from nothing.
- The motherhood test: would anyone argue the opposite, does it resolve a conflict, and name the decision it would have changed.
- All 31 principles read Status = Draft, six of them while also reading Record Status = Approved. Fix that first.
- Fill May Not Decide before May Decide in O57 Governance Body, and fill Standing honestly.
- G1 design review and G3 benefit tracking are the two gates that go missing. 44 of 50 interfaces reached production without G1.
- The maturity assessment earns its keep only if it moves a gap severity or releases a work package.
- Load internal drivers, not just regulatory ones. DRV-13, DRV-14 and DRV-15 explain the estate.
- Close the phase with a decision record, a cut baseline and zero pending approvals on the sheets you touched.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets O57 Governance Body and O58 Governance Gate
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets O30 Work Package, O59 Compliance Assessment and O60 Governance Decision
- **3.** gov.za / SARB, "Commencement of Insurance Act 18 of 2017 and Prudential Standards" (29 June 2018) https://www.gov.za/news/media-statements/south-african-reserve-bank-commencement-insurance-act-2017-and-prudential
- **4.** Insurance Act 18 of 2017, Schedule 2 — classes and sub-classes of life and non-life insurance business https://www.acts.co.za/insurance-act-2017/table_2__classes_and_sub-class
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheets O05 Principle, R05 PRN-REQ and RPT11 Principle Impact
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheets O01 Driver, O02 Goal, O03 Objective, O04 Measure and RPT08 Traceability
- **7.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method, Preliminary Phase https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 3

Architecture Vision

Two documents and one signature. Meridian produced neither, and the next three phases have been measured against the wrong baseline ever since.

Phase A is where the architecture gets a mandate or quietly fails to. It produces two documents and one agreement: a vision somebody senior will defend, a statement of work that says what is and is not in scope, and a signature that converts opinion into authority. Eleven steps, four to six weeks, and most of the effort goes on the conversations rather than the sheets.

Meridian Assurance Group is invented and its numbers are illustrative — but its Phase A record is instructive, because the repository shows the phase stalling in a very ordinary way. `DLV-10 Request for Architecture Work` in `O31 Deliverable` is owned by the Group Chief Executive, was due 2026-11, and reads `Not started` against a repository state of 2027-11-20. `DLV-11 Architecture Vision` and `DLV-12 Statement of Architecture Work` were both due 2027-02 and both read `Not started`. `BL-003 Phase A Entry Baseline`, planned for 2027-03-31, has never been cut [1]. Fourteen months of work went into the Preliminary Phase and nobody wrote the paragraph that authorises the next one.

> **BEFORE YOU START THIS PHASE**
>
> Phase A is conversation. These let you hold it without an interpreter.
>
> - **Volume 1, chapter 5, *Running the discovery session*.** Four questions that work on any role.
> - **Volume 1, chapter 4, *The canvas for a composite insurer*.** The nine blocks, already filled.
> - **Volume 1, chapter 4, *Customer journeys, done honestly*.** Four worked journeys, and the rows a map needs.
> - **Volume 1, chapter 17, *The motivation view: why the group is changing*.** A driver-to-estate argument, worked.

The shape of the phase

Steps	What they settle
3.1	The mandate, and what to do when nobody will write one
3.2 – 3.4	Who the stakeholders are, including the ones architects forget
3.5 – 3.6	The business model and the value streams, named before capabilities
3.7	Scope: breadth, depth, horizon, and which lines of business are in
3.8 – 3.9	Constraints, assumptions, and the strategic requirements
3.10 – 3.11	Vision, statement of work, and the gate

Step 3.1 — Get a Request for Architecture Work, or write one and have it signed

Produce. A one- or two-page request naming the sponsor, the business problem, the scope, the time box and the authority being granted.

You need. A sponsor. The strategy document. The drivers from step 2.10, because the request should quote two or three of them by ID.

Where it lands. `O31 Deliverable`, row `DLV-10`: `Type` = Deliverable, `ADM Phase` = "Preliminary / A", `Work Package` = the governance package, `Owner` = Group Chief Executive, `Due`, `Status`, `Repository Location`. Note the `ADM Phase` value spans two phases — the request is raised in Preliminary and consumed in A.

How.

1. Ask the sponsor for it once, in writing, with a template attached.
2. When that produces nothing — and in most insurers it does — write it yourself, in their words, in under 500 words.
3. Quote the drivers by ID and the objectives by ID. `DRV-14 Cost of run` at 71% of the technology budget and `OBJ-05` at 71% falling to 60% say more than two paragraphs of narrative.
4. State what you will not do. A request with no exclusions will be read as a blank cheque and then resented.
5. Put it in front of the sponsor as a document to sign rather than to write. Signature, date, and a named deputy.
6. Record it in `O31 Deliverable` with `Status` = Complete and the signature location in `Repository Location`.

Signed by. The Group Chief Executive or the executive who owns the problem. Not the CIO on the CEO's behalf, unless the CIO is the sponsor.

Done when. A signed document exists, the `O31 Deliverable` row is Complete, and you can name the person who would answer for it in a board meeting.

Shows up in. `RPT14 Deliverable Register`. In Meridian, `DLV-10` is depended on by the Phase A deliverables and is itself the thing nobody started — which is exactly why `DLV-11` and `DLV-12` never moved.

Goes wrong when. You start Phase A without one, on the reasoning that everybody knows what you are doing. Six weeks later the scope is contested, there is no document to point at, and the argument is conducted in a steering committee by people who were not in the original conversation. Write the two pages. They are the cheapest insurance in the ADM.

BEFORE YOU ASK FOR THE MANDATE

Write the architecture work in one sentence your sponsor would say out loud, with you out of the room.

Who owns the problem this work solves, and what are they measured on that it would move?

If nobody will sign, which fallback are you taking — minute, funded work package, or recorded silence — and what does it cost you in authority?

WHEN NOBODY WILL SIGN

Three fallbacks, in order of preference. **One:** get the request minuted instead. A decision recorded in `O60 Governance Decision` with `Body`, `Date`, `Why` and `Affects` is weaker than a signature but it is evidence. **Two:** attach the architecture work to a funded work package that already has approval — Meridian's `WP-01 Establish Architecture Governance`, approved and funded at R6.4 million, carries the Preliminary and Phase A deliverables in `R29 WPK-DLV`. **Three:** write the request, send it, and record the absence of a reply in the deliverable register as `Not started` with the date you asked. That last one is not cynical. It is the difference between a phase that is blocked and a phase that is late.

Step 3.2 — Identify the stakeholders, and find the ones who are missing

Produce. A stakeholder list that includes every person who can stop the work, not just the people who want it.

You need. Step 2.1. Volume 1, chapter 5 (Who does what), which is the insurance-specific stakeholder set: what each role does all day, the decision they own, the data they need and what they complain about. Volume 1, chapter 15 for the statutory control functions.

Look it up. Volume 1, chapter 5, *One process, every role*, for the hand-offs an org chart hides.

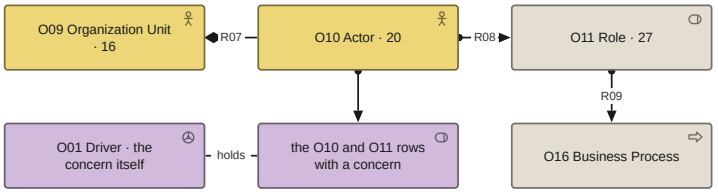

Figure 3.1 Three sheets, three ArchiMate 4 elements. An organisation unit is composed of actors (`R07`), an actor is assigned to a role (`R08`), and a role is assigned to a process (`R09`). The same `O10` and `O11` rows are drawn as `Stakeholder` in a motivation view, where the concern each one holds is a `Driver`.

Where it lands. Three object sheets and two relationship sheets.

Sheet	Columns that matter here
`O09 Organization Unit`	`Head` — the role title that speaks for the unit
`O10 Actor`	`Type` (Person or Organisation), `Internal/External`, `Organization Unit`, `Population` — the count, which tells you whether you are designing for 14 people or 1.9 million
`O11 Role`	`Domain`, `Authority Level` (Transactional, Mandated, Delegated authority, Responsible, Accountable, Advisory, Approval, Statutory), `Organization Unit`
`R07 ORG-ACT`	`Count` — how many of that actor the unit employs
`R08 ACT-ROL`	`Basis` — on what basis the actor performs the role

How.

1. Start from the 20 actors and 27 roles shipped. Read them against volume 1, chapter 5 and mark what is absent.

2. Add the statutory control functions. `ROL-16 Information Officer` is there, marked `Authority Level` = Statutory. The Head of Actuarial Function is not, and neither is the Chief Risk Officer as a role — the CRO appears only as the `Head` of `ORG-09` and as an `Owner` value in other sheets.
3. Add the intermediated set. `ACT-02 Independent Broker` covers brokers; it does not cover a binder holder or an underwriting manager, who underwrite and in many cases settle claims on your licence. They are external actors with internal authority, which is why they are easy to miss and expensive to miss.
4. Add the reinsurance manager. `ACT-08 Reinsurer` exists, and `CTR-02` and `CTR-03` record the surplus treaty and the catastrophe programme at R980 million and R640 million a year. No role owns them.
5. Fill `Population` on every actor. It changes the design.
6. Raise one change request for the whole addition. `CR-021` is the precedent: the Business Domain Architect extended the role catalogue and added two external actors, eleven logged changes, approved in seven days.

Signed by. The Chief Enterprise Architect, with the list reviewed by the Chief Operating Officer and the Chief Risk Officer. They will find the omissions you cannot.

Done when. Every statutory control function appears in `O11 Role` with `Authority Level` = Statutory; every external party that can bind the insurer appears in `O10 Actor`; and every actor has a `Population`.

Shows up in. `RPT12 Integrity Check`: `O10 Actor` should show links as both source and target, and `O11 Role` likewise through `R09 ROL-BPR`. Meridian reads 20 actors with 24 links and 27 roles with 39.

Goes wrong when. The stakeholder list is the attendee list of the kick-off meeting. The people who stop architecture in an insurer are rarely in that room: the Head of Actuarial Function who will not sign, the Information Officer who will not approve a data flow, the binder holder whose system is the one your straight-through processing has to talk to, and the reinsurance manager running a R1.6 billion programme from a spreadsheet.

THE STAKEHOLDERS WHO CAN STOP THIS

Who can stop this work who was not in the kick-off meeting? Name three, by role, not by name.

Which external party can bind your insurer — binder holder, underwriting manager, administrator — and whose system must yours talk to?

Which statutory control function will refuse to sign, on what evidence, and when will you find that out?

Step 3.3 — Record the stakeholder assessment somewhere it will survive

Produce. Per stakeholder: what they care about, what they can block, what they need from this architecture, and how often you will talk to them.

You need. Step 3.2. An hour each with six to ten people.

Where it lands. Honestly: nowhere clean. The repository has no stakeholder sheet. `O34 Architecture Deliverable` records `DLV-11 Strategic and Motivation Model` as holding eighteen stakeholders and twenty-seven assessments — but no object sheet in the metamodel holds a stakeholder record, so that content lives inside the deliverable, outside the repository, and nothing links it to anything.

How.

1. Decide, and record the decision, where the assessment lives. Two defensible options.
2. Option one: keep it in the deliverable, register the deliverable properly in `O34 Architecture Deliverable` with `Deliverable Version`, `Lifecycle Status`, `Owner Org` and `Approver`, and accept that it is not queryable.
3. Option two: raise a change request to add a stakeholder object type with its own sheet and a relationship sheet to `O11 Role`. `CR-025` is the precedent for introducing a new object type — it added Business Building Block as a type in its own right.
4. Either way, write the assessment against `ROL-` IDs rather than names, so it survives a reorganisation.
5. Set a review cadence and put it in the governance calendar.

Signed by. The Chief Enterprise Architect. This is a repository design decision, not an architecture decision.

Done when. Every stakeholder in the assessment is identified by a `ROL-` or `ACT-` ID, the assessment has a version and an owner, and a successor could find it without asking you.

Shows up in. `RPT14 Deliverable Register`, as a registered deliverable with a version and a dependency count. If you took option two, in `RPT12 Integrity Check` as a new object type — which should not read `ISOLATED` within a month.

Goes wrong when. The assessment is a slide in a pack. Six months later the pack is on a personal drive and the insight is gone. `GAP-13 No architecture repository` describes precisely this failure — architecture content held in project sites and personal drives — and it is still Critical.

Working table — the stakeholder assessment

The repository has no stakeholder sheet, which is why this one is on paper. Use `ROL-` or `ACT-` IDs, never names, so it survives a reorganisation. Six to ten rows is a first pass; file it as the registered deliverable from step 3.3 and set its review cadence.

ROL- or ACT- ID	What they care about	What they can block	What they need from this	Cadence
ROL-16	Lawful processing	A data flow	POPIA position per interface	Monthly

Step 3.4 — Build the business model canvas as a Phase A artefact

Produce. One page showing how the insurer makes money today and one showing how it intends to, with the differences marked.

You need. Step 3.2. Volume 1, chapter 4 (How the business model works), which gives the canvas for a composite insurer and the warning that it differs in every line of business. Volume 1, chapter 6 for the economics behind the numbers you will be quoted.

Look it up. Volume 1, chapter 4, *What the canvas leaves out*. Those omissions are what the CFO attacks first.

Where it lands. Two registers, two IDs, one artefact — read the warning below before you file it. In `O31 Deliverable`, `DLV-05 Business Model Canvas`, `Type` = Artifact, `ADM Phase` = Preliminary, `Status` = Complete. In `O34 Architecture Deliverable`, `DLV-10 Business Model Canvas`, `Artifact Type` = Business Model, `ADM Phase` = Phase A, `Formats` = Excel; PowerPoint, `Deliverable Version` 1.1, `Lifecycle Status` = Draft, `Approver` = Group Executive Committee.

How.

1. Draw the current canvas from the repository rather than from a workshop: `O18 Product` gives the products with `Policies In Force` and `GWP (Rm)`, `O19 Contract` gives the distribution and reinsurance agreements with values, `O10 Actor` gives the channels and their populations.
2. Do it per line of business where the model genuinely differs. Meridian's funeral book — `PRD-01`, 742,000 policies, R4,180 million of premium, collected through a 312-parlour network — has almost nothing in common with `PRD-10 SME Multi-Peril` at 38,000 policies.
3. Draw the target canvas from `O02 Goal` and `O03 Objective`. `OBJ-08` moving digital and embedded new business from 6% to 18% is a change to the channels block, not an aspiration.
4. Mark every block that changes, and nothing else.
5. Take it to the Group Executive Committee as the approver named in the register.

Signed by. The Group Executive Committee. The canvas is a claim about the business, not about the architecture.

Done when. Both canvases exist on one page each, every changed block traces to an objective ID, and the deliverable has a version above 1.0 and a `Lifecycle Status` above Draft.

Shows up in. `RPT14 Deliverable Register`. Meridian's reads version 1.1, `Draft`, and the `O31` row for the same artefact reads `Complete` — the same artefact, two registers, two different states.

Goes wrong when. The canvas is drawn in a workshop with no numbers in the room, which produces a picture everybody agrees with and nobody can act on. The repository has the numbers. Use them.

> **ONE ARTEFACT, TWO REGISTERS, CONTRADICTORY STATUS**
>
> The business model canvas is `DLV-05` in `O31 Deliverable` and `DLV-10` in `O34 Architecture Deliverable`. `O31` files it under Preliminary and calls it Complete; `O34` files it under Phase A, version 1.1, Draft. Both rows are v1.0 `Baseline` records that nobody has reconciled. This is the collision chapter 1 warns about, and this is what it costs: you cannot answer "is the canvas done?" from the repository. Decide in step 2.12 which register is authoritative, and make the other one point at it.

Step 3.5 — Name the value streams before you touch capabilities

Produce. Five to eight value streams, each with a triggering stakeholder, a value proposition in the stakeholder's words, and a stage count. The view to draw is an ArchiMate 4 strategy view: one `Value Stream` per row, the `Stakeholder` that triggers it, and the `Value` it delivers.

You need. Steps 3.2 and 3.4. Volume 1, chapter 4 for the customer journey work and chapter 8 (The process model) for where streams and processes differ.

Look it up. Volume 1, chapter 8, *L0, L1, L2, L3 — and the warning*, for the grain.

Where it lands. `O14 Value Stream`: `Name` as a verb phrase, `Description` from trigger to outcome, `Triggering Stakeholder`, `Value Proposition`, `Stages` as a count, `Owner Org` as an `ORG-` ID.

How.

1. Start from the seven Meridian ships, because they are the seven an insurer always has: `VST-01 Acquire Cover` (6 stages), `VST-02 Settle a Claim` (7), `VST-03 Service a Policy` (5), `VST-04 Renew Cover` (4), `VST-05 Launch a Product` (6), `VST-06 Onboard an Intermediary` (5), `VST-07 Report to a Regulator` (5).
2. Write `Value Proposition` in the stakeholder's words, not yours. Meridian's `VST-02` reads "Money arrives without a fight". That sentence has survived three governance reviews because nobody can argue with it.
3. Name the `Owner Org`. A stream with no owning unit will be optimised by whoever shouts loudest.
4. Count the stages. Do not name them yet; stages belong in Phase B.
5. Leave `O13 Business Capability` alone until Phase B. Streams tell you which capabilities matter; the capability map built first always mirrors the org chart.

Signed by. The Chief Operating Officer, with each stream's `Owner Org` head agreeing their stream.

Done when. Every stream has a triggering stakeholder who is an `ACT-` record, a value proposition a customer would recognise, and an owning unit.

Shows up in. `R11 VST-CAP` in Phase B, which joins streams to capabilities — 50 rows in Meridian, every one marked `Criticality to stream` = High. And `R41 APP-VST`, 77 rows, which is how you discover that a stream nobody owns is supported by eleven applications.

Goes wrong when. The streams are named after departments: "Underwriting", "Claims Operations". A value stream that shares a name with an organisation unit is an organisation unit. The test is the trigger: if the thing that starts it is an internal decision rather than a stakeholder need, it is a process, not a stream.

WHY STREAMS COME BEFORE CAPABILITIES

A capability map built first is built from the organisation you can see, and it will have a branch per department. A capability map built after the value streams is built from what the enterprise has to be able to do to deliver value, and the departments fall out of it as owners. Meridian's own history shows the correction: the baseline load held 26 capabilities, and `CR-016` replaced them fourteen months later with a four-level, 556-record map — nineteen logged changes, approved in eight days. The second attempt was derived from the streams. The first was not.

The repository also shows the cost of stopping half-way. Every one of the 50 rows in `R11 VST-CAP` has an empty `Stage` column, so the repository knows which capabilities a stream needs and not where in the stream they are needed. And `Criticality to stream` reads High on all 50 rows, which means the column is recording nothing. A column with one value is a column somebody filled in to be finished.

Step 3.6 — Settle the scope, in four dimensions, in writing

Produce. A scope statement with four answers and an explicit exclusion list.

You need. Steps 3.1 and 3.5. The sponsor's time, once, with the exclusions in front of them.

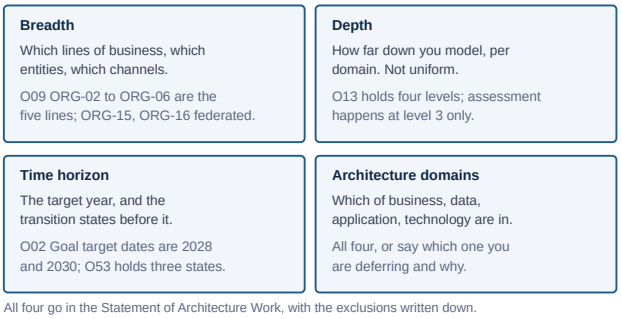

Figure 3.2 The four scope dimensions, with where each one is already half-answered in the repository. All four belong in the Statement of Architecture Work, exclusions included.

Where it lands. The scope section of `DLV-12 Statement of Architecture Work` in `O31 Deliverable`. The repository has no scope sheet; what it has is `O09 Organization Unit` to name the units in scope and `O07 Constraint` to record what bounds them.

How.

1. **Breadth.** Name the lines of business in scope by `ORG-` ID. Meridian has five — `ORG-02` Life & Funeral through `ORG-06` Investments — plus two federated subsidiaries, `ORG-15` Namibia and `ORG-16` Botswana, which `CON-08` records as federated rather than centrally governed. Federated means out of scope until somebody with authority says otherwise.
2. **Depth.** State how far down you will model, per domain, and expect it to be uneven. Meridian's capability map runs to four levels but is only assessed at level 3 — `RPT03 Capability Heatmap` says so in its own header, and levels 1 and 2 roll up by formula.
3. **Time horizon.** Take it from `O02 Goal Target Date`: Meridian's goals land in 2028 and 2030, so the target architecture is 2030 and the transition states in `O53 Transition Architecture` are the waypoints.
4. **Domains.** All four, or name the one you are deferring and why.
5. Write the exclusions as a list, each with a reason and the person who agreed it.
6. Have the sponsor initial the exclusion list specifically. It is the half of the scope that gets forgotten.

Signed by. The sponsor from step 3.1, with the Architecture Board noting it.

Done when. All four dimensions are answered in a document, the exclusions are initialled, and no line of business is ambiguous.

Shows up in. Everywhere downstream. The first symptom of unsettled scope is a Phase B capability assessment that keeps growing; the second is a gap list with entries from a line of business you thought was out.

Goes wrong when. Scope is agreed as "the group" because that sounds ambitious. A composite insurer with five lines of business, two subsidiaries, 26 applications and 1.9 million policyholders cannot be done in one cycle by fourteen people. Phasing by value stream — `VST-02 Settle a Claim` across all lines, rather than one line end to end — is usually the better cut, because it follows the money.

SETTLING THE SCOPE

Which line of business costs you the most to include — and what do you give up by leaving it out?

Which exclusion will be contested first, by whom, and whose initials are against it?

Is the cut by line of business or by value stream, and which of those follows the money in your insurer?

Step 3.7 — Confirm the constraints and test the assumptions

Produce. A constraint set that bounds the solution space and an assumption set with validation dates somebody owns.

You need. Step 3.6. The CFO on budget, the CIO on skills and vendor windows, the Chief Compliance Officer on residency.

Where it lands. Two sheets. `O07 Constraint`: `Type` (Financial, Resource, Contractual, Regulatory, Organisational), `Applies Until`, `Owner`. `O08 Assumption`: `Confidence` (High, Medium, Low), `Validate By` as a year-month, `Owner`, `Status`.

How.

1. Read the eight shipped constraints and keep the ones that are true for your insurer. Meridian's are the familiar set: `CON-01` no technology budget increase before FY2028, so change is funded from run-cost reduction; `CON-02` mainframe skills scarcity; `CON-03` bancassurance contractual service levels; `CON-04` data residency under POPIA; `CON-05` limited change appetite after two failed core replacements; `CON-07` solvency capital requirement; `CON-08` the federated subsidiaries.
2. For each constraint, record who can lift it. A constraint nobody can lift is a fact; a constraint somebody can lift is a negotiation you have not had.
3. Write every assumption as a sentence that can be false. "Run-cost reduction funds change" can be false. "The business supports the architecture" cannot.
4. Set `Confidence` and `Validate By`, and name an owner who is not you.
5. Fill `R06 CON-WPK` with `Impact` and `Mitigation`, and `R35 ASM-WPK` with `If Invalidated`.
6. Diarise the validation dates. Then actually check them.

Signed by. Each constraint's and assumption's named owner. The Chief Enterprise Architect owns the set, not the contents.

Done when. Every assumption has a `Confidence`, a `Validate By` inside the next twelve months, an owner, and at least one row in `R35 ASM-WPK`.

Shows up in. `R35 ASM-WPK`, whose `If Invalidated` column is the most useful single column in the motivation extension. Its values in Meridian are Stop, Delay, Rescope, Reschedule and Accelerate. `ASM-04 Run-cost reduction funds change` — `Confidence` = **Low** —

underpins `WP-06 Enterprise Data Platform` and `WP-08 Product Configuration Platform` with `If Invalidated` = Stop, and `WP-10 Claims Modernisation` with Delay. That is R162.4 million of work that stops and R67.9 million that slips if one low-confidence assumption fails.

Goes wrong when. The dates pass and nobody looks. Five of Meridian's six assumptions have a `Validate By` date in the past against a repository state of 2027-11-20, and all six still read `Status` = Open — including `ASM-02 Architecture Board is established in Q4 2026`, due for validation in 2026-12, which `O57 Governance Body` separately records as having never refused anything. The assumption was testable, the test date passed, and the answer was available in the same workbook.

Step 3.8 — Seed the strategic requirements

Produce. Fifteen to twenty requirements that state what the architecture must deliver, each traceable to a driver and an objective.

You need. Steps 2.10, 2.11 and 3.7. Volume 1, chapter 15 for the obligations that become requirements, and chapter 10 (The information model) for the data ones.

Look it up. Volume 1, chapter 15, *Modelling a regulatory requirement properly*, for the chain each one carries.

Where it lands. `O06 Requirement`, using the `REQ-` prefix: `Type` (Functional or Non-functional), `Priority` (Must, Should, Could), `Source Driver` as a `DRV-` ID, `Objective` as an `OBJ-` ID, `Owner`, `Status`, `Source` = "Phase A strategic", and `Acceptance Criteria`. Leave `Building Block` and `Work Package` empty — those are filled in Phase E.

How.

1. Write one requirement per obligation or objective that the architecture must satisfy, not one per idea.
2. Fill `Source Driver` and `Objective` on every row. A requirement with neither is somebody's preference.
3. Write `Acceptance Criteria` now, while the requirement is fresh and the owner is in the room. This is the step everybody skips.
4. Set `Priority` honestly. If everything is Must, the sequencing conversation in Phase F has no input.
5. Add the `R04 REQ-OBJ` rows with `Necessity`, and the `R05 PRN-REQ` rows from step 2.5.
6. Keep the count under twenty-five. The derived architecture requirements — the `ARS-` population — arrive in Phases C and D.

Signed by. The named `Owner` per requirement, with the Architecture Board accepting the set.

Done when. Every `REQ-` row has a `Source Driver`, an `Objective`, an `Owner` and a non-empty `Acceptance Criteria`.

Shows up in. `RPT32 Requirement Traceability`, columns `Building block`, `Work package`, `Traced` and `Ever tested`; and `RPT08 Traceability`, which adds `Governing Principles` and a `Chain` column.

Goes wrong when. Acceptance criteria are deferred to delivery. Look at Meridian: all 20 `REQ-` rows have an empty `Acceptance Criteria`, an empty `Building Block` and an empty `Work Package`, and `RPT32` duly reads `Traced` = no and `Ever tested` = no for every one of them. The 56 `ARS-` requirements derived later in Phases C and D all carry acceptance criteria and a building block. The strategic requirements — the ones the executives actually asked for — are the untestable ones.

Step 3.9 — Draft the Architecture Vision

Produce. A document, ten to fifteen pages, that an executive can read in one sitting and defend in the next meeting. One page of it is an ArchiMate 4 motivation view — `Stakeholder`, `Driver`, `Goal`, `Outcome`, `Principle` and the `Requirement`s they engage — which is the page that gets quoted back at you.

You need. Steps 3.1 to 3.8. One drafting day and two review rounds. Volume 1, chapter 17 (Cape Mutual, end to end) as a worked example of the argument running from driver to estate.

Where it lands. `DLV-11 Architecture Vision` in `O31 Deliverable`: `Type` = Deliverable, `ADM Phase` = A, `Owner`, `Due`, `Status`, `Repository Location`. Beware the ID: `DLV-11` in `O34 Architecture Deliverable` is the Strategic and Motivation Model, a different artefact.

How.

1. Open with the drivers, by ID, and the two or three numbers that make the case. `DRV-14` at 71% of budget on run and `OBJ-03` cost-to-income at 37.4% are the whole argument in two lines.
2. State the target in capability terms, from `O13 Business Capability` `Target Maturity`, not in system terms. Nobody approves a vision that names a vendor.
3. Show the value streams and which of them change. Four or five streams, one page.
4. Name the three or four gaps already visible. Meridian had six Critical gaps at this point, including `GAP-01 No single customer identity` with 640,000 records needing manual matching.
5. State what it will cost to find out more, not what the programme will cost. Phase A does not price the roadmap.
6. Close with the decision you want: approval to proceed to Phase B on this scope.

Signed by. The Group Executive Committee, on the Architecture Board's recommendation.

Done when. The document exists, has a version, names the gaps and the target capabilities, asks for one decision, and has been read by the sponsor before the meeting rather than in it.

Shows up in. `RPT14 Deliverable Register` with a `Status` above Draft and a `Depended on by` count above zero — the Phase B deliverables depend on it through `R40 DLV-DLV`.

Goes wrong when. The vision describes a target estate. It should describe a target *business*, with the estate as a consequence. The second failure is length: a 60-page vision is a Phase B deliverable written early, and it will be approved by people who have not read it.

THE DECISION THE VISION ASKS FOR

Write the one decision the vision asks for, and the body that takes it.

Which three numbers carry the argument, and which sheet does each come from?

Which sentence will be quoted back at you in a year — and is it a claim about the business or about the estate?

Step 3.10 — Write the Statement of Architecture Work

Produce. The agreement: scope, deliverables, dates, roles, acceptance criteria for the architecture work itself.

You need. Steps 3.6 and 3.9. The deliverable register from step 2.12.

Where it lands. `DLV-12 Statement of Architecture Work` in `O31 Deliverable`, with the deliverables it commits to listed in the same sheet and joined by `R40 DLV-DLV` (`Nature`) and `R39 ORG-DLV` (`Role`). Meridian's `DLV-12` was due 2027-02 and reads `Not started`.

How.

1. List every deliverable you are committing to, with `Due` dates from `O31 Deliverable`, and nothing you are not.
2. Carry the scope statement from step 3.6 verbatim, exclusions included.
3. State the acceptance criteria for the architecture work: what a finished Phase B looks like, in the words of chapter 1's three conditions.
4. Name the roles and the time you need from each, by `ROL-` ID and days.
5. State the governance: which body holds which gate, from `O57` and `O58`.
6. Get it signed by the sponsor and the Architecture Board chair.

Signed by. The sponsor and the Architecture Board. Two signatures, because one authorises the work and the other accepts the standard it will be held to.

Done when. Signed, dated, registered, and every committed deliverable exists as a row in `O31 Deliverable` with an owner and a due date.

Shows up in. `RPT14 Deliverable Register`, as a cluster of Phase B rows with dependencies. If the statement commits to deliverables that are not in the register, the register is wrong or the statement is fiction.

Goes wrong when. It is written as a project plan. It is a contract about scope and acceptance, and its most valuable paragraph is the one listing what you will not deliver.

Step 3.11 — Hold the Phase A gate and cut the entry baseline

Produce. A recorded decision to proceed, and a baseline that Phase B gap analysis will be measured against.

You need. Steps 3.9 and 3.10 approved. `G0` from step 2.7, held by the Investment Committee.

Where it lands. `O60 Governance Decision` for the decision (`Date`, `Decision`, `Body`, `Status`, `Why`, `Affects`), `O59 Compliance Assessment` for the verdict against each acceptance criterion (`Gate` = G0, `Verdict`, `Evidence`, `Recorded In`, `Assessed`), and `04 Baselines` for the snapshot.

How.

1. Put the vision and the statement of work to the gate together. They are one decision.
2. Record a verdict per acceptance criterion in `O59 Compliance Assessment`, with evidence that points at a sheet or a document, not at an opinion.
3. If the gate conditions a decision, record the condition and its expiry in `O61 Governance Change Request` rather than in the minutes.

4. Cut the baseline in `04 Baselines`: name it for the phase, date it, record the `Workbook Version` from `01 Version Control`, and set the previous row to Superseded.
5. Load the new extract into the comparison tool's `10 Input A Baseline` and update `01 Config`.
6. Check `RPT13 Version Control` reads zero in `Pending approval` for every sheet Phase A touched.

Signed by. The Investment Committee for `G0`, with the Architecture Board recommending.

Done when. `04 Baselines` has one `Current` row dated this month; `O60 Governance Decision` has an entry naming the body and the date; and every Phase A deliverable in `O31 Deliverable` reads Complete with a `Repository Location`.

Shows up in. `RPT31 Governance Activity` as a decision with an owner and a status, and `RPT29 Transition Readiness` once Phase F fills it.

Goes wrong when. The baseline is planned and never cut. Meridian's `BL-003 Phase A Entry Baseline` is dated 2027-03-31, reads `Planned`, carries the note that it will be cut on approval of the Architecture Vision — and the vision was never written. Eight months and 88 logged changes later, there is no fixed point for Phase B to measure against, and every gap analysis from here compares a moving repository against a baseline cut before the governance existed. One row in one sheet, and it quietly invalidates the next three phases.

The six conversations that make or break the phase

Phase A is mostly talking, and the same six conversations decide it. Book them in this order.

With	What you need from them	What they will test you on
The sponsor	A signature on the request, and the exclusions initialled	Whether you can state the problem in their words in under a minute
The Chief Financial Officer	The budget constraint, stated as a number and a date — Meridian's `CON-01` is "no increase before FY2028"	Whether you understand that change is funded from run-cost reduction
The Chief Operating Officer	Agreement on the value streams and their owners	Whether your streams match how work actually flows, not the org chart
The Chief Risk Officer and Chief Compliance Officer	The obligations that become requirements, and the residency constraint	Whether you know which obligations are already breached
The Head of Actuarial Function	What they need before they will sign — and they are not in `O11 Role`, so you will have to add them	Whether the lineage of a number can be shown, not asserted
The line of business managing directors	Which lines are in scope, and what they will give up to be in it	Whether being in scope costs them anything this year

Two rules for all six. Take the repository with you — a printed `RPT02 Application Portfolio` or `RPT03 Capability Heatmap` changes a conversation faster than a slide. And write the outcome into a sheet the same day, as a constraint, an assumption, a requirement or a stakeholder note. A conversation you did not record is a conversation you will have again.

Before you move to Phase B

Test	Where to look
Is there a signature on the mandate?	`O31 Deliverable`, `DLV-10`, `Status` and `Repository Location`
Can you name the person who will refuse to sign something, and why?	`O11 Role`, `Authority Level` = Statutory
Does every assumption have a future validation date?	`O08 Assumption`, `Validate By` against today
Does every strategic requirement have acceptance criteria?	`RPT32 Requirement Traceability`, `Ever tested`
Is there a `Current` baseline dated this month?	`04 Baselines`, `Status`

Five readings, ten minutes. Phase B is the long one, and it is much harder to go back and fix the mandate once four hundred capability records depend on it.

PHASE A — A REVIEWER'S QUESTIONS

Where is the signature on the mandate, and who would answer for it in a board meeting?

Which assumption, if it turns out false, stops the most work — and who is validating it, by when?

Which strategic requirement carries acceptance criteria that somebody other than you wrote?

Is there a `Current` baseline dated this month, and what became of the one before it?

THE CHAPTER IN TEN LINES

- If nobody will write the Request for Architecture Work, write it in 500 words and have it signed. DLV-10 in O31 Deliverable.
- The stakeholder set that matters is in volume 1 chapter 5, not in the kick-off invitation list.
- The shipped repository has no Head of Actuarial Function, no CRO role, no binder holder and no reinsurance manager. Add them under one change request.
- There is no stakeholder sheet. Decide where the assessment lives and record that decision.
- Draw the business model canvas from O18 Product and O19 Contract, not from a workshop with no numbers.
- Name the value streams before the capabilities. A stream named after a department is a department.
- Scope has four dimensions: breadth, depth, time horizon and domains. The exclusion list is the half that gets forgotten.
- R35 ASM-WPK If Invalidated is the most useful column in the motivation extension: one low-confidence assumption stops R162.4m of work.
- Write acceptance criteria while the requirement is fresh. All 20 of Meridian's strategic requirements have none.
- Cut the Phase A entry baseline. BL-003 was planned for 2027-03-31 and never cut, and Phase B has no fixed point to measure against.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets O31 Deliverable, O34 Architecture Deliverable and 04 Baselines
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets O09 Organization Unit, O10 Actor, O11 Role, R07 ORG-ACT and R08 ACT-ROL
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets O14 Value Stream, R11 VST-CAP and R41 APP-VST
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets O07 Constraint, O08 Assumption and R35 ASM-WPK
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheets O06 Requirement, RPT08 Traceability and RPT32 Requirement Traceability
- **6.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method, Phase A https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **7.** National Treasury, Annexure D3 to the final insurance conduct regulations — binder functions and binder fees https://www.treasury.gov.za/legislation/regulations/ICRAndPPR/Annexure%20D3.pdf

CHAPTER 4

Business Architecture

The phase that decides whether anybody believes the rest of the work.

Everything in Phase C, D and E hangs off two sheets you fill here [5] — `O13 Business Capability` and `O16 Business Process`. It is also the phase with the most sheets, the most meetings and the most opportunity to produce a beautiful artefact nobody uses.

Meridian Assurance Group, the insurer in the repository on your desk, is an invention; so is Cape Mutual, the insurer in volume 1. Their numbers do not mix.

> **BEFORE YOU START THIS PHASE**
>
> Most of volume 1 is business-layer content, so this is where it saves most time.
>
> - **Volume 1, chapter 5, *Underwriting* and *Claims*.** What each role decides, and where authority stops.
> - **Volume 1, chapter 8, *The L1 process map*.** Twelve processes, each with a trigger, an outcome, an owner.
> - **Volume 1, chapter 16, *Is it a service or a process?*** What keeps `O15`, `O16` and `O17` apart.
> - **Volume 1, chapter 19, *Business capabilities*, *Roles* and *Business processes*.** The pre-drafted catalogues.

What you are filling, and in what order

Phase B in this repository is eight object sheets and ten relationship sheets. The object sheets are `O09 Organization Unit`, `O10 Actor`, `O11 Role`, `O13 Business Capability`, `O14 Value Stream`, `O15 Business Service`, `O16 Business Process`, `O17 Business Function`, plus `O18 Product` and `O19 Contract` where the engagement reaches the product factory. The relationship sheets are `R07 ORG-ACT`, `R08 ACT-ROL`, `R09 ROL-BPR`, `R10 ORG-CAP`, `R11 VST-CAP`, `R12 CAP-BSV`, `R13 FUN-BPR`, `R14 BPR-BSV`, `R15 BSV-PRD`, `R16 PRD-CTR` and `R38 CAP-CAP` [1].

The order is not arbitrary.

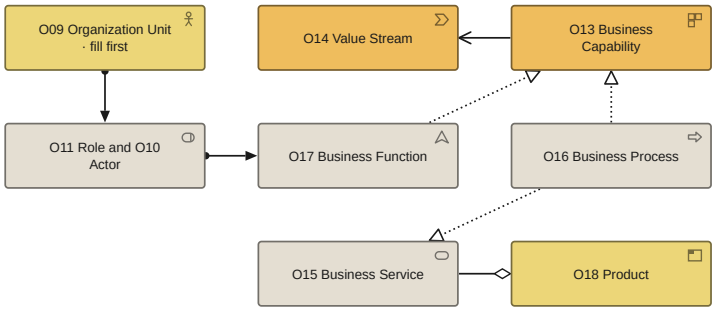

Figure 4.1 The Phase B sheets as one ArchiMate 4 business-layer view, carrying the relationships the ten `R` sheets actually hold. Nothing on it stands alone, which is why the fill order below is not negotiable.

Fill them in that order. Skip `O09 Organization Unit` and nothing downstream has an owner. Skip `O14 Value Stream` and the capability map has nothing to be tested against. Skip `O13 Business Capability` and Phases C and E have no anchor at all. Skip `O16` and `O17` and the CRUD map in Phase C has no rows. Skip `O15 Business Service` and the applications in Phase C have nothing to be assessed against.

> **THE RULE FOR EVERY SHEET IN THIS CHAPTER**
>
> Fill a relationship sheet in the same sitting as the second of its two endpoint sheets. A relationship backlog is not a backlog; it is a decision nobody made. `R11 VST-CAP` in the repository you were given has fifty rows, every one of them with a blank `Stage` and `Criticality to stream` set to `High` — which is what a relationship sheet filled in a batch at the end looks like.

The steps

Step 4.1 — Fix the Phase B boundary in writing before you open a sheet

Produce. One page: which lines of business, which value streams and which capability levels are in scope for this iteration, and which are explicitly not.

You need. The approved Statement of Architecture Work from Phase A (`031 Deliverable` , `DLV-12`), the drivers in `001 Driver` , and fifteen minutes with the sponsor.

Where it lands. Not in a sheet. It lands in the Phase B section of `DLV-14 Architecture Definition Document` on `031 Deliverable` , and the scope statement is repeated at the top of every workshop pack.

How.

1. List the organisation units from `009 Organization Unit` and strike the ones out of scope. Meridian has sixteen, including `ORG-15 Meridian Namibia` and `ORG-16 Meridian Botswana` , both federated subsidiaries — decide now whether their architecture is in or out.
2. Decide the capability level you will assess. The repository assesses at Level 3 only.
3. Name the value streams in scope from `014 Value Stream` . Seven exist; three is a defensible iteration.
4. Write what "out of scope" means operationally: no workshop, no scoring, no gap raised, and get the sponsor to say the exclusions out loud before you write them down.

Signed by. The sponsor named on the Statement of Architecture Work, countersigned by the Chief Enterprise Architect (`GB-05 Architecture Function` , `057 Governance Body`).

Done when. Every row of `009 Organization Unit` is marked in or out, with no row unmarked.

Shows up in. Nowhere in the reports — which is the point. Scope is the one Phase B artefact the repository does not hold, so it has to live in the deliverable.

Goes wrong when. Scope is set by capability count ("we will do the whole map") rather than by value stream. Five hundred and fifty-six capability rows at four levels takes four months to assess honestly; three value streams and their enabling Level 3 capabilities takes five weeks.

FIXING THE PHASE B BOUNDARY

Which three value streams are in this iteration, and what makes them the three?

For each exclusion, what is the operational consequence — no workshop, no score, no gap — and who said it out loud?

Are the federated subsidiaries in or out, and who has the authority to say so?

Step 4.2 — Load the organisation and refuse to call it the architecture

Produce. A complete `009 Organization Unit` tree with one head and one headcount per unit, and a written note of where the chart lies.

You need. The HR organisation chart, the group structure from the annual report, and volume 1, chapter 5, *Who does what*, which opens on exactly this warning.

Where it lands.

> **`009 Organization Unit`**
>
> `ID` , `Name` , `Description` , `Type` , `Parent` , `Head` , `Headcount` , `Location` . `Type` is the honest column: Meridian uses `Group` , `Line of Business` , `Group Function` and `Subsidiary` , and the distinction between the last two is what tells you whether a unit can be told what to do. `Head` is a position title, never a person's name — people move and the repository is versioned. `Location` is an `012 Location` ID, not a city name.

How.

1. Load the units top down, parents before children, so `Parent` never points at a blank.
2. Set `Type` from the group structure, not from the reporting line. A line of business that reports to a group function is still a line of business.
3. Record `Headcount` from payroll, not from the chart. Meridian's sixteen units carry 4,200 between them with `ORG-01` holding the group total.
4. Write, in `Description` , what the unit is accountable for in one sentence that does not use the unit's own name, and mark the units with no decision rights.

Signed by. The Group HR Director approves `DLV-14 Organisation Chart` on `034 Architecture Deliverable` ; the Chief Enterprise Architect approves the `009` load.

Done when. Every `Parent` resolves to an existing `ID` , every `Head` is a title, and the sum of line-of-business headcounts reconciles to the group figure.

Shows up in. `RPT12 Integrity Check` — the `O09` row should show links as source for every unit that owns a capability. A unit with zero links is a unit you have recorded and not used.

Goes wrong when. You treat `O09` as the shape of the architecture. It is the shape of the payroll. In Meridian, underwriting authority sits in `ORG-02`, `ORG-03` and `ORG-04` separately while claims authority sits in one place, `ORG-13` — three underwriting practices and one claims practice, which the chart renders as four symmetrical boxes. The asymmetry is the architecture; the symmetry is the drawing.

Step 4.3 — Separate actors from roles, and count them

Produce. A populated `O10 Actor` sheet with populations, and `R07 ORG-ACT` showing which unit employs how many of each.

You need. `O09` complete. Volume 1, chapter 5 for the external actor set that insurers consistently forget.

Look it up. Volume 1, chapter 5, *Distribution* and *The broker, properly*, for the external actors usually left off.

Where it lands.

> **`O10 Actor`**
>
> `Type` (`Person` or `Organisation`), `Internal/External`, `Organization Unit`, `Population`. An external actor has no `Organization Unit` — leave it blank rather than inventing one. `Population` is an order of magnitude, not a precise count: Meridian records 1,900,000 policyholders and six reinsurers on the same sheet, which is correct.

How.

1. Load internal actors first, each with its employing unit, then the external set.
2. Include the regulators. Meridian has `ACT-14 Prudential Authority` and `ACT-15 Financial Sector Conduct Authority` as actors, which is what lets Phase G trace an obligation to somebody.
3. Separate the parties volume 1 insists on separating: `ACT-03 Policyholder`, `ACT-04 Beneficiary` and `ACT-19 Claimant` are three actors, not one customer.
4. Fill `R07 ORG-ACT` with the `Count` column for every internal actor, and check that no actor name is a job title with authority in it — those are roles.

Signed by. Chief Enterprise Architect, with the Chief Distribution Officer confirming the intermediary and partner populations.

Done when. Every internal actor appears in `R07 ORG-ACT` with a `Count`, and every external actor does not.

Shows up in. `RPT12 Integrity Check`. Meridian's `R07 ORG-ACT` has only eight rows for twenty actors, because twelve are external — that ratio is the finding, not a defect.

Goes wrong when. `Population` gets the headcount of the team that serves the actor rather than the actor's own count, which inverts every channel sizing downstream.

Step 4.4 — Build the role catalogue and record where authority stops

Produce. A role catalogue where every role has a `Domain`, an `Authority Level` and an employing unit, plus `R08 ACT-ROL` joining people to the roles they perform.

You need. `O10` complete. Volume 1 appendix A already drafts the insurance role set as `R-01` to `R-33` — start there and argue with it. Volume 1, chapter 5 carries the underwriting authority ladder and the claims settlement mandate.

Look it up. Volume 1, chapter 19, *Roles*, drafts `R-01` … `R-33` — load it, argue with it, match by name: thirty-three roles against twenty-seven `ROL-` rows is not a renumbering.

Where it lands.

> **`O11 Role`**
>
> `Domain`, `Authority Level`, `Organization Unit`. `Domain` groups roles for reporting (Meridian uses fourteen: Claims, Underwriting, Operations, Distribution, Data, Product, Compliance, Governance and so on). `Authority Level` is the column that matters: Meridian's values are `Transactional`, `Mandated`, `Delegated authority`, `Responsible`, `Accountable`, `Advisory`, `Approval` and `Statutory`.

How.

1. Load volume 1's `R-01` to `R-33` as a candidate list, then delete every role the insurer does not actually staff. Meridian keeps twenty-seven.
2. Set `Authority Level` for each. The underwriting ladder is a pair of rows: `ROL-03 New Business Underwriter` is `Mandated` and `ROL-04 Senior Underwriter` is `Delegated authority`. Claims is the same shape: `ROL-05 Claims Assessor` is `Mandated`, `ROL-06 Claims Authoriser` is `Delegated authority`.
3. Put the rand limit in `Description`, because `Authority Level` holds a class and not a number. "Accepts, loads or declines a risk within mandate" is what the repository says; "within R2m sum assured, non-impaired lives" is what the architecture needs.

4. Mark the statutory roles. `ROL-16 Information Officer` carries POPIA and PAIA duties and is the only `Statutory` row — Phase C will need it.
5. Fill `R08 ACT-ROL` with the `Basis` column, so a role performed by a partner's staff is visible as such.

Signed by. The executive who owns the mandate. Underwriting authority is signed by the Chief Underwriter or line MD, claims settlement by the Claims Executive (`ORG-13`), never by the architect.

Done when. Every role has an `Authority Level` , every `Mandated` or `Delegated authority` role has its limit written in `Description` , and the two ladders reconcile to the signed mandate document.

Shows up in. `RPT13 Version Control` — authority changes are the ones you will be asked to prove, and an `O11` row changed without a `Change Ref` is an authority change nobody approved.

Goes wrong when. Roles are copied from job titles. "Senior Underwriter" is a role because its authority differs; "Senior Claims Administrator" usually is not, because it does not. Volume 1, chapter 5 makes the general point that authority is data; this step is where it stops being a point and becomes a column.

Step 4.5 — Write the value streams from the triggering stakeholder inwards

Produce. `O14 Value Stream` with a named triggering stakeholder, a value proposition in that stakeholder's words, and a stage count for each stream.

You need. Scope from step 4.1. The business model canvas from Phase A (`DLV-10 Business Model Canvas` , `O34 Architecture Deliverable`).

Look it up. Volume 1, chapter 4, *Customer journeys, done honestly*, for the trigger in the stakeholder's own words.

Where it lands.

> **`O14 Value Stream`**
>
> `Triggering Stakeholder` , `Value Proposition` , `Stages` , `Owner Org` . `Value Proposition` is written from outside the insurer: Meridian's `VST-02 Settle a Claim` reads "Money arrives without a fight", which is a customer's sentence, not an operations target.

How.

1. Start from the stakeholder, never from the department. If you cannot name who triggers it, it is a process, not a value stream.
2. Write `Value Proposition` as the trigger's own words. If it contains an internal metric, it is wrong.
3. Set `Stages` as a number now and name the stages in `Description` . Four to seven stages is the usable range.
4. Set `Owner Org` to one unit. Meridian spreads the seven across `ORG-07` , `ORG-13` , `ORG-12` , `ORG-03` , `ORG-02` and `ORG-08` , which is honest and awkward — no single unit owns the end-to-end anything.
5. Confirm the reporting stream exists. `VST-07 Report to a Regulator` is a value stream whose triggering stakeholder is the regulator, and insurers leave it out because it feels internal.

Signed by. The `Owner Org` head for each stream, in a single sitting so they see each other's streams.

Done when. Every stream has a triggering stakeholder who is not an employee of the owning unit, and the stage names are written.

Shows up in. `RPT23 Process Flow` , which groups the twenty-two processes by value stream. A stream with no processes under it is a stream you invented.

Goes wrong when. The streams mirror the org chart, giving you an underwriting stream, a claims stream and a finance stream. Meridian's seven cross three or four units each, which is why the next step is where the argument happens.

Step 4.6 — Map value streams to capabilities, and fill the Stage column

Produce. `R11 VST-CAP` with every row carrying a `Stage` and a differentiated `Criticality to stream` . Draw it as an ArchiMate 4 strategy view: each `Value Stream` with the `Capability` elements that serve it, one column per stage.

You need. `O14` complete and the capability map available in draft (step 4.7 can run in parallel; this relationship is what tests both).

Where it lands.

> **`R11 VST-CAP`**
>
> `Source (VST)` , `Target (CAP)` , `Stage` , `Criticality to stream` . `Stage` is the stage name from the stream's `Description` , not a number. `Criticality to stream` is `High` , `Medium` or `Low` — and it must vary, or it carries no information.

How.

1. For each stream, walk the stages in order and ask which capabilities the stage calls on.

2. Write the `Stage` name into every row as you create it. This is the step the repository skipped: all fifty rows have a blank `Stage`.
3. Set `Criticality to stream` by asking "if this capability fails, does the stream stop or slow?" Stop is `High`. Slow is `Medium`. Degrades is `Low`.
4. Count the capabilities per stream. Meridian's `VST-01 Acquire Cover` draws on thirteen; `VST-06 Onboard an Intermediary` on three. A stream with one capability is mis-scoped.
5. Look for the capability that appears in four streams. `C02.01 Customer Identity & Data` appears in `VST-01`, `VST-02` and `VST-03` — three streams stop if it fails, which is the whole business case for the identity work.
6. Raise a `Business` gap in `O29 Gap` for any stage with no capability behind it.

Signed by. Each stream's `Owner Org` head, for their own stream's rows only.

Done when. No row has a blank `Stage`, and `Criticality to stream` contains at least two distinct values.

Shows up in. Nothing reads `R11 VST-CAP` directly, which is why it rots. Its value is in the sequencing argument in Phase E: a `High` capability in three streams outranks a `High` capability in one, and that fact exists only here.

Goes wrong when. Every row is set to `High`, as in the repository you were given. Fifty `High` rows and no `Stage` is the single most common piece of Phase B theatre, and it is invisible because no report shows it.

GRADING CRITICALITY TO THE STREAM

For one stream, which capability failing stops it rather than slows it? Name the capability and the stage.

Which capability appears in three or more of your streams, and what does that do to its place in the Phase E sequence?

Which stage has no capability behind it, and is that a gap or a stage you invented?

WHAT TO DO ABOUT THE FIFTY ROWS YOU INHERITED

Do not delete them. Work `VST-01` through `VST-07` in stream order, adding `Stage` and re-grading `Criticality to stream` row by row, each change logged in `02 Change Log` against one change request. Two sittings of ninety minutes with the stream owners is enough, and the resulting sheet is the most argued-over and most useful thing in Phase B.

Step 4.7 — Accept, prune or rebuild the capability map

Produce. A capability map whose every row passes four tests, with the levels you will assess marked.

You need. `O14` and `R11` in draft. Volume 1 appendix A drafts `C-01` to `C-20` at Level 1 and 2 — start there and argue with it. Volume 1, chapter 7, *The operating model*, for the shape.

Look it up. Volume 1, chapter 19, *Business capabilities*, is where those twenty sit. Map by name: twenty against 556 dotted repository IDs is a change of grain, and `C-07` is not `C07`.

Where it lands.

> **`O13 Business Capability`**
>
> `ID`, `Name`, `Description`, `Level`, `Parent`, `L1`, `L2`, `Owner Org`. `L1` and `L2` are denormalised ancestor columns that exist so the reports can group without walking the tree — fill them on every row, including Level 3 and 4 rows, or `RPT03 Capability Heatmap` returns blanks. `ID` encodes the path: `C07.02.01` is Level 3 under `C07.02` under `C07`.

How.

1. Lay volume 1's twenty against the repository's twelve Level 1 areas and reconcile the names, not the count. Meridian's `C07 Claims Management` covers volume 1's `C-11` and `C-12`; its `C11 Data & Analytics` covers `C-20`.
2. Apply the four tests to every Level 2 and Level 3 row.
3. Delete, do not reclassify. A row that fails test 1 goes to `O16` or `O17`; a row that fails test 3 gets merged with its sibling.
4. Fill `L1` and `L2` on all 556 rows before you touch a maturity column.
5. Stop at Level 4. Meridian has 364 Level 4 rows and they are unassessed by design — they exist to make the Level 3 boundary arguable, not to be scored.

Signed by. Architecture Board (`GB-01`, `O57 Governance Body`), which holds `G1` and may grant a dispensation but cannot approve funding.

Done when. Every row has `Level`, `Parent`, `L1` and `L2` populated; `R38 CAP-CAP` row count equals the number of non-Level-1 rows; and no two siblings overlap in `Description`.

Shows up in. `RPT03 Capability Heatmap`, whose three blocks count Level 2, 3 and 4 rows per area. `DLV-13 Business Capability Map` on `O31 Deliverable` is the deliverable this becomes.

Goes wrong when. The map is built from the org chart, which produces twelve areas that match twelve departments. Meridian's `C12 Enterprise Enablement` holds five Level 2 groups including `C12.05 Enterprise Architecture` — an architecture capability inside the capability map, scored like any other, which is the test of whether the practice is serious.

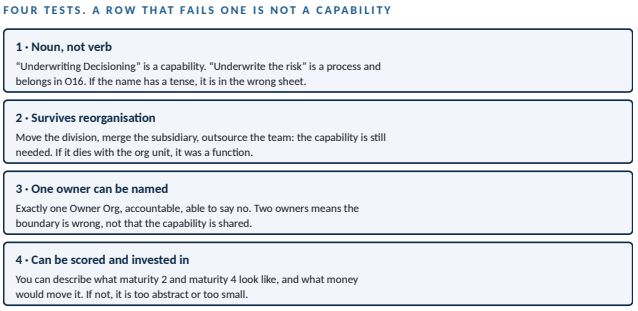

Figure 4.2 The four tests. Apply them to every Level 2 and Level 3 row before any maturity column is touched.

THE SENTENCE THAT DEFINES A CAPABILITY BOUNDARY

The map itself is a Phase B catalogue — appendix F carries the form for `O13 Business Capability`, with `Level`, `Parent`, `L1` and `L2` against each row. Do the arguing here first.

Write the one sentence that decides whether a candidate row belongs in `O13` at all.

What would a reader wrongly assume sits inside this capability, and where does it actually live?

Which two siblings overlap, and which of them are you merging?

Step 4.8 — Prove the decomposition with R38 CAP-CAP

Produce. A decomposition sheet where every parent-child link is recorded once, with the level transition named.

You need. `O13` complete.

Where it lands.

> **`R38 CAP-CAP`**
>
> `Source (CAP)`, `Target (CAP)`, `Decomposition`. `Decomposition` records the transition: Meridian uses `L1 to L2`, `L2 to L3` and `L3 to L4`. There is no cross-level link and there should not be.

How.

1. Generate the rows from `Parent` on `O13` rather than typing them.
2. Count: Meridian has 544 rows — 46 `L1 to L2`, 134 `L2 to L3`, 364 `L3 to L4` — which reconciles exactly to the `Level` counts on `O13`.
3. Look for a capability with two parents. There should be none; if there is one, the map has a matrix in it and Phase E will produce a double-counted roadmap.
4. Re-run the count after every `O13` change.

Signed by. Chief Enterprise Architect. This is a mechanical check, not a business decision.

Done when. `R38 CAP-CAP` row count equals `O13` rows minus Level 1 rows, and no `Target` appears twice.

Shows up in. `RPT12 Integrity Check`, where `R38` should show the highest links-per-record ratio in the repository.

Goes wrong when. The sheet is maintained by hand after the map changes. It drifts silently, and the first symptom is an area whose Level 3 count in `RPT03` does not match the map.

Step 4.9 — Assign one accountable owner per assessed capability

Produce. `R10 ORG-CAP` with exactly one `Accountable` organisation unit per assessed capability, and `Owner Org` on `O13` agreeing with it.

You need. `009` and `O13` complete.

Where it lands.

> **`R10 ORG-CAP`**
>
> `Source (ORG)`, `Target (CAP)`, `Ownership`. Meridian records `Accountable` on all 134 rows — one per Level 3 capability, no shared ownership anywhere. That is the discipline, not a limitation of the sheet.

How.

1. Work capability by capability, not unit by unit, or you will get volunteers.
2. Ask one question: who can stop a change to this capability? That is the owner.
3. Write the same unit into `Owner Org` on the `O13` row so the heatmap can group by owner.
4. Count the result. Meridian's `ORG-08 Group Finance` owns 32 of 134 and `ORG-09 Group Risk & Compliance` owns 23 — which says the capability map is weighted towards control, and is worth saying out loud.
5. Flag any unit owning more than a quarter of the assessed map as a concentration risk.

Signed by. Each owning unit's head, in writing, in one meeting. Ownership agreed bilaterally gets re-negotiated.

Done when. Every Level 3 row has exactly one `Accountable` row in `R10 ORG-CAP`, and `Owner Org` on `O13` matches it for all 134.

Shows up in. `RPT03 Capability Heatmap` and `RPT15 Catalogue Completeness`. A capability with no owner cannot carry a maturity score anybody will defend.

Goes wrong when. `Ownership` gets a second value — `Contributing`, `Consulted` — to settle an argument. The moment two units own a capability nobody does, and the disposition decision in step 4.11 becomes unsignable.

Step 4.10 — Score current maturity so the numbers survive contact

Produce. `Current Maturity` on every assessed capability, with the evidence for the score written down.

You need. `R10 ORG-CAP` signed. Two facilitators. Three hours per capability area with the owning unit and one consumer of the capability in the room.

Where it lands.

> **`O13 Business Capability`**
>
> `Current Maturity` (integer 1 to 5), and the evidence sentence appended to `Description`. Meridian scores 134 Level 3 rows and leaves 422 blank — Level 1, 2 and 4 are not scored, because an average of averages is not a measurement.

How.

1. Publish the scale before the workshop, with the evidence each level demands [7].
2. Score blind, then reveal. Each participant writes a number on paper and the facilitator collects them before anyone speaks. That one mechanic removes most of the anchoring.
3. Where the spread is two or more, do not average — ask the high scorer and the low scorer each to name the evidence. The score is whatever the evidence supports, which is usually the lower number.
4. Write the evidence sentence into the row as you go. A score with no evidence sentence is deleted at the end of the day, not defended.
5. Cap the day at one capability area. Twelve areas is twelve sessions and roughly five weeks.

Signed by. The `Owner Org` head for their own capabilities, in the room, at the end of the session. Not by email afterwards.

Done when. Every assessed row has an integer and an evidence sentence, and the mean lands somewhere believable. Meridian's enterprise mean is 2.51, with 23 rows at 1 and 16 at 4.

Shows up in. `RPT03 Capability Heatmap`, `Avg Current` per area and per group. A mean above 3.2 in an insurer with a mainframe policy book is a sign the blind-scoring step was skipped.

Goes wrong when. The score is negotiated instead of evidenced, which produces the flat 3 — Meridian's `C04`, `C05` and `C06` all average exactly 3.00, which is either a coincidence or the sound of a room agreeing to go home. Treat an area-level average of exactly 3 as a re-score candidate.

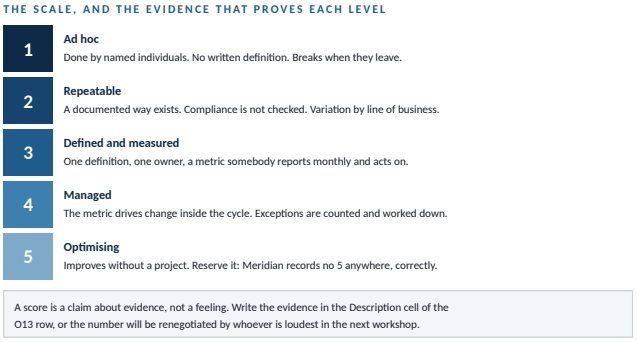

Figure 4.3 The scoring scale to publish before the workshop, with the evidence each level demands.

Working table — blind scores for one capability area

One row per capability scored in the session. Collect the written numbers before anyone speaks, then record the spread and the evidence that settled it. The agreed score goes into `Current Maturity`, the evidence sentence into `Description`, signed by the `Owner Org` head in the room.

Capability ID	Scores as written	Spread	Evidence that settled it	Agreed
C02.01	2,2,3,1	2	One definition, no owner	2

EVIDENCE FOR THE SCORE YOU HAVE JUST WRITTEN

What is the one definition this capability runs on, and where is it written down?

Which metric proves the score, who reports it, and how often?

Which area averaged exactly 3.00, and are you re-scoring it or defending it?

Step 4.11 — Set target, criticality, differentiation and disposition together

Produce. `Target Maturity`, `Criticality`, `Differentiation` and `Disposition` on every assessed capability, decided as one set.

You need. `Current Maturity` complete. The goals and objectives from Phase A (`O02 Goal`, `O03 Objective`) so that a target can be traced to something.

Where it lands.

> **`O13 Business Capability`**
> `Criticality` (`Critical`, `High`, `Medium`), `Target Maturity` (integer), `Differentiation` (`Differentiating`, `Advantage`, `Parity`, `Commodity`), `Disposition` (`Invest`, `Build`, `Sustain`, `Optimise`, `Outsource`).

How.

1. Set `Criticality` first, from failure consequence: `Critical` means a regulatory breach [8], an unpaid claim or a stopped value stream.
2. Set `Differentiation` second, and be ruthless. A capability every insurer has is `Commodity` or `Parity` however well you do it. Meridian records 24 `Differentiating` and 36 `Commodity` out of 134.
3. Set `Target Maturity` third, from the pair above. `Commodity` plus `Critical` targets 3 and stops; `Differentiating` targets 4.
4. Set `Disposition` last, because it falls out of the other three. `Build` where the gap is two or more and differentiation is high; `Outsource` where differentiation is `Commodity` and the gap is large.
5. Check the distribution of `Target Maturity`. Meridian sets 120 of 134 rows to 4, which is not a strategy — it is a wish list, and the first thing a sceptical CFO will find.

Signed by. Architecture Board for the set, with the Investment Committee (`GB-02`) noting the `Build` and `Invest` rows because those become funding requests in Phase E.

Done when. Four columns populated on every assessed row; no `Commodity` row targeting 4; and `Disposition` reconciles to the gap on every row.

Shows up in. `RPT03 Capability Heatmap` , `Critical` and `Build` counts per area, and the `Priority` verdict on the Level 2 block.

Goes wrong when. `Target Maturity` is set to 4 everywhere because nobody wants to nominate their own capability as a commodity. The repository shows what that looks like: a target mean of 3.90 against a current mean of 2.51, an enterprise gap of 1.39, and no basis for choosing between the twenty-six `Build` rows.

Step 4.12 — Read the heatmap and turn it into a shortlist

Produce. A shortlist of ten to fifteen capabilities that Phase E will have to address, with the reason each is on it.

You need. Steps 4.10 and 4.11 complete and recalculated.

Where it lands. `RPT03 Capability Heatmap` is a formula sheet — you read it, you do not type in it. The shortlist goes into the Phase B section of `DLV-14` .

How.

1. Read the Level 1 block first for the shape. Meridian: `C11 Data & Analytics` at 1.22 and `C02 Customer Management` at 1.58 against an enterprise 2.51.
2. Read the Level 2 block's `Priority` column. The rule is mechanical: `URGENT` means the group holds at least one `Critical` capability *and* averages 2.0 or below. Four qualify.
3. Read the Level 3 block, which lists only the capabilities needing attention and labels each `CRITICAL WEAK` , `NO APPLICATION` or `Large gap` .
4. Take `CRITICAL WEAK` first — 17 capabilities sit `Critical` at maturity 1. Then `NO APPLICATION` — 17 Level 3 rows have no supporting application in `R37 APP-CAP` , four of them inside `C12.05 Enterprise Architecture` .
5. Write one sentence per shortlisted capability saying what the business loses today.

Signed by. Architecture Board, as the input to Phase E. The shortlist is an architecture opinion and should be labelled as one.

Done when. Every `CRITICAL WEAK` row is either on the shortlist or has a written reason for exclusion.

Shows up in. `RPT09 Gaps and Roadmap` once the shortlist becomes gaps, and `RPT07 Capability Coverage` for the application-side half of the story.

Goes wrong when. The heatmap is presented as the finding. It is not a finding; it is a sorted list. The finding is the sentence you write about why `C01.02.03 Product Configuration` is `Critical` , at maturity 1, with a target of 4 and three applications touching it — which volume 1, chapter 12 explains and step 4.17 turns into a column.

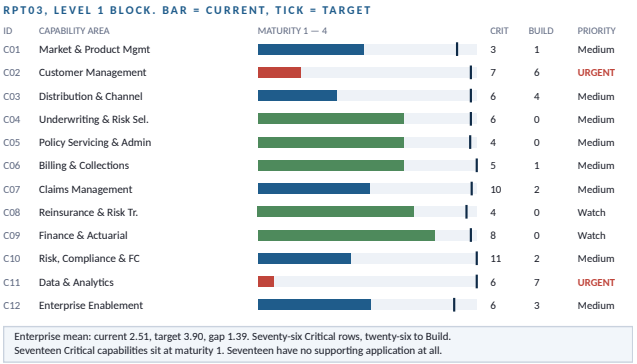

Figure 4.4 The Level 1 block of RPT03 as delivered. Two areas are urgent, and the target line is suspiciously flat.

DIFFERENTIATING, OR JUST WELL RUN?

What can a competitor not copy here in eighteen months, and what would it cost them to try?

Which of these capabilities would the business pay to be better at than a competitor — and who in the business says so?

Which capability are you about to mark `Differentiating` that every insurer in the market has?

Working table — the Phase E shortlist

Taken from the Level 3 block of `RPT03 Capability Heatmap`, with the report's own label: `CRITICAL WEAK`, `NO APPLICATION` or `Large gap`. The last column but one is one sentence in the owner's words, and it is what Phase E will quote. Signed by the Architecture Board as an architecture opinion.

Capability ID	Report label	Current → target	What the business loses today	Keep?
C01.02.03	CRITICAL WEAK	1 → 4	Rate change takes a release	Yes

Step 4.13 — Separate functions from processes before you load either

Produce. `O17 Business Function` populated with a `Type`, and a written rule for which sheet a candidate row goes in.

You need. `O09` and `O13` complete. This step exists because confusing these two is the commonest Phase B error, and it is unrecoverable once `R13 FUN-BPR` is wrong.

Where it lands.

> **`O17 Business Function`**
> `Organization Unit`, `Type` (`Core` or `Supporting`). Meridian has twelve functions: seven `Core` and five `Supporting`. A function is always attached to exactly one `O09` unit — that is the difference from a capability.

How.

1. Apply the question in the figure below to every candidate row.
2. Load the functions. Twelve to twenty is the usable range for an insurer [6]; more than twenty means you have loaded departments.
3. Set `Type`. `Core` means it touches the product, the risk or the customer; `Supporting` means the insurer would still be an insurer without it being excellent.
4. Fill `R13 FUN-BPR` immediately: which function performs which process. Meridian has twenty-two rows for twenty-two processes, one each.
5. Check for a process performed by two functions. That is either a handover you have not modelled or a function boundary in the wrong place.

Signed by. Chief Operating Officer (`ORG-12`) for the `Core` set, Chief Enterprise Architect for the classification rule.

Done when. Every `O17` row has one `Organization Unit` and one `Type`, and every `O16` row appears exactly once as a `Target` in `R13 FUN-BPR`.

Shows up in. `RPT12 Integrity Check` for `R13`, and `RPT15 Catalogue Completeness` indirectly — a process with no function has no performer.

Goes wrong when. The same word is used in three sheets with three meanings. In Meridian, `FUN-06 Claims`, `C07 Claims Management` and `VST-02 Settle a Claim` coexist. All three are right. The error is writing `Claims` into `O13` as if the department were the capability, at which point the heatmap scores an org unit and the whole Phase B assessment is about staffing.

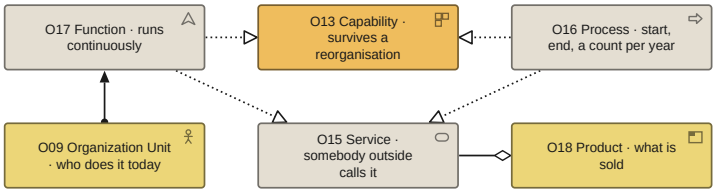

Figure 4.5 The four sheets Phase B keeps collapsing into each other, drawn as the four different ArchiMate 4 elements they are. A function and a process both realise a capability and both can realise a service — but only the process has a start, an end and a count per year.

Step 4.14 — Load the process inventory with volumes and cycle times

Produce. `O16 Business Process` complete, every row carrying a value stream, an owner role, an automation level, an annual volume and a cycle time.

You need. `O14` and `O11` complete. Volume 1, chapter 8, *The process model*, publishes twelve L1 processes `P-01` to `P-12` with triggers, outcomes and owners, plus eleven BPMN L2 models — load those as the candidate inventory. Volume 1 appendix A carries the same twelve as a table.

Look it up. Volume 1, chapter 8, *The L1 process map*, and chapter 19, *Business processes*. Map by name — twelve L1 processes against twenty-two `BPR-` rows is grain, not ten missing.

Where it lands.

> **`O16 Business Process`**
>
> `Value Stream`, `Owner Role`, `Automation Level` (`Manual`, `Partly automated`, `Automated`), `Volume p.a.`, `Cycle Time`. `Volume p.a.` and `Cycle Time` are text in this repository, which means they will not sum — accept that and keep the units consistent.

How.

1. Load volume 1's twelve as candidates, then split where the insurer's own stream boundaries differ. Meridian lands on twenty-two, having split quote from needs analysis, and having added `BPR-21 Screen Party` and `BPR-22 Score Fraud Risk` as processes in their own right.
2. Set `Value Stream` on every row. No orphans: a process with no stream is a process nobody has justified.
3. Set `Owner Role` to an `O11` ID, never a unit. The owner of a process is a person's role.
4. Get `Volume p.a.` from the operational systems, not from the workshop. The three places it actually lives: the policy administration system's transaction counts, the contact centre platform's interaction counts, and the finance system's payment counts. Ask for twelve months and divide, do not annualise a good month.
5. Get `Cycle Time` as the median, not the mean, and say which. Meridian's `BPR-12 Pursue Recovery` at 62 days is a median that a mean would have hidden.
6. Load the L2 BPMN models as attachments to the deliverable, not as rows.

Signed by. The `Owner Role`'s accountable executive per value stream; the volume figures signed by the Chief Operating Officer because they will be quoted in a business case.

Done when. Every row has all five columns, and the volumes reconcile to something a finance person recognises.

Shows up in. `RPT23 Process Flow`, which groups the twenty-two by stream and computes an `Effort hours p.a.` column where the cycle time is in minutes. Meridian's `BPR-13 Handle Servicing Request` comes out at 870,000 hours a year — a number that moves funding conversations in a way a maturity score never does.

Goes wrong when. `Volume p.a.` is left blank "for now". Without it, `RPT23` computes nothing, the automation case for `BPR-02 Perform Needs Analysis` (910,000 cases at 32 minutes each, `Manual`) is invisible, and Phase E prioritises on opinion.

Step 4.15 — Complete the RACI, not just the accountable column

Produce. `R09 ROL-BPR` with four participation values across every process, so that each process has one `Accountable` and at least one `Informed`.

You need. `O11` and `O16` complete. Volume 1 appendix A already has this as a RACI across `P-01` to `P-12` with all four columns filled — start there.

Look it up. Volume 1, chapter 19, *Process to role, as RACI*. Match by name, and keep its honesty about the rows it could not assign.

Where it lands.

> **`R09 ROL-BPR`**
> `Source (ROL)`, `Target (BPR)`, `Participation`, `Note`. `Participation` takes `Accountable`, `Responsible`, `Consulted` and `Informed`. One `Accountable` per process, no exceptions.

How.

1. Start from the `Accountable` row for each process, which the repository already has.
2. Add the `Responsible` rows — who does the work, as distinct from who answers for it.
3. Add `Consulted` where a role can block, and `Informed` where a role must know the outcome.
4. Check the roles that appear nowhere. Meridian's `ROL-24 Product Manager` and `ROL-25 Product Configuration Specialist` were added under `CR-021` and appear in no `R09` row at all, while `BPR-17 Design Product` is `Accountable` to `ROL-03 New Business Underwriter`. Either the role catalogue is wrong or the RACI is; resolve it in the room rather than in the sheet.
5. Count the result. A complete RACI for twenty-two processes runs to between eighty and a hundred and twenty rows. The repository has twenty-three.

Signed by. Each process's accountable executive signs their own process's rows; the Architecture Board notes the set.

Done when. Every process has exactly one `Accountable`, at least one `Responsible` and at least one `Informed`, and no role in `O11` is absent from the sheet without a reason recorded.

Shows up in. `RPT23 Process Flow`, `Owning role` column, and `O42 Metadata Element` `MDE-21 Access`, whose coverage is derived from `R09` and `R17` and currently reaches 11 of 27 roles. Completing this step is also what fixes that metadata gap in Phase C.

Goes wrong when. The sheet is filled with `Accountable` only, which is what you have: seventeen `Accountable`, five `Responsible`, one `Consulted`, no `Informed`. It looks populated and answers nothing. The question a RACI exists to answer — who finds out when this goes wrong — has no rows at all.

WHO ANSWERS WHEN A PROCESS GOES WRONG

Who is `Accountable` for `BPR-17 Design Product` in your insurer, and which person confirmed it?

Which roles exist in `O11 Role` and appear in no `R09 ROL-BPR` row — is the catalogue wrong, or the RACI?

For your highest-volume process, who must be `Informed` when it fails, and how do they find out today?

Step 4.16 — Define business services and test them for channel independence

Produce. `O15 Business Service` with a criticality tier and an honest `Channel Agnostic` flag, joined to capabilities and processes. The view is an ArchiMate 4 business-layer view: `Service` realised by `Process`, the `Role` assigned to the process, and the `Product` that aggregates the services.

You need. `O13` and `O16` complete.

Where it lands.

> **`O15 Business Service`**
> `Capability`, `Consumer`, `Channel Agnostic` (`Yes` or `No`), `Owner Org`, `Criticality` (`Tier 1`, `Tier 2`, `Tier 3`). `Channel Agnostic` is an assessment of the service as it is built today, not of how you would like it built.

How.

1. Name each service as something a consumer asks for, not as something a system does, and set `Consumer` in the consumer's words: "Adviser, broker, customer", not "front end".
2. Set `Capability` to exactly one `O13` ID at Level 3, which is what makes `RPT07 Capability Coverage` computable.

3. Set `Channel Agnostic` by test, not by intent: if the quote logic differs between the adviser tool and the web, the answer is `No`. Seven of Meridian's twenty are `No`, including `BSV-01 Produce Quotation`, `BSV-04 Issue Policy` and `BSV-09 Notify Claim`, which are `Tier 1`.
4. Fill `R12 CAP-BSV` with `Coverage` (`Full`, `Partial`, `None`) and `R14 BPR-BSV` with `Sequence`, so a service's position in a process is recorded.
5. Raise a gap for every `Tier 1` service that is not channel agnostic — that is principle `AP-04 Services are Channel-Agnostic` in `005 Principle` being breached in the baseline.

Signed by. Owning unit head per service; the Architecture Board for the `Channel Agnostic` assessments, because they are the ones that will be disputed.

Done when. Every service has one capability, one owner, a tier and a tested flag; every `None` in `R12 CAP-BSV` has a gap behind it.

Shows up in. `RPT07 Capability Coverage`, which joins each service to its implementing applications and labels `Supported`, `Partial only`, `Single`, `Two apps` or `DUPLICATED`. Meridian shows `BSV-01`, `BSV-02` and `BSV-04` as `DUPLICATED`.

Goes wrong when. `Channel Agnostic` is set to `Yes` as an aspiration. Thirteen `Yes` rows in Meridian's sheet, and the seven `No` rows are exactly the customer-facing ones — which is the real finding and only exists because somebody answered the question honestly.

Step 4.17 — Register the products and read the Config or Code column

Produce. `018 Product` with policies in force, gross written premium and a config-or-code verdict per product, plus `019 Contract` for the agreements that constrain them.

You need. `013` and `015` complete. Volume 1, chapter 12, *The application landscape*, explains the product factory and why this column is an architecture finding.

Look it up. Volume 1, chapter 12, *Product configuration and the product factory*, for why `Config or Code` is a finding.

Where it lands.

> **`018 Product`**
>
> `Line of Business` (an `009` ID), `Policies In Force`, `GWP (Rm)`, `Config or Code`, `Status`. `Config or Code` records whether a rating or wording change is a configuration change or a software release. There is no third value, and the temptation to add "Partly" should be resisted — the answer is whichever one a rate change actually requires.

How.

1. Load every product with a policy in force, including pilots. Meridian has eighteen, seventeen `In market` and one `Pilot`.
2. Get `Policies In Force` and `GWP (Rm)` from finance, as at a stated date.
3. Set `Config or Code` by asking the product team how the last rate change was made. Not how it is supposed to be made.
4. Read the split by line of business. Meridian's nine `Code` products are all life and group risk; its nine `Config` products are all short-term. Same group, two eras.
5. Note where the pilot went. `PRD-18 Simple Funeral (pilot)` is `Config` in a life line whose other five products are `Code` — the new product avoided the old platform, which tells you what the delivery teams believe about it.
6. Fill `R15 BSV-PRD` and `R16 PRD-CTR`, and load `019 Contract` with `Expiry` and `Notice Period`. `CTR-04 Short-term Platform Licence` expires 2027-06-30 with twelve months' notice, which means the decision date is mid-2026 regardless of what the roadmap says.

Signed by. The `Line of Business` MD for the product rows; the Chief Financial Officer for `GWP (Rm)`; the contract owner named on each `019` row for expiry and notice.

Done when. Every product has a line of business, a premium figure and a `Config or Code` verdict; every contract has an expiry and a notice period.

Shows up in. Nothing in the report set reads `018` directly, which is a real limitation of this repository. The column earns its place in Phase E, where the nine `Code` products become `MIG-04 Product catalogue externalisation` on `039 Data Migration`.

Goes wrong when. `Config or Code` is filled from the vendor's brochure. Every modern policy platform is sold as configurable. The question is whether *this* insurer's last rate change needed a developer, and that is a matter of record.

Step 4.18 — Close Phase B and hand over the three things Phase C needs

Produce. A signed Phase B section of the Architecture Definition Document, a baselined capability map, and a written handover of the three artefacts Phase C depends on.

You need. Steps 4.2 to 4.17 complete, with `02 Change Log` entries for everything changed.

Where it lands.

031 Deliverable

DLV-13 Business Capability Map (ADM Phase = B, Status moves to Complete) and DLV-14 Architecture Definition Document (ADM Phase = B, C, D — it stays open through Phase D). Note the collision: O34 Architecture Deliverable uses the same DLV- prefix for a different list, where DLV-13 is the Organisation Map and DLV-14 the Organisation Chart. Always say which sheet [4].

How.

1. Run RPT12 Integrity Check and resolve every Phase B sheet showing zero links.
2. Read RPT07 Capability Coverage and RPT03 Capability Heatmap together: the first says what is supported, the second says what is weak, and a capability that is both weak and duplicated is the Phase E shortlist.
3. Read RPT23 Process Flow for the effort numbers [3] and attach it to the deliverable.
4. Hand Phase C three things by name: the O16 process list (which becomes the rows of R17 BPR-DEN), the O11 role list with ROL-09 Data Owner and ROL-16 Information Officer identified, and the O13 Level 3 capabilities marked Critical.
5. Raise the business-domain gaps. O29 Gap in the repository holds eighteen gaps — seven Application, six Data, three Technology, two Realization, and **none** in the Business domain. Phase B with no business gap recorded is Phase B that found nothing.

Signed by. Architecture Board (GB-01) baselines the capability map; the Group Executive Committee notes the Phase B section. The Investment Committee (GB-02) does not sign Phase B and should not be asked to.

Done when. DLV-13 is Complete on O31 Deliverable, every Phase B sheet has a Change Ref, and at least one gap with Domain = Business exists in O29 Gap.

Shows up in. RPT14 Deliverable Register and RPT09 Gaps and Roadmap.

Goes wrong when. Phase B closes with the capability map baselined and the relationship sheets half-filled. The map is the artefact people look at; the relationships are the artefact the next three phases run on. R11 VST-CAP, R09 ROL-BPR and R13 FUN-BPR are the three to check before you let anybody call Phase B finished.

What a finished Phase B looks like in the numbers

Sheet	Rows you should have	What the repository shipped
O09 Organization Unit	12–20 units, all Parent resolved	16, complete
O11 Role	25–35, every Authority Level set	27, limits not recorded
O14 Value Stream	5–8 streams, stages named	7, stage names in Description only
R11 VST-CAP	Every row with a Stage	50 rows, Stage blank throughout
O13 Business Capability	Assessed level fully scored	556 rows, 134 assessed
R10 ORG-CAP	One Accountable per assessed row	134, exactly right
O16 Business Process	15–25 with volumes	22, complete
R09 ROL-BPR	80–120 rows, four values	23 rows, no Informed
O15 Business Service	15–25 with tested flags	20, 7 not channel agnostic
O18 Product	Every in-force product	18, split 9 Code / 9 Config
O29 Gap	At least some Business gaps	0 business gaps

Three rows of that table are the work: R11 VST-CAP, R09 ROL-BPR and the empty business-gap domain. The capability map is already good — resist improving it.

WHERE THIS VOLUME AND VOLUME 1 DISAGREE

Volume 1 appendix A publishes twelve L1 processes, P-01 to P-12; the repository holds twenty-two in O16. Volume 1 publishes thirty-three roles; the repository holds twenty-seven. Neither is wrong: volume 1 is a reference inventory at the grain an insurer can be taught, the repository one insurer's own at the grain it can be governed. Record the mapping in DLV-14 so the next architect does not re-run the reconciliation.

PHASE B — A REVIEWER'S QUESTIONS

Which business gap did this phase raise? If `O29 Gap` holds none, what did five weeks of assessment find?

Pick a capability at random: name its owner, its score, the evidence, and who signed it.

Do `R11 VST-CAP`, `R09 ROL-BPR` and `R13 FUN-BPR` carry information, or only rows?

What are you handing Phase C, by sheet and column, and who have you handed it to?

THE CHAPTER IN TEN LINES

- Phase B is eight object sheets and ten relationship sheets. The relationships are the part that rots.
- Fill a relationship sheet in the same sitting as the second of its endpoints, never in a batch.
- `O09 Organization Unit` is the shape of the payroll, not the shape of the architecture.
- `Authority Level` on `O11 Role` holds a class; put the rand limit in `Description` or it is lost.
- `R11 VST-CAP` carries `Stage` and `Criticality to stream`. Both are empty in the repository. Fix them.
- Four tests for a capability: noun not verb, survives reorganisation, one nameable owner, scorable.
- Score maturity blind, reveal, then argue from evidence. A `Target Maturity` of 4 on 120 of 134 rows is a wish list.
- Function, process, capability and service: ask whether it has a start, an end and a count per year.
- `Volume p.a.` on `O16` is what makes `RPT23 Process Flow` compute hours, and hours move funding.
- A RACI with no `Informed` rows answers nothing. The repository has twenty-three rows; you need a hundred.
- `Config or Code` on `O18 Product` is an architecture finding wearing a data field's clothes.
- Phase B that records no `Business` gap in `O29 Gap` found nothing. The repository records none.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, Phase B sheets O09–O19, R07–R16 and R38 CAP-CAP
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT03 Capability Heatmap (three blocks; Priority and Finding formulae)
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT07 Capability Coverage, RPT23 Process Flow, RPT12 Integrity Check
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets O31 Deliverable and O34 Architecture Deliverable (colliding DLV- prefixes)
- **5.** The Open Group — TOGAF Standard, 10th Edition: ADM Methods, Phase B Business Architecture https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **6.** APQC — Process Classification Framework and process frameworks overview https://www.apqc.org/process-performance-management/process-frameworks
- **7.** ACORD — Reference Architecture (Capability Model published without predefined maturity scoring) https://www.acord.org/standards-architecture/reference-architecture
- **8.** National Treasury — Release of final insurance conduct regulations and policy, 15 December 2017 https://www.treasury.gov.za/comm_media/press/2017/2017121501%20Media%20statement%20release%20of%20final%20insurance%20conduct%20regulations%20and%20policy.pdf

CHAPTER 5

Data Architecture

One column decides this phase. The other ninety are description.

Phase C has two halves and they behave differently. The data half, this chapter, is the one that gets skipped, because an entity catalogue produces no demonstrable thing and an application portfolio produces a slide the CIO will look at. Skip it and every subsequent decision is made without knowing where the truth is kept.

Meridian Assurance Group is an invention, as is Cape Mutual in volume 1; do not mix their figures.

The repository hands you the data layer in reasonable shape: twenty-four entities in `O20 Data Entity`, thirty-one relationships in `O36 Data Entity Relationship`, 161 attributes on `O37 Data Attribute`, twelve logical components, eighteen physical stores, twelve master data rules and twenty-two metadata elements [1]. What it does not hand you is agreement. Three entities carry `Authoritative Source` = `CONTESTED`, seven say `SPLIT`, every `Owner Role` but one says `ROL-09 Data Owner`, and not one of the forty-nine CRUD rows contains a `D`.

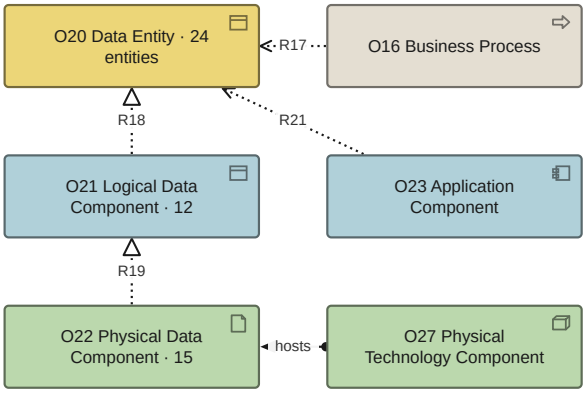

Figure 5.1 The three data layers in ArchiMate 4. An artifact realises a data object, which realises a business object: `R19` and `R18`. The process and the application reach the entity by access, never by realisation — `R17` and `R21`. Getting that distinction wrong is what produces a logical layer nobody believes.

> **THE ONE THING TO GET RIGHT IN PHASE C**
>
> `Authoritative Source` on `O20 Data Entity`. Every other column in this chapter is description. That one is a decision, it is the decision principle `DP-02 One Authoritative Source per Data Entity` exists to force, and it is the column every Phase C, D and E argument eventually reduces to.

> **BEFORE YOU START THIS PHASE**
>
> The data half of Phase C is the half volume 1 says most about.
>
> - **Volume 1, chapter 9, *The spine: a party, a contract, a thing*.** The six entities that carry the weight.
> - **Volume 1, chapter 10, *Master data and the single view of customer*.** Why insurers are structurally bad at this.
> - **Volume 1, chapter 15, *POPIA, and the two sections that catch insurers*.** The regulation behind step 5.9.
> - **Volume 1, chapter 19, *Conceptual data entities* and *Process to data, as CRUD*.** `D-01` … `D-20`, and a filled CRUD map.

The steps

Step 5.1 — Inherit Phase B and set the entity scope

Produce. A one-page Phase C data scope: which subject areas, which entities and which value streams, with the Phase B artefacts you are building on named.

You need. The three Phase B handover items from step 4.18: the `O16 Business Process` list, the `O11 Role` list with `ROL-09 Data Owner` and `ROL-16 Information Officer` identified, and the `Critical` Level 3 capabilities.

Look it up. Volume 1, chapter 9, *Three tests for whether something is an entity*, before setting scope.

Where it lands. The Phase C section of `DLV-14 Architecture Definition Document` on `O31 Deliverable`, and the scope line at the top of `DLV-16 Data Architecture (Target)`.

How.

1. List the subject areas already in `O20 Data Entity`: Customer, Policy, Product, Finance, Claim, Conduct, Distribution, Reinsurance, Regulatory, Risk, Content. Mark each in or out.
2. Pull the entities touched by the in-scope processes from `R17 BPR-DEN`. That is your minimum set, and it will be shorter than the catalogue.
3. Add the entities a `Critical` capability depends on even if no in-scope process touches them. `DEN-01 Party` qualifies on that test alone.
4. State the attribute rule now: attribute level only where step 5.8 says it earns it.
5. Name the Data Governance Council (`GB-03`, `O57 Governance Body`) as the signing body and check it can reach quorum — the repository records that it never has.

Signed by. Chief Data Officer, who chairs `GB-03`. If the Council cannot be convened, the Architecture Board (`GB-01`) signs and the inability is itself recorded as a gap.

Done when. Every subject area is marked in or out, and every in-scope entity has a reason recorded for being in.

Shows up in. `RPT15 Catalogue Completeness`, which walks six links per entity. Entities you declared out of scope should not be counted against you.

Goes wrong when. Scope is set as "all data". Twenty-four entities sounds small until each needs an owner found, an authority settled and a retention basis confirmed by legal.

SETTING THE DATA SCOPE

Which subject areas are in, and which value stream justifies each one?

Which entity is in scope only because a `Critical` capability depends on it?

Can `GB-03 Data Governance Council` reach quorum? If not, who signs instead, and is that recorded as a gap?

Step 5.2 — Read the entity catalogue, then reconcile it against volume 1

Produce. A reconciled entity list with every difference from volume 1's twenty explained, not silently resolved.

You need. `O20 Data Entity` as delivered. Volume 1, chapter 9, *The conceptual data model*, publishes fifteen entities plus five it admits as borderline; volume 1 appendix A carries them as `D-01` to `D-20`.

Look it up. Volume 1, chapter 19, *Conceptual data entities*, carries `D-01` … `D-20` with the attributes that matter. Load it, argue with it, and reconcile by name: `D-05` is not `DEN-05`.

Where it lands.

> **`O20 Data Entity`**
> `ID`, `Name`, `Description`, `Subject Area`, `Tier` (`Tier 1` or `Tier 2`), `Data Category` (`Master`, `Contract`, `Transactional`, `Reference and content`), `Master Count`, `Quality` (`Poor`, `Fair`, `Good`). `Master Count` is how many systems hold a master copy today, which is a fact to be counted rather than estimated.

How.

1. Read all twenty-four rows before changing anything. Fifteen are `Tier 1` [8].
2. Lay volume 1's twenty beside them and mark the differences in both directions. The repository has `DEN-02 Customer`, `DEN-19 Underwriting Decision`, `DEN-20 Scheme`, `DEN-22 Regulatory Return` and `DEN-23 Model`, which volume 1 does not. Volume 1 has `D-02 Party Role`, `D-09 Loss Event`, `D-11 Reserve` and `D-13 Recovery`, which the repository does not.
3. Decide each difference on its merits, in writing. The absence of a loss event entity means one event with many claims cannot be modelled — that is a real gap, not a naming choice.
4. Set `Data Category` on every row. It drives whether the entity needs a master data rule in step 5.11.
5. Set `Master Count` by counting rows in `O22 Physical Data Component`, not by asking.
6. Set `Quality` as a judgement and say so. Five entities are `Poor`, and three of those five are the customer entities.

Signed by. Chief Data Officer for the list; the Chief Enterprise Architect for the reconciliation note.

Done when. Every row has a subject area, tier, category, master count and quality, and every difference from volume 1's twenty has a one-line decision recorded.

Shows up in. `RPT15 Catalogue Completeness` and `RPT17 Master Data`, which both read `Data Category`.

Goes wrong when. The repository list is accepted as complete because it is populated. Twenty-four well-filled rows are more dangerous than ten blank ones, because nobody goes looking for what is not there.

Step 5.3 — Settle the authoritative source, entity by entity

Produce. `Authoritative Source` resolved on every in-scope entity, with the three `CONTESTED` rows either decided or escalated with a date.

You need. Step 5.2 complete. `O23 Application Component` loaded (the application half of Phase C, chapter 6). `R21 APP-DEN` with its `Authoritative` flag. Principle `DP-02` in `O05 Principle`.

Look it up. Volume 1, chapter 12, *System of record versus system of engagement*, for what a `CONTESTED` row is really about.

Where it lands.

> **`O20 Data Entity`**
>
> `Authoritative Source`. One application ID, or one of the three honest non-answers: `SPLIT`, `CONTESTED`, `NONE`. Never two IDs in one cell, and never blank.

How.

1. For each entity, list every system that holds it from `O22 Physical Data Component` and `R21 APP-DEN`.
2. Ask the owning role which one the business treats as right when two disagree. That system is the authority, whatever the architecture intends.
3. Where no answer exists, write `CONTESTED` and raise a gap. Do not write a preference.
4. Where two systems each master a genuinely different part, write `SPLIT` and record the division in `Description`. `DEN-06 Policy` is `SPLIT` between the life and short-term platforms, which is a fact about the merger, not a defect in the sheet.
5. Cross-check against `R21 APP-DEN`: a named `Authoritative Source` with no `Authoritative` flag on the matching `R21` row is an unevidenced claim.
6. Give each `CONTESTED` row an escalation date and a named decider. The repository shows `CHG-001` moving `DEN-01` from `APP-01` to `CONTESTED` under `CR-002` — which is the correct direction of travel: honesty first, then a decision.

Signed by. The Data Governance Council (`GB-03`), whose stated power is to designate a named individual owner per data entity. Where it cannot sit, the Architecture Board decides and the Council's failure to convene goes in the gap log.

Done when. No in-scope entity has a blank `Authoritative Source`; every `CONTESTED` row has a decider and a date; every named source is corroborated in `R21 APP-DEN`.

Shows up in. `RPT05 App-Data Matrix`, whose `Masters (all)`, `Baseline masters`, `Recorded authority` and `Status` columns exist to catch exactly the mismatch in step 5 above. Also `RPT22 Portfolio Data Risk`.

Goes wrong when. Somebody fills the cell with the system they think should be authoritative after the programme. That turns the baseline into a target and makes `RPT05` agree with itself while disagreeing with the business. The target belongs in `DLV-16`, not in `O20`.

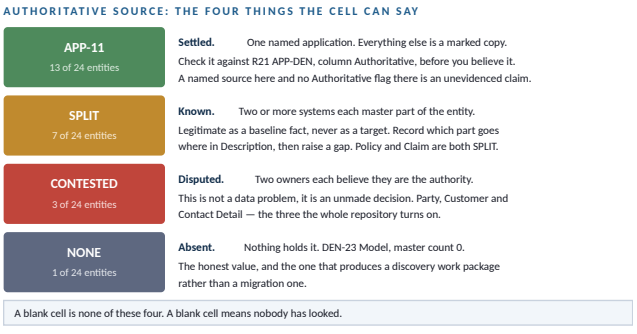

Figure 5.2 The four things Authoritative Source can legitimately say, with the repository's counts. A blank cell is none of them.

THE CONTESTED SOURCE, DECIDED

The entity catalogue itself is a Phase C form in appendix F — one row per entity, with source, owner, classification and retention. These are the questions that fill the `Authoritative Source` column honestly.

When two systems disagree about a customer, which one does the business act on today?

Who can decide this, by what date, and what does the losing system become?

Where you have written `SPLIT`, what exactly is the division — and would an engineer read it the same way?

Working table — deciding the contested sources

One row per entity whose source is `CONTESTED`, `SPLIT` or `NONE`. `Treated as right` is what the business acts on today, not what the architecture intends. Transcribe the outcome into `Authoritative Source` on `O20 Data Entity` and cross-check `R21 APP-DEN`. Signed by `GB-03`, or by the Architecture Board where the Council cannot sit.

DEN- ID	Systems holding it	Treated as right today	Verdict	Decider / date
DEN-01	APP-01, APP-04, APP-11	APP-01	CONTESTED	CDO / 2027-04

Step 5.4 — Put a person behind Owner Role

Produce. An owner per entity that is a role somebody actually performs, with the named individual recorded in the governance log.

You need. `O11 Role` and `R08 ACT-ROL` from Phase B.

Where it lands.

> **`O20 Data Entity`**
> `Owner Role`, an `O11` ID. The repository sets `ROL-09 Data Owner` on 23 of 24 entities and `ROL-17 Model Owner` on `DEN-23`.

How.

1. Check whether the role in the cell is performed by any actor. `O42 Metadata Element` `MDE-03` records the answer: 23 of 24 name a role no actor performs.
2. Replace the generic role with the business role that already owns the function. Policy data belongs to the role that owns policy administration, not to a role called Data Owner.
3. Where the generic role is right, name the individual in `O60 Governance Decision` so the accountability is attributable.
4. Keep `ROL-16 Information Officer` on the entities carrying special personal information — it is the only `Statutory` authority level in `O11 Role`.
5. Check `MDE-04 Steward`, which the repository records as `Absent`. An owner with no steward resolves no disputes.

Signed by. Chief Data Officer, with each business owner accepting their own entities in writing.

Done when. Every entity's `Owner Role` maps through `R08 ACT-ROL` to at least one actor, and every `Tier 1` entity has a named individual in the governance log.

Shows up in. `RPT18 Metadata Coverage`, business metadata category. `MDE-03` should move from `Recorded, not held` to `Full`, and `MDE-04` from `Absent` to at least `Partial`.

Goes wrong when. `ROL-09 Data Owner` is left on every row because it is defensible on paper. It satisfies principle `DP-03 Data is Owned and Stewarded` in letter and defeats `IP-01 Information is Owned by the Business`, which states the owner must be a person and not a role nobody performs.

Step 5.5 — Draw the relationships with cardinality on every one

Produce. `036 Data Entity Relationship` complete, every row carrying a verb and a cardinality.

You need. Step 5.2 complete. Volume 1, chapter 9 for the spine of the model.

Look it up. Volume 1, chapter 9, *The cardinality that carries meaning*, for the three that are decisions.

Where it lands.

> **`036 Data Entity Relationship`**
>
> `From Entity`, `To Entity`, `Relationship`, `Cardinality`. `Relationship` is a verb phrase read from source to target — Meridian uses `is known as`, `has`, `may be`, `holds`, `provides`, `covers`, `is written from`, `is claimed against`. `Cardinality` is written `1 : 0..*` style, with the optionality explicit.

How.

1. Start from the spine: party, customer, policy, benefit, risk object, claim, payment [7].
2. Write the verb before the cardinality. A relationship you cannot name in a verb is two relationships or none.
3. Make optionality explicit. `DER-01` records `DEN-01 Party` *is known as* `DEN-02 Customer` at `1 : 0..1` — a party need not be a customer, which is the distinction the whole duplication problem turns on.
4. Check every `Tier 1` entity appears at least once. Thirty-one relationships across twenty-four entities is thin; the sparse ones are where the model has not been tested.
5. Do not model many-to-many without an entity in the middle. If you need one, add it.

Signed by. Chief Data Officer. This is a modelling judgement, not a business negotiation, and it should not go to a forum.

Done when. Every row has a verb and a cardinality, and no entity in scope is isolated.

Shows up in. `RPT18 Metadata Coverage`, `MDE-07 Relationship`, recorded as `Full` with 31 relationships.

Goes wrong when. Cardinality is written as "many", or left blank. One character in that cell separates an optional child from a mandatory one, and it is the difference between a policy that can exist without a benefit and one that cannot.

Step 5.6 — Place every entity in a logical component

Produce. `021 Logical Data Component` with an explicit `Entities Held` list and a complete `R18 LDC-DEN`. Draw it as an ArchiMate 4 information structure view: each `Data Object` realising the `Business Object` entities it holds.

You need. Steps 5.2 and 5.5 complete.

Where it lands.

> **`021 Logical Data Component`**
>
> `Subject Area`, `Entities Held`, `State` (`Baseline`, `Target`, `Baseline & Target`), `Owner Role`. `Entities Held` is a comma-separated list of `DEN-` IDs. `State` says whether the component exists today, is only in the target, or both.

How.

1. Group entities by the thing that must hold them together, not by the system that holds them now. `LDC-02 Policy Store` holds `DEN-06`, `DEN-07`, `DEN-18` and `DEN-19` because a policy and its underwriting decision are inseparable.
2. Write the list out. `LDC-09 Enterprise Data Platform` says `All` in `Entities Held` and has exactly one row in `R18 LDC-DEN` — replace `All` with the enumeration or the join is a fiction and `RPT15` will keep reporting entities as unplaced.
3. Set `State` honestly. Four of twelve components are `Target` only: `LDC-03 Product Catalogue`, `LDC-06 Conduct Record Store`, `LDC-10 Model Register` and `LDC-12 Regulatory Reporting Store`. Those four are Phase E work packages in waiting.
4. Fill `R18 LDC-DEN` so every entity appears exactly once as a target.
5. Look for the entity in two components. That is either a legitimate analytical copy or a design that has not been decided.

Signed by. Chief Data Officer; the Data Domain Architect maintains it, and the repository notes that post is vacant.

Done when. Every in-scope entity appears exactly once as a `Target (DEN)` in `R18 LDC-DEN`, and no `Entities Held` cell reads `All`.

Shows up in. `RPT15 Catalogue Completeness`, the `Placed` column. Meridian's `DEN-04 Life Assured` and `DEN-05 Beneficiary` both fail it today.

Goes wrong when. Logical components are named after products. `LDC-01 Customer Master` is a logical statement; "CustomerHub" is a physical one and belongs in `022`.

Step 5.7 — Record the physical estate as it is, not as it should be

Produce. `O22 Physical Data Component` for every store that holds an in-scope entity, with `R19 PDC-LDC` marking baseline against target.

You need. Step 5.6 complete. The technology inventory from `O27 Physical Technology Component`.

Where it lands.

> **`O22 Physical Data Component`**
> `Realises` (an `LDC-` ID), `Technology`, `Hosted On` (a `PTC-` ID), `Volume`, `State`, `Owner Role`. `Volume` is free text with the unit stated: "1.1m parties", "contract level".

How.

1. One row per store, not one per server. Five rows realise `LDC-01 Customer Master`: four `Baseline` and `PDC-05 CustomerHub MDM Store` as `Target`.
2. Set `Technology` precisely enough to matter — `DB2 for z/OS`, `Oracle 19c`, `SQL Server 2019`, `PostgreSQL 15`. Phase D reads this.
3. Set `Hosted On` to a `PTC-` ID so the resilience tiering in `O49 Resilience Tier` can reach the data.
4. Fill `R19 PDC-LDC` with `State`. Meridian has 18 rows, 15 `Baseline` and 3 `Target`.
5. Count the stores per logical component. That count is the duplication finding, and it is more persuasive than any maturity score.

Signed by. Application Owner (`ROL-11`) per store; Chief Information Officer for the set.

Done when. Every `Realises` resolves to an `O21` row, every `Hosted On` to an `O27` row, and the per-component store counts reconcile to `Master Count` on `O20`.

Shows up in. `RPT22 Portfolio Data Risk` and `RPT16 Retention and Disposal`, whose `Physical stores` column counts these rows. Note that `RPT16` shows zero stores for `DEN-04 Life Assured` and `DEN-12 Advice Record` — entities with a retention obligation and no recorded store, which is worse than it looks.

Goes wrong when. The target store is entered with `State` = `Baseline` because the project is funded. Phase C records what is; `DLV-16` records what will be.

Step 5.8 — Go to attribute level only where it earns it

Produce. `O37 Data Attribute` populated for the entities that pass the test, and explicitly empty for the rest.

You need. Steps 5.2 to 5.5 complete.

Where it lands.

> **`O37 Data Attribute`**
> `Entity`, `Attribute`, `Type`, `Key` (`PK`, `BK`, `FK`, or blank), `Personal Information` (`Yes` or `No`), `Note`. The repository holds 161 attributes across 165 rows, averaging under seven per entity — this is a key-and-contention list, not a schema.

How.

1. Apply the test. An attribute earns a row if it is a key, a matching attribute, a regulatory trigger, a contested definition, or the thing a migration will fail on. Nothing else.
2. Set `Key`. Meridian records 24 `PK`, 8 `BK` and 34 `FK`. The business keys are the interesting ones: `DEN-01 Party` carries three — legal name, date of birth and national identifier — which is exactly the set `MIG-01` has to match on.
3. Set `Personal Information` on every attribute, not on the ones you remember. The repository flags 11 of 161, and that number is too low for an insurer: contact values, health disclosures and identifiers are flagged, bank details and claim narratives are not.
4. Use `Note` for the finding. `ATT-001 Party identifier` reads "Does not exist today" and `ATT-014 Vulnerability indicator` reads "Not captured today" — an attribute catalogue that records absences is doing real work.
5. Stop. Do not complete the schema. Four entities with forty attributes each will consume the iteration and nobody will read them.

Signed by. Data Steward (`ROL-10`) where one exists, Chief Data Officer otherwise.

Done when. Every `Tier 1` entity has its keys recorded, every attribute has a `Personal Information` value, and no entity has more than about a dozen rows.

Shows up in. `RPT18 Metadata Coverage`, `MDE-06 Attribute` and `MDE-17 Personal information flag`, both recorded as `Full`.

Goes wrong when. The sheet becomes a data dictionary. A hundred and sixty-one attributes is useful; four thousand is a copy of the database with worse tooling and no owner.

Step 5.9 — Treat POPIA as four columns, not as a policy document

Produce. `Classification`, `Retention Period`, `Retention Basis` and `Data Category` set on every entity, with the health-information entities identified.

You need. Steps 5.2 and 5.8 complete. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, has the regulation — do not restate it here, point at it. `DLV-19 Regulatory Obligation Register` on `O34 Architecture Deliverable` carries the obligations.

Look it up. Volume 1, chapter 15, *POPIA, and the two sections that catch insurers*. `Retention Basis` comes from the obligation, not a policy document.

Where it lands.

> **`O20 Data Entity`**
> `Classification` (`Restricted`, `Confidential`, `Internal`), `Retention Period` as a sentence, `Retention Basis` naming the instrument and section, `Data Category`. Meridian's basis cells read `POPIA s14 and FICA s23`, `FAIS General Code s3(2)`, `POPIA s26, special personal information` — section-level, which is what makes them auditable.

How.

1. Set `Classification` from handling consequence, not from sensitivity feeling. `Restricted` is the special-category tier: Meridian uses it on four entities.
2. Write `Retention Period` as the obligation, not as a number of years where the obligation is conditional. "5 years after the relationship ends" and "With the policy, then 5 years" are both correct and both unambiguous.
3. Name the instrument *and the section* in `Retention Basis`. A basis of "POPIA" is not a basis.
4. Identify the health-information entities. The Regulations Relating to the Processing of Data Subjects' Health Information took effect on 6 March 2026 and name insurers expressly [2]. For a composite with a group risk book that reaches `DEN-04 Life Assured` (health disclosure), `DEN-10 Claim` (cause of claim), `DEN-19 Underwriting Decision` (the medical basis of a loading) and `DEN-20 Scheme` (member health data held for an employer). Of those four, only `DEN-04` is `Restricted` today. The other three are `Confidential`, and that is the re-classification the regulations force.
5. Record the consequence of re-classification rather than only the classification: a `Restricted` entity needs an access control in `O41 Security Control` and an enforcement point in `O65 Enforcement Point`.

Signed by. Information Officer (`ROL-16`), the statutory role, countersigned by the Chief Risk Officer. Not the architect, and not the Chief Data Officer alone.

Done when. Every entity has all four columns; every `Restricted` entity has a named control; and the four health-information entities are classified consistently.

Shows up in. `RPT16 Retention and Disposal`, which pairs each entity's obligation against whether anything disposes of it, and `RPT22 Portfolio Data Risk`.

Goes wrong when. Classification is set once and never revisited after a regulation changes. Re-classifying three entities from `Confidential` to `Restricted` sounds like paperwork; it changes which applications may hold them, which is Phase C and D work with cost attached.

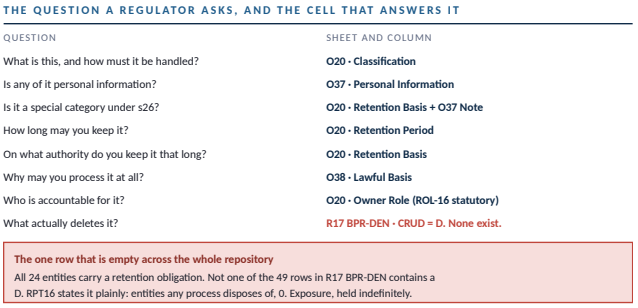

Figure 5.3 POPIA as columns. Seven questions a regulator asks, and the cell that answers each — plus the one that answers none.

CLASSIFICATION, AND WHAT IT COSTS

Which section of which instrument puts this entity in `Restricted`, and what handling changes as a result?

For each entity you move to `Restricted`, which control in `041 Security Control` and which enforcement point now has to exist?

Which attribute carries personal information and is not flagged — bank details, claim narrative, anything else?

Step 5.10 — State the lawful basis per data set, then read the disposal report

Produce. `038 Data Set` with a `Lawful Basis` per set, and a written answer to the zero in `RPT16 Retention and Disposal`.

You need. Step 5.9 complete.

Where it lands.

> **`038 Data Set`**
> `Purpose`, `Entities`, `Refresh`, `Consumer`, `Classification`, `Lawful Basis`, `State`. `Lawful Basis` names the POPIA ground: Meridian uses `Legal obligation`, `Contract performance`, `Legitimate interest` and combinations, with `DS-09 Financial crime screening` reading `Legal obligation under FICA`.

How.

1. One row per assembled set, with the purpose written before the entities. Meridian has nine, four of them `Target` only.
2. Set `Lawful Basis` for the purpose as stated. A set assembled for one basis cannot be reused under another without a new row — that is principle `IP-05 Purpose is Stated Before Information is Used`.
3. Look hard at `DS-08 Model feature`, whose basis is `Legitimate interest, subject to` conditions. A feature set feeding pricing and fraud models is where purpose limitation either holds or quietly fails.
4. Note that `Lawful Basis` lives only at data set level. `042 Metadata Element MDE-18` records this as `Partial` for exactly that reason. If you need it per entity, that is a metamodel change request, not a cell to overwrite.
5. Read `RPT16 Retention and Disposal`. Twenty-four entities carry a retention obligation; zero are disposed of by any process; every `Exposure` cell reads `Held indefinitely`.
6. Raise one gap, not twenty-four. The missing thing is a disposal process, and it is one work package that touches every entity.

Signed by. Information Officer (`ROL-16`) for the lawful bases; Chief Risk Officer for the disposal gap, because it is a risk acceptance until it is closed.

Done when. Every data set has a purpose and a basis; the disposal gap exists in `029 Gap` with a severity and an owner.

Shows up in. `RPT16 Retention and Disposal` and `RPT09 Gaps and Roadmap`.

Goes wrong when. The disposal finding is recorded as a data quality issue. It is not. It is a POPIA section 14 exposure across the whole estate, and the sheet already says so.

Step 5.11 — Write the master data rules, including the survivorship

Produce. `043 Master Data Rule` for every `Master` and `Contract` category entity, with a golden record definition and a survivorship rule in plain language.

You need. Steps 5.2, 5.3 and 5.7 complete. This is where the party and customer duplication that runs through volume 1 is either fixed or formally accepted.

Where it lands.

> **`043 Master Data Rule`**
> `Entity`, `Category`, `Mastering Style` (`Registry`, `Consolidation`, `Centralised`, `Governed`, `Not mastered`), `Golden Record Definition`, `Survivorship Rule`, `Mastering Status`, `Implemented`. `Mastering Style` also appears on `020 Data Entity` and the two must agree.

How.

1. Write a rule for every `Master` and `Contract` entity. The repository has twelve rules for twenty-four entities, which is right: the twelve `Transactional` entities are mastered by the process that creates them.
2. Write `Golden Record Definition` as a sentence a business person would accept. `MDR-01` reads "One enterprise party identifier per legal or natural person" — a definition, not a rule about systems.
3. Write `Survivorship Rule` as a decision procedure. `MDR-01` : national identifier wins where present and valid. `MDR-03` : most recently verified wins, and an unverified value never beats a verified one. These sentences are what an engineer implements.
4. Set `Mastering Status` from fact: Meridian records `Contested` on the three customer entities, `Three masters` on `DEN-06 Policy` and `DEN-08 Product Definition` , `Two masters` on `DEN-07` and `DEN-18` , `Single source` on five.
5. Keep `Implemented` honest. All twelve read `No` , and `RPT17 Master Data` states "Survivorship rules implemented, 0". A written rule nobody has built is still worth writing, because it is what the build is tested against.
6. Check that `Mastering Style` matches between `O43` and `O20` for every entity.

Signed by. Data Governance Council (`GB-03`) for the golden record definitions, which are business definitions. The survivorship rules are signed by the entity owner.

Done when. Every `Master` and `Contract` entity has a rule; every rule has both a definition and a survivorship sentence; `Mastering Style` agrees across both sheets.

Shows up in. `RPT17 Master Data` , whose three footer figures — twelve mastered, three contested, zero survivorship rules implemented — are the clearest single statement of the problem in the repository.

Goes wrong when. `Survivorship Rule` is written as "the most recent record wins". That rule is why the duplication exists: the most recent record is usually the one created by the latest channel that could not find the existing party.

THE SURVIVORSHIP RULE, WRITTEN TO BE BUILT

When two party records match, the record that survives is the one that — finish the sentence so an engineer can build it.

What does your rule do when the most recent record is the least verified one?

How many records will not resolve automatically, and who stewards them by hand?

Step 5.12 — Record the metadata position, including what is absent

Produce. `042 Metadata Element` reflecting the real coverage, with every `Absent` element either scheduled or accepted.

You need. Steps 5.2 to 5.11 complete, because metadata coverage is a verdict on them.

Where it lands.

> **`042 Metadata Element`**
>
> `Category` (`Business` , `Technical` , `Operational` , `Governance` metadata), `Element` , `What It Records` , `Held In` (a sheet name, or `Nowhere`), `Coverage` (`Full` , `Partial` , `Recorded, not held` , `Absent`), `Note` .

How.

1. Set `Held In` to the actual sheet, or to `Nowhere` . `Nowhere` is a valid and useful value.
2. Set `Coverage` against evidence. `MDE-21 Access` reads `Partial` and covers 11 of 27 roles, which is a direct consequence of the incomplete `R09 ROL-BPR` in Phase B.
3. Read the eight `Absent` elements and rank them. `MDE-14 Lineage` is the one IFRS 17 and COFI both assume exists; `MDE-22 Consent` is the one POPIA assumes; `MDE-10 Schema version` is the one every migration assumes.
4. Attach each `Absent` element to a work package or accept it in writing. Eight absences with no plan is a governance finding, not a metadata one.
5. Re-read `RPT18 Metadata Coverage` after each change. The category split — business 2 of 5 `Full` , technical 4 of 5, operational 0 of 5, governance 4 of 7 — is the shape of the problem: Meridian knows what its data means and nothing about how it moves.

Signed by. Chief Data Officer. Metadata coverage is a self-assessment and should be labelled as one.

Done when. Every element has a `Held In` and a `Coverage`; every `Absent` element has a work package or a recorded acceptance.

Shows up in. `RPT18 Metadata Coverage`, both the category summary and the element detail.

Goes wrong when. The sheet is filled aspirationally, with `Partial` where the honest answer is `Absent`. `Absent` is the value that produces a work package; `Partial` is the value that produces nothing.

Step 5.13 — Complete the process-to-data CRUD map

Produce. `R17 BPR-DEN` with a `CRUD` value for every real process-entity pair, including the `D`s. The ArchiMate 4 view is one `Process` per row with an access relationship to each `Business Object` it touches, labelled with the CRUD letter.

You need. `O16 Business Process` from Phase B and `O20 Data Entity` complete. Volume 1 appendix A drafts this map across `P-01` to `P-12`, with the creating process marked for each entity — start there.

Look it up. Volume 1, chapter 19, *Process to data, as CRUD*. Match by name — the `BPR-` and `DEN-` numbering is unrelated to volume 1's.

Where it lands.

> **`R17 BPR-DEN`**
>
> `Source (BPR)`, `Target (DEN)`, `CRUD`, `Note`. `CRUD` is a string of the letters that apply: Meridian uses `C`, `R`, `U`, `RU`, `CR`, `CRU` — and no value anywhere contains `D`.

How.

1. Walk process by process, not entity by entity. A process owner can tell you what their process creates; an entity owner cannot tell you who creates it.
2. Mark exactly one creating process per entity where you can. Volume 1's version bolds the creator for each entity and that column is the one worth reading alone.
3. Find the entities with no creator. `DEN-02 Customer` has three processes reading or updating it and none creating it — which `STM-10` on `040 Source To Target Mapping` confirms in words: customer is created by no function.
4. Find the entities no process touches at all: `DEN-15 Commission Transaction`, `DEN-16 Reinsurance Treaty` and `DEN-20 Scheme`. Either three processes are missing from `O16` or those entities belong to an out-of-scope line of business.
5. Add the `D`s. Every entity with a retention obligation needs a process that disposes of it, and today none has one.
6. Do not read the footer of `RPT04 CRUD Matrix`. Its `Accessing processes` row counts non-blank cells over formula cells that return empty strings, so it reports 22 for every one of the twenty-four entities. Count from `R17 BPR-DEN` [3].

Signed by. Each process's accountable role for its own rows; Chief Data Officer for the set.

Done when. Every in-scope entity has at least one creating process or a recorded reason why not, and at least one `D` exists somewhere in the sheet.

Shows up in. `RPT04 CRUD Matrix`, read as a grid and not as a total, and `RPT16 Retention and Disposal`, whose `Disposed by any process` column is computed from this sheet.

Goes wrong when. The map is built from system documentation rather than from process owners, which produces a complete-looking grid in which every system reads everything and nothing is created anywhere.

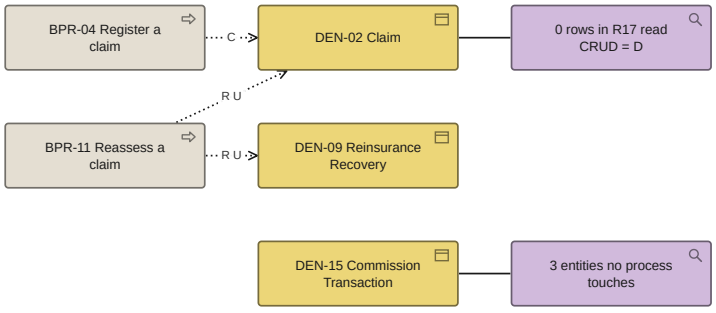

Figure 5.4 `R17 BPR-DEN` drawn as what it is — access relationships, labelled with the CRUD letter. Two processes create `DEN-02 Claim`. `DEN-15 Commission Transaction` has no process at all. And no row anywhere in the sheet carries a `D`.

THE CRUD MAP, READ HONESTLY

Which in-scope entity has no creating process, and what does that say about where it really comes from?

Which entities no process touches at all — are processes missing from `016`, or are those entities out of scope?

Where will the first `D` come from, and which process will own disposal?

Step 5.14 — Stub the migrations while the data is fresh in the room

Produce. `039 Data Migration` and `040 Source To Target Mapping` populated for every logical component whose `State` is `Target`, with a quality gate per migration.

You need. Steps 5.6, 5.7 and 5.11 complete. These are Phase C stubs that Phase F will finish — the reason they belong here is that nobody will remember the detail by Phase F.

Where it lands.

> **`039 Data Migration`**
>
> `Sources`, `Target`, `Entities`, `Volume`, `Approach`, `Quality Gate`, `Gate Status`, `Wave`, `Work Package`, `Risk`. `Quality Gate` is a testable condition, not an intention: `MIG-01` reads "95% of presented parties resolve". `Gate Status` is `Not met` on all six migrations, which is the correct status for a gate nobody has run.

How.

1. One migration per target logical component. Meridian has six, matching the four `Target` components plus the analytical estate and the model register.
2. Write `Volume` with both sides. `MIG-01` reads "2.6m source records to 1.9m resolved parties" — the shrinkage is the point.
3. Write `Quality Gate` as a number somebody can measure and fail. The gate on `MIG-04` is that a rating factor changes in under a stated time, which is a business test, not a data test.
4. Record the `Risk` sentence. `MIG-01`: roughly 640,000 records cannot be resolved automatically. That sentence sizes the stewardship team.
5. Fill `040 Source To Target Mapping`, one row per source-entity pair, with `Quality Issue To Resolve First` populated on every row. That column is the whole value of the sheet: "Date of birth quality poor in the funeral book" is a work item, not a note.
6. Where there is no source, say so. `MIG-03` records "No source. Models are held in spreadsheets" and its first mapping row reads "No inventory exists" — which turns a migration into a discovery exercise with a different shape and cost.

Signed by. Chief Data Officer for the gates; the receiving work package owner for the approach; the Investment Committee (`GB-02`) sees them in Phase E.

Done when. Every `Target` logical component has a migration row with a measurable gate, and every migration has at least one `040` row with a quality issue named.

Shows up in. `RPT09 Gaps and Roadmap` once the migrations attach to work packages, and `RPT27 Programme Sequencing` in Phase F.

Goes wrong when. `Quality Gate` is written as "data quality is acceptable". A gate that cannot fail is not a gate, and `Gate Status` becomes a formality rather than the thing that stops a cutover.

Step 5.15 — Close the data half of Phase C

Produce. `DLV-16 Data Architecture (Target)` complete, and the data checkpoint run and recorded.

You need. Steps 5.1 to 5.14 complete, with `02 Change Log` entries throughout.

Where it lands.

> **`031 Deliverable`**
>
> `DLV-16 Data Architecture (Target)`, `ADM Phase` = C, `Work Package` = WP-04, owner Chief Data Officer, `Status` from `Not started` to Complete. Beware the prefix collision: `034 Architecture Deliverable` also uses `DLV-` IDs, and its `DLV-16` is the Architecture Governance Framework. Name the sheet whenever you cite a `DLV-` number [4].

How.

1. Run the checkpoint: `RPT15 Catalogue Completeness`, which walks six links per entity — Named, Defined, Placed, Stored, Sourced, Consumed. Meridian completes all six on 11 of 24.
2. Work the `Where it breaks` column rather than the score. Three entities break on `Contested source`, two on `No logical component`, several on `No physical store`.
3. Read `RPT17 Master Data` and `RPT16 Retention and Disposal` together. The first says the truth is in several places; the second says none of it ever leaves. Those two reports are the Phase C data story in two sentences.
4. State the target in `DLV-16` as deltas against the baseline you recorded, never by editing `O20` to describe the future.
5. Hand Phase D two things: the `Classification` and `Retention Period` columns, which set the control and storage requirements, and the `O22` technology list.
6. Check that every data gap is in `O29 Gap`. Six exist today — `GAP-01` to `GAP-04`, `GAP-12`, `GAP-18` — four of them `Critical` or `High` and all pointing at the customer entities.

Signed by. Chief Data Officer owns `DLV-16`; the Architecture Board (`GB-01`) baselines it; the Information Officer signs the POPIA columns separately so the regulatory sign-off is traceable on its own.

Done when. `DLV-16` is Complete on `O31 Deliverable`, `RPT15` shows no entity failing on `Sourced` without a dated decision behind it, and the disposal gap has an owner.

Shows up in. `RPT14 Deliverable Register`, `RPT15 Catalogue Completeness` and `RPT09 Gaps and Roadmap` [5].

Goes wrong when. Phase C closes with the catalogue complete and the three `CONTESTED` rows still contested. A data architecture that cannot say where the customer is mastered has not been done, however many other columns are full.

What a finished Phase C data layer looks like

Sheet	What it should say	What the repository says
`O20 Authoritative Source`	One source per entity	3 `CONTESTED`, 7 `SPLIT`, 1 `NONE`
`O20 Owner Role`	A role an actor performs	23 of 24 say `ROL-09`
`O21 Entities Held`	An enumerated list	`LDC-09` says `All`
`O37 Personal Information`	Every attribute assessed	11 of 161 flagged
`O38 Lawful Basis`	Per data set, per purpose	9 sets, all populated
`O43 Implemented`	Rules built and tested	12 rules, 0 implemented
`O42 Coverage`	Honest, including `Absent`	10 `Full`, 4 `Partial`, 8 `Absent`
`R17 CRUD`	Includes `D` where retention bites	49 rows, no `D`
`O39 Gate Status`	Measurable, and some met	6 migrations, all `Not met`
`RPT15` six links	Most entities complete	11 of 24 complete

Four of those rows are the Phase C data agenda: settle the three contested sources, put people behind the owner column, add a disposal process, and write survivorship rules somebody intends to build. Everything else on the sheet is already adequate.

> **WHAT THIS REPOSITORY CANNOT DO, AND WHAT TO DO INSTEAD**
>
> `Lawful Basis` exists only on `O38 Data Set`, so there is no place to record the basis for an entity held outside any data set — raise a change request against the metamodel rather than overloading `Retention Basis`. `O20` has no column for the data subject's consent state; `MDE-22 Consent` records this as `Absent` and `ATT-020 Consent basis` on `DEN-03 Contact Detail` is the only trace of it. And `Mastering Style` is duplicated on `O20` and `O43`, so the two drift — pick `O43` as the authority for it and make `O20` the copy.

PHASE C, THE DATA HALF — A REVIEWER'S QUESTIONS

Where is the customer mastered? One answer, one system, or a dated decision saying when there will be one.

Which owner in `Owner Role` is a role an actual person performs, and who is that person?

What disposes of anything? Name the process, or name the accepted risk and who accepted it.

Which migration gate could fail, and what happens to the cutover if it does?

THE CHAPTER IN TEN LINES

- `Authoritative Source` on `O20 Data Entity` is the one decision in Phase C. Everything else describes.
- `SPLIT`, `CONTESTED` and `NONE` are honest answers. A blank cell is not one of them, and three rows are contested: Party, Customer, Contact Detail.
- An owner must be a role an actor performs. Twenty-three of twenty-four name a role nobody performs.
- `Entities Held` reading `All` breaks `R18 LDC-DEN` and `RPT15` silently. Enumerate it.
- Attributes earn a row if they are a key, a matcher, a regulatory trigger or a migration risk.
- POPIA is four columns on `O20` plus one flag on `O37` plus `Lawful Basis` on `O38`.
- The health information regulations have been in force since 6 March 2026 and name insurers.
- Twenty-four entities carry a retention obligation and nothing disposes of any of them.
- Twelve master data rules, zero implemented. Write them anyway — the build is tested against them.
- No `CRUD` value in the repository contains a `D`, and `RPT04`'s footer row is wrong. Count from `R17 BPR-DEN`.
- `DLV-16` means Data Architecture on `O31` and Governance Framework on `O34`. Always name the sheet.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, Phase C data sheets O20, O21, O22, O36, O37, O38, O39, O40, O42, O43 and R17–R19
- **2.** Moonstone — New POPIA regulations on health information now in force (Regulations Relating to the Processing of Data Subjects' Health Information, effective 6 March 2026; named parties include insurers) https://www.moonstone.co.za/new-popia-regulations-on-health-information-now-in-force/
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT04 CRUD Matrix, footer row formula COUNTIF(range,"<>") over formula cells
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets O31 Deliverable and O34 Architecture Deliverable (colliding DLV- prefixes)
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT15 Catalogue Completeness, RPT16 Retention and Disposal, RPT17 Master Data, RPT18 Metadata Coverage
- **6.** The Open Group — TOGAF Standard, 10th Edition: ADM Methods, Phase C Information Systems Architectures https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **7.** Len Silverston and Paul Agnew — The Data Model Resource Book, Volume 3 (the Party supertype and role pattern) https://www.oreilly.com/library/view/the-data-model/9780470178454/chapter-32.html
- **8.** ACORD — Reference Architecture (the Information Model as a conceptual representation) https://www.acord.org/standards-architecture/reference-architecture

CHAPTER 6

Application Architecture

The half of Phase C where the arguments get settled, because an application has a cost centre, a vendor and a person whose name is on it.

The ADM puts data and application architecture in one phase [10], and they are not equal halves. The data half, chapter 5, is where the arguments happen. The application half is where they get settled, because an application has a cost centre, a vendor, a contract and a person whose name is on it. You can be vague about who owns Party. You cannot be vague about who owns the policy system.

Seventeen steps, most of them `O23 Application Component` and the nine relationship sheets that hang off it. Both insurers named here are inventions: **Meridian Assurance Group** is the insurer in the repository you were handed, **Cape Mutual** is volume 1's worked example. Their numbers are not interchangeable.

BEFORE YOU START THIS PHASE

The section below says to walk the conceptual model first. It lives in four places.

- **Volume 1, chapter 11, *What goes in a box, and what does not.*** Four tests for an application capability.
- **Volume 1, chapter 11, *The patterns.*** Systems per model entry, and the *usually missing entirely* list.
- **Volume 1, chapter 12, *The shape of the estate.*** The four groups a long inventory falls into.
- **Volume 1, chapter 19, *Conceptual applications*, *Application to data*, *with the system of record* and *Capability to application — the coverage map.*** `A-01` … `A-25`, and two filled maps.

Start at the model, not the estate

The commonest way to waste Phase C is to begin with a list of systems. Someone sends an export from the service-management tool, it has 340 rows, 90 of them printers, and you spend five weeks cleaning it. At the end you have a tidy list and no way to say whether it is complete, because nothing tells you what *should* be there.

Volume 1 chapter 11 tells you. The conceptual application model names twenty-five applications, `A-01` to `A-25`, by what they do, with no product name in it. Volume 1 appendix A indexes all twenty-five against capabilities and data entities and marks the system of record for each. Volume 1 chapter 12 names what insurers actually run, class by class, and how each goes bad.

Use them in that order. Walk `A-01` through `A-25` and for each one ask *which real system does this here, and how well*. A model entry with no system behind it is a gap you can name before the inventory is finished. A system that maps to no model entry is either something you do not understand yet or something nobody needs.

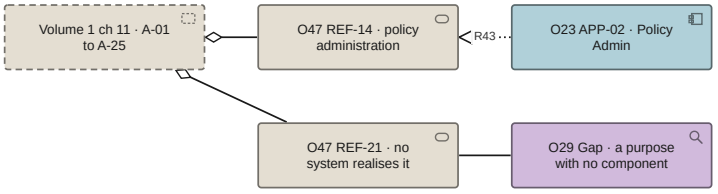

Figure 6.1 The direction of travel in Phase C, in ArchiMate 4. The conceptual model from volume 1 chapter 11 aggregates the purposes held in `047`; a real application realises one of them. A purpose nothing realises is a gap you can name before the inventory is finished.

The repository has a place for this reasoning. `047 Reference Model Entry` holds thirty-four purposes across two domains — eighteen `Application`, sixteen `Technology` — each with a `Purpose` phrased as a job ("Hold a party or customer record"), a `Standard Today`, a `Status` and an `At Target`. Five of Meridian's thirty-four have no approved answer [7]. That sheet is the conceptual model with the organisation's answers filled in, and step 6.15 reconciles the two.

THE SPINE OF PHASE C APPLICATION WORK

Walk the conceptual model. Map real systems onto it. Record what you find in `O23 Application Component`. Never let the inventory define the model: it has survivor bias built in, because the systems easiest to see are the ones somebody already watches.

Step 6.1 — Lay the conceptual model across the estate

Produce. A one-page mapping from `A-01` … `A-25` to the systems you believe deliver each one, blanks left visibly blank.

You need. Volume 1 chapter 11 and appendix A for the application-to-capability and application-to-data maps; volume 1 chapter 12 to recognise what you are shown. Ninety minutes each with the heads of distribution, underwriting, claims, finance and data.

Look it up. Volume 1, chapter 19, *Conceptual applications*, indexes all twenty-five by domain. Map them onto the `APP-` rows by name: `A-08` is not `APP-08`.

Where it lands.

> **`047 Reference Model Entry`**
>
> One row per purpose. `Domain` is `Application` here. `Purpose` is the job, as a verb the business would recognise. `The Need` is why the job must be done. `Standard Today` is the system that does it now, or `No standard` — a legitimate and useful answer. `Status` from `Current`, `Contained`, `Retiring`, `Unsupported`, `Emerging`, `No standard`. `At Target` is what should do it. `When To Use It` and `When Not To` make the sheet usable by a project architect who was not in the room.

How.

1. Copy the twenty-five conceptual applications from volume 1 chapter 11 into a working list.
2. For each, write the system you expect and the person who would know. Do not research yet — you are recording your prior so you can be wrong on the record.
3. Hold the five interviews. One question per entry: *what does this job here?*
4. Two systems, record two. A spreadsheet, record the spreadsheet. No answer, `No standard`.
5. Collapse into `047` rows. Meridian's eighteen application entries are the shape to aim for.

Signed by. Chief Enterprise Architect, with the Chief Information Officer acknowledging the `Standard Today` column.

Done when. Every `047` application row has a `Standard Today` or an explicit `No standard`, and every conceptual application from volume 1 chapter 11 is mapped or recorded as out of scope with a reason.

Shows up in. `RPT25 Reference Model Conforman` — Excel truncates the Reference Model Conformance report's tab name, and this spelling is what you type. Its footer counts `Purposes`, `No approved answer` and `Past vendor support`; Meridian reads 34, 5 and 3 [7]. A `No approved answer` count of zero means you were too generous.

Goes wrong when. You map the model to the estate instead of the estate to the model. The tell: every entry has one system and nothing says `No standard`, which means the purposes were written *after* the answers.

The inventory itself

`023 Application Component` is twenty-five columns wide: eighteen content, seven the version block every object sheet carries. Meridian holds twenty-six rows [1] — the right order of magnitude for a composite insurer counting applications rather than servers. The eighteen content columns fill at four moments, by four sets of people.

Group	Columns	Who fills it	When
Identity	`ID`, `Name`, `Description`, `Type`	Architect	Step 6.2
Accountability	`Owner Org`, `Owner Role`, `Criticality`	Architect with CIO	Step 6.2
State and intent	`Lifecycle`, `Disposition`, `Year Live`	Owning exec	Step 6.3
Size and cost	`Users`, `Annual Run Cost (Rm)`, `Hosting`, `Vendor`	Finance with IT	Step 6.3
Assessment	`Business Value`, `Technical Fitness`, `Portfolio Quadrant`, `Disposition Agrees`	Workshop	Steps 6.5–6.7

`Type` deserves a word. Meridian records twenty-one `Physical` and five `Logical`. A `Logical` row is an application that does not exist yet, given an identity so requirements, capabilities and work packages can point at it — `APP-20 API Gateway` through `APP-24 ModelOps`, all five with `Vendor` = `To be selected`, `Annual Run Cost (Rm)` = 0 and a `Year Live` in 2027 or 2028. That is how to hold a target application: a named row with zero cost, not a box on a slide.

> **Step 6.2 — Open a row for every application you can name**
>
> **Produce.** `023 Application Component` populated with identity and accountability for every application, declared and target.
>
> **You need.** Step 6.1's mapping. The service-management tool's list, if one exists. `009 Organization Unit` and `011 Role` populated from Phase B — you cannot fill `Owner Org` or `Owner Role` against sheets that do not exist.
>
> **Where it lands.**

023 Application Component

`ID` sequential `APP-nn`, never reused, never renumbered. `Name` is what people in the building call it, not the vendor's product name. `Description` is one sentence on what it administers — Meridian's `APP-01` reads "Life and funeral policy administration on the mainframe", which gives line of business and platform in eleven words. `Type` is `Physical` for something running, `Logical` for something designed. `Owner Org` is an `ORG-` id, `Owner Role` a `ROL-` id. `Criticality` is `Tier 1` to `Tier 3`, the same tier language as `O12 Location` and `O49 Resilience Tier`.

How.

1. One row per entry from step 6.1, named as the business names it.
2. Fill `Owner Org` from `O09`. If two units claim it, pick the one that pays and log the dispute in `02 Change Log`.
3. Fill `Owner Role` from `O11`. Expect lopsidedness: Meridian has twenty-three of twenty-six on `ROL-11`, which is honest and also a finding.
4. Set `Criticality` by one test: if it stops for a working day, does the insurer breach a policyholder obligation or a regulatory deadline? Yes is `Tier 1`.
5. Leave the assessment columns empty until steps 6.5 to 6.7.

Signed by. Chief Information Officer for `Owner Org` and `Criticality`; Chief Enterprise Architect for the rest.

Done when. Every row has the seven identity and accountability columns filled, and every `Owner Org` resolves to a live `O09` row.

Shows up in. `RPT02 Application Portfolio`, a live view over this sheet, and `RPT12 Integrity Check`, which will show `O23` under-linked until steps 6.9 to 6.14 are done.

Goes wrong when. `Owner Role` is a committee. A role is one person's job. Twenty-three applications on one role is a resourcing problem you can state; "the Technology Steering Group" is one you cannot.

Step 6.3 — Classify, size and cost each application

Produce. The state, intent, size and cost columns filled for every row, with run cost reconciled to the ledger.

You need. Step 6.2. Run cost by application from finance. The licence and support contracts — Meridian holds four technology agreements in `O19 Contract`, `CTR-04` to `CTR-07`. Volume 1 chapter 12 for what a system class should cost relative to its peers.

Where it lands.

023 Application Component

`Lifecycle` is factual: `Current`, `Legacy`, `Emerging`, `Declining`. `Disposition` is intentional: `Sustain`, `Extend`, `Consolidate`, `Replace`, `Retire`, `Build`. Keep the two vocabularies separate — the value of the pair is that a `Current` system can carry `Replace` and a `Legacy` one `Sustain`. `Year Live` is first live, not the last upgrade. `Users` is named human users, zero for an anonymous public surface. `Annual Run Cost (Rm)` is licence plus support plus infrastructure plus the run team. `Hosting` is `On-premise`, `Cloud` or `SaaS`. `Vendor` is a class, not a name: `In-house`, `Commercial vendor`, `ERP vendor`, `To be selected`.

How.

1. Take `Year Live` from the contract start or the project closure report, not from memory.
2. Get run cost from the ledger, not the budget. Budget is intent; ledger is fact.
3. Reconcile the column total against the IT run line. Meridian's twenty-six rows total **R320.5m**, and `RPT02` computes each application's `% of Run` from it [3]. Unreconciled, the next step has nothing to measure against.
4. Set `Hosting` from where the compute runs, not who operates it. A vendor-managed box in your data centre is `On-premise`.
5. Keep `Vendor` as a class. Product names turn the catalogue into a procurement document and date it within a year.

Signed by. Chief Financial Officer for `Annual Run Cost (Rm)`; Chief Information Officer for the rest.

Done when. `Annual Run Cost (Rm)` reconciles to the ledger's IT run line within five per cent, and no `Legacy` row carries `Sustain`.

Shows up in. `RPT02 Application Portfolio`: `Age (yrs)`, `% of Run`, and the two summary blocks at the foot. Meridian reads `Current` 17 at R214.6m, `Legacy` 4 at R105.9m, `Emerging` 5 at R0 [3]. Four legacy systems carrying a third of the run cost is the sentence that funds Phase E.

Goes wrong when. Run cost is allocated rather than measured. A per-user share of the IT budget makes the quadrant a function of headcount, and `APP-01` stops looking expensive.

THE RUN-COST RECONCILIATION

The inventory itself transcribes onto the Phase C application form in appendix F — identity, accountability, state and cost. These are the questions that make the cost columns true.

What is the ledger IT run line for the last full year, and what does `Annual Run Cost (Rm)` in `O23` sum to?

What is the difference made of — allocations, applications you have not recorded, or spend nobody owns?

Which row is an allocation rather than a measurement, and who can measure it properly?

Finding the applications nobody declares

Every estate has applications no inventory contains. They are invisible because the inventory was built from what IT operates, and these are what the business operates. Three rules find most of them, and all three produce evidence you can put in a sheet.

Figure 6.2 The three discovery rules, what each one reads, and the sheet the finding lands in. Run all three: each misses a different class of undeclared application.

The spreadsheet rule. A spreadsheet is an application when three things are true: more than one person edits it, it has a refresh rhythm, and something downstream depends on its output. Ask the owner what happens if it is deleted on Friday afternoon, and give it an `APP-` id if the answer names a regulatory return, a payment run, a reserving figure or a reinsurance recovery. Meridian already has one: `APP-25 LegacyReporting`, departmental spreadsheet and Access-based reporting, 380 users, R3.2m a year, `Tier 3`, `Legacy`, `Retire`, with `GAP-12 Spreadsheet regulatory assembly` against it [1]. Volume 1 chapter 12 says which classes most often sit in a spreadsheet in a South African insurer: reinsurance recoveries and exposure accumulation lead.

The invoice rule. Pull twenty-four months of accounts payable from the general ledger — at Meridian, `APP-11 FinERP`. Filter two ways: cost centre equals IT, and then — the half that finds things — every software, licence, subscription and professional-services account code *across all cost centres*. Shadow applications are paid for by the business, not by IT, which is why the IT asset register does not have them. Every vendor with a recurring payment and no `O23` row is one of three things: an application you have not recorded, a contract you have not recorded in `O19 Contract`, or a service nobody uses. All three are findings. R500,000 a year is a defensible materiality threshold in a group of Meridian's size.

The Active Directory rule. Export security groups, service accounts and the identity provider's application registrations — at Meridian, `PTC-11 Entra ID Tenant`. Three signals. A security group named after a system with no `O23` row means the system exists and someone grants access to it. A service account with database rights and no owning application is almost always an interface, and belongs in `R23 APP-APP` with `Managed = No`. And the identity provider's list of federated applications will be longer than your `Hosting = SaaS` count: Meridian records three of twenty-six, and a tenant export in a real insurer that size routinely returns thirty or more.

Step 6.4 — Run the three discovery rules and close the inventory

Produce. A dated discovery log, new `O23` rows, and a written statement of what you chose not to record and why.

You need. Step 6.3's reconciliation. Read access to accounts payable, the identity provider and the finance and actuarial shared drives. One analyst for four days.

Where it lands.

> **`023 Application Component`**
>
> New rows, each with `Description` saying how it was found — provenance matters more here than anywhere else in the sheet, because these rows will be challenged. Where `Hosting` and `Vendor` are unknown, record a best guess and raise a change request to confirm, rather than leaving the row out.

How.

1. Spreadsheet rule: shared drives, external links, distinct editors, top twenty interviewed.
2. Invoice rule: twenty-four months, IT cost centre plus software account codes everywhere else, ranked by annual spend.
3. Active Directory rule: groups, service accounts, identity-provider application list.
4. Each candidate gets one of three outcomes: new `023` row, new `019 Contract` row, or rejected with a reason.
5. Log every rejection. The rejections make the inventory defensible six months later.
6. Re-run the step 6.3 reconciliation. The gap should have closed.

Signed by. Chief Information Officer, with the Chief Financial Officer confirming the ledger extract was complete.

Done when. The run-cost reconciliation closes to within five per cent and the count of applications discovery found that the original list did not have is written down. Zero means the discovery was not run properly.

Shows up in. `RPT02 Application Portfolio` row count and total, and `RPT21 Software Distribution`, whose Meridian footer reads "Five applications cannot be traced to a location. Three are SaaS, where no Meridian location is the right answer and the register should say so" [8].

Goes wrong when. You find twelve spreadsheets and add twelve rows. An application is a capability boundary, not a file. Twelve finance workbooks that together assemble the statutory returns are one application, badly implemented — which is what `APP-25` is.

WHAT THE THREE DISCOVERY RULES TURNED UP

How many applications did discovery find that the original list did not have? If the answer is zero, which rule did you not really run?

Which spreadsheet would stop a regulatory return, a payment run or a reinsurance recovery if it were deleted on Friday?

Which vendor takes a recurring payment from a business cost centre with no `023` row and no `019 Contract` row?

Working table — the discovery log

One row per candidate, including the rejections — they are what make the inventory defensible later. `Found by` is the rule: spreadsheet, invoice or directory. Signed by the Chief Information Officer, with finance confirming the ledger extract was complete.

Candidate	Found by	Evidence	Outcome	Reason
Recoveries workbook	Spreadsheet	4 editors, monthly	New `023` row	Feeds reinsurance recovery

Scoring the portfolio

Two columns carry the assessment, `Business Value` and `Technical Fitness`, and two are derived from them, `Portfolio Quadrant` and `Disposition Agrees`. The repository stores the results, not the working, so the method is yours to supply — and must be the same every year, or the trend means nothing. Meridian's scores run 2 to 11 and 1 to 10 [1][2], ranges that come from a sum of sub-scores. That is the only method that survives a workshop: a participant can argue with one sub-score without rejecting the whole number.

Business Value, out of 12. Four sub-scores, 0 to 3, scored by the business owner:

Sub-score	0	3
Premium or claims exposure touched	none	the majority of gross written premium
Regulatory or contractual necessity	none	a return cannot be produced without it
Differentiation	commodity	the reason customers choose you
Breadth of capability support	no `R37 APP-CAP` rows	ten or more

Take differentiation from `O13 Business Capability`'s `Differentiation` column for the capabilities the application supports, not from the owner's opinion of their own system. Take breadth from the row count in `R37 APP-CAP` once step 6.10 is done, which is why the scoring workshop runs a second pass after the capability mapping.

Technical Fitness, out of 10. Five sub-scores, 0 to 2, scored by the owning engineer, each with evidence named:

Sub-score	Evidence	0	2
Support currency	`O27 Support Ends`	past or inside twelve months	beyond five years
Standards conformance	`R31 STD-PTC Conformance`	any `Non-conformant`	`Conformant` throughout
Integration style	`R23 APP-APP Style`, `Managed`	batch file or direct database read	managed API or event
Cost of change	release history	code change and a platform release	configuration in days
Operability	`O49 Last Tested`, `STD-11`	neither tested nor instrumented	both

Score the axes separately and never let one person score both. An owner scoring value has an incentive to score fitness low to attract investment; an engineer scoring fitness has an incentive to score value high to protect the team.

Step 6.5 — Score business value with the owner

Produce. A `Business Value` score, 0 to 12, for every `O23` row, with sub-scores in the workshop pack.

You need. Step 6.4. `O13 Business Capability` with `Differentiation` from Phase B. `O18 Product` for the premium each application's line of business carries. The named business owner for each application, not their delegate.

Where it lands.

> **`O23 Application Component`**
>
> `Business Value` as an integer. The sub-scores have no column in the repository: keep them in the workshop pack attached to `DLV-17`, record the pack's date in `02 Change Log` against the `Business Value` field, and say in the catalogue that the sheet holds the total and the pack the working.

How.

1. Pre-fill exposure and necessity from `O18 Product` and `O63 Control Objective`, so the room argues about two sub-scores rather than four.
2. Run the room in `APP-` order, ninety minutes for twenty-six rows. Anything over three minutes is parked, not debated.
3. Resolve parked items offline with the two who disagreed, circulate, write the total into `O23`.

Signed by. The owning exec per application, recorded in `02 Change Log` as `Changed By`.

Done when. Every row has an integer, the parked list is empty, and no two applications doing the same job are more than two apart without a written reason.

Shows up in. `RPT20 Portfolio Assessment`, column `Business value`. Meridian's distribution is top-heavy on purpose: the two policy platforms at 11 and 10, nothing else above 8 [2].

Goes wrong when. Everything scores 7 or 8 — the signature of a room scoring importance rather than value, which makes the quadrant useless because nothing falls below the threshold.

Step 6.6 — Score technical fitness with the engineer

Produce. A `Technical Fitness` score, 0 to 10, for every row, each sub-score backed by named evidence from the repository.

You need. Step 6.5. `O27 Physical Technology Compone` with `Support Ends` — if Phase D has not started, this step and step 7.4 run in parallel and trade data. `R31 STD-PTC`. Each application's change history, for the cost-of-change sub-score.

Look it up. Volume 1, chapter 12, *The mainframe reality in life insurance*, before anybody proposes replacing one.

Where it lands.

023 Application Component

`Technical Fitness` as an integer. Where the evidence does not exist, score 0 and note it: a missing recovery test is a fitness problem, not an unknown.

How.

1. Extract `Support Ends` per application by joining `R24 APP-TSV` to `027` through `R25 PTC-LTC`. No traceable physical component means a support sub-score of 0.
2. Pull `Conformance` verdicts from `R31 STD-PTC` for every component the application runs on.
3. Score integration style from `R23 APP-APP`. Any `Direct DB read` row caps it at 0.
4. Score cost of change from the last four releases: days from approved request to production.
5. Score operability from `049 Resilience Tier` `Last Tested` and conformance to `STD-11 Logging and Monitoring Standard`.

Signed by. Chief Information Officer, on the record of the domain architect who assembled the evidence.

Done when. Every row has an integer and every sub-score of 0 or 2 names its evidence row. Middle scores can be judgement; the extremes cannot.

Shows up in. `RPT20 Portfolio Assessment`, column `Technical fitness`. Meridian's five `Logical` target applications all score 10, which is correct and worth saying in the catalogue: an application that does not exist has no technical debt, and that is not an achievement.

Goes wrong when. Fitness tracks age. Age is one input to one sub-score. `APP-11 FinERP` is four years old and scores 5; `APP-17 ScreenGate` is three years old and scores 9 [2]. The difference is integration style and operability, which is what the score exists to expose.

Figure 6.3 The quadrant as the repository computes it, with Meridian Assurance Group's twenty-six applications counted into each cell. Business Value of 6 or more is high; Technical Fitness of 5 or more is high. Meridian is an invented insurer.

Step 6.7 — Derive the quadrant and test the disposition

Produce. `Portfolio Quadrant` for every row, `Disposition Agrees`, an explanation for every `No`.

You need. Steps 6.3, 6.5 and 6.6, and a `Disposition` column set *before* the scores were revealed — otherwise this step tests nothing.

Where it lands.

023 Application Component

`Portfolio Quadrant` from the two scores on these thresholds: `Business Value` of 6 or more is high, `Technical Fitness` of 5 or more is high. High and high is `Invest`; high value, low fitness is `Migrate`; low value, high fitness is `Tolerate`; low and low is `Eliminate`. `Disposition Agrees` is `Yes` when the recorded `Disposition` sits in the quadrant's permitted set — `Invest` permits `Invest`, `Extend`, `Build`; `Migrate` permits `Replace`, `Consolidate`; `Tolerate` permits `Sustain`, `Build`; `Eliminate` permits `Retire` — and `No` otherwise.

How.

1. Compute the quadrant mechanically. Never adjust a score to move an application into a quadrant somebody prefers.
2. Compare each `Disposition` against the permitted set; write `Yes` or `No`.
3. For every `No`, hold a ten-minute conversation and write one sentence of explanation.
4. Change the score, change the disposition, or keep both and record why. Silence is the only wrong answer.
5. Carry the `No` rows into Phase E as a named input to `054 Architecture Alternative`.

Signed by. Architecture Board (`GB-01`), monthly, which may approve dispensations but cannot approve funding.

Done when. Every row has a quadrant and a `Disposition Agrees` verdict, and every `No` has an explanation in `02 Change Log`.

Shows up in. `RPT20 Portfolio Assessment`, columns `Quadrant`, `Disposition` and `Agrees`, footer counting `Disposition disagrees`. Meridian reads six of twenty-six, and the sheet's own closing note is the most useful sentence in the report: *disagreement is not error — the disposition may reflect a contract or a dependency the scoring cannot see, and none of the six has an explanation recorded against it* [2].

Goes wrong when. The disposition is set after the scores. Then `Disposition Agrees` reads `Yes` everywhere and the lie detector only ever agrees with itself. Meridian's six are the sheet working: `APP-04 MotorClaims` scores into `Invest` and carries `Consolidate`; `APP-11 FinERP` scores into `Invest` and carries `Sustain`; four score into `Tolerate` and carry `Extend` [2].

WHERE THE QUADRANT AND THE DISPOSITION DISAGREE

Were the dispositions locked before the scores were revealed? If not, what does `Disposition Agrees` actually test?

For each `No`, is the score wrong, the disposition wrong, or is there a contract or dependency the scoring cannot see?

Which application scores into `Eliminate` and has a defender? Who is it, and what are they protecting?

Working table — the disposition disagreements

One row per `Disposition Agrees` = `No`. The explanation is one sentence, written in the ten-minute conversation, and it goes into `02 Change Log` against the row. Carry what survives into `054 Architecture Alternative` in Phase E. Signed by the Architecture Board.

APP- ID	Quadrant	Disposition	Score, value / fitness	Why they disagree
APP-11	Invest	Sustain	8 / 5	ERP upgrade contracted to 2029

THE WORKSHOP THAT PRODUCES ALL OF THIS

One session, three hours, nine people: the Chief Enterprise Architect chairing, four domain architects, the CIO, the CFO's finance business partner, two business owners rotating by application group. Agenda: dispositions read out from `023` and locked; business value scored; break; technical fitness scored; quadrant computed live on the sheet; disagreements listed but not resolved in the room. Resolution is a two-week follow-up with named owners. Locking the dispositions before scoring is the one mechanic that makes `Disposition Agrees` worth having.

Services, capabilities and value streams

An application component is a thing you buy and run. An information system service is a thing other things call. Separate sheets are what let you replace the first without renegotiating the second.

Step 6.8 — Name the information system services

Produce. `024 Information System Service` populated, each service realising a business service from Phase B.

You need. `015 Business Service` from Phase B. Volume 1 chapter 11 for granularity: the conceptual model's "what it does" column is close to the right grain.

Where it lands.

`O24 Information System Service`

`Name` ends in `Service`: `Quotation Service`, `Policy Issue Service`, `Identity Resolution Service`. `Description` is one imperative sentence — Meridian's `ISS-01` reads "Rate a risk and return a binding premium". `Realises Business Service` is a `BSV-` id. `Provided By` is one `APP-` id, not several. `Interface Style` is what it is today, including the ugly truth: Meridian records `Green screen` for `ISS-10 Claim Assessment Service` and `REST (target)` for three services that do not exist yet. `State` is `Baseline` or `Target`.

How.

1. Walk `O15 Business Service`. For each, ask what automated element delivers it.
2. Name one service per business service where you can, and say so where you cannot.
3. Set `Provided By` to the single accountable application. Two providers means two services, or a duplication finding — not one service with two providers.
4. Set `Availability Target` from the business service's criticality, not the current platform's capability. Meridian: nine at 99.5%, seven at 99.9%, three at 99.0%, one at 99.95%.
5. Mark `State = Target` for services that do not exist. Six of Meridian's twenty are `Target` [1].

Signed by. Chief Enterprise Architect, with the business service owner confirming `Availability Target`.

Done when. Every `O15 Business Service` row has at least one `O24` row realising it, or is recorded in `O29 Gap` as unserviced.

Shows up in. `RPT15 Catalogue Completeness`, and `RPT07 Capability Coverage` indirectly.

Goes wrong when. Services are named after applications. `LIFE/400 Service` is not a service; it is an application with a word after it.

Step 6.9 — Attach services and business services to applications

Produce. `R22 APP-ISS` and `R20 APP-BSV` populated, coverage honestly graded.

You need. Steps 6.2 and 6.8, and `O15 Business Service`.

Where it lands.

`R22 APP-ISS`

`Source (APP)`, `Target (ISS)`, and the `Key` column as `APP-nn|ISS-nn`. One row per provision. Meridian holds twenty.

`R20 APP-BSV`

`Source (APP)`, `Target (BSV)`, `Coverage` as `Full` or `Partial`, `State` as `Baseline` or `Target`, and `Note` — which is where `Partial` earns its keep. Meridian's `APP-02` against `BSV-01 Produce Quotation` carries `Partial` with the note "Personal lines only; other channels have their own path".

How.

1. Fill `R22` from `O24`'s `Provided By`. If the two disagree, `O24` wins and the relationship sheet is corrected.
2. Fill `R20` from the Phase B capability and process work: which applications take part in delivering each business service.
3. Grade `Coverage`. `Full` means end to end, for the whole book. Anything else is `Partial` with a note saying what is missing.
4. Set `State` so baseline and target read separately. Meridian holds thirty-three `Baseline` and two `Target` rows in `R20`.
5. Count `Partial` against `Full`. Meridian reads sixteen to nineteen — the expected shape for a composite insurer with four customer masters, not a data-quality problem.

Signed by. Chief Enterprise Architect.

Done when. Every `O24` row appears in `R22` and every `Partial` row in `R20` has a note.

Shows up in. `RPT07 Capability Coverage`: `Implementing Apps`, `Full`, `Partial`, `Coverage` and `Duplication`.

Goes wrong when. Everything is `Full`. Four applications delivering quotation, each for its own channel, is four `Partial` rows and a duplication finding — not four `Full` rows and a healthy portfolio.

Step 6.10 — Map applications to capabilities and value streams

Produce. `R37 APP-CAP` populated at the lowest capability level you hold, and `R41 APP-VST` derived from it. The view is an ArchiMate 4 capability realisation view: `Application Component` realising `Capability`, serving `Value Stream`.

You need. Step 6.2. `O13 Business Capability` from Phase B — Meridian holds 556 capability rows across three levels — and `O14 Value Stream`, of which Meridian holds seven.

Look it up. Volume 1, chapter 19, *Capability to application — the coverage map*, grades each capability full, partial or gap. Take the grading, not the IDs — match by name.

Where it lands.

> **`R37 APP-CAP`**
>
> `Source (APP)`, `Target (CAP)` as a levelled id like `C03.04.02`, `Support` — Meridian uses the single value `Supports` across all 181 rows — and `Note` where the support is conditional.

> **`R41 APP-VST`**
>
> `Source (APP)`, `Target (VST)`, `Basis` and `Note`. Meridian's `Basis` carries `Reach` for fifty-two rows and `Practice` for twenty-five, and every `Note` says "Composed via capability roll-up": the sheet is derived from `R37 APP-CAP` through `R11 VST-CAP`, and says so.

How.

1. Map capabilities at level 3. At level 1 every application supports everything and nothing can be read.
2. Work application by application, not capability by capability. An application's owner knows what it supports; a capability's owner rarely knows what supports it. Expect six to ten rows each — Meridian averages seven.
3. Use `Note` for conditional support: personal lines only, one line of business, read-only.
4. Derive `R41` from `R37` through `R11 VST-CAP`, and record the derivation in `Note`. A derived relationship that does not say so will be mistaken for an observation.
5. Use `Basis` to separate a demonstrated role in the stream from mere reachability through the capability graph, then delete what is reachable but not material.

Signed by. Chief Enterprise Architect.

Done when. Every `Physical` application has an `R37` row, every `Tier 1` level-3 capability has an application or a `O29 Gap` row, and every value stream has at least one application.

Shows up in. `RPT07 Capability Coverage`, `RPT03 Capability Heatmap`, and Phase E sequencing through `O52 Implementation Factor`: an application touching five value streams cannot be taken offline in a weekend.

Goes wrong when. An architect fills `R37` from the capability side alone — even coverage, no duplication, no gaps, a map drawn from the model rather than the estate. Or `R41` is drawn by hand: seventy-seven hand-drawn rows contradict `R37` within a quarter and nobody knows which is right.

Step 6.11 — Read coverage and duplication

Produce. A one-page reading of `RPT07 Capability Coverage`, with duplication and partial coverage raised as `O29 Gap` rows.

You need. Steps 6.9 and 6.10.

Where it lands.

> **`O29 Gap`**
>
> `Domain` = `Application`. `Gap Type` from `New`, `Replace`, `Eliminate`. `Severity` `Critical`, `High` or `Medium`. `Affected Object` is the `APP-`, `ISS-` or `BSV-` id. `Traces To` is the `REQ-` id, `Addressed By` the `WP-` id once Phase E exists.

How.

1. Read `Duplication`. `DUPLICATED` means three or more applications implement one business service. Meridian has four: `BSV-01`, `BSV-02`, `BSV-04` and `BSV-14 Maintain Customer Record` [6].
2. Read `Coverage`. `Partial only` means no single application delivers the service end to end. Meridian has five.
3. Read `Services with no implementing application`. Meridian's is zero, which is a finding about the *mapping*, not a clean bill of health.
4. Raise one gap per duplication cluster, not one per application in it.
5. Name it in business terms. "Each channel has its own quotation path" is `GAP-07`; "four applications touch quotation" is an observation nobody will fund.

Signed by. Architecture Board (`GB-01`).

Done when. Every `DUPLICATED` and `Partial only` row has either a gap or a written reason why the duplication is deliberate.

Shows up in. `RPT09 Gaps and Roadmap`, which is the gap report — not `RPT07`, which is coverage.

Goes wrong when. Duplication is treated as automatically wrong. Two claims systems for two very different books can be correct. Meridian's `GAP-10 Two claims systems` is `Medium`, not `Critical`, for that reason.

DUPLICATION, AND WHICH OF IT IS DELIBERATE

Which duplication is deliberate — two books, two regimes, two reasons — and who will confirm that in writing?

For each `DUPLICATED` cluster, what is the one sentence in business terms that somebody would fund?

`Services with no implementing application` reads zero. Is that a clean estate or an incomplete mapping?

Data authority, interfaces, requirements

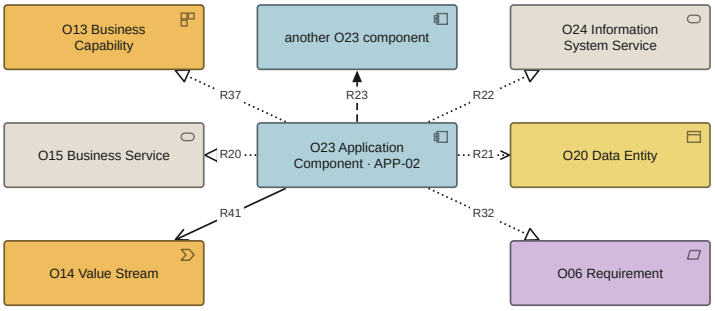

Figure 6.4 Everything that hangs off one `O23 Application Component`, drawn as the ArchiMate 4 relationships the relationship sheets actually mean: realisation to services, capabilities and requirements, access to data, flow to another component, serving to a value stream. Fill these before you expect anything from the reports.

Step 6.12 — Record which application masters which data

Produce. `R21 APP-DEN` populated with `Role` and `Authoritative`, `RPT05 App-Data Matrix` read for contested authority.

You need. `O20 Data Entity` from chapter 5, and volume 1 appendix A, whose application-to-data map marks the system of record per entity — start there and argue with it.

Look it up. Volume 1, chapter 19, *Application to data, with the system of record*, bolds the record-master per entity. Match by name.

Where it lands.

> **`R21 APP-DEN`**
>
> `Source (APP)`, `Target (DEN)`, `Role` as `Master` or `Copy`, `Authoritative` as `Yes`, `No`, `Contested` or `Target authority`, and `Note`. Meridian's forty-nine rows split forty-four `Master` to five `Copy`; `Authoritative` reads twenty-six `Yes`, thirteen `Contested`, five `Target authority`, five `No` [5].

How.

1. For every `O20` entity, ask which applications hold a writeable copy. Each is a row with `Role` = `Master`.
2. Mark read-only replicas `Copy`.
3. Set `Authoritative` = `Yes` only where one application masters the entity and nobody disputes it. Where two or more claim it, all of them are `Contested`.
4. Where the target designates an authority that does not exist yet, use `Target authority`. Meridian uses it five times, all for `APP-21 CustomerHub`.
5. Read `RPT05 App-Data Matrix`: `M` and `C` per application, then four derived columns, `Masters (all)`, `Baseline masters`, `Recorded authority` and `Status`.

Signed by. Chief Data Officer, through the Data Governance Council (`GB-03`) — which at Meridian has never reached quorum, a fact that belongs in the catalogue.

Done when. Every `O20` entity appears in `RPT05` with a `Status`, and no entity has two `Authoritative` = `Yes` rows.

Shows up in. `RPT05 App-Data Matrix`. Meridian's footers: ten entities with more than one baseline master, one with none at all, five where the target introduces a designated authority [5]. `DEN-01 Party` reads five masters, four baseline, `Recorded authority` = `CONTESTED`. `DEN-23 Model` reads one master, zero baseline, `NO BASELINE MASTER`. Those two rows are the argument for `APP-21` and `APP-24`, one line each.

Goes wrong when. Somebody resolves the contest in the sheet. Writing `APP-21` into `Authoritative` for `DEN-01` because that is the plan erases the finding — four systems master Party today — which is the thing the Data Governance Council has to decide.

DATA AUTHORITY, BEFORE ANYBODY RESOLVES IT IN THE SHEET

Which entity has four masters today, and which of them does the business act on when they disagree?

Where you were tempted to write the target application into `Authoritative`, what finding would that erase?

`GB-03` has to decide these. If it cannot sit, who decides, and how is that recorded?

Step 6.13 — Build the interface register and count the unmanaged

Produce. `R23 APP-APP` complete, and the unmanaged count — the headline finding of Phase C application work in most insurers. Draw it as an ArchiMate 4 application cooperation view: a `Flow` relationship per interface, with the unmanaged ones marked.

You need. Step 6.2. The scheduler's job list, the broker's queue list, the file transfer schedule, and the database link and service-account inventory from step 6.4.

Look it up. Volume 1, chapter 12, *The overnight batch, and why it still exists*: count the nightly file transfers too.

Where it lands.

> **`R23 APP-APP`**
> `Source (APP)`, `Target (APP)`, `Interface ID` as `IF-nnn`, `Style`, `Direction`, `Frequency`, `Managed`, `Data Entity` as the `DEN-` id it carries, and `Note`. `Managed` means the interface traverses a managed integration layer, is published in a catalogue, is versioned and is monitored — all four, or it is `No`.

How.

1. Collect from four sources — scheduler, broker, file transfer, database links — because each one misses a different class.
2. One row per logical interface, not per file. Twenty files on one nightly schedule between two systems is one interface at `Frequency` = `Nightly`.
3. Set `Data Entity` to the primary entity carried. That is what makes the register readable next to `RPT05`.
4. Apply the four-part `Managed` test strictly. A REST call with no gateway, no catalogue entry and no monitoring is `No`.
5. Count. Meridian: fifty interfaces, forty-four unmanaged, eighty-eight per cent [4]. All twenty batch files, all seven `MQ`, all four `Direct DB read`, the one `SOAP` and twelve of sixteen `REST` are unmanaged. Only the two `CDC` rows are managed.
6. Halve the `Unmanaged` total in `RPT02 Application Portfolio` before quoting it: `RPT02` counts each interface at both endpoints, so its 88 is 44 seen twice [3].

Signed by. Chief Information Officer, with the integration lead as named evidence owner.

Done when. Every interface has an `IF-` id, a `Managed` verdict and a `Data Entity`, and the `Direct DB read` rows are listed separately for the Architecture Board.

Shows up in. `RPT06 Interface Register`, including the `Summary by style` block with `Count`, `Unmanaged` and `% unmanaged` per style and a total row.

Goes wrong when. The count flatters. Why governance gate `G1 Design reviewed` exists at Meridian is recorded in `O58 Governance Gate` in one sentence: "This is the gate that was missing. Forty-four unmanaged interfaces reached production without one" [9]. A soft `Managed` test hides the forty-four and the gate never gets created.

THE UNMANAGED INTERFACES

The register itself is the Phase C interface form in appendix F — one row per logical interface, with style, direction, frequency and the `Managed` verdict. These questions decide what you write in that last column.

Applying the four-part test strictly, what is your unmanaged percentage — and what was it before you applied it strictly?

Which direct database read would make a replacement cost twice its estimate, and which system has no contract to hold a replacement to?

Which unmanaged interface carries the entity you are about to re-master, and who owns both ends?

FOUR DIRECT DATABASE READS

Meridian's register holds four `Direct DB read` interfaces, one of them `IF-004`, where `APP-03 ClaimsOne` reads `APP-01 LIFE/400` policy tables directly [4]. Name them individually in the catalogue. They are what makes a replacement programme cost twice its estimate, because the consuming system has no contract to hold the replacement to. `RPT30 Compliance Position` carries two acceptance criteria about them, both `Non-compliant`.

Step 6.14 — Trace requirements to applications

Produce. `R32 REQ-APP` populated, `State` distinguishing what the baseline already does from what the target must do.

You need. Steps 6.2 and 6.7, and `O06 Requirement` from the Requirements Management work that runs through every phase.

Where it lands.

> **`R32 REQ-APP`**
>
> `Source (REQ)`, `Target (APP)`, `State` as `Baseline`, `Target` or `Baseline & Target`, and `Note`. Meridian's twenty-one rows read thirteen `Target`, five `Baseline & Target`, three `Baseline` [1].

How.

1. For each requirement, name the applications that realise it. Several will point at the same target: four of Meridian's point at `APP-21 CustomerHub`.
2. Set `State` = `Baseline` where the requirement is met by remediating something that exists — `REQ-14 Attributable access to customer data` against `APP-01`, noted "Remediation on the existing platform".
3. Use `Baseline & Target` where both are needed, and name the dependency in `Note`.
4. Leave untraceable requirements untraced rather than inventing a realiser. The untraced count is a finding.

Signed by. Chief Enterprise Architect, with the requirement's owner from `O06`.

Done when. Every requirement is traced, or listed as untraced with a reason.

Shows up in. `RPT08 Traceability`, column `Realising Apps`, and `RPT32 Requirement Traceability`, columns `Traced` and `Ever tested`.

Goes wrong when. Every requirement traces to the target application that was going to be built anyway. The trace then proves the programme is self-consistent and nothing else.

Step 6.15 — Assess reference model conformance

Produce. `R43 O47-APP` populated and `RPT25 Reference Model Conforman` read, with the no-standard purposes raised.

You need. Step 6.1's `O47` rows, and volume 1 chapter 13 (the reference architecture), which covers what ACORD publishes, what it costs, and the standard mistake of treating its capability model as a target architecture [11]. Do not re-derive that chapter.

Look it up. Volume 1, chapter 13, *The Reference Architecture: seven facets*, for what ACORD publishes per `O47` purpose.

Where it lands.

> **`R43 O47-APP`**
>
> `Source (O47)`, `Target (APP)`, `Status` as `Current`, `Contained` or `Retiring`, and `Domain` = `Application`. Meridian holds seventeen rows: thirteen `Current`, three `Contained`, one `Retiring` [7].

How.

1. For each `O47` application entry, link the applications that currently answer it.
2. Set `Status` per link: `Current` is approved for new work, `Contained` is no new workload, `Retiring` is going.
3. Where two applications answer one purpose and both are `Contained` — Meridian's `REF-A-07 Handle a claim`, answered by `ClaimsOne` and `MotorClaims` — say so in `When Not To`: "Both carry Consolidate and neither disposition names the survivor".
4. Count the purposes with no link. Meridian has five `No standard`, including "Hold a party or customer record" and "Integrate two systems" [7].
5. Where ACORD is your reference, keep the mapping in `O47`'s `Purpose` wording and cite the ACORD capability in `The Need`. Do not reproduce ACORD content in the repository.

Signed by. Chief Enterprise Architect.

Done when. Every `O47` application row has a `R43` link or an explicit `No standard`, and every `No standard` has a `When Not To` sentence telling a project architect what to do meanwhile.

Shows up in. `RPT25 Reference Model Conforman`: thirty-four purposes, five with no approved answer, three past vendor support [7].

Goes wrong when. The reference model is filled in as aspiration. `Standard Today` is a fact about this month. If it reads like a target, you have two target architectures and no baseline.

Step 6.16 — Read portfolio data risk and software distribution

Produce. Two readings: what happens to the data when each disposition is executed, and where each application runs.

You need. Steps 6.3, 6.7 and 6.12. `O27 Physical Technology Compone` and `R26 PTC-LOC` from Phase D, which is why this step often runs twice.

Where it lands. Nothing new — both reports read sheets you have filled, and the findings land in `O29 Gap`.

How.

1. Read `RPT22 Portfolio Data Risk` entity by entity: `Mastered by`, `At risk from`, `Surviving master`, `Position`. Meridian: twenty-four entities, four with `no surviving master`, nine with `authority unrecorded` [8].
2. Act on the four first. `DEN-04 Life Assured`, `DEN-05 Beneficiary`, `DEN-10 Claim` and `DEN-14 Intermediary` are each mastered only by an application carrying `Replace` or `Consolidate`, and the report's closing note says no work package names a destination for any of them.
3. Read `RPT21 Software Distribution` for nodes, annual cost and `Out of support in 2027`. Meridian: eighteen nodes, R146.8m, three out of support in 2027 [8].
4. Reconcile the two cost figures in one sentence. R320.5m of application run cost over R146.8m of node cost is not a contradiction — applications carry licence, support and people, nodes carry infrastructure — but say so, or someone will add them.
5. Raise the untraceable applications. Meridian's five, three `SaaS`, are a register problem to fix, not a mystery to investigate.

Signed by. Chief Enterprise Architect, with the Chief Data Officer on the data-risk reading.

Done when. Every `no surviving master` entity has a named destination or a `O29 Gap` row, and every application traces to a location or is recorded as `SaaS` with none.

Shows up in. `RPT22 Portfolio Data Risk` and `RPT21 Software Distribution`.

Goes wrong when. Disposition and data authority are approved in different meetings. Deciding to replace `APP-01` is also deciding to move four masters, and `RPT22` exists to make that unavoidable.

Step 6.17 — Publish the Application Portfolio Catalog

Produce. `DLV-17 Application Portfolio Catalog`, the Phase C application deliverable.

You need. Steps 6.1 to 6.16, and `O31 Deliverable` row `DLV-17` — at Meridian `Artifact` type, phase `C`, work package `WP-02`, owner Chief Information Officer, due 2027-03, status `In progress`, location `Repository/PhaseC` [9].

Where it lands.

> **`O31 Deliverable`**
> Update `DLV-17`: `Status` to `Complete`, `Repository Location` to the published path. Never move the `Due` date to match the day you finished.

How.

1. Assemble six sections: inventory from `O23`, assessment from `RPT20`, coverage and duplication from `RPT07`, data authority from `RPT05`, interfaces from `RPT06`, reference conformance from `RPT25`.

2. Write a two-page front section with five numbers: applications, total run cost, unmanaged interface percentage, disposition disagreements, contested entities. Meridian: 26, R320.5m, 88%, 6, 10.
3. Attach the scoring packs from steps 6.5 and 6.6. The repository holds totals; the packs hold the working.
4. List what is not in it: the applications with no location, the rejected discovery candidates, the parked scores.
5. Take it to Architecture Board (`GB-01`) for approval and to Investment Committee (`GB-02`) as information — `GB-02` cannot judge architecture compliance and `GB-01` cannot approve funding.

Signed by. Chief Information Officer as owner, approved by Architecture Board (`GB-01`).

Done when. `DLV-17` reads `Complete` in `O31` , `RPT14 Deliverable Register` reflects it, and every number on the front page traces to a sheet in one step.

Shows up in. `RPT14 Deliverable Register` , `RPT15 Catalogue Completeness` .

Goes wrong when. The catalogue is a slide deck. A catalogue is a register with a reading. If the numbers are not live from the sheets, the first question in the room is whether they are current, and you cannot answer it.

THE HANDOVER TO PHASE D

Two of the `Technical Fitness` sub-scores — support currency and operability — are inputs from chapter 7, not outputs of chapter 6. They depend on `O27 Physical Technology Compone` , `O49 Resilience Tier` and `R31 STD-PTC` . Run step 6.6 with steps 7.4, 7.7 and 7.12 as one piece of work with two owners, or accept that one of them scores against evidence that does not exist.

PHASE C, THE APPLICATION HALF — A REVIEWER'S QUESTIONS

What are your five front-page numbers, and can each be traced to a sheet in one step?

Which applications did the business declare that IT does not operate, and who owns them now?

Which disposition commits you to moving a data master, and does any work package name the destination?

What is deliberately not in the catalogue, and is that written down inside it?

THE CHAPTER IN TEN LINES

- Walk the conceptual model from volume 1 chapter 11; map real systems onto it, never the reverse.
- O23 is eighteen content columns in four groups, filled at four moments by four sets of people.
- Keep Lifecycle factual and Disposition intentional; the pair is only useful if they can disagree.
- Reconcile Annual Run Cost (Rm) to the ledger before you trust anything derived from it.
- Three discovery rules — the spreadsheet, the invoice, the Active Directory. Run all three.
- Business Value out of 12, Technical Fitness out of 10, scored by different people, sub-score by sub-score.
- Quadrant thresholds: value 6 or more is high, fitness 5 or more is high.
- Lock the disposition before revealing the scores, or Disposition Agrees tests nothing.
- R21 APP-DEN read through RPT05 catches two systems both claiming to master one entity.
- Managed means gateway, catalogue, version and monitoring, all four; and halve RPT02's Unmanaged total before quoting it.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets O23 Application Component, O24 Information System Service and R32 REQ-APP
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT20 Portfolio Assessment
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT02 Application Portfolio
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets R23 APP-APP and RPT06 Interface Register
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheets R21 APP-DEN and RPT05 App-Data Matrix
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheets R20 APP-BSV, R37 APP-CAP, R41 APP-VST and RPT07 Capability Coverage
- **7.** Meridian_TOGAF_Metamodel.xlsx, sheets O47 Reference Model Entry, R43 O47-APP and RPT25 Reference Model Conforman
- **8.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT21 Software Distribution and RPT22 Portfolio Data Risk
- **9.** Meridian_TOGAF_Metamodel.xlsx, sheets O31 Deliverable, O57 Governance Body and O58 Governance Gate
- **10.** The Open Group, the TOGAF Standard — Architecture Development Method overview https://www.opengroup.org/togaf
- **11.** ACORD, "Reference Architecture" (as covered in volume 1, chapter 13) https://www.acord.org/standards-architecture/reference-architecture

CHAPTER 7

Technology Architecture

Four sheets, four questions: what the estate offers, what kind of thing offers it, which product on which contract, and where it sits.

Phase D is the phase architects either skip or drown in. Skipped, it becomes a server list. Drowned in, it becomes a configuration management database with worse tooling. The repository keeps you out of both by forcing a separation the estate itself does not have: service, logical component, physical component, place. Four sheets, four questions. What does the estate offer. What kind of thing offers it. Which actual product, on which actual contract, with which support end date. And where does it sit.

Fourteen steps. **Meridian Assurance Group**, the insurer in your repository, is an invention, as is volume 1's **Cape Mutual** — keep their data apart.

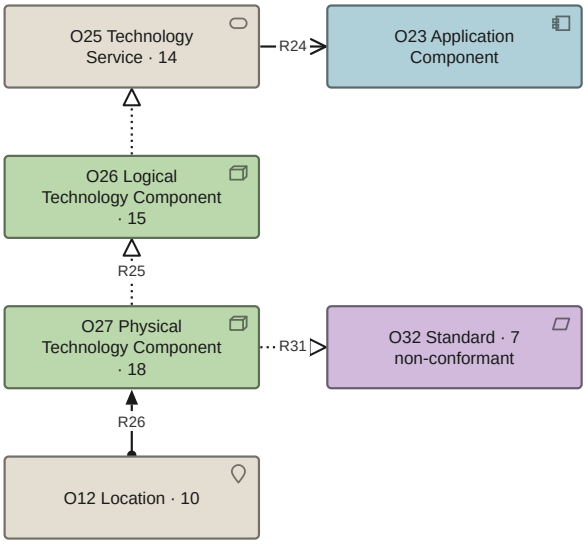

Figure 7.1 The Phase D layers in ArchiMate 4. A physical node realises a logical node, which realises a technology service, which serves an application. The location is assigned to the physical node, and the standard is realised by it — or the node is one of the seven non-conformant.

BEFORE YOU START THIS PHASE

Volume 1 says less about this layer than about business and data, deliberately.

- **Volume 1, chapter 12, *The mainframe reality in life insurance*.** The oldest node carries the largest book.
- **Volume 1, chapter 13, *AL3, and why insurance integration looks the way it does*.** Where the standards came from.
- **Volume 1, chapter 14, *What rots first*.** Which vendor facts to distrust, and how fast.
- **Volume 1, chapter 15, *What the regulator actually inspects*.** Six questions, answered here or nowhere.

Services before things

`O25 Technology Service` is the sheet a business reader can follow. Fourteen rows at Meridian, each a technical capability the estate offers rather than a product that offers it: `Transaction Processing`, `Message Queuing`, `API Management`, `Identity and Access Management`, `Backup and Recovery` [1]. `Category` groups them — Meridian runs four `Integration`, three `Compute`, three `Data`, two `Security`, one `Resilience` and one `Network`.

The value of this sheet is that it survives a product change. When the message broker is replaced, `TSV-04 Message Queuing` does not move; `O27 Physical Technology Compone` does. Every application dependency, every availability commitment and every resilience target points at the service, not the product, which is the whole reason the layer exists.

Step 7.1 — Name the technology services

Produce. `O25 Technology Service` populated, with an availability target on every row.

You need. `O23 Application Component` from chapter 6, because the consumer list is drawn from it. The operations team's service catalogue if one exists. Volume 1 chapter 14 (technology providers) for what the market calls each service.

Where it lands.

> **`O25 Technology Service`**
>
> `Name` is the capability, not the product: `Relational Database`, not the database engine. `Description` is one sentence. `Category` from `Compute`, `Data`, `Integration`, `Security`, `Resilience`, `Network`. `Availability Target` is the commitment the service makes, as a percentage. `Consumers` is a short phrase, not an exhaustive list — Meridian uses `All applications`, `Most applications`, `Target estate` and, where it is genuinely narrow, `APP-01, APP-03`. `State` is `Baseline` or `Target`. `Owner Org` is the `ORG-` id that runs it.

How.

1. Start from what applications need, not from what infrastructure owns. Twelve to twenty rows is the right size for an insurer; Meridian has fourteen.
2. Set `Availability Target` from the most demanding consumer. Meridian runs eight services at 99.9%, four at 99.95% and two at 99.5%.
3. Keep `Consumers` coarse. An exhaustive list here duplicates `R24 APP-TSV` and goes stale.
4. Mark the services that do not exist yet as `State` = `Target`. Meridian has four — `API Management`, `Analytics Compute`, `Event Streaming` and `Container Orchestration` — and recording them as services rather than projects is what lets requirements point at them.

Signed by. Chief Information Officer, with `Availability Target` acknowledged by the owner of each consuming business service.

Done when. Every service has a category, an availability target and an owning organisation, and no row names a vendor product.

Shows up in. `RPT12 Integrity Check`, which will show `O25` under-linked until step 7.2 is done, and `RPT15 Catalogue Completeness`.

Goes wrong when. The sheet is the product list with the products renamed. `WebSphere Hosting` is not a technology service; `Application Hosting` is, and WebSphere is one physical realisation of it.

Step 7.2 — Record what each application depends on

Produce. `R24 APP-TSV` populated, with the dependency graded.

You need. Steps 6.2 and 7.1.

Where it lands.

> **`R24 APP-TSV`**
>
> `Source (APP)`, `Target (TSV)`, `Dependency` and `Note`. `Dependency` says what happens to the application if the service is unavailable: `Critical` means the application stops, `High` means it degrades. Meridian holds forty-two rows, thirty-seven `Critical` and five `High` [1].

How.

1. Walk each application and name the services it cannot run without. Expect one or two rows per application; Meridian averages 1.6.
2. Grade honestly. A ratio of thirty-seven `Critical` to five `High` looks lopsided until you notice it is correct: an application without its runtime or its database does not degrade, it stops.
3. Where an application traces to no service, say so. That application also has no `Support Ends` evidence for the fitness score in step 6.6.
4. Cross-check against `RPT21 Software Distribution`. Applications with no traceable node are usually the same ones with no traceable service.

Signed by. Chief Information Officer.

Done when. Every `Physical` application has at least one row, and every `Tier 1` application has a `Critical` row.

Shows up in. The fitness scoring in step 6.6, which joins through here to `O27`, and Phase E sequencing: a service with eleven `Critical` consumers cannot be migrated in one wave.

Goes wrong when. Everything is graded `Critical` because grading takes judgement. Then the column carries no information and the resilience work in step 7.12 has nothing to prioritise against.

GRADING WHAT THE ESTATE DEPENDS ON

Which `Tier 1` application has no `Critical` dependency recorded, and is that honest or unfinished?

For one Tier 1 application, which single service failing takes it down — and who would tell you?

Which dependency exists only in somebody's head, and what evidence would put it in a sheet?

Logical and physical, kept apart

Two sheets, and the names are nearly identical because Excel truncates them. Spell them as they appear: `O26 Logical Technology Componen` and `O27 Physical Technology Compone`. The missing letters are not a typo in your writing — they are what the tab says.

A logical component is a kind of thing: `Application Server`, `Message Broker`, `Identity Provider`, `Lakehouse Platform`. A physical component is a specific thing with a version, a location, a support end date and an annual cost. Fifteen logical rows at Meridian, eighteen physical [1]. The join is `R25 PTC-LTC`, nineteen rows — more than eighteen, because one physical node can realise more than one logical component.

Both sheets carry `Standard Status`, and that is where the governance lives. Meridian's logical sheet reads nine `Current`, four `Emerging`, two `Contained`; the physical sheet reads nine `Current`, five `Contained`, four `Emerging` [1]. `Contained` is the most useful word in Phase D: it means the thing works, it is staying for now, and nothing new may be built on it.

> ONE TRAP IN O27
>
> `O27 Physical Technology Compone` has two columns headed `Version`. The first is the product version (`19c`, `9.0`, `2.5`, `rolling`); the second is the record version in the version block every object sheet carries. When you extract from this sheet, take the column by position, not by name.

Step 7.3 — Write the logical component catalogue

Produce. `O26 Logical Technology Componen` populated, each row providing a service.

You need. Step 7.1.

Where it lands.

> **`O26 Logical Technology Componen`**
>
> `Name` is the class of technology. `Category` matches `O25`'s vocabulary. `Provides Service` is the `TSV-` id — one service per logical component keeps the sheet readable, and where two logical components provide one service, that is a real finding about diversity. `Standard Status` from `Current`, `Emerging`, `Contained`. `Owner Org` is the `ORG-` id.

How.

1. One row per class of technology you deliberately run. Not per product.
2. Set `Provides Service` to the `O25` row it realises. Meridian's `LTC-02 Application Server` and `LTC-15 Hypervisor` both point at `TSV-02 Application Hosting`, which is an honest record of two ways of doing one thing.
3. Set `Standard Status`. `Emerging` means approved and not yet widespread; `Contained` means present and closed to new work.
4. Expect roughly one logical component per service, plus the duplicates you are containing.

Signed by. Chief Enterprise Architect, with the domain architect for each category.

Done when. Every `O25` service has at least one logical component, and every logical component has a `Standard Status`.

Shows up in. `RPT10 Standards Conformance` indirectly, through `R25` and `R31`.

Goes wrong when. The sheet becomes a second product catalogue. If a row has a version number in it, it belongs in `O27`.

Step 7.4 — Record the physical estate and join it to the logical

Produce. `O27 Physical Technology Compone` populated and `R25 PTC-LTC` joined, with support end dates from the contracts rather than from memory. The ArchiMate 4 view is a technology view: `Node`, `Device` or `System Software` for the physical row, realising the logical `Node` above it.

You need. Step 7.3. The support and licence contracts — Meridian holds four in `O19 Contract`, including `CTR-04 Short-term Platform Licence` expiring 2027-06-30 with twelve months' notice and `CTR-06 Mainframe Support Agreement` expiring 2027-12-31. The infrastructure cost breakdown from finance. `O12 Location` from step 7.9, or run the two steps together.

Look it up. Volume 1, chapter 14, *What rots first* — ownership rots fastest, and the support roadmap travels with it.

Where it lands.

> **`O27 Physical Technology Compone`**
>
> `Name` is the node as operations names it: `Mainframe LPAR PROD1`, `AKS Production Cluster`. `Realises` is the `LTC-` id. `Vendor / Product` is the actual product, and this is the one sheet where naming it is correct. The first `Version` column is the product version. `Location` is the `LOC-` id. `Lifecycle` from `Current`, `Emerging`, `Declining`, `Legacy`. `Support Ends` is a `YYYY-MM` date, or `rolling` for a subscription with no end date, or blank for something not yet selected. `Standard Status` as in `O26`. `Annual Cost (Rm)` is the infrastructure cost of the node.

> **`R25 PTC-LTC`**
>
> `Source (PTC)`, `Target (LTC)`, `Standard Status` and `Note`. The `Standard Status` here must agree with `O27`'s; if it does not, `O27` wins.

How.

1. One row per node or cluster, not per server. Eighteen rows is the right order for a composite insurer; three hundred means you have built an asset register.
2. Take `Support Ends` from the contract, and write `rolling` only where the contract genuinely has no end — Meridian uses it for the four cloud-native nodes.
3. Leave `Support Ends` blank where no product is selected. Meridian's `PTC-16 API Gateway Platform` and `PTC-17 Event Streaming Cluster` both read `To be selected`, blank support, `Annual Cost (Rm)` zero.
4. Fill `Annual Cost (Rm)` from the infrastructure ledger. Meridian's eighteen nodes total **R146.8m**, against R320.5m of application run cost — different things, counted separately, and never added [1][2].
5. Join `R25`. Meridian's nineteen rows against eighteen nodes is the clue that one node realises two logical components.

Signed by. Chief Information Officer, with the Chief Financial Officer on `Annual Cost (Rm)`.

Done when. Every node has a `Realises`, a `Location`, a `Lifecycle` and either a `Support Ends` date, `rolling`, or a blank with `To be selected` in `Vendor / Product`.

Shows up in. `RPT21 Software Distribution`, which groups nodes by location and counts `Out of support in 2027`. Meridian reads eighteen nodes, R146.8m, three [2].

Goes wrong when. `Support Ends` is filled from the vendor's public lifecycle page rather than from your contract. Extended support you are paying for and extended support you could buy are different facts, and only one of them is yours.

The standards catalogue, and the finding it produces

`O32 Standard` is sixteen rows at Meridian and spans more than technology: six `Technology`, four `Application`, two `Security`, two `Data`, one `Risk`, one `Governance` [3]. Ten of the sixteen carry `Status` = `Emerging` and six `Current`, which is the honest signature of a practice that has just written its standards down and not yet enforced them.

Conformance is a relationship, not an attribute. `R31 STD-PTC` holds nineteen assessments: ten `Conformant`, seven `Non-conformant`, two `Partial` — a conformance rate of 52.6% as `RPT10 Standards Conformance` computes it [3].

Step 7.5 — Write the standards catalogue

Produce. `O32 Standard` populated, each row with a named owner and a review date.

You need. `005 Principle` from the Preliminary Phase — a standard with no principle behind it is a preference. `O11 Role` for owners.

Look it up. Volume 1, chapter 13, *AL3, and why insurance integration looks the way it does*, and *The rest of the messaging families*.

Where it lands.

> **`O32 Standard`**
>
> `Name` states the subject: `Relational DBMS Standard`, `Cloud Placement Standard`, `Exit Assessment Standard`. `Description` is the rule in one sentence. `Domain` from `Technology`, `Application`, `Security`, `Data`, `Risk`, `Governance`. `Status` is `Current` or `Emerging` — `Emerging` means published and not yet enforced, which is a position, not a failure. `Owner` is a `ROL-` id, never an organisation. `Applies To` is the scope, and it is the column that decides whether the standard can be assessed: `All new interfaces`, `New workloads`, `Cloud workloads`, `All persistence`. `Review Due` is `YYYY-MM`, within eighteen months.

How.

1. Write the standard only where you can name what it applies to. `Applies To` = "everything" is an unassessable standard.
2. Set `Owner` to a role from `O11`, and check the role is filled. Meridian's `GB-05 Architecture Function` records a vacant Data Domain Architect post, which makes two standards ownerless in practice.
3. Use `Status` = `Emerging` freely. Ten of Meridian's sixteen are `Emerging`, and claiming otherwise would make the conformance assessment in step 7.6 dishonest.
4. Stagger `Review Due` so the Architecture Board does not review sixteen standards in one month. Meridian spreads them across four quarters of 2027.

Signed by. Architecture Board (`GB-01`), which publishes criteria and may grant dispensations.

Done when. Every standard has an owner who exists, an `Applies To` that names a testable scope, and a `Review Due` inside eighteen months.

Shows up in. `RPT10 Standards Conformance` and `RPT11 Principle Impact`.

Goes wrong when. The catalogue is written as aspiration and marked `Current`. Then step 7.6 reports breaches against standards nobody agreed to, and the Board rejects the report rather than the estate.

Step 7.6 — Assess conformance component by component

Produce. `R31 STD-PTC` populated and `RPT10 Standards Conformance` read.

You need. Steps 7.4 and 7.5.

Where it lands.

> **`R31 STD-PTC`**
>
> `Source (STD)`, `Target (PTC)`, `Conformance` as `Conformant`, `Partial` or `Non-conformant`, and `Note` — which is mandatory on anything other than `Conformant`. Meridian's notes are the model: "DB2 for z/OS outside the current standard", "Generic accounts persist for batch", "RTO not tested within 12 months".

How.

1. Assess only where the standard's `Applies To` covers the component. Do not assess a cloud placement standard against a mainframe.
2. Write a `Note` for every `Partial` and `Non-conformant`. A verdict without a note is a complaint.
3. Do not assess everything. Nineteen assessments across sixteen standards and eighteen nodes is deliberate coverage of what matters, not a matrix.
4. Read `RPT10`. Its columns put `Conformance` next to `Lifecycle`, `Support Ends` and `Standard Status`, which is the join that produces step 7.7.

Signed by. Architecture Board (`GB-01`), on evidence assembled by the domain architect.

Done when. Every `Tier 1` node is assessed against every standard whose `Applies To` covers it, and every non-conformance has a note.

Shows up in. `RPT10 Standards Conformance`, footer: `Assessments recorded`, `Non-conformant`, `Partially conformant`, `Conformance rate`. Meridian reads 19, 7, 2 and 52.6% [3].

Goes wrong when. The conformance rate is reported as a score. It is a coverage-weighted ratio over whichever assessments somebody chose to run. Report the count and the list, and let the ratio be a footnote.

CONFORMANCE, REPORTED HONESTLY

Which standards did you not assess, and what makes that deliberate rather than convenient?

Which non-conformance has a note a reader could act on, and which is still a complaint?

Which standard is marked `Current` that nothing in the estate conforms to — and is the standard wrong or the estate?

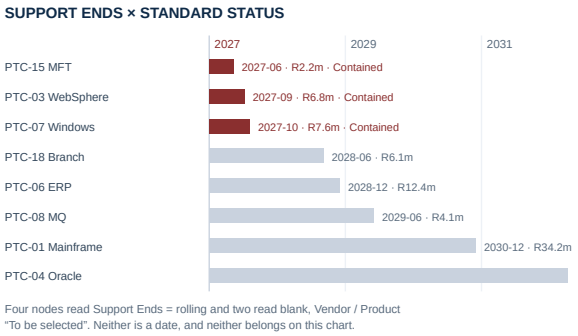

Figure 7.2 The technology-currency finding, drawn from four columns of O27 Physical Technology Compone. Three nodes lose vendor support in 2027, all three already Contained, together carrying R16.6m of annual cost.

Step 7.7 — Produce the technology-currency finding

Produce. A one-page paper, by node, of what falls out of vendor support and when, with the annual cost and the applications affected. This is the Phase D artefact that gets budget approved.

You need. Step 7.4 for `Support Ends` and `Standard Status`, step 7.6 for conformance, step 7.2 for the application dependency, and `O19 Contract` for the notice periods.

Look it up. Volume 1, chapter 14, *What a core replacement actually costs*, and *The one-platform trap*.

Where it lands.

> **`O29 Gap`**
>
> `Domain` = `Technology`, `Gap Type` = `Replace`, `Severity` from the tier of the applications affected. `Affected Object` is the `PTC-` id. Meridian holds this as `GAP-15 Contained technology past support`, `High`, affected object `PTC-03`, tracing to `REQ-13` and addressed by `WP-11` [4].

How.

1. Filter `O27` on `Support Ends` inside thirty-six months. Sort ascending.
2. Cross `Standard Status`. A node that is both out of support and `Contained` is a decision already taken and unfunded; a node out of support and `Current` is a standard you have not updated.
3. Pull the applications per node from `RPT21 Software Distribution` and the dependency grade from `R24 APP-TSV`.
4. Add the notice period from `O19 Contract`. A support contract with twelve months' notice and a 2027-06 expiry is a decision due this quarter, not next year.
5. Total the `Annual Cost (Rm)` of the affected nodes. That number, next to the applications, is the paper.

Signed by. Chief Information Officer presenting, Investment Committee (`GB-02`) deciding — it holds `G0` funding approval and the Architecture Board cannot approve funding.

Done when. Every node with `Support Ends` inside thirty-six months appears in the paper with its cost, its applications and the contract notice period, and each has either a funded work package or a recorded decision to accept the risk.

Shows up in. `RPT21 Software Distribution` (`Out of support in 2027`) and `RPT09 Gaps and Roadmap`.

Goes wrong when. The paper leads with the standard rather than the date. "Non-conformant with `STD-05`" is an architecture opinion. "`PTC-03 WebSphere Cluster`, support ends 2027-09, R6.8m a year, seven applications on it, `Contained`, and `REF-T-02 Host an application runtime` already reads `Unsupported`" is a fact with a deadline [2][3][5].

THE CURRENCY PAPER, BEFORE YOU PRESENT IT

Which date on this page is the earliest decision date once you count the contract notice period back from expiry?

Which node is out of support and still `Current` in the standards — which means the standard was not updated?

For each node, what is the cost of doing nothing for one more year, stated in rands and in applications?

Working table — the technology currency paper

One row per node with `Support Ends` inside thirty-six months, sorted by date. `Notice` is from the matching row in `O19 Contract`; the decision date is expiry minus notice. The last column is funded work package or recorded risk acceptance — not "monitor". Presented by the Chief Information Officer, decided by `GB-02`.

PTC- ID	Support ends	Cost (Rm)	Apps on it	Notice	Funded or accepted
PTC-03	2027-09	6.8	7	6 months	WP-11

MERIDIAN'S CURRENCY POSITION, IN THREE ROWS

Three nodes lose vendor support in 2027: `PTC-15 MFT Server` in June (R2.2m), `PTC-03 WebSphere Cluster` in September (R6.8m) and `PTC-07 Legacy Windows Estate` in October (R7.6m). All three are `Declining` and `Contained`. Together they carry R16.6m of annual cost, and `RPT25 Reference Model Conforman` — Excel's truncation of the Reference Model Conformance report — records that the runtime-hosting purpose they answer is already `Unsupported` with `AKS Production Cluster` as the target [2][5]. That is the paper, and it writes itself from four columns.

Step 7.8 — Publish the platform services and record who consumes them

Produce. `O28 Platform Service` populated and `R36 PSV-APP` joined. Draw it as an ArchiMate 4 technology usage view: `Service` realised by the platform `Node`, serving each `Application Component` that consumes it.

You need. Steps 7.1 and 7.4. A view on which services are offered self-service and which are requested.

Where it lands.

`O28 Platform Service`

`Name` is what a delivery team asks for: `Managed Database Platform`, `Integration Platform`, `Observability Platform`. `Technology Service` is the `TSV-` id it packages. `Consumption Model` is `Self-service` or `Managed` — Meridian records seven `Self-service` and one `Managed`, the `Resilience Platform`. `State` is `Baseline` or `Target`; three of Meridian's eight are `Target`.

`R36 PSV-APP`

`Source (PSV)`, `Target (APP)`, `Consumption` as `Direct` or `Indirect`, and `State`. Meridian holds twenty-eight rows, twenty-seven `Direct` and one `Indirect`, split eighteen `Baseline` to ten `Target` [1].

How.

1. Define a platform service only where a delivery team can actually obtain it without a project. Otherwise it is a technology service with a brochure.
2. Set `Consumption Model` from how it is obtained today, not from the target operating model.
3. Join `R36` for the applications that use it. `Indirect` means consumed through another platform.
4. Use `State = Target` on the rows that depend on a product not yet selected. Meridian's ten `Target` rows mostly hang off `PSV-03 Integration Platform`, whose gateway product is still `To be selected`.

Signed by. Chief Information Officer.

Done when. Every platform service names a technology service and a consumption model, and every `Self-service` claim can be demonstrated.

Shows up in. `RPT12 Integrity Check` and the Phase E work-package shaping: a `Target` platform service with ten consuming applications is a dependency, not a deliverable.

Goes wrong when. `Self-service` is recorded as an intention. If obtaining it needs a ticket and a conversation, it is `Managed`, and saying so is what makes the gap visible.

Where it all actually sits

Step 7.9 — Place the estate

Produce. `O12 Location` populated, with `R26 PTC-LOC` and `R27 ORG-LOC` joined.

You need. Step 7.4. The facilities list. `O09 Organization Unit` from Phase B with headcounts.

Where it lands.

> **`O12 Location`**
> `Name`, `Type` from `Office`, `Contact Centre`, `Data Centre`, `Cloud Region`, `Branch`, then `City`, `Country`, `Headcount` and `Criticality` in the same tier language as `O23`. Meridian holds ten rows: two data centres, one cloud region, a branch network standing for the whole estate, four offices and contact centres, and two foreign offices in Namibia and Botswana.

> **`R26 PTC-LOC`**
> `Source (PTC)`, `Target (LOC)`, `Role` from `Primary`, `Secondary`, `Replica`, `Disaster recovery`, `Distributed`. Twenty rows at Meridian, sixteen `Primary` [6].

> **`R27 ORG-LOC`**
> `Source (ORG)`, `Target (LOC)`, `Type` and `Headcount`. Eighteen rows, which is how you find the business units split across sites.

How.

1. One row per place where something runs or someone works. A cloud region is a location, with `Headcount` zero.
2. Set `Criticality` from what happens if the place is unavailable, not from how many people sit in it. Meridian's data centres are `Tier 1` on forty and twenty-two staff; the branch network is `Tier 3` on 640.
3. Join `R26` for every node. Mark the recovery site's role explicitly — Meridian's `PTC-02 Mainframe LPAR DR` is `Disaster recovery` at `LOC-06`.
4. Join `R27` for every organisation unit, with headcount per site. The sum must match `O09`.
5. List the nodes with no location and the applications with no node. Meridian has five of the latter, three of them `SaaS`, where no Meridian location is the right answer and the register should say so [2].

Signed by. Chief Information Officer, with facilities confirming the headcounts.

Done when. Every `O27` node has a location or a recorded reason, and `R27` headcounts reconcile to `O09`.

Shows up in. `RPT21 Software Distribution`, grouped by location.

Goes wrong when. Cross-border sites are recorded as offices and nothing else. Meridian's Windhoek and Gaborone offices are also a data-transfer position: `CO-06` in `O63 Control Objective` records POPIA section 72 as `Partial` because in-country cloud is met and the two foreign sites are not assessed [7].

Step 7.10 — Record the environments, and say where the knowledge came from

Produce. `O48 Environment` populated with `Evidence In The Repository` and `Provenance` filled honestly, and `R44 O48-LOC` joined.

You need. Step 7.9. The non-production estate's actual configuration, which usually nobody holds centrally.

Where it lands.

> **`O48 Environment`**
> `Name`, `Purpose`, `Where It Runs`, `Data It Holds`, then the two columns that make this sheet unusual. `Evidence In The Repository` says which repository records support the claim — Meridian's `ENV-01 Production` reads "Recorded indirectly: every physical component in O27 is a production component", and `ENV-03 Non-production` reads "Implied by SEC-C07 Non-production masking, which is recorded as Partial". `Provenance` says how the row came to exist; Meridian's four rows all read `Authored`.

> **`R44 O48-LOC`**
> `Source (O48)`, `Target (LOC)`, `Provenance` and `Note`. Six rows at Meridian, all `Authored` [6].

How.

1. Write four rows: production, disaster recovery, non-production, analytics. Add more only if they differ in data classification.

2. Fill `Data It Holds` with the classification, not the volume. Meridian's `ENV-03` reads "Masked data on two platforms of nine", which is a control finding written as a fact.
3. Fill `Evidence In The Repository` by naming the rows that support the claim. Where the evidence is indirect, say *indirectly*.
4. Fill `Provenance` with `Authored` where an architect wrote it down from conversation, and change it only when somebody verifies it against a system.

Signed by. Chief Enterprise Architect, with the Chief Information Security Officer on non-production.

Done when. Every environment names its evidence and its provenance, and every `Authored` row has a named person who would have to verify it.

Shows up in. `RPT30 Compliance Position` and `RPT18 Metadata Coverage`.

Goes wrong when. `Provenance` is left blank because the architect is confident. The column exists because half of what an architect records about the estate is hearsay until somebody checks it — and the honest thing is to publish it marked as hearsay rather than wait a quarter for certainty. Meridian publishes all four environments, all ten network links and all twelve resilience tiers as `Authored`, and that is the correct state of a repository in its first quarter.

Step 7.11 — Record the network links

Produce. `O50 Network Link` populated, each row saying what it carries.

You need. Step 7.9. The network team's site and partner connection list.

Where it lands.

> **`O50 Network Link`**
>
> `From`, `To`, `Type` from `Data centre interconnect`, `Private cloud connection`, `Branch wide-area network`, `Corporate wide-area network`, `Cross-border wide-area network`, `Partner connection`, `Public internet`, `Secure submission`. `What It Carries` is a sentence about business content, not bandwidth — Meridian's `LNK-06` reads "3,400 brokerages reaching BrokerNet". `Provenance` as in `O48`.

How.

1. Record inter-site links and perimeter links. Ten rows covers an insurer of Meridian's size.
2. Write `What It Carries` in business terms: who is at the far end, how many of them, which application they reach.
3. Include the regulator. Meridian's `LNK-10` runs from the perimeter to the Prudential Authority and the FSCA, which makes the submission path an architecture object rather than a finance process.
4. Set `Provenance`.

Signed by. Chief Information Officer.

Done when. Every external party that reaches an application has a link, and every link says what it carries.

Shows up in. `RPT25 Reference Model Conformance`, whose `REF-T-16 Connect a partner` reads `No standard` at Meridian with the note that no link, zone or control is recorded for any external connection [5].

Goes wrong when. The sheet becomes a network diagram in table form. Three thousand funeral parlours notifying claims is an architecture fact; the circuit type is not.

Resilience, and the regulator

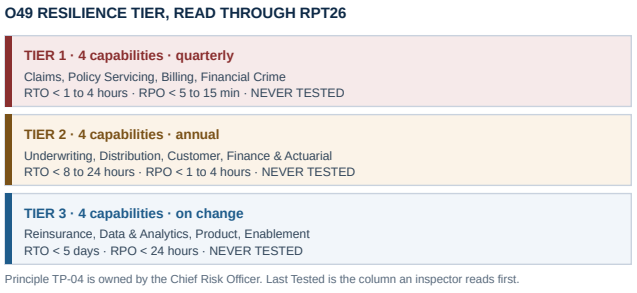

Figure 7.3 Meridian's twelve resilience tiers, by tier, with recovery objectives and test status. Every row reads Never tested, and every row's Provenance reads Authored.

`O49 Resilience Tier` is the sheet that makes principle `TP-04 Resilience Proportionate to Criticality` testable. The principle, owned by the Chief Risk Officer, reads "Resilience engineered against criticality tiers defined once and applied consistently" [4]. Without `O49` that is a sentence. With it, it is twelve rows with recovery objectives and a last-tested date.

Volume 1 chapter 15 sets out what the Prudential Authority and the FSCA expect and why they issue Joint Standards together, and covers **Joint Standard 1 of 2024 on outsourcing by insurers**, effective 1 December 2024, which replaced Prudential Standard GOI 5 [8][9]. Read that chapter before this step rather than re-deriving it. Two things follow for Phase D.

For resilience, the evidence columns are `Tier`, `RTO`, `RPO`, `Test Cadence` and `Last Tested`. Of those, `Last Tested` is the only one an inspector cannot be talked out of. The repository records Meridian's resilience obligation against `CO-09` in `O63 Control Objective`, sourced to `Joint Standard 1 of 2023`, with `Evidence Today` reading "None. Twelve tiers, none ever tested" [7]. Note the year: the repository's three technology control objectives — attributable access, logging, and resilience testing — all cite the 2023 Joint Standard on information technology governance and risk, not the 2024 outsourcing standard. Both apply, and they apply to different columns.

For outsourcing, the evidence is in `O19 Contract` and `O27`'s `Vendor / Product` and `Location`: which material functions run on somebody else's platform, where that platform sits, what the notice period is, and whether an exit has been assessed. Meridian holds `STD-14 Exit Assessment Standard`, `Emerging`, scoped to `Platform selection`, which is the right place to hang that obligation [3].

Step 7.12 — Set the resilience tiers and read the test status

Produce. `049 Resilience Tier` populated, and a reading of `RPT26 Resilience Test Status` that names the untested Tier 1 capabilities.

You need. `O13 Business Capability` with `Criticality`, step 6.2 for the applications, and the business continuity team's recovery objectives if they exist. Volume 1 chapter 15 for the regulatory frame.

Look it up. Volume 1, chapter 15, *What the regulator actually inspects*, question six — one external data flow off, business still running.

Where it lands.

> **`049 Resilience Tier`**
>
> `Capability` is the business capability, not the application — tiering by capability is what makes the sheet survive an application replacement. `Tier` from `Tier 1` to `Tier 3`. `RTO` and `RPO` as durations: Meridian's Tier 1 rows read `< 1 hour` and `< 5 min`, Tier 3 `< 5 days` and `< 24 hours`. `Test Cadence` from `Quarterly`, `Annual`, `On change`. `Last Tested` is a date or `Never tested`. `Applications` lists the `O23` names in scope. `Provenance` as in `O48`.

How.

1. Tier by capability. Meridian has twelve rows, four per tier.
2. Set `RTO` and `RPO` from the business consequence, then check whether the estate can meet them. A Tier 1 RPO of five minutes against a nightly batch is a gap, and recording both is the point.
3. Set `Test Cadence` from the tier: quarterly for Tier 1, annual for Tier 2, on change for Tier 3.
4. Write `Never tested` where that is true. Every one of Meridian's twelve rows reads it [10].
5. Raise the gap. Meridian holds `GAP-16 Untested resilience tiers`, `High`, funded under `WP-11`.

Signed by. Chief Risk Officer, who owns `TP-04`, with the Chief Information Officer on feasibility.

Done when. Every `Tier 1` capability has an RTO, an RPO, a cadence and either a test date or an explicit `Never tested`, and the four Tier 1 rows are named in the Phase D checkpoint pack.

Shows up in. `RPT26 Resilience Test Status`, footer: `Tiers` 12, `Never tested` 12, `Tier 1 capabilities` 4. Its closing note is the sentence to carry into the regulator conversation: until a test runs, the recovery position of the whole estate is asserted rather than demonstrated [10].

Goes wrong when. The tiers are copied from the disaster recovery plan without the test dates. A tier with an RTO and no `Last Tested` is a target, and an inspector reads it as one.

RESILIENCE YOU CAN DEMONSTRATE

Which Tier 1 capability has never been tested, and what would a test cost to run next quarter?

Where the RPO is minutes and the data moves in a nightly batch, which number is wrong — the target or the estate?

If one external data flow were switched off tomorrow, what stops? Name the flow and the consequence.

A CONTRADICTION TO RESOLVE, NOT TO HIDE

Meridian's `GAP-16` describes the problem as "Only 41% of Tier 1 capabilities have a tested RTO and RPO", while `049 Resilience Tier` and `RPT26` both record `Never tested` on all twelve rows, and `RPT26` 's footer counts twelve of twelve [4][10]. One of the two is wrong. Raise it as a change request against `029 Gap` rather than quietly aligning the gap text to the report — the discrepancy is itself evidence about how the baseline was assembled.

Security as architecture

Security in this repository is not a control list bolted on at the end. It is five sheets that answer five different questions, and the chain between them is what makes a compliance position auditable.

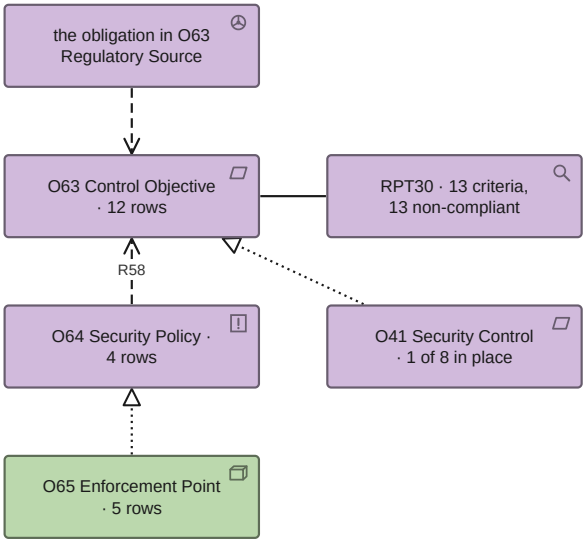

Figure 7.4 The security chain in ArchiMate 4. The obligation and the security policy both influence the control objective; the enforcement point realises the policy; the security control realises the objective. `RPT30` assesses the chain, and at Meridian every one of the thirteen criteria fails.

Sheet	Question it answers
`063 Control Objective`	What must be true, and which regulation says so
`064 Security Policy`	What the organisation has committed to
`041 Security Control`	What is actually implemented, and the gap
`065 Enforcement Point`	Where the control is applied in the architecture
`R58 064-065`	Whether the policy is in fact enforced at that point

Step 7.13 — Record security as architecture

Produce. `063` , `064` , `041` , `065` , `R58 064-065` and `R60 063-REQ` populated, and `RPT30 Compliance Position` read.

You need. Volume 1 chapter 15 for the regulatory sources. `006 Requirement` . Steps 7.4, 7.9 and 7.10. The Chief Information Security Officer and the Chief Compliance Officer in the same room.

Look it up. Volume 1, chapter 15, *FICA, and its argument with POPIA*, for the one conflict a control set must resolve rather than restate.

Where it lands.

> **`063 Control Objective`**
>
> `Obligation` in plain words. `Regulatory Source` names the instrument — Meridian's twelve rows cite `POPIA` four times, `Joint Standard 1 of 2023` three times, and `COFI Bill` , `FAIS` , `IFRS 17` , `FICA` and the `Insurance Act` once each. `Control Objective` is what must be true. `Control` names the control or the `SEC-C` id. `Evidence Today` is the honest position: Meridian's reads `None` for six objectives. `Requirement` is the `REQ-` or `ARS-` id with its funding state.

064 Security Policy

`Policy` as a statement. `Principles`, `Standards` and `Controls` as id lists. `Enforced At` names the enforcement points. `Position` is the verdict: all four of Meridian's read `Breached` or `Unenforceable until now` [7].

065 Enforcement Point

`Enforcement Point`, `Acts On`, `Enforces` as the `SEC-P` id, and `Position`. Meridian's `EP-03 API gateway` reads that no product is selected, so nothing is enforced there.

How.

1. Start from `063`. One row per obligation with a named regulatory source. Twelve rows is enough for an insurer; thirty means you have copied a compliance register.
2. Write `Evidence Today` before you write the control. `None` is the most useful value in the sheet.
3. Fill `041 Security Control` with `State` from `In place`, `Partial`, `Absent`, and a `Gap` sentence. Meridian's eight controls read one `In place`, four `Partial`, three `Absent`.
4. Join `R58 064-065` with `Position`. Meridian has two rows: the identity provider enforces `SEC-P02`, and the design review gate `G1` carries `SEC-P04` as `Not enforced today`.
5. Join `R60 063-REQ` with `Evidence Today`. Ten rows at Meridian: six `None`, three `Partial`, one `Not yet due` [7].
6. Read `RPT30 Compliance Position`: thirteen criteria assessed, thirteen non-compliant, none compliant [7]. Report that number without softening it.

Signed by. Chief Information Security Officer for `064`, `041` and `065`; Chief Compliance Officer for `063`'s regulatory sources; Architecture Board (`GB-01`) for the position.

Done when. Every control objective names a regulatory source, an evidence position and either a requirement or an explicit "no requirement covers this", and every security policy names an enforcement point with a `R58` verdict.

Shows up in. `RPT30 Compliance Position`, columns `Verdict`, `Evidence` and `Contract status`.

Goes wrong when. The enforcement point is named but not assessed. `EP-01 Design review, G1` at Meridian records that the gate did not exist until Phase G, which is why forty-four interfaces reached production unmanaged. A policy enforced at a gate that does not exist is not enforced.

THE COMPLIANCE POSITION, UNSOFTENED

Which control objective reads `None` in `Evidence Today`, and which requirement would change that?

Which policy is enforced at a point that does not exist yet, and what is happening meanwhile?

Thirteen criteria assessed and thirteen non-compliant: who in the business has been told, in those words?

Step 7.14 — Publish the Technology Standards Catalogue and pass the checkpoint

Produce. `DLV-18 Technology Standards Catalogue`, and a Phase D checkpoint pack the Architecture Board can approve or refuse.

You need. Steps 7.1 to 7.13. `031 Deliverable` row `DLV-18` — at Meridian `Artifact`, phase `D`, work package `WP-11`, owner Chief Information Officer, due 2027-06, status `In progress`, location `Repository/PhaseD` [11].

Where it lands.

031 Deliverable

Update `DLV-18`: `Status` to `Complete`, `Repository Location` to the published path.

How.

1. Assemble the catalogue in four sections: the standards from `032`; conformance from `RPT10`; the currency paper from step 7.7; the resilience position from `RPT26`.

2. Put six numbers on the front page. Meridian's: 16 standards, 10 `Emerging`, 19 assessments, 52.6% conformance, 3 nodes out of support in 2027, 12 resilience tiers never tested.
3. State the provenance. Environments, network links and resilience tiers are all `Authored`; say so on the front page rather than in a footnote.
4. Take it to Architecture Board (`GB-01`) with the currency paper going on to Investment Committee (`GB-02`) for funding.
5. Check the Phase D checkpoint list below before you present, with the ADM's own phase objectives as the backstop [12]. A checkpoint that fails on a missing join wastes a month of Board cadence.

Signed by. Chief Information Officer as owner, approved by Architecture Board (`GB-01`).

Done when. `DLV-18` reads `Complete` in `O31`, every front-page number traces to a sheet in one step, and the Board has either approved the catalogue or named what is missing.

Shows up in. `RPT14 Deliverable Register` and `RPT15 Catalogue Completeness`.

Goes wrong when. The catalogue is published without the currency paper. The standards catalogue on its own is a governance document; with the three 2027 dates and R16.6m attached, it is a budget submission.

PHASE D — A REVIEWER'S QUESTIONS

Does every `O25` service have both a logical and a physical realisation, and which one is missing?

Which node has no location, and what is the recorded reason?

Which standard cannot be tested as written, and who is rewriting `Applies To`?

Which rows are `Authored` rather than observed, and who would verify each one?

THE CHAPTER IN TEN LINES

- Four layers: service, logical component, physical component, location. Keep them in four sheets.
- O25 names capabilities, never products. The service survives the product change.
- The sheet names are truncated in Excel: O26 Logical Technology Componen, O27 Physical Technology Compone.
- O27 has two columns headed Version. Take the product version by position.
- Contained is the most useful word in Phase D: it works, it stays, nothing new goes on it.
- Support Ends comes from your contract, not the vendor's public lifecycle page.
- Support Ends crossed with Standard Status, costed and tied to applications, is the budget paper.
- Provenance exists because half of what an architect records about the estate is hearsay until checked.
- Tier resilience by capability, not application; Last Tested is the column an inspector reads first.
- The repository cites Joint Standard 1 of 2023 for access, logging and resilience; volume 1 chapter 15 covers Joint Standard 1 of 2024 on outsourcing. Both apply, to different columns.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets O25 Technology Service, O26 Logical Technology Componen, O27 Physical Technology Compone, O28 Platform Service, R24 APP-TSV, R25 PTC-LTC and R36 PSV-APP
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT21 Software Distribution and RPT02 Application Portfolio
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets O32 Standard, R31 STD-PTC and RPT10 Standards Conformance
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets O05 Principle and O29 Gap
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheets O47 Reference Model Entry, R46 O47-PTC and RPT25 Reference Model Conforman
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheets O12 Location, O48 Environment, O50 Network Link, R26 PTC-LOC, R27 ORG-LOC and R44 O48-LOC
- **7.** Meridian_TOGAF_Metamodel.xlsx, sheets O41 Security Control, O63 Control Objective, O64 Security Policy, O65 Enforcement Point, R58 O64-O65, R60 O63-REQ and RPT30 Compliance Position
- **8.** Prudential Authority and FSCA, Joint Standard 1 of 2024: Outsourcing by Insurers (effective 1 December 2024) https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-public-awareness/covid-19-response/2024/joint-comms-1-of-2024/Joint%20Standard%201%20of%202024%20Outsourcing%20by%20Insurers.pdf
- **9.** Bowmans, "South Africa: new requirements for outsourcing by insurers published in Joint Standard 1 of 2024" https://bowmanslaw.com/insights/south-africa-new-requirements-for-outsourcing-by-insurers-published-in-joint-standard-1-of-2024/
- **10.** Meridian_TOGAF_Metamodel.xlsx, sheets O49 Resilience Tier and RPT26 Resilience Test Status
- **11.** Meridian_TOGAF_Metamodel.xlsx, sheets O31 Deliverable, O19 Contract, O57 Governance Body and O58 Governance Gate
- **12.** The Open Group, the TOGAF Standard — Architecture Development Method overview https://www.opengroup.org/togaf

CHAPTER 8

Opportunities and Solutions

You have a tool most enterprise architecture practices do not have. Used properly it tells you which differences are gaps, which are drift somebody approved, and which are changes nobody logged — and those three things need three different responses.

Phase E is where the architecture stops being a description and becomes a programme [8]. You leave with a gap register, building blocks, work packages, three transition states and a roadmap somebody has funded.

You have one advantage most practices do not. You were handed `Meridian_Architecture_Comparison_Tool.xlsx`, a twelve-sheet workbook that takes two extracts of the repository and separates real gaps from drift the change log explains and from changes nobody logged [1]. Most practices do this in a meeting, from memory. Everything downstream is only as good as the gap register, so the tool comes first.

Meridian Assurance Group is an invented South African composite insurer and its data is real within this workbook. Cape Mutual, volume 1's insurer, appears only when pointing back at volume 1.

> **BEFORE YOU START THIS PHASE**
>
> The tool finds differences. What an insurer is structurally missing comes from volume 1.
>
> - **Volume 1, chapter 4, *The same five failures, every time*.** Identity, as-at, document against system, hand-off, status.
> - **Volume 1, chapter 11, *The patterns*.** The *usually missing entirely* list, and what you find instead.
> - **Volume 1, chapter 14, *Build versus buy, per capability — and the forgotten third option*.** Sourcing per capability.
> - **Volume 1, chapter 17, *The change view*.** Three plateaus, drawn after the architecture, not before.

The machine, in one picture

Five sheets take input, two calculate, one takes your judgement, two present the result. Pale yellow marks a cell you fill; everything else is calculated, and overwriting a formula breaks it [1].

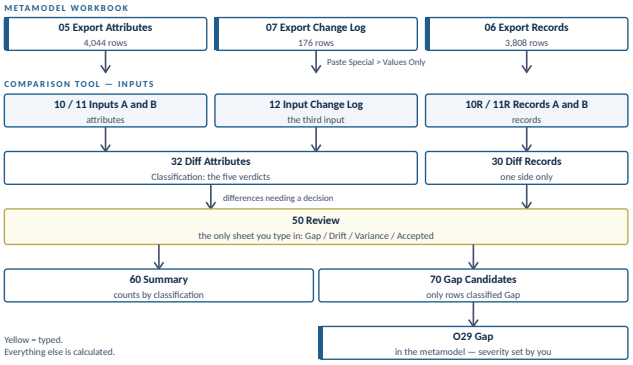

Figure 8.1 The comparison tool, end to end. Three export sheets in the metamodel become five pasted inputs; two sheets calculate; one sheet takes a human verdict; two present the result. Nothing reaches `O29 Gap` without passing through `50 Review`.

> **`00 HOW TO USE` OMITS TWO INPUT SHEETS**
>
> Its sheet list does not mention `10R Records A` or `11R Records B`, which are pasted inputs too: they carry the record lists `30 Diff Records` walks [1]. Paste only the three it names and record differences come out empty while attribute differences come out right — the worst kind of failure, because it looks like a result.

Part one — running the comparison

Step 8.1 — Produce the three extracts

Produce. Five blocks of pasted values: baseline attributes, target attributes, both record lists, the change log.

You need. The metamodel workbook; a named baseline from `04 Baselines` for snapshot A; agreement on what snapshot B is.

How.

1. On `05 Export Attributes`, select from row 5 down, copy, then Paste Special > Values Only into `11 Input B Target` at row 5. Repeat from `06 Export Records` into `11R Records B` and from `07 Export Change Log` into `12 Input Change Log`.
2. Run the same two exports on the baseline workbook — the one `04 Baselines` records as `BL-001` — into `10 Input A Baseline` and `10R Records A`.
3. Check the counts in the tool's own input headers: the shipped example reads 1,439 attribute rows across 1,204 records in A, 4,040 across 3,804 in B [1].
4. If your extract is longer, fill the formula columns of `30 Diff Records` and `32 Diff Attributes` down to match. The ranges are fixed and will not grow.

Signed by. Chief Enterprise Architect, recorded as `Prepared by` on `01 Config`.

Done when. The input counts match the export sheets, and no input cell holds a formula.

Shows up in. `60 Summary`: `Records in A`, `Records in B`, `Attribute rows compared`, `Change log entries supplied`. If the last reads zero, every difference will come out as a gap.

Goes wrong when. Somebody links the sheets to the source workbook instead of pasting. External references break on rename, need both files at fixed paths, and depend on users enabling link updates [1]. Pasting is far more reliable.

Where it lands.

`05 Export Attributes · 06 Export Records · 07 Export Change Log`

- `05 Export Attributes` — `Key | Kind | Type | Type Name | ID | Name | Attribute | Value`, 4,044 rows, material attributes only. `O13` is assessed at Level 3, so Levels 1, 2 and 4 contribute nothing. Into `10` and `11`.
- `06 Export Records` — `Key | Kind | Type | Type Name | ID | Name`, one row per object and per relationship instance. Into `10R` and `11R`.
- `07 Export Change Log` — `Key | Change | Date | Sheet | Record | Field | Previous Value | New Value | Change Request | Approved By`. Into `12`.

THE COMPOSITE `KEY` IS THE WHOLE TRICK

`O20|DEN-01|Authoritative Source` is three things joined by a pipe: sheet, record, attribute. Every match in the tool is a lookup on that one string. The record-level key drops the third part: `O20|DEN-01`.

Two consequences. A change log entry explains a difference only if its `Key` is built the same way, which is why `07 Export Change Log` exists rather than pointing the tool at `02 Change Log`. And if an ID scheme changes between snapshots, every key changes and the whole catalogue reports as removed and re-added — the trap in step 8.3.

Step 8.2 — Set the configuration before reading any result

Produce. A dated, labelled, attributed comparison.

You need. The baseline reference from `04 Baselines`; the date the target was drawn; a decision on mode.

How.

1. Set `Snapshot A label` to the baseline's name as `04 Baselines` records it — the shipped example reads `BL-001 Preliminary Phase Baseline` [1].
2. Set `Snapshot A date` to that baseline's date. Entries after it are candidates to explain a difference; entries before it are already inside the baseline.
3. Set `Snapshot B label` and `Snapshot B date`: `Repository as at 2027-11-20, v1.23` and `2027-11-20` in the shipped example.
4. Set `Comparison mode`. `Baseline vs Target` means unexplained differences are gap candidates; `Snapshot vs Snapshot` means they are ungoverned changes [1]. The mode changes no formula — it changes what you may conclude.
5. Set `Prepared by` and `Prepared on`. A comparison with no preparer is not evidence.

Signed by. Chief Enterprise Architect.

Done when. `60 Summary` echoes all four snapshot fields and the mode, and the two dates bracket the change log you pasted.

Goes wrong when. The dates are out by a quarter. The change log explains differences only inside the window, so a `Snapshot A date` set too late pushes governed drift into the `UNGOVERNED CHANGE` column and manufactures a governance crisis out of arithmetic.

Where it lands.

`01 Config`

`Snapshot A label` and `Snapshot A date` — the 'from' side; entries after the date can explain a difference. `Snapshot B label` and `Snapshot B date` — the 'to' side; entries up to the date are in scope. `Comparison mode` — `Baseline vs Target` or `Snapshot vs Snapshot`. `Prepared by` and `Prepared on` — the attribution that makes the output usable in a governance pack.

Step 8.3 — Read the record-level differences

Produce. Whole records present on one side only, triaged into additions, removals and renumbering noise.

You need. Steps 8.1 and 8.2.

How.

1. Filter `30 Diff Records` on `Status`: `ADDED in B`, `REMOVED in B`, `Present both`.
2. Group `REMOVED in B` by `Type`. Where nearly every record of a type is removed and a similar number added, compare the `ID` columns: a changed identifier shape means renumbering.
3. Separate the real removals. `O23|APP-27` QuoteLegacy is genuinely absent: it was decommissioned in 2026 [3].
4. Read the `ADDED in B` block. Whole records present only in the target are the usual shape of a design delta.

Signed by. Chief Enterprise Architect.

Done when. Every `REMOVED in B` record is accounted for as decommissioning, omission or renumbering, and the renumbered ones are excluded from any count you quote.

Shows up in. `60 Summary`: `Records added` 2,628 and `Records removed` 28, against 1,204 records in A. Those numbers size a catalogue build, not a gap register.

Goes wrong when. You quote the raw count. The shipped comparison reports `O13|CAP-01` Product Management as `REMOVED in B`, with the whole `CAP-nn` series, because the capability catalogue was renumbered to the four-level `C01.01.01` scheme between snapshots [1]. Nothing was removed, and a register built off that count raises fifteen capability gaps that do not exist.

Where it lands.

`30 Diff Records`

`Scope | Key | Kind | Type | Type Name | ID | Name | In A | In B | Status`

`Scope` — `From A` walks the baseline; `From B` walks the target and reports only what the baseline lacked, so nothing is double-counted. `In A` and `In B` are 1 or 0, from a `MATCH` on the record key. `Status` is `Present both`, `REMOVED in B` or `ADDED in B`. Relationships are compared here and only here [1].

Step 8.4 — Read the classification column

Produce. The attribute-level difference list, sorted by the tool's verdict.

You need. Step 8.3, so you already know which record-level differences are noise.

How.

1. Filter `32 Diff Attributes` on `Classification` and work the verdicts in this order: `UNGOVERNED CHANGE`, `Governed drift + further change`, `Governed drift`, `Record removed in B`, `Added in B`. Ignore `Same`.
2. For each `UNGOVERNED CHANGE` read `Value in A` against `Value in B`. `O23|APP-08|Hosting` moves from `On-premise` to `Cloud` with nothing logged [2].
3. For each `Governed drift` read `Change Request` and `Logged new value`. `O05|TP-01|Tier` moved 2 to 3 under `CR-012`, logged new value 3: fully explained [2].
4. Read the `Governed drift + further change` rows closely. `O21|LDC-01|State` has `CR-002` logging a move to `Target`, but the value now reads `Baseline & Target` [2]. Treat `** not in B **` and `** not in A **` as sentinels, not values.

Signed by. Chief Enterprise Architect.

Done when. Every row not classified `Same` has been read and every `UNGOVERNED CHANGE` row copied to `50 Review`.

Shows up in. `60 Summary`: `Same` 1,264, `Governed drift` 31, `Governed drift + further change` 1, `UNGOVERNED CHANGE` 5, `Added in B` 2,739, `Record removed in B` 138 [4].

Goes wrong when. Somebody filters on `Differs = 1` rather than `Classification`, merging drift into gaps, or reads the `From B` block as new findings. That block marks anything already seen `(already reported)` — a sixth verdict whose only job is to stop double counting.

Where it lands.

32 Diff Attributes

`Scope | Key | Type | Type Name | ID | Name | Attribute | Value in A | Value in B | Differs | Logged change | Change Request | Logged new value | Classification`

`Differs` is 1 or 0 from a text comparison, so `2` and `"2"` compare equal. `Logged change` is a `MATCH` of the composite key against `12 Input Change Log`; `Change Request` and `Logged new value` come from the matching entry. `Classification` is the machine verdict — material attributes only, because a diff across descriptions would bury real findings under rewording [1].

The verdicts, in language an executive accepts

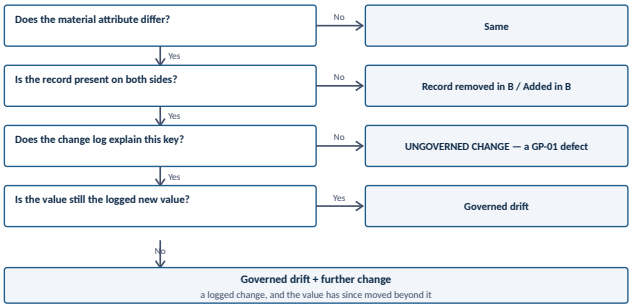

Figure 8.2 What the `Classification` column is actually deciding. Four questions, six answers. The tool applies no judgement about significance — that is step 8.5.

Verdict	What the machine found	What it means to a board
`Governed drift`	The value moved and an approved change request explains it	The baseline moved under governance. Not a gap, but your *target* may now be stale
`Governed drift + further change`	A change request explains part of the move; the value has since moved further	A governed change followed by an ungoverned one
`UNGOVERNED CHANGE`	The value moved, nothing explains it	A governance defect under `GP-01`, not an architecture gap. Usually the most valuable thing the tool finds
`Added in B`	The attribute exists only in the target	The usual shape of a design delta — a real gap candidate
`Record removed in B`	The record exists only in the baseline	Decommissioning, omission or renumbering

A **design delta** is deliberate: somebody decided the target should say something the baseline does not. An API gateway appears in the target because the architecture says there should be one. That is a gap, and it belongs in `029 Gap` [6].

An **ungoverned change** is the opposite. `APP-08` moved to cloud hosting and neither architecture reflects where it now runs. There is no architectural gap: there is a repository that no longer describes reality, and a decision taken outside `GP-01`, which requires context, options, rationale and consequences recorded for every significant decision [7]. Raise it as a gap and you ask a programme to fund something a change request would close in a week. Raise it as a `GP-01` defect and it goes to the Architecture Board as a governance failure, with a retrospective change request as the remedy.

Step 8.5 — Classify each difference on `50 Review`

Produce. A reviewed list where every row has a verdict, a rationale, a named owner and an action.

You need. `32 Diff Attributes` filtered; the domain architect for each affected sheet; the executive in the record's `Owner Role` or `Owner Org`.

How.

1. Copy `Key`, `Type Name`, `ID`, `Attribute` and `Tool classification` across for each difference needing a decision.
2. Set `Your classification` to `Gap`, `Drift`, `Variance` or `Accepted`, by the rules below.
3. Write `Rationale` as a sentence that survives being read aloud in a governance meeting. The shipped standard: "Backup platform promoted to Current standard status outside the standards process. TP-02 breach." [3]
4. Set `Owner` to a role in `011 Role` filled by an actual person. An owner who is a vacant role is not an owner.
5. Set `Action` to something a named person can do by a date — `Raise CR; assess against STD-16`, or `Re-run assessment and log it` [3].

Signed by. Chief Enterprise Architect, with the domain architect countersigning the rationale.

Done when. No row has a blank `Your classification`, `Owner` or `Action`, and `60 Summary` shows `Reviewed` equal to the rows you brought across.

Shows up in. `60 Summary`'s human classification block, and `70 Gap Candidates`.

Goes wrong when. Everything is classified `Gap`, because `Gap` is what gets funded. Four of the eight shipped rows are `Gap`, one `Drift`, one `Variance`, two `Accepted` [3] — about the right ratio for a practice doing this honestly.

Where it lands.

`50 Review`

`Key | Type Name | ID | Attribute | Tool classification | Your classification | Rationale | Owner | Action`

The only sheet you type in. Columns A to E come from `32 Diff Attributes`; F to I are yours. Only rows classified `Gap` flow to `70 Gap Candidates` [3].

CLASSIFYING A DIFFERENCE

For the difference in front of you: did the target move, did the baseline move, or did neither — and which of the four verdicts does that make it?

Which rows are you tempted to call `Gap` because `Gap` is what gets funded?

Write one rationale as you would read it aloud in a governance meeting. Does it survive?

THE DECISION RULES

`Gap` — the target deliberately differs from the baseline and closing it needs design or delivery. Also where an ungoverned change reveals something neither architecture accounts for.

`Drift` — the baseline moved under an approved change request, so your *target* is now out of date. The action is to reissue the target architecture, not to fund a package: `DEN-01 Authoritative Source` moved to `CONTESTED` under `CR-002`, so the target's assumption of a single named master needs revisiting [3].

`Variance` — a modelling difference with no architectural meaning. Fold it into whichever deliverable settles the convention.

`Accepted` — deliberate and fine; the record is what needs fixing. `BSV-12 Recover Claim` was downgraded to Tier 3 with no change request, and the reviewer's verdict is that it is defensible on the leakage review: log it retrospectively rather than reverse it [3].

PRESENTING IT

Lead with the classification split, not the totals, and name the `UNGOVERNED CHANGE` rows: that count measures the practice, not the estate, and five in 3,804 records is nearly in control. Claim the `Governed drift` count as evidence the change log works. State `Differences not yet reviewed` honestly: 2,906 against 8 [4]. A pack that leads with "2,739 differences found" is true, meaningless, and the last time that audience reads a gap analysis.

Step 8.6 — Lift `70 Gap Candidates` and set severity

Produce. Gap candidates ready to paste into `O29 Gap`, with a severity you can defend.

You need. `50 Review` complete; `O04 Measure` to judge consequence; `O63 Control Objective` to spot a regulatory obligation behind a gap.

How.

1. Read `70 Gap Candidates`: only the rows a person classified `Gap`, already shaped for `O29 Gap`.
2. Set `Severity`, which arrives as `** set **` on every row — the tool has no basis for judging it and marks the cell so it cannot be pasted unnoticed [5]. Use `Critical`, `High` or `Medium`; a fourth value breaks `RPT09 Gaps and Roadmap`.
3. Check `Gap Type`. The tool writes `New` on everything; `O29 Gap` also uses `Replace` and `Eliminate`, and the distinction drives what the package does.
4. Renumber. The tool issues `GAP-C02`, `GAP-C03` from the review row position; `O29 Gap` runs to `GAP-18`, so your first candidate is `GAP-19`.
5. Reconcile before pasting. `APP-20 Criticality`, present only in the target, is the managed integration layer gap already recorded as `GAP-06` [3]. Pasting it again double-counts R42.8m.

Signed by. Chief Enterprise Architect; the executive in `Owner` accepts the severity.

Done when. No row carries `** set **`, no row duplicates an existing gap, and every severity has a one-line justification in `Description`.

Shows up in. `RPT09 Gaps and Roadmap`: `Gap | Name | Domain | Severity | Principles engaged | Work packages | Cost (Rm) | Benefit (Rm) | Status`. A new gap shows `Status` blank until `R28 GAP-WPK` gives it a package.

Goes wrong when. Severity is set by who complained loudest. Meridian's 6 `Critical`, 10 `High` and 2 `Medium` is a defensible shape; thirty critical gaps is a practice with no prioritisation.

Where it lands.

`70 Gap Candidates`

`Gap ID | Name | Description | Domain | Gap Type | Severity | Affected Object | Source difference | Owner`

`Name` arrives as `<Type Name> <ID> — <Attribute>`, machine-readable and useless to a reader: rewrite it. `Domain` is the object type, not one of `O29 Gap`'s five domain values, so map `Application Component` to `Application`. Keep `Source difference` — it is the trail back to the comparison.

SETTING SEVERITY DEFENSIBLY

Answer all four, take the highest band any one reaches, and write the band and the reason into `Description`.

What breaks if nothing is done, and when? A dated exposure — support end, regulatory deadline, treaty renewal — is `Critical` whatever it costs.

Which `O04 Measure` moves, and by how much? A gap with no measure behind it is at most `High`.

Which principle does it engage, and is it violated or merely engaged? A directly violated Tier 1 principle is `Critical`.

How many other gaps wait on this one, and what does the cheapest of them cost?

Part two — the gaps the tool cannot find

The tool finds differences between two things you wrote down. It is blind to anything missing from both, and the blindness has a shape: the capabilities nobody owns are the capabilities nobody modelled.

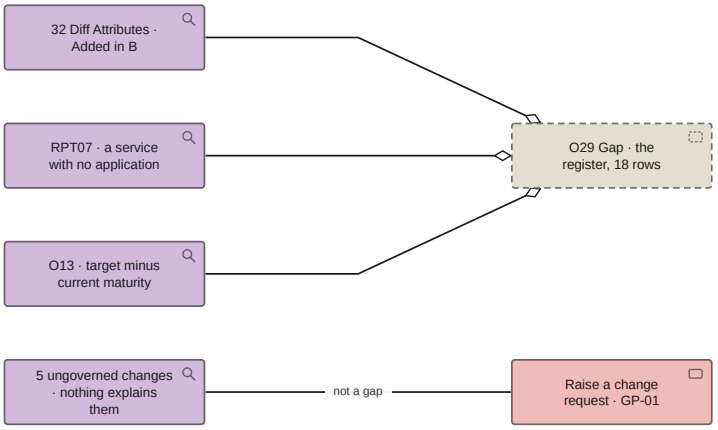

Figure 8.3 Three sources of gap, aggregated into the gap register, and one thing that looks like a gap and is not. An ungoverned change is a `GP-01` defect: it earns a change request, not a work package.

Step 8.7 — Find capability gaps from coverage and the maturity delta

Produce. Capability gaps that exist on neither side of the comparison.

You need. `RPT07 Capability Coverage`; `O13 Business Capability` filtered to `Level` 3; volume 1 chapter 4, how the business model works.

Look it up. Volume 1, chapter 4, *The same five failures, every time*. Each hides behind a `Supported` grade on `RPT07 Capability Coverage`.

How.

1. On `RPT07 Capability Coverage` filter `Coverage` to anything but `Supported`. A service with no implementing application is a capability gap; one with `Partial only` and four applications is a duplication finding, and `DUPLICATED` against `BSV-01 Produce Quotation` is the shape that produced `GAP-07`.
2. On `O13 Business Capability` filter `Level` to 3; assessment is recorded at Level 3 only. In Meridian that is 134 capabilities.
3. Compute `Target Maturity` minus `Current Maturity`, sort descending. A delta of 3 on a `Critical` capability is a gap whether or not an application exists. Meridian has 23, including `C11.01.01 Data Ownership & Stewardship` at 1 against a target of 4.
4. Cross-read `Criticality`, `Disposition` and `Differentiation` first. A delta of 3 on a `Commodity` capability dispositioned `Outsource` is a sourcing decision, not an architecture gap.

Signed by. Chief Enterprise Architect with the capability's `Owner Org` head.

Done when. Every Level 3 capability with a delta of 2 or more and a `Criticality` of `Critical` or `High` is traced to an existing gap or raised as new.

Shows up in. `RPT03 Capability Heatmap`, where a raised gap should coincide with a cell already cold; then `RPT09 Gaps and Roadmap`.

Goes wrong when. Every non-zero delta becomes a gap. Twenty-six of the 134 have a delta of zero and 53 a delta of one; treating one point as a gap produces a roadmap nobody can fund.

Where it lands.

`O13 Business Capability · RPT07 Capability Coverage`

`O13` — `Level`, `Criticality`, `Current Maturity`, `Target Maturity`, `Disposition`, `Differentiation`. IDs run `C01`, `C01.01`, `C01.01.01`, `C01.01.01.01`; assessment sits on the third.

`RPT07` — `Business Service | Name | Capability | Implementing Apps | Full | Partial | Coverage | Duplication`. Note the name: this is the coverage report. The gap report is `RPT09 Gaps and Roadmap`.

Step 8.8 — Test the five capabilities that are usually missing entirely

Produce. Five verdicts — present, partial or absent — against the capabilities volume 1 says insurers almost always lack.

You need. Volume 1 chapter 11, the conceptual application model, and its *usually missing entirely* list; `R37 APP-CAP`; `O19 Contract` for the delegated-authority question.

Look it up. Volume 1, chapter 11, *The patterns*, holds that list — and the "what you find instead" column that funds the gap.

How.

1. Take the five in turn: `A-03` intermediary and delegated authority, `A-05` product definition, `A-11` exposure and accumulation, `A-17` reinsurance administration, `A-25` identity and entitlement as a service.
2. For each, search `R37 APP-CAP` for an application supporting the matching capability in `O13`. An absent link is a finding; a link to a spreadsheet-based component is a worse one.
3. Ask volume 1's question: what do you find instead? Intermediary identity mastered in the commission system, licence status in a spreadsheet, mandates in email. Product spread across the policy system, rating, portal and templates.
4. Record each as a gap with `Gap Type` of `New`, or write down why it is not a gap here. Keep the argument that gets them funded: each is owned by somebody without a seat at the architecture table — distribution operations, the product committee, the chief risk officer, one reinsurance manager, internal audit.

Signed by. Chief Enterprise Architect; Chief Risk Officer for `A-11` and `A-17`.

Done when. All five have a recorded verdict, and any "we have this" names the application in `O23 Application Component` that provides it.

Shows up in. `RPT02 Application Portfolio`, and the gap count by `Domain` on `RPT09 Gaps and Roadmap`.

Goes wrong when. `A-17` reinsurance administration comes back as "covered by the finance system". It is usually two spreadsheets and two people, and the largest concentration of knowledge risk in the insurer.

Where it lands.

`O29 Gap`

Five candidate rows, `Domain` of `Application`. `Affected Object` names the application that should exist, and `Description` carries the "what you find instead" sentence — the one that makes the gap real to an executive.

Working table — the five usually missing entirely

One row per conceptual application from volume 1's *usually missing entirely* list. `What you find instead` is the sentence that makes the gap real to an executive — mastered in the commission system, held in a spreadsheet, agreed by email. Signed by the Chief Enterprise Architect, with the Chief Risk Officer on exposure and reinsurance.

A-nn	Capability it serves	Verdict	What you find instead	Gap ID
A-17	Reinsurance administration	Absent	Two spreadsheets, two people	GAP-21

Step 8.9 — Write the gap register and engage the principles

Produce. `029 Gap` complete, each gap traced to a requirement and linked to the principles it engages.

You need. Candidates from steps 8.6 to 8.8; `006 Requirement`; `005 Principle`. Volume 1 appendix A does not help here: its seven maps join capability, process, role, data and application — none is a gap-to-requirement map. Derive the trace from the requirement each gap satisfies.

How.

1. Set `Gap Type`: `New` for something that must be created, `Replace` for something unfit, `Eliminate` for something that must stop.
2. Set `Affected Object` to the single object the gap is about, by ID. `GAP-01` points at `DEN-01`, `GAP-15` at `PTC-03`. Two gaps about one object is usually one gap.
3. Set `Traces To` to the requirement the gap satisfies. A gap with no requirement behind it has no sponsor.
4. Leave `Addressed By` blank until step 8.13: a gap addressed by a package that does not exist is the commonest fiction in a gap register.
5. Add rows to `R34 PRN-GAP` with `Engagement` of `Directly violated` or `Engaged`. Four customer masters is the direct negation of `DP-02 One Authoritative Source per Data Entity`. For gaps that came from an `UNGOVERNED CHANGE`, add the `GP-01` link too — the governance defect travels with the gap.

Signed by. `GB-01 Architecture Board`, which may refuse a design; the `Approved By` column records it.

Done when. Every row has `Gap Type`, `Severity`, `Affected Object` and `Traces To`, and `RPT11 Principle Impact` shows a non-zero `Gaps engaging it` against every Tier 1 principle you expected.

Shows up in. `RPT11 Principle Impact`, where `Direct violations` is the column an auditor reads first.

Goes wrong when. `R34 PRN-GAP` reads `Engaged` everywhere because `Directly violated` feels accusatory. Sixteen of Meridian's twenty-two principle-gap links are `Directly violated`, which is the honest number for an insurer at this stage.

Where it lands.

`029 Gap`

`ID | Name | Description | Domain | Gap Type | Severity | Affected Object | Traces To | Addressed By`

`Domain` is `Business`, `Data`, `Application`, `Technology` or `Realization` — the last for gaps in the practice itself, such as `GAP-13 No architecture repository`. `Gap Type` is `New`, `Replace` or `Eliminate`; `Severity` is `Critical`, `High` or `Medium`; `Affected Object` is one object ID, blank only for `Realization` gaps; `Traces To` is a requirement ID. Then `R34 PRN-GAP`: `Source (PRN) | Target (GAP) | Engagement`.

THE SENTENCE THAT DEFINES A BUILDING BLOCK

Take your three highest-severity gaps. If you cannot write the sentence, the gap is not yet understood — and the catalogue of blocks in appendix F cannot be filled from it.

What must this block do? One imperative sentence, with no product name in it.

What is the acceptance criterion somebody could answer yes or no to, without you in the room?

What does the block deliberately not do, so that the next architect does not grow it?

Part three — from gaps to building blocks

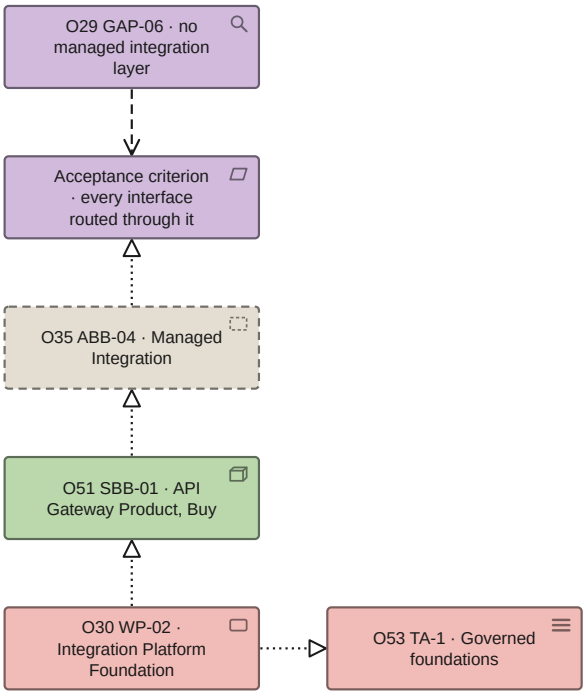

Figure 8.4 The Phase E ladder in ArchiMate 4, one chain worked all the way down. The gap influences the acceptance criterion, the building block realises it, the solution building block realises the block, and the work package realises both the solution and the transition architecture.

> **`055 BUILDING BLOCK` IS A COMPOSED VIEW, NOT A STORE**
>
> Its own header says so: rows are read from `035`, `044`, `045` and `046` at load, and nothing is entered in `055` directly. You author in the domain sheet and read in `055`, where `Held In` names the sheet a row came from.
>
> One quirk. In `R42 055-055` and `R48 051-055` the `Source Name` and `Target Name` lookup columns resolve to the block's `Domain` rather than its `Name`, because `055`'s name sits in the fourth column and the lookup takes the second. The IDs are right; the resolved names are not.

Step 8.10 — Author the architecture building blocks in their domain sheets

Produce. Fourteen building blocks per domain, each naming one mechanism that must exist. Each block is a `Grouping` in ArchiMate 4, realising the `Requirement` that is its acceptance criterion.

You need. `029 Gap`; the four architecture definition documents from Phases B, C and D; volume 1 chapter 13, the reference architecture, for the shape of the mechanisms.

Look it up. Volume 1, chapter 13, *The Reference Architecture: seven facets*, and *What ACORD does not give you*.

How.

1. Start from the gap. Ask what mechanism, stated without a product name, would make it not a gap. That is the building block.
2. Write it in the right domain sheet: business mechanisms in `O35` with `Capabilities Realised`; data, application and technology in `O44`, `O45` and `O46`, each with `Serves` naming the block above it.
3. Write `What It Must Do` as one imperative sentence with no technology in it. The repository's standard: "Resolve any presented party to one enterprise identity across all masters, without replacing any master."
4. Set `Closes Gap`. A block closing no gap is a mechanism nobody asked for, or a gap you have not raised — `RPT24 Building Block Coverage` reports twelve, every one a governance mechanism.
5. Leave `Work Package` and `Wave` alone; both are derived from `O30 Work Package`.

Signed by. Domain architect for the sheet; `GB-01 Architecture Board` approves the set.

Done when. Every gap of `High` severity or above has a block naming the mechanism that closes it, and every block has a `Serves` or `Capabilities Realised` value.

Shows up in. `RPT24 Building Block Coverage` for all four domains — `Domain | Ref | Building block | What it must do | Closes gap | Work package | Wave | Funded`, 66 rows — and `RPT19 Building Block Coverage`, which despite the identical name reports the fourteen *data* blocks only, with an eighth column carrying the acceptance criterion.

Goes wrong when. A block is written as a product. "API gateway product" is a solution building block and belongs in `O51`; "provide the single route by which applications exchange data, authenticated, logged and versioned" is the architecture building block.

Where it lands.

`O35 / O44 / O45 / O46 — then read in O55`

Each domain sheet: `ID | Name | What It Must Do | Serves (or Capabilities Realised) | Depends On | Acceptance Criteria | Closes Gap | Work Package | Wave`. ID prefixes are set by sheet and must not be mixed: `ABB-nn` in `O35`, `DBB-nn` in `O44`, `APP-BB-nn` in `O45`, `TEC-BB-nn` in `O46`.

A BLOCK WITHOUT AN ACCEPTANCE CRITERION IS NOT A BUILDING BLOCK

`G2 Build complete` tests every acceptance criterion with evidence, and requires a dated remediation for partial compliance. So write each criterion so somebody with access to the running system could answer `Yes` or `No` without asking an opinion: "95% of presented parties resolved in under 2 seconds; never auto-merges below a stated confidence; queues the remainder for a steward." Three short clauses, semicolon-separated, assess cleanly against `O59 Compliance Assessment`. Include the negative where it carries the design decision — "no direct database reads"; "no product change requires a code deployment" — and take any number from `O04 Measure`, so the gate and the benefit case use the same figure. A criterion that restates `What It Must Do` in the future tense is an intention, and `G2` has nothing to assess. `GB-05 Architecture Function` publishes the criteria; `RPT19 Building Block Coverage` carries them in its last column.

Step 8.11 — Record the dependencies between blocks

Produce. A dependency graph at block level, and the package-level verdicts that fall out of it.

You need. All four domain sheets; `R42 O55-O55`; `R47 WPK-WPK` once packages exist.

How.

1. For each block, ask which block must be accepted first. Write it in `Depends On` and as a row in `R42 O55-O55`. That is all the relationship carries: a block cannot be accepted before the block it depends on.
2. Walk the chains. `ABB-06 Channel-Agnostic Quotation` depends on `ABB-05` and `ABB-04`; `APP-BB-06` depends on `APP-BB-05` and `APP-BB-01`. One dependency seen from two domains.
3. After step 8.13, derive `R47 WPK-WPK` and set `Verdict`: `Clean` if the prerequisite finishes first, `Overlapping` if the windows overlap, `Impossible` if the dependent package finishes before its prerequisite delivers.
4. Put the derivation in `Derived via`, naming the block pair. "ABB-06 depends on ABB-05; APP-BB-06 depends on APP-BB-05" is why `WP-07` against `WP-08` reads `Impossible`.

Signed by. Chief Enterprise Architect.

Done when. Every `Impossible` verdict is either resequenced or carried into `O52 Implementation Factor` as a factor with a deduction, in chapter 9.

Shows up in. `RPT27 Programme Sequencing`, where the graph produces `Earliest feasible start` and `Slip months`. Meridian's reads a critical path of 62 months against 50 available.

Goes wrong when. Dependencies are recorded between packages rather than between blocks. Package dependencies are opinions; block dependencies are architecture, and they let you say *why* a sequence is impossible rather than only that it is.

Where it lands.

R42 O55-O55 · R47 WPK-WPK

`R42 O55-O55` : `Source (O55) | Target (O55) | Domain | Note | Key` , fifty-two rows. `R47 WPK-WPK` : `Source (WPK) | Target (WPK) | Verdict | Derived via | Key` .

Part four — solutions, packages and states

Step 8.12 — Decide sourcing for each solution building block

Produce. `O51 Solution Building Block` complete, with a sourcing recommendation per block.

You need. All architecture blocks; `AP-01 Reuse, then Buy, then Build` ; volume 1 chapter 14, technology providers, for what kind of thing each vendor sells; `O47 Reference Model Entry` for what the estate has.

Look it up. Volume 1, chapter 14, *Build versus buy, per capability — and the forgotten third option*, and *The one-platform trap*.

How.

1. Group architecture blocks into the solutions that would realise them. Meridian's fifty-six blocks resolve to twenty-nine solutions: one product often serves a business, an application and a technology block at once.
2. Set `Sourcing` in `AP-01` order — reuse an enterprise asset first, then `Buy` a proven product, then `Configure` an existing platform, and `Build` only with a written justification. Meridian's split is 17 `Buy` , 8 `Build` , 4 `Configure` .
3. Write `Realises` as block IDs across domains: `ABB-04, APP-BB-01, TEC-BB-05` for the API gateway. Set `Work Package` , or write `No work package` — the honest value for six of the twenty-nine, and what `RPT28 Solution Sourcing` exists to surface.
4. Use `Note` for the decision not yet taken. `SBB-01 API Gateway Product` carries "No product selected. Seven packages wait on this decision and it costs nothing to take" — worth more than the rest of the row. Then add rows to `R48 O51-O55` for each realisation and `R49 WPK-O51` for each delivery.

Signed by. Chief Information Officer owns `AP-01` ; `GB-01 Architecture Board` approves the recommendation; `GB-02 Investment Committee` funds it.

Done when. Every architecture block appears in a `Realises` list, and every `Build` has a reason that is not "no product exists".

Shows up in. `RPT28 Solution Sourcing` — `Ref | Solution building block | Sourcing | Realises | Work package | Funded | Note` . `Funded` should read `no` only where you intend it to.

Goes wrong when. Everything becomes `Build` because the team is capable, or `Buy` because procurement is easier than design. Volume 1 chapter 11 names the pattern that works: insurers buy the applications that master data and build the ones that mediate experience — and it reverses under pressure.

Where it lands.

O51 Solution Building Block

`ID | Name | What It Is | Sourcing | Realises | Work Package | Note | Provenance` . `Sourcing` carries `Buy` , `Build` or `Configure` ; reuse of an existing asset appears as `Configure` . Then `R48 O51-O55` and `R49 WPK-O51` , both carrying `Sourcing` .

Step 8.13 — Group blocks into work packages

Produce. `O30 Work Package` complete, each package closing named gaps and delivering named blocks.

You need. `O29 Gap` ; `O55 Building Block` ; `O51 Solution Building Block` ; a cost estimate per package; `O04 Measure` for benefit.

How.

1. Group by what must be bought, built or configured together — not by domain, not by organisational unit. `WP-02 Integration Platform Foundation` carries gateway, broker, catalogue and interface lifecycle because they are one procurement and one team.
2. Set `Wave` . Wave 1 is foundations with no business-visible change — a feature, and the hardest wave to fund.
3. Set `Start` and `End` as `YYYY-MM` , `Cost (Rm)` from the delivery estimate, and `Benefit (Rm p.a.)` only where it traces to a measure — chapter 9 step 9.5 makes that trace or removes the number.
4. Set `Owner` to an executive role, `Status` to `Candidate` , `Proposed` or `Approved` , and `Transition State` to the `O53` state it belongs to.
5. Set `Delivery Status` separately from `Status` : it was added under `CR-051` because a package that had not landed looked identical to one that had.
6. Link it up: `R28 GAP-WPK` with `Closure` , `R30 WPK-APP` with `Action` , `R29 WPK-DLV` for the deliverables.

Signed by. Package `Owner` accepts the scope; `GB-02 Investment Committee` holds funding at `G0`.

Done when. Every gap of `High` severity or above has an `R28 GAP-WPK` row, and every package has a cost, owner, wave and transition state.

Shows up in. `RPT09 Gaps and Roadmap`, where `Status` reads `Addressed`, and `RPT24 Building Block Coverage`, where `Funded` flips to `yes`.

Goes wrong when. `Closure` reads `Full` everywhere. Three of Meridian's twenty-one links are `Partial`, including `GAP-02` against `WP-04`, noted "Timeline depends on identity resolution" — and a partial closure recorded as full is how a gap gets marked closed while still open.

Where it lands.

`030 Work Package`

`ID | Name | Description | Wave | Start | End | Cost (Rm) | Benefit (Rm p.a.) | Owner | Status | Transition State | Delivery Status`. `Transition State` takes `TA-1`, `TA-2` or `TA-3`. `R28 GAP-WPK` carries `Closure` of `Full` or `Partial`; `R30 WPK-APP` carries `Action` of `Build`, `Change`, `Replace` or `Retire`.

Step 8.14 — Define transition architectures you can stop at

Produce. Three transition states, each with a window, an exit test and an honest statement of what remains broken. The view is an ArchiMate 4 implementation and migration view: one `Plateau` per state, with the `Work Package` elements that realise it.

You need. `030 Work Package` with `Transition State` populated; `R50 053-WPK`.

How.

1. Write each state as one the enterprise can stop at safely. If funding stopped the day it was reached, the insurer must still be operable and defensible.
2. Set `Window` as `YYYY-MM → YYYY-MM` and `Transition` as `Baseline → TA-1`, `TA-1 → TA-2`, `TA-2 → Target`.
3. Write `What Is True When Reached` in business language with no package IDs. Meridian's `TA-1`: "The estate is governed, integrable, supportable and access-controlled. No new capability, and no business-visible change."
4. Write `Exit Criteria` as separate testable clauses. "The Architecture Board has met, has a quorum rule, and has refused at least one thing" is the model: a body that has never refused anything has not been tested.
5. Write `Still Broken Here` — the column that makes the document honest. Name what a reader would assume is fixed and is not: "Four customer masters remain. Nothing has been merged or matched." Meridian's `TA-3` reads "Nothing, if it is reached", then names the six unfunded compliance blocks absent from the target state too.
6. Set `Standing` to what is true now. Meridian's `TA-2` reads "Running before TA-1 was reached", six packages worth R489.2m in flight against foundations that do not exist.

Signed by. `GB-01 Architecture Board` approves the states; `GB-04 Executive Committee` is the escalation point when one is declared reached without evidence.

Done when. Every exit criterion can be answered today as `Fails today`, `Not yet testable` or met, and every package sits in exactly one state.

Shows up in. `RPT29 Transition Readiness` — `State | Name | Window | Exit criterion | Standing`. Meridian's reads seventeen criteria across three states, all six `TA-1` criteria failing, `TA-2` already running.

Goes wrong when. `Still Broken Here` is left blank or hedged. Its value is that it stops a programme claiming a transition state delivers more than it does, and an architect who writes it honestly is believed about everything else.

Where it lands.

`053 Transition Architecture`

`ID | Name | Window | Transition | What Is True When Reached | Work Packages | Cost (Rm) | Exit Criteria | Still Broken Here | Standing`. Three states at R105.4m, R303.6m and R405.0m. Populate `R50 053-WPK` from the `Transition State` column of `030`.

WHAT IS STILL BROKEN AT EACH STATE

For each state, name the thing a reader would reasonably assume is fixed and is not. This is the column that makes the document believable.

`TA-1` — assumed fixed, actually not?

`TA-2` — assumed fixed, actually not? And is `TA-2` already running before `TA-1` was reached?

`TA-3` — what does the target architecture still not cover, funded or otherwise?

For each state, can the enterprise stop here safely if the money stops? If not, it is not a transition state.

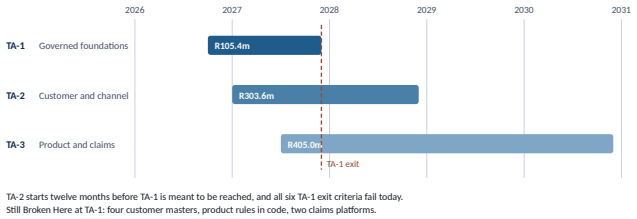

Figure 8.5 Meridian's three transition states against the calendar. The overlap is the finding: `TA-2` is already running against foundations `TA-1` has not delivered.

Step 8.15 — Put three options to the board, not one plan

Produce. `054 Architecture Alternative` — named options, costed, with one recommendation.

You need. `RPT27 Programme Sequencing`; `052 Implementation Factor` from chapter 9 step 9.2; the committed end date, and whether it is external.

Look it up. Volume 1, chapter 14, *Why a shortlist is an architecture decision*, and *And now it is also a regulatory filing*.

How.

1. Write the alternatives as responses to a stated problem. Meridian's five exist because the roadmap cannot be delivered in dependency order: 62 months of critical path against 50 available.
2. Always include the do-nothing option, named honestly. `A Hold the plan` keeps every date and accepts that three packages finish before their prerequisites deliver — and its `Fits 2030` reads `Nominally`, the most useful word on the sheet.
3. Fill `Committed Cost (Rm)`, `Benefit (Rm p.a.)`, `Ends`, `Deliverable In Order` and `Fits 2030`. Two yes-or-no columns and two numbers decide most board conversations.
4. Fill `Gives` and `Costs` as three clauses each, and make `Costs` specific. Option `C` defers claims modernisation, and its cost entry says it cuts the package with the second-best payback: the most expensive cut per rand saved.
5. Write `When To Choose It` as a condition on the outside world, not a preference: "Choose if the December 2030 date is internal rather than committed, and the Board can carry two more years of spend before net benefit." Then set `Recommended` to `Yes` on exactly one, and keep the options not chosen so they are not reopened later as though they were new.

Signed by. Chief Enterprise Architect recommends; `GB-02 Investment Committee` decides — explicitly in that body's `May Decide` column.

Done when. Exactly one option reads `Recommended Yes`, every option has a non-empty `Costs`, and the minute records which was chosen and why.

Shows up in. Nowhere automatically — `054` has no report sheet. Carry it into the Phase E pack and record the decision in `060 Governance Decision`.

Goes wrong when. Five options are presented and four are obviously wrong, which is one plan wearing a costume. Each should be choosable by a reasonable executive with a different attitude to risk, cost or date.

Where it lands.

054 Architecture Alternative

`ID | Name | What It Is | Committed Cost (Rm) | Benefit (Rm p.a.) | Ends | Deliverable In Order | Fits 2030 | Gives | Costs | When To Choose It | Recommended`. Meridian records five, `A` to `E`, from R240m to R814m, with `D Foundations first, then rebaseline` recommended.

THREE OPTIONS A REASONABLE EXECUTIVE COULD CHOOSE

Which option would a board member with a different attitude to risk choose, and why is it not yours?

Name the do-nothing option honestly. What does it actually accept?

For your recommendation, what is the condition in the outside world that would make it the wrong one?

Working table — the alternatives, side by side

One row per option, including do-nothing. Two numbers and two yes-or-no columns decide most board conversations; `Costs` goes on the back of the sheet in three clauses. Exactly one row reads `Yes` in the last column. Recommended by the Chief Enterprise Architect, decided by `GB-02`.

Option	Cost (Rm)	Benefit (Rm p.a.)	Ends	In order?	Recommended
A Hold the plan	240	61	2030-12	No	No

Part five — reading the result

Step 8.16 — Read the reports and issue the roadmap

Produce. `DLV-19 Architecture Roadmap`, issued from the repository rather than written beside it.

You need. Everything above; `O31 Deliverable`; `RPT09`, `RPT19`, `RPT24`, `RPT27`, `RPT28` and `RPT29`.

How.

1. Read `RPT09 Gaps and Roadmap` first. Every gap should carry a severity, a principle engaged, a work package, a cost, a benefit and `Status` of `Addressed`.
2. Read `RPT24 Building Block Coverage` for all four domains, and `RPT19 Building Block Coverage` for data with its acceptance criteria. The two share a name: `RPT24` has 66 rows across four domains, `RPT19` has 25 and covers data only.
3. Read `RPT29 Transition Readiness` and expect discomfort. Exit criteria that all read `Fails today` for the first state are a finding, not a formatting problem.
4. Read `RPT28 Solution Sourcing` for `Funded`, and `RPT27 Programme Sequencing` for `Slip months`, which chapter 9 turns into a plan.
5. Assemble `DLV-19`, recorded in `O31 Deliverable` against `ADM Phase E`, owned by the Chief Enterprise Architect.

Signed by. `GB-01 Architecture Board` approves `DLV-19`; `GB-02 Investment Committee` takes the alternative decision.

Done when. `RPT14 Deliverable Register` shows `DLV-19` with a status other than `Not started`, and every gap on `RPT09` reads `Addressed` or carries a written acceptance of the exposure.

Shows up in. `RPT14 Deliverable Register` and `RPT31 Governance Activity`.

Goes wrong when. The roadmap is written as a slide deck and the repository updated afterwards, or never. The repository is the roadmap; the deck is a view of it.

Where it lands.

031 Deliverable

`ID | Name | Type | ADM Phase | Work Package | Owner | Due | Status | Repository Location`. `DLV-19 Architecture Roadmap`, `ADM Phase E`, due `2028-03`, carried by `WP-01`. Beware the ID collision: `O34 Architecture Deliverable` also has a `DLV-19`, where it is the Regulatory Obligation Register. When you cite a deliverable ID, name the sheet.

THE HONEST LIMITS OF THIS CHAPTER

`O30 Work Package` carries `Cost (Rm)` and nothing in the repository records how any figure was derived: record the basis in `O52 Implementation Factor`. Every package carries a benefit and none carries a realisation date, logged as `RSK-08` in `O62 Delivery Risk`. Chapter 9 fixes both or deletes the numbers.

PHASE E — A REVIEWER'S QUESTIONS

Which gap in the register did the tool find, and which did you have to go looking for?

Which gap is addressed by a work package that does not exist yet?

Which transition state is already running before the one before it was reached?

Where did the cost figures come from, and is that basis written down anywhere?

THE CHAPTER IN TEN LINES

- Phase E turns differences into a funded programme. The gap register sets the quality of everything after it.
- The comparison tool takes five pasted inputs, not three: `10`, `11`, `10R`, `11R` and `12`. `00 How to Use` omits the two record sheets.
- The composite `Key` — sheet, record, attribute — is how every match in the tool works, and why a renumbered catalogue reports as wholly removed and re-added.
- Set `01 Config` first. The change log explains a difference only inside the window the two dates define.
- A design delta is a gap. An ungoverned change is a `GP-01` governance defect, and the count of them is the practice's health metric.
- A difference is not a finding until a named person classifies it on `50 Review` as Gap, Drift, Variance or Accepted.
- `Severity` arrives as `** set **` on purpose. Set it from the dated exposure, the measure that moves, the principle engaged and the gaps that wait on it.
- The tool is blind to what is missing from both sides. `RPT07 Capability Coverage`, the `O13` maturity delta and volume 1's five usually-missing capabilities fill the hole.
- Author building blocks in `O35`, `O44`, `O45` and `O46`; `O55 Building Block` only composes them. A block with no acceptance criterion is an intention, and `G2` has nothing to assess.
- Transition states must be states you can stop at. `Still Broken Here` is the column that makes the roadmap believable, and `DLV-19` is what you issue at the end.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_Architecture_Comparison_Tool.xlsx, sheets 00 How to Use, 01 Config, 10 Input A Baseline, 11 Input B Target, 12 Input Change Log, 30 Diff Records
- **2.** Meridian_Architecture_Comparison_Tool.xlsx, sheet 32 Diff Attributes
- **3.** Meridian_Architecture_Comparison_Tool.xlsx, sheet 50 Review
- **4.** Meridian_Architecture_Comparison_Tool.xlsx, sheet 60 Summary
- **5.** Meridian_Architecture_Comparison_Tool.xlsx, sheet 70 Gap Candidates
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheet O29 Gap
- **7.** Meridian_TOGAF_Metamodel.xlsx, sheet O05 Principle (GP-01, AP-01, DP-02, TP-02)
- **8.** The Open Group, TOGAF Standard, 10th Edition — ADM Phase E, Opportunities and Solutions https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 9

Migration Planning

A roadmap can be wrong quietly. A plan cannot: the moment you write a date against a project with a named lead, somebody can check. This chapter is about making the plan checkable before somebody else checks it.

Phase F takes the roadmap and turns it into something a delivery organisation can be held to [11]. The roadmap says what will change and roughly when. The plan says who is chartered to do it, in what order, with what money, against which assumptions, and what happens to the regulatory calendar while it is happening.

The difference matters because a roadmap can be wrong quietly and a plan cannot. The moment you write `Start` and `End` against a project with a named lead, somebody can check. Meridian Assurance Group — an invented South African composite insurer whose repository data is real within this workbook — is a useful case precisely because the check has already been run on it and it failed: the critical path is sixty-two months against fifty available [3].

BEFORE YOU START THIS PHASE

Volume 1 works through five of the judgements this chapter asks you to make. Read them before the first step.

- **Chapter 7, *The operating model*** — *10. The reporting calendar*: the dates the plan has to fit around, and who owns each one.
- **Chapter 10, *The information model*** — *Bitemporality, and why this is the hardest temporal problem you will meet*: why a policy history will not migrate as a flat extract.
- **Chapter 14, *Technology providers*** — *What a core replacement actually costs*: the cost shapes to test every `Cost (Rm)` against.
- **Chapter 15, *Regulation, risk and what the regulator inspects*** — *A calendar worth carrying*: the statutory dates a slipped data feed moves.
- **Chapter 12, *The application landscape*** — *The overnight batch, and why it still exists*: why the cutover window is a night, not a weekend.

Part one — from packages to projects

Step 9.1 — Shape work packages into projects

Produce. `O56 Project` complete: a fundable, staffable, chartered unit of delivery per row.

You need. `O30 Work Package` from chapter 8; `R47 WPK-WPK` with its verdicts; the delivery function's view of team size and availability.

How.

1. Start one project per work package, then look for the two cases that force a change.
2. Combine packages where separating them creates a dependency you would otherwise have to manage. Meridian's `PRJ-08 Analytics Platform` carries `WP-06` and `WP-12` together because model governance has nothing to observe without the lakehouse [1].
3. Create a project where no package exists but work does. `PRJ-12 Compliance Mechanisms` has `Work Packages` of `none` and carries the six unfunded solution blocks — retention, masking, consent, glossary, reference data and the registers.
4. Set `Months`, `Team` and `Lead`. A lead is a person's role, not a committee.
5. Set `Depends On` from `R55 O56-O56`, and set `Override` to `Yes` where you are deliberately ignoring a derived dependency.
6. Write `Why This Shape` for every row. One or two sentences on why this is a project rather than two, or half of one.

Signed by. Project `Lead` accepts the charter; `GB-02 Investment Committee` funds it at `G0`.

Done when. Every work package appears in exactly one project's `Work Packages`, every project has a `Lead` who exists, and no row has an empty `Why This Shape`.

Shows up in. `R54 O56-WPK`, where `Project Type` should match the project's own `Type`, and `RPT27 Programme Sequencing` once the dates move.

Goes wrong when. Sixteen packages become sixteen projects because that is the least work. Meridian's sixteen resolve to fifteen: two pairs combine and one project exists for work no package covers [1].

Where it lands.

056 Project

`ID | Name | Type | Work Packages | Start | End | Months | Team | Cost (Rm) | Benefit (Rm p.a.) | Lead | Depends On | Override | Why This Shape`

`Type` is `Enablement`, `Platform`, `Remediation`, `Core change` or `Decommissioning`. `Override` is `Yes` or `No`. `R54 056-WPK` links project to package and `R55 056-056` project to project, the latter carrying `Override` and the reason.

Draw the result as an ArchiMate 4 project view. Both a project and a work package are the Work Package element; the project aggregates the packages it carries, and `R55` becomes a Triggering relationship from prerequisite to dependant.

`WHY THIS SHAPE` IS THE COLUMN THAT SURVIVES A CHANGE OF CIO

Dates move, costs are re-estimated, leads are replaced. The argument for why `WP-07` and `WP-09` run as one project does not change, and it is the thing nobody can reconstruct eighteen months later. Meridian's `PRJ-10 Quotation and Channel` reads: "Two packages combined. The intermediary portal needs the quotation service; running them as one project removes a dependency rather than managing it." [1]

WHY THE PROGRAMME IS CUT THIS WAY

Take your three largest projects. If you cannot answer the first question, the project is the wrong shape.

Why is this one project rather than two? Write the sentence you would say to a new executive.

What does combining or splitting it remove from the dependency graph?

What would it cost to shape it the obvious way instead?

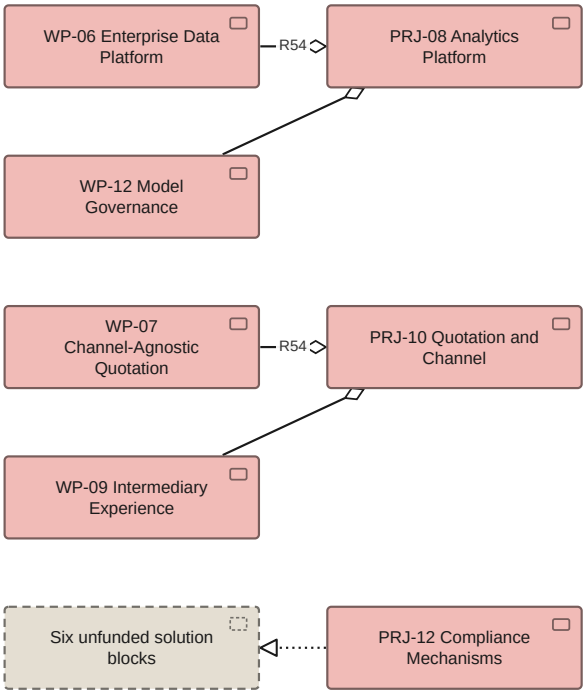

Figure 9.1 Sixteen work packages, fifteen projects. A project aggregates the packages it carries, so `PRJ-08` and `PRJ-10` each hold two and remove a dependency rather than manage it. `PRJ-12` aggregates nothing and realises six unfunded solution blocks instead. `R54 056-WPK` carries the link; `Why This Shape` carries the argument.

Step 9.2 — Record the implementation factors

Produce. `052 Implementation Factor` complete — everything that will shape delivery, each with the deduction that follows.

You need. Phase B to E findings; `O07 Constraint`; `O08 Assumption`; `RPT27 Programme Sequencing`; the three `Impossible` verdicts from `R47 WPK-WPK`.

How.

1. Set `Category` to `Issue`, `Risk`, `Driver`, `Constraint` or `Assumption`. Meridian records thirty factors, six of each of the first three [2].
2. Write `Factor` as a fact with a number in it: "The critical path is sixty-two months against fifty available to December 2030."
3. Write `Source` as the artefact the fact came from — `Solution Dependency Matrix`, `Technology Portfolio Catalogue`, `Consolidated Gaps, October 2027`.
4. Write `Deduction`, which is the whole point of the sheet. A factor with no deduction is an observation. `IF-01`'s deduction: "Wave 1 must be reconciled before Wave 2 is replanned. A package cannot be marked complete on an approval status."
5. Set `Affects` to the packages involved, and `Changes Sequence` to `Yes` only where the factor moves something. Six of Meridian's thirty do.

Signed by. Chief Enterprise Architect; the owner of each affected package acknowledges the deduction.

Done when. No row has an empty `Deduction`, and every `Changes Sequence` of `Yes` is reflected in `O56 Project` dates or recorded as an accepted exception.

Shows up in. `RPT27 Programme Sequencing` and in the `O54 Architecture Alternative` options, which exist because of factors `IF-02`, `IF-03` and `IF-04`.

Goes wrong when. The sheet becomes a risk register. A risk is something that might happen; a factor is something that is true and changes the plan. Four of Meridian's delivery risks are recorded as `Certain`, which means they are issues with a mitigation.

Where it lands.

`O52 Implementation Factor`

`ID | Category | Factor | Source | Deduction | Affects | Changes Sequence`

This is the sheet that records why the ideal sequence is not the real one. Constraints and assumptions cross-reference `O07` and `O08`; issues, risks and drivers are drawn from the Phase B to E findings.

Step 9.3 — Read the slip

Produce. A dependency-honest sequence, and a number for how far the current plan is from it.

You need. `R47 WPK-WPK`; `O30 Work Package` dates; `RPT27 Programme Sequencing`.

How.

1. Read `RPT27 Programme Sequencing` sorted on `Slip months` descending. `Slip months` is `Earliest feasible start` minus `Planned start`, and `—` means the package can start when it says it will.
2. Read `Depends on` and `Needed by` together on each slipping row. `WP-09` slips forty months, depends on `WP-02` and `WP-07`, and nothing depends on it — so the slip is absorbable.
3. Find the rows where something *does* depend on the slipping package. `WP-04` slips five months and is `Needed by` `WP-05` and `WP-10`, so its five months become somebody else's five months.
4. Identify the critical path. Meridian's is sixty-two months and runs through four packages: `WP-03`, `WP-04`, `WP-05`, `WP-10` [3].
5. Read the report's own footnote on the three `Impossible` dependencies: `WP-07` needs `WP-08`, `WP-09` needs `WP-07`, `WP-12` needs `WP-06` [3].

Signed by. Chief Enterprise Architect produces it; `GB-02 Investment Committee` is the audience.

Done when. Every package with a non-blank `Slip months` has either a new date in `O56 Project` or a recorded decision to proceed anyway.

Shows up in. `RPT27 Programme Sequencing`, where the summary block reads sixteen packages, ten would slip, critical path sixty-two months.

Goes wrong when. The slip is read as a scheduling problem. A forty-month slip on `WP-09` is not late delivery; it is a package whose prerequisites were never in the plan.

Where it lands.

`RPT27 Programme Sequencing`

`Ref | Work package | Wave | Planned start | Planned end | Earliest feasible start | Slip months | Depends on | Needed by`

Derived, not typed. `Earliest feasible start` comes from the dependency graph in `R47 WPK-WPK`, which in turn comes from the building block dependencies in `R42 O55-O55`. If a slip looks wrong, the error is in the block graph, not here.

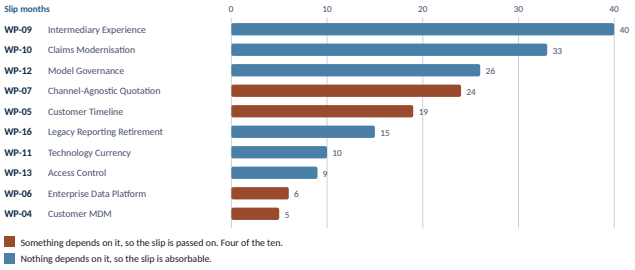

Figure 9.2 `RPT27 Programme Sequencing`, sorted on `Slip months`. The slip itself matters less than whether anything is `Needed by` the slipping package: four of these ten pass their slip straight on.

Step 9.4 — Decide what to do about each slip

Produce. A decision per slipping project, recorded where somebody will find it.

You need. Step 9.3; `O54 Architecture Alternative` and the option `GB-02 Investment Committee` chose; `O52 Implementation Factor`.

How.

1. For each slip take one of four actions, and only one: move the date, combine the projects, descope the dependent project explicitly, or override the dependency with a written reason.
2. Move the date where nothing downstream cares. That is most slips.
3. Combine where the dependency is really internal sequencing. `PRJ-10` combines `WP-07` and `WP-09` for exactly this reason.
4. Descope explicitly where the date cannot move. Write what the project will deliver *without* its prerequisite, in the charter, in advance. `IF-02`'s deduction names the alternative honestly: stating it is the only defence against half a package being delivered quietly.
5. Override only with a reason that beats the dependency. Meridian overrides twice, both on dated regulatory exposure: `PRJ-04` because three platforms are already past vendor support, `PRJ-05` because `STD-06` is a Current standard in breach with `GAP-17` open [1].

Signed by. `GB-01 Architecture Board` approves an override; `GB-02 Investment Committee` approves a descope, because it changes the benefit case.

Done when. Every `Override` of `Yes` in `R55 O56-O56` has a reason in `Note` naming the exposure that justifies it.

Shows up in. `O56 Project` dates, `R55 O56-O56 Override`, and `O60 Governance Decision` for anything the Board decided.

Goes wrong when. The dependency is overridden silently by moving the dependent project earlier and saying nothing. `WP-07` rating against a product catalogue that arrives twelve months later delivers quotation against rules in code, which is the problem it exists to solve [5].

Where it lands.

`R55 O56-O56`

`Source (O56) | Target (O56) | Override | Note | Key`

`Override` of `No` means the dependency stands and the dates honour it. `Override` of `Yes` means you have decided to run anyway, and `Note` carries the argument. Twelve of Meridian's fourteen project dependencies are `No`.

DECIDING WHAT TO DO ABOUT A SLIP

For your worst slip: does anything downstream depend on it? If nothing does, why is it a problem?

Which of the four actions are you taking — move, combine, descope, override — and who approves that one?

Where you are overriding, which dated exposure beats the dependency? Name the date.

Part two — money, increments and risk

Step 9.5 — Put a basis under every cost

Produce. `Cost (Rm)` on every project, with a recorded derivation.

You need. `O30 Work Package` costs; `O19 Contract` for anything already committed; `O23 Application Component` `Annual Run Cost (Rm)` for anything being retired; `O07 Constraint`.

Look it up. Volume 1, chapter 14, *Technology providers*, *What a core replacement actually costs* — the shapes to test a number against.

How.

1. Roll the package costs into the project. `PRJ-08` carries `WP-06` at R74.1m and `WP-12` at R18.7m, and reads R92.8m [1].
2. Separate committed from estimated. Spend already under contract is a different kind of number from a rough order of magnitude, and `O54 Architecture Alternative` uses `Committed Cost (Rm)` for exactly this reason.
3. Record the basis as an `O52 Implementation Factor` row where it shapes the plan. Nothing in the repository records how any cost figure was derived, which is a gap in the practice, not in the sheet.
4. Check the costs against the constraint. `CON-01 No technology budget increase before FY2028` has `Impact` of `Critical` against `WP-15`, which is why the largest package is a `Candidate` rather than approved.
5. Flag any project whose cost is large and whose architectural content is thin. `PRJ-15` carries R214.6m against nine months and a team of four, and is chartered as a feasibility study rather than a delivery project [1].

Signed by. Chief Financial Officer owns the numbers; `GB-02 Investment Committee` approves them at `G0`.

Done when. Every project cost reconciles to the sum of its packages, and every cost over R50m has a recorded derivation.

Shows up in. `RPT09 Gaps and Roadmap`, `Cost (Rm)` per gap, and the `Committed Cost (Rm)` column of whichever `O54` option was chosen.

Goes wrong when. A project carries a cost nobody can explain, which survives until the first external review and then takes the whole plan's credibility with it.

Where it lands.

> **`O56 Project` · `O52 Implementation Factor`**
>
> `Cost (Rm)` on `O56 Project` is the roll-up of its packages' `Cost (Rm)` from `O30 Work Package`. Where the basis of an estimate shapes the plan — scarce skills, a fixed licence, a supplier change window — record it as a factor with a `Deduction`, not as a comment.

Step 9.6 — Make every benefit trace to a measure, or delete it

Produce. `Benefit (Rm p.a.)` that an auditor can follow to a number somebody already reports.

You need. `O03 Objective`; `O04 Measure`; `R03 OBJ-MSR`; the executive who owns the measure.

Look it up. Volume 1, chapter 6, *How insurers make money*, *Reading the ratios* — the measures an insurer already reports, the only place a credible benefit lands.

How.

1. For each project's benefit, name the objective it serves and the measure that tracks that objective. The chain is `O03 Objective` to `O04 Measure` through `R03 OBJ-MSR`, and the relationship carries `Type` of `Leading` or `Lagging` and a `Reporting` frequency.
2. Check the measure has a `Baseline`, a `Current` and a `Target` with a `Unit` and a `Source System`. `MSR-02 Records requiring manual match` reads 640,000 against a target of 0, monthly, from `APP-21` [4].
3. Convert the measure movement into rands, and show the arithmetic. A benefit that cannot be expressed as a delta on a reported measure is not a benefit; it is a hope.
4. Record a realisation date. Meridian records R375.6m a year of claimed benefit and not one realisation date, logged as `RSK-08` with the mitigation "Record a realisation date per package before the next investment review" [6].
5. Delete any `Benefit (Rm p.a.)` that survives none of the above, and say in the plan that you deleted it.

Signed by. The measure's `Owner` from `O04 Measure`; Chief Financial Officer accepts the total; `GB-02 Investment Committee` confirms benefit at `G3`.

Done when. Every non-zero `Benefit (Rm p.a.)` traces through `R03 OBJ-MSR` to a measure with a source system, and carries a realisation date.

Shows up in. `RPT09 Gaps and Roadmap`, `Benefit (Rm)` per gap, and `RPT08 Traceability`, where the chain from requirement to objective to goal becomes visible.

Goes wrong when. Benefit is assumed to start when delivery ends. It never does, and `G3` is the gate that would have caught it — no package in Meridian's estate has ever passed `G3`, because the gate did not exist.

Where it lands.

004 Measure · R03 OBJ-MSR

`004 Measure`: `ID | Name | Objective | Unit | Baseline | Current | Target | Frequency | Source System | Owner`.

`R03 OBJ-MSR`: `Source (OBJ) | Target (MSR) | Type | Reporting`. `Type` is `Leading` or `Lagging`. A project whose only benefit measure is `Lagging` cannot be tracked inside its own delivery window, which is worth knowing before you promise quarterly reporting on it.

THE BENEFIT TRACE

For each project's benefit, which objective does it serve and which measure tracks that objective?

Show the arithmetic: what does the measure move from, to what, and what is that worth in rands?

Which benefit is tracked only by a `Lagging` measure, and what will you report inside the delivery window instead?

Working table — benefit, traced to a measure

One row per project. An empty arithmetic or realisation cell means the benefit comes out of `056 Project`, and the plan says so. The measure's `Owner` signs its own row, the Chief Financial Officer accepts the total, `GB-02` confirms it at `G3`.

Project	Objective	Measure	Arithmetic	Realisation date
PRJ-03	OBJ-02	MSR-02	640k → 0 matches, R18m p.a.	2029-09

Step 9.7 — Write the architecture increments

Produce. `066 Architecture Increment` — the readable version of the roadmap, for the people who will not read a Gantt.

You need. `056 Project` with dates; `053 Transition Architecture`; `R59 066-056`.

How.

1. Write each increment as what is *added* to the architecture, not as what each element looks like at each state. Increments do not map one to one to projects and should not.
2. Set `Lands` to the month the last contributing project finishes, and `Projects` to the contributors.
3. Write `What Is Added` as a list of mechanisms that will exist: "An API gateway carrying production interfaces; the four direct database reads removed; event streaming selected."
4. Write `What Becomes Possible` in the language of the business, and let it be unflattering where it is. `INC-1` reads "Nothing a customer notices. The estate becomes governable and supportable"; `INC-4` reads "The first business-visible increment, twenty-six months in" [8].
5. Use `Note` for the conditional ones. `INC-3 Compliant by mechanism` carries "Conditional. PRJ-12 has no budget, so this increment may never appear."
6. Populate `R59 066-056` with `Lands` carried on the relationship.

Signed by. Chief Enterprise Architect; `GB-04 Executive Committee` is the audience that most needs it.

Done when. Every project appears in at least one increment, and every increment has a non-empty `What Becomes Possible`.

Shows up in. Nowhere automatically — `066` has no report sheet. It is the slide that goes in front of the executive, and it should be the only slide.

Goes wrong when. Increments are drawn one per project, which reproduces the Gantt in prose and tells nobody anything. Six increments against fifteen projects is the right order of magnitude.

Where it lands.

`066 Architecture Increment`

`ID | Increment | Lands | What Is Added | Projects | What Becomes Possible | Note`

Meridian records six, `INC-1 Governed and supportable` landing 2028-10 through `INC-6 Product and claims` landing 2033-02. The second, `INC-2 Integrable`, has the most downstream dependants and is held up by an unmade product decision.

Draw it as an ArchiMate 4 migration view: one Plateau per increment, each Triggering the next, with the Work Package that realises it drawn against the increment it lands in. Nothing else belongs on that view.

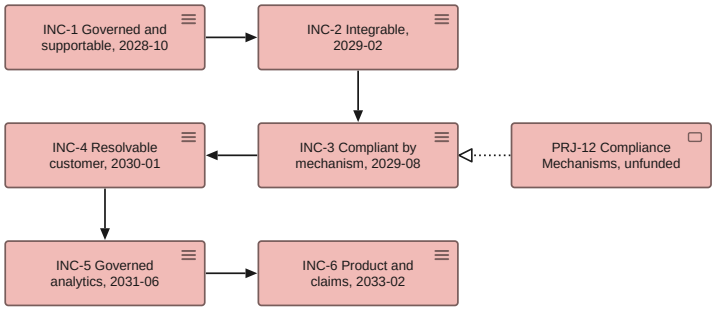

Figure 9.3 The six architecture increments as plateaus, each triggering the next. This is the version of the roadmap to put in front of an executive. `INC-3` is realised by one project, `PRJ-12`, which has no budget — so everything from `INC-3` onward is conditional. `R59 O66-O56` carries `Lands` on the relationship.

Step 9.8 — Record delivery risk with a consequence, not an adjective

Produce. `O62 Delivery Risk` complete, each risk owned and mitigated.

You need. `O56 Project`; `O52 Implementation Factor`; `O08 Assumption`; the project leads.

Look it up. Volume 1, chapter 14, *Technology providers*, *The one-platform trap* — the delivery risk that reads lowest on a register and costs most.

How.

1. Set `Likelihood` to `Certain`, `Likely` or `Possible`, and `Impact` to `Severe` or `Major`.
2. Where `Likelihood` reads `Certain`, stop and reclassify. A certain risk is an issue, and Meridian carries four of them — including `RSK-01 No gateway product selected`, which blocks seven projects and has no licence cost.
3. Write `Consequence` as what happens to the plan, with a number: "R96.4m of customer identity work is gated on a meeting."
4. Write `Mitigation` as an action with an owner, not a posture. "Escalate to the Executive. Reconstitute the Council with a smaller quorum if necessary."
5. Set `Project` to the project at risk, or `All` where it is programme-wide. `RSK-03`, unrecorded delivery capacity peaking at ninety people, is `All`.

Signed by. Risk `Owner`; `GB-02 Investment Committee` accepts the residual position.

Done when. No risk has an empty `Mitigation`, no `Certain` risk is still called a risk, and every risk names a project or `All`.

Shows up in. `RPT31 Governance Activity`, which lists governance items by `Kind` with their owner and status.

Goes wrong when. The register fills with risks whose mitigation is "monitor". A mitigation that nobody has to do anything about is a sentence, not a control.

Where it lands.

`062 Delivery Risk`

`ID | Risk | Project | Likelihood | Impact | Consequence | Mitigation | Owner | Status`

Eight risks in Meridian, three `Certain`, six `Severe`. `RSK-07` is the instructive one: `WP-02` refused at `G2` stops seven dependent projects, so the correct governance action has a programme-wide consequence — and the mitigation is to re-baseline rather than refuse outright.

Step 9.9 — Tie assumptions and constraints to the packages they hold up

Produce. Two relationship sheets populated, so that an invalidated assumption has a known blast radius.

You need. `O08 Assumption`; `O07 Constraint`; `O30 Work Package`; `R35 ASM-WPK`; `R06 CON-WPK`.

How.

1. For each assumption, add an `R35 ASM-WPK` row per package that rests on it, and set `If Invalidated` to `Stop`, `Delay` or `Rescope`.
2. Read the `Stop` rows as a list. `ASM-02 Architecture Board is established in Q4 2026` against `WP-01` reads `Stop`, with the note "Without a Board there is nothing to establish."
3. Note where the same assumption gives different answers. `ASM-02` against `WP-02` reads `Delay`, with "Platform work proceeds but ungoverned" — which is a decision disguised as a note.
4. Set `Validate By` on each assumption in `O08`, and treat a passed date with no validation as an issue.
5. For each constraint, add an `R06 CON-WPK` row with `Impact` of `Critical`, `High`, `Medium` or `Low`, and a `Mitigation` that is specific to that package.

Signed by. Assumption `Owner` from `O08 Assumption`; Chief Enterprise Architect for the relationship set.

Done when. Every assumption has at least one `R35 ASM-WPK` row, every `Stop` row has a named validation owner and date, and every constraint-package pair has a `Mitigation`.

Shows up in. `RPT12 Integrity Check`, where a relationship sheet with no rows against a populated object sheet shows up as a links-per-record problem.

Goes wrong when. Assumptions are listed once in the Statement of Architecture Work and never linked to anything. An assumption with no `R35 ASM-WPK` row cannot invalidate anything, which is why nobody notices when it does.

Where it lands.

> **`R35 ASM-WPK · R06 CON-WPK`**
>
> `R35 ASM-WPK`: `Source (ASM) | Target (WPK) | If Invalidated | Note`. Values are `Stop`, `Delay` and `Rescope`.
>
> `R06 CON-WPK`: `Source (CON) | Target (WPK) | Impact | Mitigation`. `CON-01 No technology budget increase before FY2028` carries `High` against `WP-04` and `WP-06`, and `Critical` against `WP-15`.

Part three — data migration, and the insurer's year

In an insurer the data migration is usually the critical path, and it is usually planned last. Policy history and claims history have to survive the move intact, and volume 1 chapters 9 and 10 explain why that is harder than it sounds: an insurer needs both time axes at once — valid time, when a fact was true in the world, and transaction time, when the insurer knew it [12]. A migration that collapses the two produces a target store that can answer what is true today and cannot answer what was believed on the day a claim was declined.

Step 9.10 — Plan each migration as its own piece of work

Produce. `O39 Data Migration` complete, each movement with an approach, a quality gate and a package that carries it.

You need. `O21 Logical Data Component` and `O22 Physical Data Component`; `O20 Data Entity` with `Quality` and `Mastering Style`; the work package that funds it; volume 1 chapters 9 and 10.

Look it up. Volume 1, chapter 10, *The information model, Bitemporality, and why this is the hardest temporal problem you will meet* — why policy and claims history need two time axes through the move.

How.

1. One row per movement from one or more source stores to a target store. Name the stores by their `O22` IDs, not by nickname.
2. Write `Approach` as the method and the rule it turns on: "Match and link. Deterministic on national identifier, probabilistic on name, date of birth and contact."
3. Write `Quality Gate` as a measurable condition that must be met *before* cutover, and set `Gate Status` honestly. All six of Meridian's read `Not met` [7].
4. Write `Risk` as the specific thing that will bite. `MIG-01` reads "Roughly 640,000 records cannot currently be matched. The 95% target is an estimate."
5. Record the migrations that are not migrations. `MIG-03 Model register establishment` has no source system: "The first task is finding out how many models are in production."
6. Set `Wave` and `Work Package`, and check the wave matches the package's own.

Signed by. Chief Data Officer; the target store's `Owner Role` from `O22`.

Done when. Every target store in the target architecture has a migration row or a recorded reason it needs none, and every row has a `Quality Gate` and a `Gate Status`.

Shows up in. `RPT17 Master Data` and `RPT22 Portfolio Data Risk`, and in `O62 Delivery Risk` where `RSK-06` records four data entities with no surviving master if the recorded dispositions execute.

Goes wrong when. The migration is treated as a task inside the project rather than a plan with its own gate. In an insurer the FAIS and POPIA retention obligations travel with the records, so `MIG-05` is a regulatory event if it loses an advice record, not a data quality issue.

Where it lands.

`O39 Data Migration`

`ID | Name | Sources | Target | Entities | Volume | Approach | Quality Gate | Gate Status | Wave | Work Package | Risk`

Six migrations in Meridian. The largest, `MIG-01 Customer identity resolution`, moves 2.6 million source records to 1.9 million resolved identities across four party masters, and no master is decommissioned.

Draw each migration as an ArchiMate 4 implementation and migration view: the source and target stores from `O22` as Artifacts, the migration itself as a Work Package with an Access relationship to each, and the quality gate as a Requirement the package has to realise.

Step 9.11 — Map source to target, entity by entity

Produce. `O40 Source To Target Mapping` — one row per entity per migration, with the quality issue that has to be fixed before the rule can run.

You need. `O39 Data Migration`; `O20 Data Entity`; `O37 Data Attribute` with its `Personal Information` flag; the data steward for each source.

Look it up. Volume 1, chapter 9, *The conceptual data model*, *The spine: a party, a contract, a thing* — the three entities to map first, and the cardinality that carries meaning.

How.

1. One row per entity per source. `MIG-01` has six rows, four of them `DEN-01 Party` from the four masters.
2. Write `Transformation Rule` as the rule, including the ones that are really modelling corrections. `STM-03` reads "Membership is not a party. A member resolves to a party; the membership stays with the scheme."
3. Write `Quality Issue To Resolve First`, which is the column that decides whether the migration lands: "Identifier absent or malformed on roughly 18% of funeral book records."
4. Flag the rules that are new behaviour rather than transformation. `STM-05` creates `DEN-02 Customer` from a derivation because "Customer is created by no function today, so the rule is new."
5. Carry the lawful basis with the record where a partner agreement limits retention, as `STM-04` does.
6. For any entity with history, state in the rule which time axis the target preserves. A policy version or a reserve series that arrives as a current-state row has been destroyed, not migrated.

Signed by. Chief Data Officer; `GB-03 Data Governance Council` designates the owner of each target entity.

Done when. Every entity in every migration's `Entities` list has at least one mapping row, and every mapping row has a `Quality Issue To Resolve First` or an explicit "none known".

Shows up in. The `Gate Status` on `O39 Data Migration`, which cannot move to met while any mapping row's quality issue is open.

Goes wrong when. The mapping is written by the integration team from the source schema. The hard rows — a member who is not a party, a customer nobody creates today, a reserve series rather than a reserve — are modelling decisions, and they belong to the architect.

Where it lands.

`O40 Source To Target Mapping`

`ID | Migration | Entity | Source | Target | Transformation Rule | Quality Issue To Resolve First`

Eighteen rows in Meridian. The last column is the one that decides whether a migration lands, and it is the column most plans leave empty.

WHAT MUST SURVIVE THE MOVE

For each entity with history, which question must the target store still answer as at a past date?

Who tests that, with what evidence, before the quality gate is called met?

Which records will not resolve automatically, and who works them by hand during cutover?

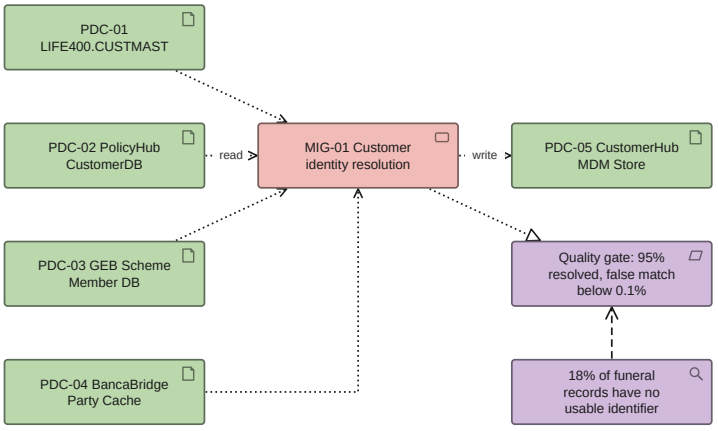

Figure 9.4 One migration, end to end. The work package reads four source stores, writes one target, and has to realise a quality gate written as a requirement. The assessment beneath the gate is why `Gate Status` reads `Not met`, and it sits in `Quality Issue To Resolve First` on `040`.

Step 9.12 — Fit the plan to the insurer's year

Produce. A sequence whose cutovers fall in windows the business will actually give you.

You need. Volume 1 chapters 6 and 7, which carry the calendar; `018 Product` for renewal shape; `019 Contract` for treaty dates; the actuarial and finance calendars.

Look it up. Volume 1, chapter 7, *The operating model*, *10. The reporting calendar* — the month-end, quarter-end and statutory dates that decide which weeks you can have.

How.

1. Mark the fixed dates first. Monthly close, quarterly returns to the Prudential Authority, quarterly actuarial valuation, half-year and annual results with IFRS 17 disclosures, and the annual own risk and solvency assessment [13].
2. Mark reinsurance renewal at 1 January and often 1 July, and work backwards. Treaty terms set what the policy and claims systems must record, so a change to either inside the three months before a renewal competes with the renewal itself.
3. Mark open enrolment for group schemes, during which the group business cannot absorb a membership or billing change.
4. Mark the actuarial close and year-end. Any migration touching policy, claim or ledger history is unavailable in that window, which removes roughly one quarter in four from the migration calendar.
5. Place each cutover in what is left, then check the result against `056 Project` dates and move the dates rather than the calendar.
6. Record each calendar constraint as an `052 Implementation Factor` row with `Category` of `Constraint` and a `Deduction`.

Signed by. Chief Financial Officer and the Head of Actuarial for the reporting windows; the line executive for a renewal or enrolment window.

Done when. No cutover falls inside a reporting, renewal, actuarial close or enrolment window, and every exception has a named approver.

Shows up in. `056 Project` `Start` and `End`, and `052 Implementation Factor` rows with `Changes Sequence` of `Yes`.

Goes wrong when. The plan assumes twelve usable months a year. It has about seven, and the ones it loses are the ones a programme most wants: January, because of renewals, and the two months around year-end.

Where it lands.

O52 Implementation Factor · O56 Project

Each calendar window becomes a factor with `Category` of `Constraint`, a `Deduction` naming the months lost, and `Affects` listing the packages whose cutover falls in it. The dates then move in `O56 Project`, not in a separate plan document.

Window	When	What it blocks	Why
Monthly close	First working week	Ledger, subledger and reporting change	Numbers in flight
Quarterly return	Month after quarter end	Anything feeding the Prudential Authority templates	A statutory date
Actuarial valuation	Quarterly, heaviest at year end	Policy, claim and reserve history	Reproducibility as at a date
Reinsurance renewal	1 January, often 1 July	Treaty terms, cession logic, bordereau formats	The treaty sets what the systems must record
Group open enrolment	Scheme-specific, usually Q4	Membership, billing, benefit configuration	Members are transacting
Year-end and results	Two months around the financial year end	Any migration touching history	Audit and disclosure

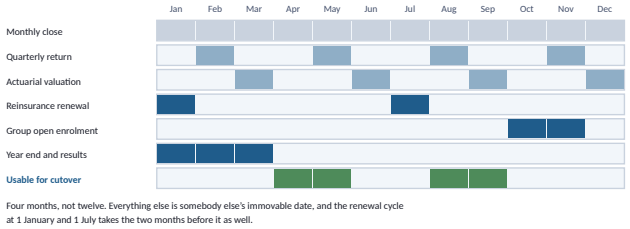

Figure 9.5 An insurer's year, from the point of view of somebody who needs a cutover window. The calendar is not negotiable, so the plan moves instead.

A MISSED DATA FEED MOVES A STATUTORY DATE

Volume 1 chapter 7 makes the point and it belongs in the migration plan: the pipelines feeding the actuarial and finance cycle are production systems with service levels, not analytics plumbing. A migration that degrades one of them during a quarter end is not a delivery issue. It is a conversation with a supervisor.

Step 9.13 — Assemble the plan and take it through `G0`

Produce. `DLV-20 Implementation and Migration Plan`, approved and funded.

You need. Everything above; `O31 Deliverable`; `O58 Governance Gate`; `O57 Governance Body`.

How.

1. Assemble the plan from the repository: `O56 Project`, `O52 Implementation Factor`, `O66 Architecture Increment`, `O62 Delivery Risk`, `O39 Data Migration` and the sequencing report.
2. Check what `G0` tests before you submit: scope, acceptance criteria inherited from the building blocks, dependencies and exclusions all stated. The Board refuses a charter whose criteria were written rather than inherited.
3. Carry the `O54 Architecture Alternative` decision into the plan as its first page, so the plan is visibly the chosen option rather than the only one.
4. Take it to `GB-02 Investment Committee`, which holds `G0` funding approval and `G3` benefit confirmation, and which cannot judge architecture compliance — a project it funds still passes `G1` and `G2` [9].
5. Record the outcome in `O60 Governance Decision` and update `O31 Deliverable` `Status` on `DLV-20`, due `2028-06`, `ADM Phase` `F` [10].
6. Name the exclusions in the plan itself. Meridian's life platform replacement sits outside the Statement of Architecture Work while `PRJ-15` carries R214.6m against it, and a plan that does not say so is misleading.

Signed by. `GB-02 Investment Committee` at `G0`; Chief Enterprise Architect owns the deliverable.

Done when. `RPT14 Deliverable Register` shows `DLV-20` past `Not started`, every project has a `G0` outcome, and every unfunded project is listed as unfunded with a named owner for the exposure.

Shows up in. `RPT14 Deliverable Register`, `RPT31 Governance Activity`, and `O33 Architecture Contract`, which chapter 10 uses to hold delivery to the plan.

Goes wrong when. The plan is approved with `PRJ-12` still at zero budget and nobody records that the six compliance mechanisms are therefore absent from every transition state, including the target.

Where it lands.

031 Deliverable

`DLV-20 Implementation & Migration Plan`, `ADM Phase F`, owner Chief Enterprise Architect, due `2028-06`, carried by `WP-01`, `Repository Location` `Repository/PhaseF`.

The same ID collision applies as in chapter 8: `O34 Architecture Deliverable` has its own `DLV-20`, the Architecture Governance Log. Name the sheet when you cite the ID.

WHAT PHASE F HANDS TO PHASE G

A funded project list with leads and dates. A sequence that honours the dependency graph or records why it does not. A benefit case that traces to measures somebody already reports. A migration plan with quality gates that are not yet met and are named. And a set of acceptance criteria, inherited from the building blocks, that `G1` and `G2` will test one project at a time. Chapter 10 is what happens when a project arrives at `G1` with a design that does not comply.

PHASE F — A REVIEWER'S QUESTIONS

Which project would you drop first if the budget fell by a fifth, and what breaks when you do?

Which cost over R50 million has a derivation somebody other than you could explain?

Which benefit has a realisation date, and who reports the measure it rests on?

Which cutover falls in a close, renewal, valuation or enrolment window — and who approved the exception?

THE CHAPTER IN TEN LINES

- Phase F charters the work. One project per work package, except where combining removes a dependency or where work exists that no package covers.
- `Why This Shape` on `O56 Project` is the column that survives a change of CIO. Write it for every row.
- `O52 Implementation Factor` records why the ideal sequence is not the real one. A factor with no `Deduction` is an observation.
- Read `RPT27 Programme Sequencing` on `Slip months`, then on `Needed by`. A slip nobody depends on is absorbable; a slip with dependants is passed straight on.
- Four responses to a slip and only four: move the date, combine the projects, descope explicitly, or override with a reason that beats the dependency.
- Cost rolls up from `O30 Work Package`. Record the basis as a factor, and separate committed from estimated.
- `Benefit (Rm p.a.)` is fiction unless it traces through `R03 OBJ-MSR` to a measure with a source system and a realisation date. Meridian claims R375.6m a year and records no realisation date at all.
- `O66 Architecture Increment` is the readable roadmap: what is added, and what becomes possible. Six increments against fifteen projects is the right order of magnitude.
- Tie assumptions to packages through `R35 ASM-WPK` with `If Invalidated`, or nobody notices when one fails.
- The migration is usually the critical path. `O39 Data Migration` carries the quality gate; `O40 Source To Target Mapping` carries the quality issue that decides whether it lands; and the insurer's calendar leaves about four usable cutover months a year.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets O56 Project, R54 O56-WPK and R55 O56-O56
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheet O52 Implementation Factor
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT27 Programme Sequencing
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets O04 Measure and R03 OBJ-MSR
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheet O54 Architecture Alternative
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **7.** Meridian_TOGAF_Metamodel.xlsx, sheets O39 Data Migration and O40 Source To Target Mapping
- **8.** Meridian_TOGAF_Metamodel.xlsx, sheets O66 Architecture Increment and R59 O66-O56
- **9.** Meridian_TOGAF_Metamodel.xlsx, sheets O57 Governance Body and O58 Governance Gate
- **10.** Meridian_TOGAF_Metamodel.xlsx, sheet O31 Deliverable
- **11.** The Open Group, TOGAF Standard, 10th Edition — ADM Phase F, Migration Planning https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **12.** Martin Fowler, "Bitemporal History" (valid time and record time) https://www.martinfowler.com/articles/bitemporal-history.html
- **13.** SARB Prudential Authority, "Selected South African Insurance Sector Data", March 2026 (quarterly return cycle); Moonstone, "COFI at the heart of the FSCA's 2026 regulation plan", 6 July 2026 https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-selected-south-african-insurance-sector-data/2026/Selected%20South%20African%20Insurance%20Sector%20March%202026.pdf

CHAPTER 10

Implementation Governance

The phase where architecture either refuses something, or admits it cannot.

Phase G is where architecture either has teeth or does not. Everything before it produced documents. This phase produces refusals, dispensations with expiry dates, and signed evidence that a thing was built the way it was designed. If the practice cannot refuse, nothing earlier in the method mattered.

The repository you were handed is a worked case of what that looks like when it has never been done. `058 Governance Gate` carries a note on its own header row: none of its four gates existed before Phase G, forty-four interfaces reached production with no design gate, and four work packages passed their end date with no build gate [1]. `059 Compliance Assessment` holds thirteen verdicts, all `Non-compliant`, all taken at one gate, all dated 2027-11 [3]. `RPT30 Compliance Position` totals it in three cells: assessed 13, non-compliant 13, compliant 0 [5]. And until version 1.23 of the workbook, four architecture contracts in `033` claimed `Compliant` while the assessment behind them said otherwise [2].

Meridian Assurance Group, the insurer in the repository, is invented. The failure pattern is not.

This chapter builds two gates that do not work yet — G1, the design gate the repository names as the one that was missing, and G3, which no package in the estate has ever passed — and gives you the instrument that makes a refusal survivable: the architecture contract.

> **BEFORE YOU START THIS PHASE**
>
> Phase G is where the regulator and the architecture practice meet. Volume 1 carries the obligations; this chapter carries the gates.
>
> - **Chapter 15, *Regulation, risk and what the regulator inspects*** — *What the regulator actually inspects*: what your gate evidence has to survive.
> - **Chapter 15** — *Joint Standard 1 of 2024: vendor selection becomes a regulatory filing*: why a sourcing decision now needs a gate and a file.
> - **Chapter 5, *Who does what*** — *Risk, compliance and legal*: the three statutory control functions step 10.13 connects the gates to.
> - **Chapter 14, *Technology providers*** — *And now it is also a regulatory filing*: what a shortlist has to show before G1 will pass it.
> - **Chapter 16, *Modelling insurance in ArchiMate*** — *Motivation: from a rule to the system that satisfies it*: the chain a gate walks when it asks what a criterion came from.

What you are building in this phase

Step	Produce	Lands in
10.1	Four gates with holders	`058 Governance Gate`
10.2	An architecture contract per package	`033 Architecture Contract`, `R33 ACO-WPK`
10.3	A charter that passes G0	`030 Work Package`, `055 Building Block`
10.4	A design review that can refuse	`059 Compliance Assessment` at `G1`
10.5	One verdict per acceptance criterion	`059`, `R56 059-055`
10.6	Contract status derived, not asserted	`033` `Compliance Status`, `Status Basis`
10.7	The two governance reports read correctly	`RPT30`, `RPT31`
10.8	A dispensation that expires	`061`, `060`, `R57 060-061`
10.9	A decision-rights boundary that holds	`057 Governance Body`
10.10	A G3 that tests benefit, not delivery	`058`, `030`
10.11	Funding checked before the gate	`RPT24`, `RPT19`
10.12	Traceability checked at the gate	`RPT32 Requirement Traceability`
10.13	The statutory control functions wired in	`057`, `011 Role`, `063`
10.14	A gate calendar that runs monthly	`R33` `Review Cadence`, `RPT31`

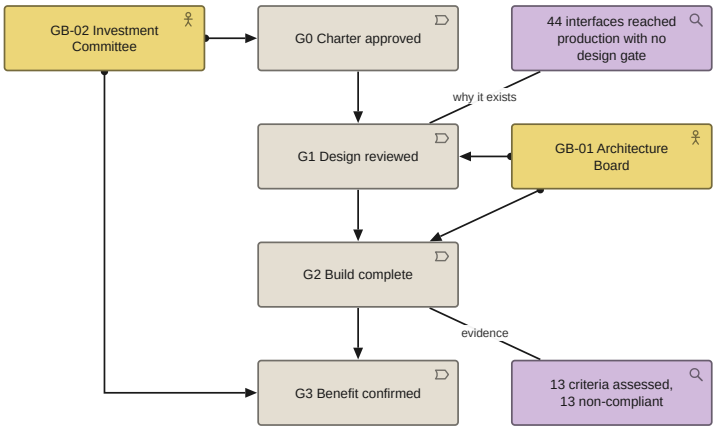

Figure 10.1 The four gates in `058 Governance Gate` as events in the delivery sequence, each triggering the next, with the body assigned to it: money gates to `GB-02`, architecture gates to `GB-01`. The two assessments are the evidence state today. `G1` and `G3` are the two that do not yet work. Meridian Assurance Group is invented.

The gates

Step 10.1 — Constitute the four gates as records, with a named holder each

Produce. Four rows in `058 Governance Gate`, each with a body that can refuse at it.

You need. The governance framework deliverable (`034 Architecture Deliverable`, `DLV-16 Architecture Governance Framework`, currently `Draft` with the Group Executive Committee as approver), the bodies in `057 Governance Body`, and volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, for why a financial services board cannot delegate this silently.

Where it lands. `058 Governance Gate` — fill `ID`, `Name`, `When`, `What It Tests`, `Why It Exists`, `Held By`.

`When` is a point in the delivery sequence, not a date: *before work starts*, *before build*, *before production*, *after deployment*. `What It Tests` must be a testable proposition, not a topic. `Why It Exists` is the thing that happened because the gate did not — in Meridian's case, forty-four unmanaged interfaces and four packages past their end date. `Held By` names the body that can say no.

Draw the four as an ArchiMate 4 implementation and migration view: each gate an Event Triggering the next, and each body a Business Actor with an Assignment relationship to the gates it holds. `Held By` is that assignment, which is why an empty one leaves the gate unowned.

How.

1. Copy `058`'s four rows as your starting set. G0 charter approved, G1 design reviewed, G2 build complete, G3 in production and benefit tracked.
2. Set `Held By` to `GB-02 Investment Committee` for G0 and G3, and `GB-01 Architecture Board` for G1 and G2. Money gates to the funders; architecture gates to the architects.
3. Write `What It Tests` as a sentence you could lose an argument about.
4. Record the adoption as a dated row in `060 Governance Decision` — Meridian's is `DEC-G01`, dated 2027-11-06 [6].
5. Raise the metamodel change request that introduces the sheet if it does not exist; Meridian's was `CR-055`.

Signed by. `GB-01 Architecture Board`, with the framework itself approved by the Group Executive Committee.

Done when. Every gate has a non-empty `Held By`, and that body's `May Decide` column in `057` names the gate it holds.

Shows up in. `RPT31 Governance Activity` as a `Decision` row with status `Approved`.

Goes wrong when. `Held By` holds a body *name* rather than a body `ID`. In the repository as delivered it reads `Architecture Board`, not `GB-01`, and `RPT12 Integrity Check` reports `058 Governance Gate` as `ISOLATED` with zero links [10]. A rename then breaks the join and nothing tells you. Add the `ID` alongside the name, or accept that the gate sheet is a document rather than a model.

Step 10.2 — Write the architecture contract for each work package

Produce. One row in `033 Architecture Contract` per package in delivery, with a signature and an expiry.

You need. `030 Work Package`, the building blocks in `055` that the package delivers, the delivery partner, and the sponsoring executive.

Where it lands. `033 Architecture Contract` — fill `ID`, `Name`, `Work Package`, `Parties`, `Signed`, `Expiry`, `Compliance Status`, `Last Review`, `Owner`, `Project`, `Status Basis`.

`Parties` is two names separated by a slash: the delivering organisation and the body that holds the gates. Meridian uses `Group Technology / Architecture Board`, `Group Data / Architecture Board`, `Security / Architecture Board`, `Distribution / Architecture Board`.

`Signed` is the date a human signed, not the date the package started. `Expiry` is the date the contract stops governing anything — set it past the package end date, not on it. `Project` carries the `O56` project that actually does the work, because a package can be delivered by a project that was not the one it was planned under. `Status Basis` is one sentence saying where `Compliance Status` came from.

How.

1. For each package with a delivery team, create the contract row. Meridian has seven, `ACO-01` to `ACO-07`, covering `WP-02`, `WP-03`, `WP-11`, `WP-13`, `WP-04`, `WP-09` and `WP-07` [2].
2. Set `Compliance Status` to `Not yet assessable` until the package reaches a gate. Four of Meridian's seven read that, with `Status Basis` reading *Package has not reached its end date*.
3. Link the contract to the package in `R33 ACO-WPK` and set `Review Cadence`. All seven of Meridian's read `Monthly`.
4. Set `Owner` to the role that chairs the gate — `ROL-14 Architecture Board Member` in the repository — and not to the delivery lead.
5. Note the exception in `R33`'s `Note` column where one exists; `ACO-03` carries *Observation raised on resilience testing evidence*.

Signed by. Both parties named in `Parties`. A contract with one signature is a memo.

Done when. Every package with a `Delivery Status` other than `Not started` has a contract, and `WP-01` either has one or has a recorded reason why not. It does not: `RPT30` shows `CA-01` against `WP-01` with `Contract status` reading `no contract` [5].

Shows up in. `RPT30 Compliance Position`, right-hand column. A package with an assessment and no contract is an assessment nobody agreed to be bound by.

Goes wrong when. The contract is treated as a procurement document and filed with the vendor agreement. It is not a commercial instrument. It is the record that lets you refuse a build without the conversation becoming a negotiation about scope.

WHAT THE CONTRACT MAKES REFUSABLE

In one sentence: what can you refuse with this contract that you could not refuse without it?

Who are the two parties, and has each actually signed — or is one of them a distribution list?

Which package in delivery has no contract, and what is the recorded reason?

Step 10.3 — Make G0 refuse a charter whose criteria were written rather than inherited

Produce. A charter test with one rule that does most of the work.

You need. `O55 Building Block` with its `Acceptance Criteria` column populated, and the package record in `O30`.

Where it lands. `O30 Work Package` — read `ID`, `Name`, `Wave`, `Start`, `End`, `Cost (Rm)`, `Benefit (Rm p.a.)`, `Owner`, `Status`, `Transition State`, `Delivery Status`.

You are not filling this sheet at G0. You are testing it against `O55`. The charter's acceptance criteria must be the same strings as the `Acceptance Criteria` of the building blocks the package delivers.

How.

1. List the building blocks the package claims to deliver, from `O55`'s `Work Package` column.
2. Copy their `Acceptance Criteria` into the charter verbatim. Do not improve the wording.
3. Refuse the charter if any criterion in it does not appear in `O55`, or if any `O55` criterion for those blocks is missing from it.
4. Refuse the charter if `Benefit (Rm p.a.)` is non-zero and no date is given for when the benefit starts. `WP-01` carries `0`, honestly; most of the others carry a figure and no start.
5. Record the rule once, as a decision, rather than arguing it per charter. Meridian's is `DEC-G02`: acceptance criteria are inherited from building blocks, never written fresh [6].

Signed by. `GB-02 Investment Committee`, which holds G0.

Done when. The charter's criteria list and the `O55` criteria for its blocks are the same set.

Shows up in. `RPT24 Building Block Coverage` — every block in the package appears with a `Work package` value and a `Funded` value of `yes`.

Goes wrong when. A project writes its own acceptance criteria because the architecture's were inconvenient. The decision record in `O60` states the consequence plainly: a charter with its own criteria drifts from the architecture within a quarter.

Step 10.4 — Build the G1 design review, the gate that was missing

Produce. A design verdict per criterion, before any build starts.

You need. The design, `O55`'s `Acceptance Criteria`, the standards in `O32 Standard` that apply, and `O06 Requirement` filtered to the package.

Look it up. Volume 1, chapter 12, *The application landscape*, *Integration patterns that actually show up* — the four patterns a design review has to ask about by name.

Where it lands. `O59 Compliance Assessment` — fill `ID`, `Work Package`, `Building Block`, `Acceptance Criterion`, `Gate`, `Verdict`, `Evidence`, `Recorded In`, `Assessed`.

Set `Gate` to `G1`. `Evidence` at G1 is a design artefact and a statement about it, not a test result — *the sequence diagram routes every call through the gateway*, not *44 of 50 interfaces are unmanaged*. `Recorded In` names the deliverable where a reader can check you.

Draw the package as an ArchiMate 4 requirements realisation view before the review sits: the building block as a Grouping aggregating one Requirement per acceptance criterion, and the design element that realises each. A criterion with nothing realising it is the first refusal.

How.

1. Take the package's criteria from `O55` and create one `O59` row per criterion with `Gate` set to `G1`.
2. For each, read the design and set `Verdict` to `Compliant`, `Non-compliant` or `Partial`. A `Partial` needs a dated remediation or it is `Non-compliant`.
3. Where the design departs deliberately, stop and go to step 10.8. A departure is a dispensation, not a verdict.
4. Name the deliverable in `Recorded In` — `ADD Application Architecture`, `Technology Standards` and `Application Communication` are the values the repository uses.
5. Link each row to its block in `R56 O59-O55`, carrying `Verdict` and `Gate`.
6. Refuse the build if any `Must` requirement for the package has a `Non-compliant` design verdict and no dispensation.

Signed by. `GB-01 Architecture Board`. Its `May Decide` column says it outright: *holds G1 and G2, approves dispensations with an expiry, refuses a design or a build* [4].

Done when. Every `O55` criterion for the package has a `G1` row. In the repository as delivered, none does: all thirteen `O59` rows carry `Gate` = `G2` [3]. Adding the `G1` rows is the single highest-value edit available to you in this phase.

Shows up in. `RPT30 Compliance Position`, which lists criterion-level verdicts. Today it shows thirteen rows and no `G1` among them.

Goes wrong when. G1 is run as a slide review with an attendance list. The output of G1 is rows in `O59`, not minutes. If the review produced no rows, it did not happen.

THE DEPARTURE YOUR NEXT DESIGN REVIEW WILL FIND

Name the departure you expect to find, and the criterion it fails.

Is it a refusal or a dispensation — and what makes it one rather than the other?

Which `Must` requirement has a non-compliant design and no dispensation behind it?

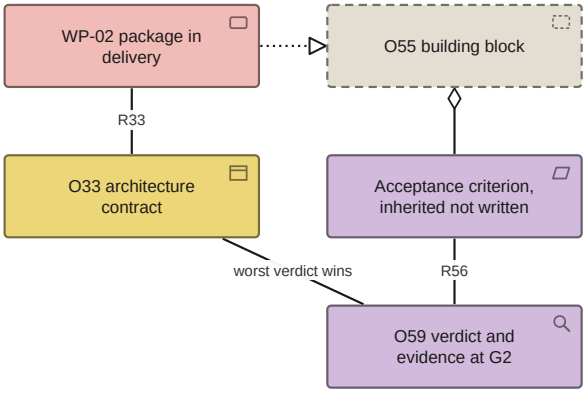

Figure 10.2 How a contract's compliance status is derived rather than asserted. The criterion belongs to the building block and is inherited, never rewritten; `O59` holds one verdict per criterion, linked through `R56` ; `O33` takes the worst verdict and records where it came from in `Status Basis` .

Assessing compliance

Step 10.5 — Assess one acceptance criterion at a time, never one project at a time

Produce. A verdict row per criterion at G2, with evidence a sceptic could check.

You need. The delivered thing, the `G1` rows from step 10.4, and the evidence sources named in `Recorded In` .

Where it lands. `O59 Compliance Assessment` — same columns, with `Gate` set to `G2` and `Assessed` set to the month of assessment.

`Evidence` at G2 is a count or a measurement. The repository's are worth copying as a style: *44 of 50 interfaces are unmanaged*; *Twenty batch files remain. Claim settlement takes 7.2 days of process time against a 48-hour promise*; *23 of 24 data entities are owned by a role with no actor performing it*.

How.

1. Create one row per criterion. `WP-02` has eight in the repository, `CA-02` through `CA-09` , because `WP-02` delivers eight criteria across four building blocks.
2. Measure. Write the number in `Evidence` . If you cannot produce a number, the verdict is `Non-compliant` because the criterion is untestable as written, and that is a finding against the criterion, not the builder.
3. Name the dependency where one criterion gates others. `CA-07` carries it: *This is the single criterion the other six depend on*.
4. Set `Assessed` to the month. Meridian's thirteen all read `2027-11` .
5. Link in `R56 O59-O55` .

Signed by. `GB-01 Architecture Board` .

Done when. The count of `O59` rows for the package equals the count of its `O55` criteria.

Shows up in. `RPT30` 's three summary cells. Assessed, non-compliant, compliant.

Goes wrong when. Somebody writes one row per project with a verdict of *mostly compliant*. Count what that costs you. `WP-02` at project level is one amber row in a steering pack. `WP-02` at criterion level is eight non-compliant rows, of which one — no gateway product selected — blocks the other six and has, per `RPT31` , no licence cost [6]. The second version tells you what to do on Monday. The first tells you to schedule another meeting.

Step 10.6 — Derive the contract's compliance status instead of asserting it

Produce. A `Compliance Status` in `O33` that no one can set by hand.

You need. The `O59` rows for the package and a rule for combining them.

Where it lands. `O33 Architecture Contract` — update `Compliance Status` , `Last Review` , `Status Basis` .

The rule: worst verdict wins. Any `Non-compliant` criterion makes the contract `Non-compliant` . All `Compliant` makes it `Compliant` . No assessed criteria and no gate reached makes it `Not yet assessable` . `Status Basis` records which assessment produced it.

How.

1. For each contract, gather its package's `O59` rows.
2. Apply the rule and write the result. Three of Meridian's seven read `Non-compliant` ; four read `Not yet assessable` .
3. Write `Status Basis` as a sentence with a gate and a date — *Assessed at G2, November 2027. Every criterion failed* — or *Package has not reached its end date*.

4. Set `Last Review` to the date of the assessment, and leave it empty where none has happened. Three of Meridian's seven have an empty `Last Review`, which is correct and should stay empty.
5. Raise the change request if the sheet previously allowed a hand-typed status. Meridian's is `CR-062`, *Derive contract compliance status from assessment*, approved 2027-11-20.

Signed by. `GB-01 Architecture Board`.

Done when. No contract carries a `Compliance Status` that its `O59` rows do not support.

Shows up in. `RPT30`'s `Contract status` column, which reads straight off `O33`. The repository's own note on that report records what the correction found: four contracts previously claimed `Compliant`, including those for `WP-02` and `WP-03`, both of which failed every criterion [5].

Goes wrong when. The status is set at the start of a package to reflect intent. Intent is what `Signed` records. `Compliance Status` records measurement.

Step 10.7 — Read the two governance reports and act on the gap between them

Produce. A one-page governance position for the Board pack.

You need. `RPT30 Compliance Position` and `RPT31 Governance Activity`.

Where it lands. `RPT30 Compliance Position` — Columns `Ref`, `Work package`, `Building block`, `Acceptance criterion`, `Verdict`, `Evidence`, `Contract status`, with three summary cells below the body.

How.

1. Read `RPT30`'s summary first: assessed, non-compliant, compliant. Meridian's reads 13, 13, 0.
2. Filter `RPT30` to `Non-compliant` and sort by `Work package`. The package with the most rows is the one to take to the Board, not the one with the largest budget.
3. Read `RPT31`'s `Kind` column. It holds three kinds in one sheet: `Decision`, `Change request` and `Risk`. Meridian has seven, five and eight.
4. Filter `RPT31` to `Status` = `Outstanding`. Two decisions sit there: `DEC-G06` (refuse `WP-02` at G2 or re-baseline it) and `DEC-G07` (fund `PRJ-12` or not).
5. Put both numbers on one slide. Thirteen non-compliant criteria and two undecided decisions is a governance position. Either number alone is a status update.

Signed by. Chief Enterprise Architect, presented to `GB-01`.

Done when. Every `Outstanding` row in `RPT31` appears on the next Board agenda with a named decision owner.

Shows up in. `RPT31` itself, whose closing note is the finding: every escalation trigger in the governance model is live today and none has been escalated [6].

Goes wrong when. `RPT31` is read as an activity log and reported as *seven decisions taken*. Seven approved and two outstanding is the number, and the two outstanding are the chapter.

Dispensations

Step 10.8 — Grant a dispensation that expires on a date, or do not grant it

Produce. A dated, owned, expiring exception, recorded in two sheets and linked.

You need. Principle `GP-02` (*Compliance is Assessed and Dispensations are Time-Bound*, Governance, Tier 1, owned by the Architecture Board), the design departure, and the thing whose delivery would make the departure unnecessary.

Where it lands. `O61 Governance Change Request` — fill `ID`, `Title`, `Against`, `Type`, `Why`, `Status`, `Expires`, `Consequence`.

`Against` is the project, package or `Repository` the request is raised against. `Type` is one of `Dispensation`, `Scope`, `Structure`, `Attribute` or `Compliance` — the five values the sheet uses. `Expires` is a year and month. `Consequence` says what happens on the expiry date if nothing has changed.

Use this template. Every row of it maps to a column.

Field	Column	Meridian's worked example, CR-G01
What is being allowed	`Title`	Dispensation: point-to-point file transfer in `PRJ-04`
Against what	`Against`	`PRJ-04`
Kind	`Type`	`Dispensation`
Why it cannot comply now	`Why`	`PRJ-04` must replace the MFT server before the gateway exists
Which principle it departs from	`Why` , second clause	`AP-02` managed interfaces
Expiry	`Expires`	`2029-06`
What ends it early	`Consequence`	Expires when `PRJ-03` delivers
What happens if nothing changes	`Consequence`	The dispensation lapses and the interface is non-compliant
Who granted it	`O60` row via `R57`	`DEC-G04` , Architecture Board, 2027-11-13
Named executive owner	`O30` `Owner` of the affected package	Chief Information Officer

How.

1. Create the `O61` row. Refuse to create it without a value in `Expires` .
2. Create the matching `O60 Governance Decision` row with `Date` , `Decision` , `Body` , `Status` , `Why` and `Affects` .
3. Link them in `R57 O60-O61` .
4. Write `Consequence` as two sentences: what ends the dispensation early, and what the state becomes if the expiry arrives first.
5. Put the expiry date in the gate calendar from step 10.14, not in somebody's diary.

Signed by. `GB-01 Architecture Board` . `O57` gives it this power explicitly and withholds the power to waive a regulatory control objective — read that limit before granting anything that touches `O63` .

Done when. No row in `O61` with `Type` = `Dispensation` has an empty `Expires` . The sheet's own header note states the rule: a dispensation without an expiry date is a permanent exception [7].

Shows up in. `RPT31 Governance Activity` , as a `Change request` row with `Status` = `Granted with expiry` .

Goes wrong when. The dispensation is granted verbally at a meeting and the `O61` row is never written. Six months later the departure is the design, and the only record that it was ever an exception is in the memory of two people, one of whom has left.

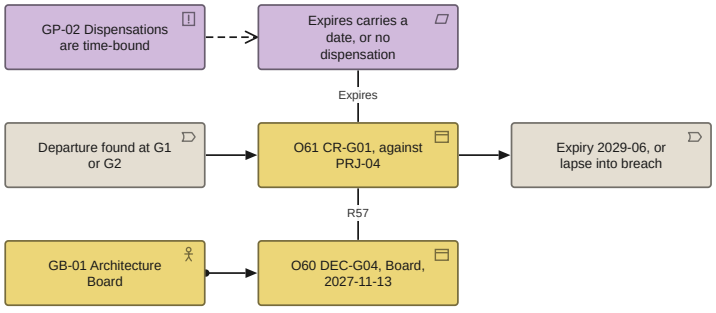

Figure 10.3 The dispensation, from the principle that forbids an open-ended one to the date it lapses. A departure found at a gate triggers the `O61` row; the Board's `O60` decision is joined to it by `R57` ; `Expires` is the whole mechanism. Without it the exception is the design.

BEFORE YOU GRANT A DISPENSATION

What date does it expire on, and what would end it earlier?

What does the estate become on the expiry date if nothing has changed — and who has been told that?

Does this departure touch a regulatory control objective in `O63` ? If so, who may actually grant it, because the Board may not?

Step 10.9 — Hold the decision-rights boundary, and fix it when the Executive crosses it

Produce. A decision-rights table that produces an escalation rather than a shrug.

You need. `O57 Governance Body`, columns `May Decide` and `May Not Decide`, and the `Standing` column, which is the one that matters.

Look it up. Volume 1, chapter 5, *Who does what*, *Risk, compliance and legal* — whose mandate the Executive is crossing when it decides an architecture question.

Where it lands. `O57 Governance Body` — fill `ID`, `Body`, `Cadence`, `Membership`, `May Decide`, `May Not Decide`, `Standing`.

`May Not Decide` is the load-bearing column. `GB-01`'s reads: cannot approve funding, cannot change a work package budget, cannot waive a regulatory control objective. `GB-02`'s reads: cannot judge architecture compliance; a project it funds still passes G1 and G2. `GB-04`'s reads: should not decide architecture — if it does, the Board has failed.

How.

1. For each body, write `May Not Decide` before `May Decide`. The exclusions are harder and more useful.
2. Fill `Standing` with what is true, not what is intended. Meridian's five rows are a lesson in this: the Board *is constituted on paper and has never refused anything*; the Data Governance Council *has never reached quorum*, with ten requirements and R96.4m gated on it; the Executive Committee has *never been used, because nothing has escalated — which given thirteen non-compliant criteria is itself the finding*.
3. When the Executive Committee decides an architecture question, do not contest the decision. Record it as an `O60` row with `Body` = `Executive Committee`, then raise an `O61` row of `Type` = `Structure` against `Repository` whose `Why` names the `O57` boundary it crossed and whose `Consequence` is *the Architecture Board's refusal is not operative*.
4. Take that pair to the next `GB-01` meeting as a single agenda item: either the Board asks for the decision back, or `O57`'s `May Decide` for `GB-01` is amended to match reality.
5. Amend one or the other. Leaving both is the failure.

Signed by. Group Executive Committee, which approves the governance framework `DLV-16`.

Done when. Every architecture decision in `O60` has a `Body` whose `May Decide` column covers it, or an `O61` row explaining why not.

Shows up in. `RPT31`, where decisions carry a `Who / owner`. An architecture decision owned by `Investment Committee` is the symptom.

Goes wrong when. The architect treats the crossing as a political defeat and says nothing. The repository's own wording is the better posture: *if it does, the Board has failed* is a statement about the Board's design, not about the Executive's behaviour. Fix the design.

WHEN THE DECISION WAS TAKEN SOMEWHERE ELSE

What was the last architecture decision taken outside the Architecture Board, who took it, and what does `O57` say about that?

Are you asking for the decision back, or amending `May Decide` to match reality? One or the other.

Which escalation trigger is live today and has never been escalated?

Step 10.10 — Make G3 test benefit, which no package has ever reached

Produce. A G3 assessment with a benefit realisation date and a closed gap.

You need. `O30 Work Package` (`Benefit (Rm p.a.)`, `End`, `Delivery Status`), `O29 Gap` (`Status`), and the measure in `O04 Measure` that the benefit is read from.

Look it up. Volume 1, chapter 6, *How insurers make money*, *Two results, not one* — which result a benefit claim asserts, which decides who signs G3.

Where it lands. `O59 Compliance Assessment` — create rows with `Gate` = `G3`. `Acceptance Criterion` at G3 is the benefit statement and the gap closure, not the build criteria. `Evidence` is a reading from `O04 Measure`'s `Current` column against its `Target`.

How.

1. Read `O30`'s `Delivery Status`. Four of Meridian's sixteen packages read *Past end date, not evidenced* — `WP-01`, `WP-02`, `WP-03`, `WP-13`. Those four are G3 candidates by date and failures by evidence.
2. Create a `G3` row per package with two criteria: the benefit is being realised at the rate in `Benefit (Rm p.a.)`, and the gap in `O29` the package addressed reads `Closed` with evidence.

3. Refuse G3 where the benefit start date does not exist. It does not exist anywhere in `O30` — `CR-G04` in `O61` records exactly this, and its `Consequence` reads: the benefit curve overstates the early years by assuming benefit begins when delivery ends [7].
4. Close `CR-G04` before you run a single G3, or every G3 verdict is measured against a date nobody set.
5. Set the gap's `Status` to `Closed` only from a G3 `Compliant` verdict, never from a delivery report.

Signed by. `GB-02 Investment Committee`, which holds G3 and `May Decide` benefit confirmation.

Done when. At least one package has a `G3` row in `O59`. In the repository, none does — `O58` says why: *No package in the estate has ever passed this gate, because it did not exist* [1].

Shows up in. `RPT09 Gaps and Roadmap`, where a gap `Status` of `Closed` becomes defensible, and `RPT31` where `CR-G04` moves from `Raised` to decided.

Goes wrong when. G3 is run as a project closure review. Project closure asks whether the work finished. G3 asks whether the money arrived.

Checks to run before you sit down at the gate

Step 10.11 — Check that every building block in scope is covered and funded

Produce. A funding position per package, before the gate rather than after it.

You need. `RPT24 Building Block Coverage` and `RPT19 Building Block Coverage`. These are two different sheets with the same name — a trap worth naming once. `RPT24` covers all fifty-six building blocks across the business, data, application and technology domains, and has columns `Domain`, `Ref`, `Building block`, `What it must do`, `Closes gap`, `Work package`, `Wave`, `Funded`. `RPT19` is titled *Data Building Block Coverage* on its own first row, covers the fourteen data building blocks only, and adds `Serves a business block` and `Acceptance criteria`.

Where it lands. `RPT24 Building Block Coverage` — read `Funded` and `Work package` together. `Funded = no` always pairs with `Work package = No work package` and `Wave = Unscheduled`; the three say one thing three times.

How.

1. Filter `RPT24` to the package in front of you and confirm every block it claims appears.
2. Filter `RPT24` to `Funded = no`. Meridian has twelve of fifty-six, and zero of them are in the business domain.
3. Read `RPT24`'s closing note, which explains why that distribution matters: a business block with no funding is a capability the business asked for and did not pay for, and there are none. The twelve unfunded blocks are technical and data blocks, which is a different conversation [8].
4. Cross-read `RPT19` for the data domain. Five of fourteen data blocks have no work package, and three serve no business block at all — `DBB-07 Retention and Disposal`, `DBB-08 Consent and Purpose Registry` and `DBB-13 Reference Data Management`.
5. Raise the unfunded blocks as gaps if they are not already. `CR-G03` in `O61` does precisely this, and its `Consequence` is the reason: until accepted, retention, masking and consent remain invisible to portfolio governance.

Signed by. `GB-02 Investment Committee` for the funding position; `GB-01` for the coverage.

Done when. No block in the package reads `Funded = no`, or each that does has an `O61` row against it.

Shows up in. `RPT28 Solution Sourcing`, which also carries a `Funded` column, and `RPT09 Gaps and Roadmap` once the unfunded blocks become gap records.

Goes wrong when. Somebody reads `RPT19` as the whole picture because the tab name matches. Fourteen blocks instead of fifty-six is a four-times understatement of the portfolio.

Step 10.12 — Check requirement traceability before you assess anything

Produce. A list of requirements the gate cannot test, produced before the gate sits.

You need. `RPT32 Requirement Traceability`.

Look it up. Volume 1, chapter 16, *Modelling insurance in ArchiMate*, *Motivation: from a rule to the system that satisfies it* — the chain drawn, so a broken trace is visible.

Where it lands. `RPT32 Requirement Traceability` — read `Ref`, `Requirement`, `Source`, `Priority`, `Building block`, `Work package`, `Traced`, `Ever tested`. The last two are yes/no.

How.

1. Filter `RPT32` to the package. Any row with `Traced = no` is a requirement the gate has no criterion for.
2. Count `Ever tested = no`. Across all seventy-six requirements in the repository the count is seventy-six: the summary cell reads `Ever tested 0` [9].
3. For each `Must` requirement with `Traced = no`, either bind it to a building block (chapter 12, step 12.7) or record that it is out of scope for this package.

4. Do not fix the whole register before the gate. Fix the rows for the package in front of you.
5. Report the count honestly in the gate paper. Twenty-three of seventy-six requirements reach nothing at all.

Signed by. Chief Enterprise Architect.

Done when. Every `Must` requirement mapped to the package has a matching `O59` row.

Shows up in. `RPT32`'s summary block: requirements, reaching nothing, ever tested.

Goes wrong when. The gate assesses the criteria it has and declares the package compliant. It is compliant with what was testable, which is not the same claim.

THE CRITERION YOU COULD NOT MEASURE TODAY

Which acceptance criterion in your portfolio could you not measure if you were asked this afternoon?

Is that a finding against the criterion's wording or against the thing that was built?

How many `Must` requirements for the package in front of you reach nothing at all?

The insurance overlay

An insurer's architecture governance sits inside a governance system the regulator already inspects. You do not get to design it; you get to connect to it. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, sets out the instruments — the Insurance Act and its prudential standards, the statutory control functions including the Head of Actuarial Function, Joint Standard 1 of 2024 on material outsourcing and its thirty-day notification, and what an on-site inspection asks for [11][12]. Read it there. This step says which repository columns are the answer when the question arrives.

Step 10.13 — Connect the architecture gates to the statutory control functions

Produce. A governance map the Prudential Authority and the FSCA can follow from board-approved framework to evidence.

You need. Volume 1, chapter 15 for the obligations; `O57 Governance Body`, `O11 Role`, `O63 Control Objective`, `O34 Architecture Deliverable`.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *What the regulator actually inspects* — the eight things asked for, in the order they are asked.

Where it lands. `O57 Governance Body` — add rows for the bodies the statutory framework already requires, and fill `Standing` honestly for each.

Regulatory expectation	Repository evidence	State today
Board-approved governance framework	`O34 DLV-16 Architecture Governance Framework`, `Approver` = Group Executive Committee	`Lifecycle Status = Draft`
Board-approved terms of reference	`O34 DLV-17 Architecture Board Terms of Reference`	`Draft`
Obligation register maintained	`O34 DLV-19 Regulatory Obligation Register`, `Approver` = Risk Committee	`Baselined`
Decisions recorded and traceable	`O60 Governance Decision`, `02 Change Log`	7 decisions, 172 logged changes
Control objectives mapped to controls	`O63 Obligation`, `Control`, `Evidence Today`	12 rows, 7 with no evidence
Head of actuarial control, head of risk, head of compliance, internal audit	`O11 Role`	**Not present.** `O11` has 27 roles and none of the four
Material outsourcing and material change	no sheet holds the arrangement or the notification	**Not modelled**
Independent assurance over the architecture function	`O57`	**Not present.** No audit body row

How.

1. Add the four statutory control functions to `O11 Role` with `Authority Level = Statutory`. The repository has one such row today, `R0L-16 Information Officer`, for POPIA and PAIA.

2. Add an `O57` row for the Risk Committee, which `O34` already names as the approver of `DLV-19` but which has no body record, and one for internal audit as an independent assurance body whose `May Decide` is empty and whose `May Not Decide` says it does not decide architecture — it reports on whether the gates ran.
3. For each of the four control functions, record in `O57`'s `Membership` for `GB-01` whether they attend the Architecture Board. Today `GB-01`'s membership is the Chief Enterprise Architect, four domain architects, a CIO delegate and a CISO delegate. No risk, compliance or actuarial voice sits on the gate that approves designs touching reserving and conduct.
4. Add `O63` rows for material outsourcing and material change. The sheet's `Regulatory Source` column today holds `COFI Bill`, `FAIS`, `POPIA s.4`, `POPIA s.11`, `POPIA s.14`, `POPIA s.72`, `Joint Standard 1 of 2023`, `IFRS 17`, `FICA` and `Insurance Act` — and nothing for Joint Standard 1 of 2024. Volume 1, chapter 15 has the obligation; the repository has no row for it.
5. Point the regulator at four columns and no slides: `O63 Evidence Today`, `O59 Evidence`, `02 Change Log Approved By`, and `O60 Date`. Those four answer *what must be true*, *is it true*, *who approved the change* and *when was it decided*.

Signed by. Group Executive Committee for the framework; Chief Risk Officer for the control objectives; `GB-01` for the gate connection.

Done when. Every row in `O63` with `Evidence Today` = `None` appears in a Board paper with a named owner and a date. Seven of twelve read `None` today: five have no mechanism at all and two have a mechanism in breach [13].

Shows up in. `RPT31 Governance Activity` for the decisions, and in whatever inspection request arrives. An inspection asks for evidence, and `O63 Evidence Today` is the only column in the repository that answers it in the regulator's own terms.

Goes wrong when. The architecture governance framework is written as though the insurer had no governance system before it. It had one, with statutory roles and a board that already approves frameworks. An architecture board that does not connect to it is a fifth wheel with its own minutes.

> Two cautions on the overlay. First, whether a particular arrangement is *material* outsourcing is a question for the standard's text and for legal advice, not for an architecture repository — volume 1, chapter 15 makes the same point and flags the unresolved references in the practitioner analyses [12]. Record the answer you are given; do not derive it. Second, the repository's `O63` attributes three control objectives to `Joint Standard 1 of 2023` on IT governance and risk management, which is a different instrument from the outsourcing standard volume 1 covers. Both exist. Keep the `Regulatory Source` strings exact, because that column is what a reviewer sorts on.

Step 10.14 — Run the gate calendar monthly, from the cadence you already recorded

Produce. A standing monthly agenda that is generated, not assembled.

You need. `R33 ACO-WPK Review Cadence`, `O61 Expires`, `O30 End`, `O27 Physical Technology Compone Support Ends`.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *A calendar worth carrying* — the statutory dates to lay the gate calendar over.

Where it lands. `R33 ACO-WPK` — read `Source (ACO)`, `Target (WPK)`, `Review Cadence`, `Note`. All seven of Meridian's rows read `Monthly`, which means seven contract reviews a month and one `Last Review` date to update in `O33` per contract.

How.

1. Build the month's agenda from four queries: contracts due a review per `R33`; dispensations in `O61` expiring within ninety days; packages whose `O30 End` has passed with `Delivery Status` not evidenced; and components in `O27` whose `Support Ends` falls within twelve months.
2. For Meridian the fourth query returns three rows for 2027 — `PTC-15 MFT Server` at `2027-06`, `PTC-03 WebSphere Cluster` at `2027-09` and `PTC-07 Legacy Windows Estate` at `2027-10` — and `PTC-15` is the component the `CR-G01` dispensation exists because of.
3. Update `O33 Last Review` for every contract reviewed, whether or not the status changed.
4. Close the loop on every `Outstanding` row in `RPT31` or carry it with a reason.
5. Cut the monthly minute as rows, not prose: `O60` for what was decided, `O59` for what was assessed, `O61` for what was raised.

Signed by. `GB-01 Architecture Board`, monthly per its `Cadence` column.

Done when. `O33`'s `Last Review` dates are all within one cadence period of today.

Shows up in. `RPT13 Version Control`, where `O33 Architecture Contract` should show a recent `Latest change`, and `RPT31` where the decision count should rise each month.

Goes wrong when. The cadence is recorded and not run. `RPT12` reports `O57 Governance Body` and `O58 Governance Gate` both `ISOLATED` with zero relationships [10] — the governance model is recorded but not connected to anything that moves. A cadence that never fires looks identical in the repository to one that does not exist.

Working table — this month's gate agenda

Four queries, run in this order, before the Board sits. Each row ends in an action with a date, or it does not belong on the agenda. Cut the minute as rows afterwards: `O60` for decided, `O59` for assessed, `O61` for raised. Chaired by `GB-01` at its recorded cadence.

Query	What it returned	Owner	Action this month	By
Dispensations expiring in 90 days	CR-G01	CIO	Confirm PRJ-03 date	2027-12

When the answer is to refuse

The hardest decision in this phase is in front of Meridian's Board right now, and it is worth working through because yours will look like it. `DEC-G06` is `Outstanding`: refuse `WP-02` at G2, or re-baseline it. Eight of eight criteria are non-compliant. Seven projects depend on the package. `RSK-07` in `RPT31` states both halves of the trap: refusing it stops seven dependent projects; accepting it without evidence makes the gate meaningless [6].

The repository also contains the answer, in `RSK-07`'s mitigation column: re-baseline `WP-02` rather than refuse it outright, with the eight criteria as the new plan's acceptance criteria. That is not a softer option. It converts eight failures into eight dated commitments, keeps the criteria intact, and leaves the gate's authority intact because the package did not pass. A refusal that stops seven projects will be overturned by the Executive Committee within a month, and then `O57`'s note about `GB-04` comes true.

Three rules for the refusal you will actually face:

- **Refuse the thing, not the programme.** A verdict belongs to a criterion. Eight criteria give you eight places to be precise instead of one place to be dramatic.
- **Offer the re-baseline in the same paper.** A refusal with no alternative is an escalation waiting to happen, and the escalation body is the one that should not decide architecture.
- **Record the dependency count.** Seven dependent projects is the number that makes the decision the Board's rather than yours.

PHASE G — A REVIEWER'S QUESTIONS

What has this practice refused, and where is the refusal recorded as rows rather than minutes?

How many `G1` rows exist in `O59 Compliance Assessment`? If none, what is being reviewed before build?

Which dispensation is past its expiry date and still in force?

Has any package passed `G3`? If not, what is the benefit claim resting on?

THE CHAPTER IN TEN LINES

- Phase G produces refusals, dispensations and evidence. If the practice cannot refuse, nothing earlier mattered.
- Four gates in O58: G0 charter and G3 benefit to GB-02, G1 design and G2 build to GB-01.
- G1 is the gate the repository says was missing; all thirteen O59 rows carry Gate = G2 and none carries G1.
- No package has ever passed G3, because G3 did not exist. Fix the missing benefit start date first.
- The architecture contract in O33 is what makes a refusal survivable: parties, signed, expiry, status basis.
- One O59 row per acceptance criterion, never one per project. Eight rows tell you what to do; one amber row does not.
- Derive Compliance Status from O59. Four contracts once claimed Compliant while failing every criterion.
- A dispensation with an empty Expires is a permanent exception, and GP-02 forbids it.
- RPT24 is all 56 building blocks; RPT19 is the 14 data blocks only. Same tab name, different scope.
- When the Executive decides architecture, record it and amend O57 or take the decision back. Leaving both is the failure.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheet O58 Governance Gate
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheet O33 Architecture Contract, and R33 ACO-WPK
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheet O59 Compliance Assessment, and R56 O59-O55
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT30 Compliance Position
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT31 Governance Activity and O60 Governance Decision
- **7.** Meridian_TOGAF_Metamodel.xlsx, sheet O61 Governance Change Request
- **8.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT24 Building Block Coverage and RPT19 Building Block Coverage
- **9.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT32 Requirement Traceability
- **10.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT12 Integrity Check
- **11.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Phase G: Implementation Governance https://pubs.opengroup.org/togaf-standard/adm/
- **12.** Bowmans, "South Africa: new requirements for outsourcing by insurers published in Joint Standard 1 of 2024" https://bowmanslaw.com/insights/south-africa-new-requirements-for-outsourcing-by-insurers-published-in-joint-standard-1-of-2024/
- **13.** Meridian_TOGAF_Metamodel.xlsx, sheet O63 Control Objective, and R60 O63-REQ

CHAPTER 11

Architecture Change Management

Every change is either governed or discoverable later as a defect. There is no third state.

Phase H is the phase that decides whether the repository is an asset or an archive. Everything in it is either maintained against a change request or quietly wrong, and the difference is visible in one column: `Change Request` on `02 Change Log`.

The repository you were handed is in unusually good shape on this one discipline and unusually bad shape on another. All one hundred and seventy-two rows in `02 Change Log` carry a change request reference — not one is blank [1]. One hundred and sixty-four of them were approved by the Architecture Board. And yet the comparison tool, run across the same estate, finds five attribute values that moved with nothing in the log at all [5]. Both things are true: the governed changes are governed properly, and there is a stream of ungoverned change running alongside them that the log never saw.

This chapter teaches the pipeline that catches the first kind and the cadence that catches the second. Meridian Assurance Group is invented; the two failure shapes are not.

TOGAF places this work in Phase H, where the method loops back rather than ending [7]. Chapter 8 teaches the comparison tool itself — how to load an extract, how the diff sheets work, what each classification means. This chapter assumes you can run it and teaches you when.

> **BEFORE YOU START THIS PHASE**
>
> Phase H fires on things that happen in the business, not in the repository. Volume 1 is where you learn which events those are.
>
> - **Chapter 19, appendix A, *The cross-maps*** — *Keeping these alive*: the three maintenance rules step 11.7's cadence comes from.
> - **Chapter 12, *The application landscape*** — *2. Product configuration and the product factory*: why a product launch is a change trigger and what it touches.
> - **Chapter 2, *The lines of business*** — *Sasria, and why every South African policy system has a field for it*: a statutory change that arrives disguised as a product change.
> - **Chapter 14, *Technology providers*** — *What rots first*: which components will raise a support-expiry change request before the others.
> - **Chapter 15, *Regulation, risk and what the regulator inspects*** — *A calendar worth carrying*: the dated instruments that force a cycle regardless of capacity.

What you are building in this phase

Step	Produce	Lands in
11.1	A change request with a scope and a type	`03 Change Requests`
11.2	A log row per field change	`02 Change Log`
11.3	A dated decision at a body	`O60`, `O61`, `R57 O60-O61`
11.4	A classification: simplification, incremental, re-architecting	`03 Change Requests` `Type`, `Scope`
11.5	A drift sweep in Snapshot vs Snapshot mode	comparison tool `01 Config`, `60 Summary`
11.6	A human classification per unexplained difference	comparison tool `50 Review`
11.7	A cadence that fires without being chased	`R33 Review Cadence`, calendar
11.8	A baseline, cut at the right moment	`04 Baselines`
11.9	Three health numbers with targets	`RPT12`, `RPT13`, `RPT15`
11.10	The cross-maps kept alive	volume 1 appendix A, `R37`, `R38`, `R09`
11.11	A trigger-to-action table for insurance events	`O18`, `O19`, `O27`, `O63`
11.12	A support-expiry response	`O27 Support Ends`
11.13	An annual review pack the board can act on	`RPT01`, `RPT09`, `RPT30`, `RPT31`

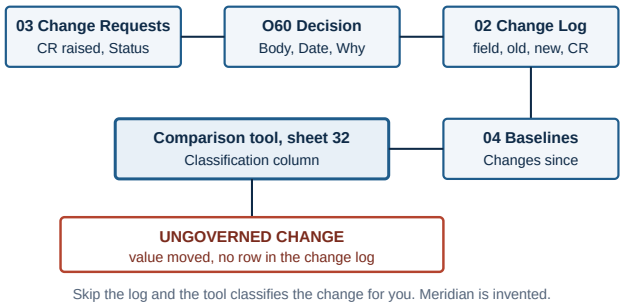

Figure 11.1 The change pipeline, and the one place it is enforced. A field value that moves without a change log row carrying a CR reference is classified UNGOVERNED CHANGE by the comparison tool.

The pipeline

Step 11.1 — Raise the change request before you touch a cell

Produce. One row in `03 Change Requests`, open, with a scope narrow enough to decide.

You need. The thing that prompted the change, the sheets it will touch, and a decision record reference.

Where it lands. `03 Change Requests` — fill `CR`, `Title`, `Raised By`, `Raised`, `Type`, `Scope`, `Status`, `Decided`, `Decision`, `Decision Record`, `Changes`.

`Scope` lists the object types affected, not the sheets — `Data Entity, Application Component`, `Work Package, Gap`, `Requirement, Gap, Work Package`. `Type` is the kind of change: the register uses `Addition`, `Correction`, `Reassessment`, `Restructure`, `Structure`, `Attribute`, `Extension`, `Convention`, `Report`, `Reinstate`, `Record` and `Baseline`. `Status` is `Proposed`, `Raised`, `Under review`, `Approved` or `Implemented`. `Changes` is the count of `02 Change Log` rows the request produced, and it is the only honest measure of a request's size.

How.

1. Write `Title` as the change, not the problem. *Record customer master authority as contested*, not *customer data is a mess*.
2. Set `Type` from the list above. If two apply, the register permits a compound value — `CR-055`'s neighbours include a row typed `Structure, Attribute`.
3. Set `Scope` to the object types. A request whose scope is *everything* has not been assessed.
4. Set `Status` to `Proposed` or `Raised`. Records affected by an open request read `Draft` in their `Record Status` column until it is decided — the register's own header note says so [2].
5. Point `Decision Record` at the architecture decision record that carries the reasoning. The register uses `ADR-001` upward, one per decision, several change requests sometimes sharing one.

Signed by. `Raised By` is a named executive role, not a team. The register's raisers are the Chief Enterprise Architect, the Chief Data Officer, the CIO, the CFO, the Chief Risk Officer, the Chief Distribution Officer and the Claims Executive.

Done when. The request has a `Type`, a `Scope` and a `Decision Record`, and nothing in the repository has changed yet.

Shows up in. `RPT13 Version Control`'s lower block, which lists every change request with its `Status`, `Raised`, `Decided` and `Changes` count, and three summary cells at the bottom: records in `Draft`, changes awaiting approval, open change requests. Meridian's read 4, 4 and 4.

Goes wrong when. The change is made first and the request written afterwards to tidy up. The log will look identical. The difference only shows when the request's `Decided` date is later than the log row's `Date`, which is a one-formula check worth having.

Step 11.2 — Log every field change, with its change request, or the tool will classify it for you

Produce. One row in `02 Change Log` per field that moved, with the before and after values.

You need. The approved change request, and the records you are about to change.

Where it lands. `02 Change Log` — fill `Change`, `Date`, `Sheet`, `Record`, `Field`, `Previous Value`, `New Value`, `Reason`, `Change Request`, `Changed By`, `Approved By`.

`Sheet` is the sheet prefix only — `O20`, `O13`, `O30` — not the full tab name. `Record` is the object `ID`. `Field` is the exact column header. `Previous Value` for a new record reads `(new record)`. `Reason` is a sentence explaining the change to a reader two years from now who has no context: *Naming LIFE/400 as the authority was an assumption, not a decision. Four systems master this entity.*

How.

1. One row per field. A change to three fields on one record is three rows, not one.
2. Set `Change Request` on every row. This is the enforcement mechanism and it is not optional: see the warning below.

3. Write `Reason` at the level of the change. *As above for Customer* is acceptable where a preceding row carries the full reasoning, and the register uses it.
4. Set `Approved By` to the body that approved, not the person who typed. One hundred and sixty-four of Meridian's rows read `Architecture Board`; four read `Chief Enterprise Architect`; four are empty and those four are the `Pending approval` count in `RPT13`.
5. Never truncate the log. `01 Version Control` states why in terms you can quote to a reviewer: the log is the evidence base for `GP-01` and for Joint Standard 1 of 2023 on IT governance [2].

> *This is the step the rest of the chapter rests on. The comparison tool's `32 Diff Attributes` sheet has a `Classification` column, and it works by matching each differing attribute against the supplied change log. A difference with a matching log row is classified `Governed drift`. A difference with no matching log row is classified `UNGOVERNED CHANGE` [5]. There is no third option and no appeal. If you move a value and skip the log, the tool will name it later, in front of whoever you present it to.*

Signed by. `Approved By` — a governance body, per the request's decision.

Done when. The count in the request's `Changes` column equals the number of log rows carrying that `CR`.

Shows up in. `RPT13 Version Control`, per object type: `Changes logged`, `Records changed`, `% changed`, `Latest change`, `Pending approval`.

Goes wrong when. Somebody fixes a typo without logging it, on the reasonable view that a typo is not a change. The tool does not share that view. Meridian's five ungoverned changes include a hosting value, a standard status, a capability maturity score, a role authority level and a business service criticality — and on review, three of the five turned out to be real [6].

Step 11.3 — Decide it at a body, and record the decision as a record

Produce. A dated row in `O60 Governance Decision`, and where the decision answers a request, a link.

You need. The change request, the body whose `May Decide` column covers it, and the papers.

Where it lands. `O60 Governance Decision` — fill `ID`, `Date`, `Decision`, `Body`, `Status`, `Why`, `Affects`.

`Date` empty means the decision has not been taken — the sheet's header note says it outright. `Affects` is scoped as broadly as the decision really is: `All projects`, `All charters`, `WP-02 and seven dependents`, `PRJ-05, PRJ-03`.

How.

1. Create the `O60` row when the item goes on the agenda, with `Status` = `Outstanding` and `Date` empty. The outstanding decision is itself a governance fact worth reporting.
2. Fill `Date` and set `Status` = `Approved` when it is taken.
3. Where the decision resolves a governance change request, create the `R57 O60-O61` link carrying `Status` and a `Note`.
4. Write `Why` as the consequence of not deciding. Meridian's best example is `DEC-G02`'s: *a charter with its own criteria drifts from the architecture within a quarter*.
5. Update the `03 Change Requests` row's `Status`, `Decided` and `Decision` in the same sitting.

Signed by. The body named in `Body`.

Done when. Every `03 Change Requests` row with `Status` = `Approved` has a `Decided` date, and every governance decision in `O60` either has a `Date` or appears on the next agenda.

Shows up in. `RPT31 Governance Activity`, filtered to `Kind` = `Decision`.

Goes wrong when. `R57 O60-O61`'s `Source Name` column resolves to the decision's `Date` rather than its text, because `O60`'s second column is `Date` and the lookup takes column two. `DEC-G06`, whose date is empty, shows as `0`. Read `R57` for the pairing and `O60` for the meaning; do not quote `R57`'s `Source Name` in a paper.

Step 11.4 — Classify the change as simplification, incremental, or re-architecting

Produce. A classification that determines who decides and how much assessment is needed.

You need. The change request's `Scope`, and the repository itself to answer four questions.

The three classes are not a theory exercise. They are answered by looking at the repository.

Question about the repository	If yes	Class
Does any object change its `Record Status` from `Baseline`?	no	Simplification
Does the change touch only `Description`, `Note`, `Reason` or a report?	yes	Simplification
Does any new object appear in a catalogue sheet without a new relationship type?	yes	Incremental
Does a `Work Package`, `Wave` or `Cost (Rm)` value move?	yes	Incremental
Does a relationship *type* get added or removed — a new `R` sheet?	yes	Re-architecting
Does a building block's `Acceptance Criteria` change?	yes	Re-architecting
Does a target-state object become baseline, or a baseline object leave?	yes	Re-architecting

Where it lands. `03 Change Requests` — Record the class in `Type` and the evidence for it in `Scope`. The register's `Type` values map cleanly: `Correction` and `Convention` are simplifications; `Addition`, `Attribute`, `Reassessment` and `Report` are incremental; `Structure`, `Restructure` and `Extension` are re-architecting.

How.

1. Run the seven questions above in order and stop at the first yes in the re-architecting block.
2. A simplification is decided by the Chief Enterprise Architect alone. Four of Meridian's log rows carry `Approved By` = `Chief Enterprise Architect`, and all four are of this kind.
3. An incremental change goes to `GB-01 Architecture Board` at its monthly cadence.
4. A re-architecting change goes to `GB-01` and then to `GB-02 Investment Committee`, because it will move money. `CR-011`, *Split claims consolidation from claims modernisation*, has sat at `Under review` since 2026-12-18 for exactly this reason: its sequencing impact on Wave 3 is the assessment.
5. Record the class once. Re-arguing it per change is how a register becomes a queue.

Signed by. Chief Enterprise Architect for simplifications; `GB-01` for the other two; `GB-02` in addition where cost moves.

Done when. Every open request has a `Type` and the decision route implied by it.

Shows up in. `RPT13`'s change request block, where `Changes` counts give you the size distribution. Meridian's largest is `CR-016`, *Load the four-level Business Capability Map*, at nineteen log rows; most are one to six.

Goes wrong when. Every change is treated as re-architecting, so the Board agenda fills with corrections and the real structural changes get ten minutes at the end. Twenty-seven of Meridian's sixty-three requests are typed `Structure`, which is a high proportion and reflects a repository still being built out rather than one being maintained — expect that ratio to invert once the metamodel settles.

THE CHANGE YOU MADE WITHOUT A REQUEST

What was the last change you made to the repository without raising a change request first, and why?

Which class was it — simplification, incremental, re-architecting — and who should have decided it?

What would make raising the request first the easier path next time?

Drift you were not told about

Step 11.5 — Sweep for ungoverned change in Snapshot vs Snapshot mode

Produce. A count of changes that happened without a change request.

You need. Two extracts of the same repository taken at different dates, the change log between them, and the comparison tool. Chapter 8 covers loading it.

Where it lands. `01 Config` — set `Snapshot A label`, `Snapshot A date`, `Snapshot B label`, `Snapshot B date`, `Comparison mode`, `Prepared by`, `Prepared on`.

`Comparison mode` is the setting that changes the meaning of every result. `Baseline vs Target` treats unexplained differences as gap candidates. `Snapshot vs Snapshot` treats them as ungoverned changes. The tool as delivered is set to `Baseline vs Target`, comparing `BL-`

`001 Preliminary Phase Baseline` at 2026-09-30 against the repository at 2027-11-20 [5]. For this step, change it.

How.

1. Export `05 Export Attributes` and `06 Export Records` from the repository as it stands into `11 Input B Target`, and the same two sheets from your last snapshot into `10 Input A Baseline`.
2. Paste the change log for the interval into `12 Input Change Log`. Without it every difference reads as ungoverned and the sweep is worthless.
3. Set `Comparison mode` to `Snapshot vs Snapshot` and set both snapshot dates.
4. Read `60 Summary`'s classification block. Meridian's reads: `Same` 1264, `Governed drift` 31, `Governed drift + further change` 1, `UNGOVERNED CHANGE` 5, `Added in B` 2739, `Record removed in B` 138.
5. Note the one row classified `Governed drift + further change`. A logged change exists but the current value has moved beyond it — the quietest failure the tool detects, and the one most worth reading every time.

Signed by. Chief Enterprise Architect, as `Prepared by` on `01 Config`.

Done when. `60 Summary` shows a number in the `UNGOVERNED CHANGE` cell and you have looked at every one of them.

Shows up in. `60 Summary`, and in the trend you keep yourself: the tool does not store history.

Goes wrong when. The sweep is run in `Baseline vs Target` mode and the ungoverned count is read anyway. In that mode the 2739 `Added in B` rows are the target architecture, which is correct and expected. The same number in `Snapshot vs Snapshot` mode would mean the repository grew by two and a half thousand attributes between snapshots, which is a different and much worse story. The mode is not a label. Read it before you read anything else.

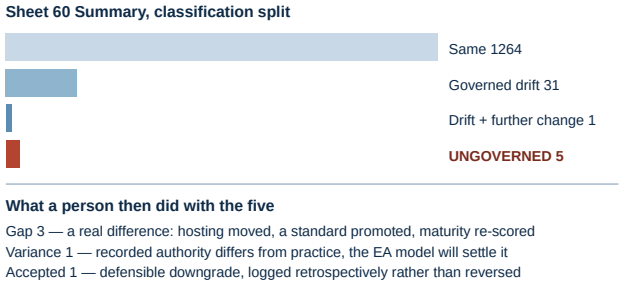

Figure 11.2 The classification split from sheet 60 Summary, to scale, with what a person then decided about the five ungoverned changes. Three of five were real.

Step 11.6 — Classify what the sweep found, and treat a rising count as a practice failure

Produce. A classification and an owner per unexplained difference.

You need. `32 Diff Attributes` filtered on `Classification`, and `50 Review`.

Where it lands. `50 Review` — fill `Key`, `Type Name`, `ID`, `Attribute`, `Tool classification`, `Your classification`, `Rationale`, `Owner`, `Action`.

`Your classification` is one of `Gap`, `Drift`, `Variance` or `Accepted`. Only `Gap` flows through to `70 Gap Candidates`. This is the only sheet in the tool you type in, and its header states the rule: a difference is not a finding until someone has classified it and taken it.

How.

1. Filter `32 Diff Attributes` to `Classification` = `UNGOVERNED CHANGE` and copy every row across.
2. Classify each. Meridian's five resolved as three `Gap`, one `Variance`, one `Accepted` — and the `Accepted` one carries the right action: *log it retrospectively rather than reverse it* [6].
3. Name an `Owner` who is a person's role, and write an `Action` that is a verb. *Raise CR; assess against STD-16. Re-run assessment and log it.*
4. Raise the retrospective change request for anything classified `Accepted`. The value stays; the log gains the row it should have had.
5. Record the count. Not the list — the count. Five last quarter, eleven this quarter is the signal.

Signed by. `GB-01 Architecture Board`, which receives the classified list.

Done when. Every `UNGOVERNED CHANGE` row has a `Your classification` and an `Owner`.

Shows up in. `60 Summary`'s human classification block, which counts `Gap`, `Drift`, `Variance`, `Accepted`, `Reviewed` and — the number that matters most — `Differences not yet reviewed`. Meridian's reads 2906, which in `Baseline vs Target` mode is ordinary and in `Snapshot vs Snapshot` mode would be alarming.

Goes wrong when. A rising ungoverned count is reported as an estate problem. It is not. The estate did what it always does. The count measures how often people changed the repository without going through the pipeline, which is a measure of the practice. Reporting it as *the estate is drifting* moves the accountability to the people who are not at fault, and nothing changes.

READING THE UNGOVERNED COUNT

What is your count this quarter, and what was it last quarter? One number each.

Of the changes the sweep found, which turned out to be real findings rather than tidying?

Which one are you logging retrospectively rather than reversing, and who agrees?

Step 11.7 — Set the cadence, and tie each run to something that already happens

Produce. Three recurring sweeps that fire without being chased.

You need. The release calendar, the Board cadence from `057` (`Monthly` for `GB-01`, `Quarterly` for `GB-02`), and the review cadence in `R33 ACO-WPK`.

Look it up. Volume 1, chapter 19, appendix A, *The cross-maps*, *Keeping these alive* — the three rules, of which the second and third decide this cadence.

When	Mode	What you read	What the result is for
Monthly, before `GB-01`	`Snapshot vs Snapshot`	`UNGOVERNED CHANGE` count only	The Board's standing governance item
Quarterly, before `GB-02`	`Snapshot vs Snapshot`	Full `60 Summary` plus `RPT13`	The investment review's data quality statement
After every major release	`Snapshot vs Snapshot`	`30 Diff Records` and `32 Diff Attributes`	Catching what delivery changed and did not tell you
Before cutting a baseline	`Baseline vs Target`	`70 Gap Candidates`	Confirming the gap catalogue before it freezes

How.

1. Attach the monthly sweep to the Board paper deadline, not to a date in the month. Volume 1 appendix A's third rule applies here and is worth repeating: review on a trigger, not a date, because a date will be skipped.
2. Keep the monthly sweep to one number. A monthly run that produces a twenty-page pack will stop happening by the third month.
3. Run the post-release sweep within five working days, while the delivery team can still explain what they changed.
4. Store each run's `60 Summary` as a dated copy. The tool holds one comparison; the trend is yours to keep.
5. Put the quarterly run before the investment review, so that a funding decision is taken against a repository whose health is stated rather than assumed.

Signed by. Chief Enterprise Architect.

Done when. The last three months each have a stored `60 Summary` with a date.

Shows up in. `RPT31 Governance Activity` if you record the sweep as a decision input, which is worth doing — otherwise the work is invisible.

Goes wrong when. The cadence is quarterly only. A quarter is long enough for an ungoverned change to become the thing everyone now relies on, at which point the classification is `Accepted` by default rather than by decision.

Baselines and health

Step 11.8 — Cut a baseline when a decision needs a fixed reference, and not otherwise

Produce. A named, dated, approved snapshot in `04 Baselines`.

You need. The repository at a stable point, the workbook version, and an approver.

Where it lands. `04 Baselines` — fill `Baseline`, `Name`, `Date`, `Workbook Version`, `Approved By`, `Status`, `Note`, `Changes since`.

`Status` is `Superseded`, `Current` or `Planned`. `Note` says what the baseline is the reference point *for* — this is the field that decides whether cutting it was justified. `Changes since` counts log rows after its date, and is the number that tells you whether the next diff will be readable.

How.

1. Cut a baseline when a decision will be measured against the repository as it stands. Meridian's three show the pattern: `BL-001` is the reference point for the maturity assessment at Level 2; `BL-002` is the reference for Phase A gap analysis, cut at the point the Architecture

Board was constituted; `BL-003` is `Planned`, to be cut on approval of the Architecture Vision [4].

2. Record the `Workbook Version`. `BL-001` is 1.0, `BL-002` is 1.9. `BL-003` has none because it has not been cut, which is correct.
3. Set `Approved By`. `BL-001` was approved by the Chief Enterprise Architect; `BL-002` by the Architecture Board. The approver rises with the baseline's consequence.
4. Set the previous baseline's `Status` to `Superseded` in the same action.
5. Watch `Changes since`. Meridian's reads 172 for `BL-001` and 130 for `BL-002`.

Signed by. `GB-01 Architecture Board` for any baseline a gap analysis will be measured against.

Done when. Exactly one row reads `Status` = `Current`.

Shows up in. The comparison tool's `01 Config`, where a baseline's name becomes `Snapshot A label`.

Goes wrong when. The frequency is wrong in either direction, and both failures are quiet.

- **Too often.** Each baseline absorbs the drift since the last one as the new normal. The `Changes since` count stays small and comfortable, the diffs stay short, and nothing ever looks wrong — because every ungoverned change was baselined before anybody compared. A baseline cut monthly is a baseline that cannot detect anything.
- **Too rarely.** `Changes since` grows past a few hundred and the diff becomes unreadable. Meridian's `BL-001` is at 172 after fourteen months and the tool's `32 Diff Attributes` sheet already runs to 5,483 rows with 2,906 differences nobody has reviewed. Two years of that and the honest answer is that nobody knows what changed.

The test: cut a baseline when a named decision needs a fixed reference, and never because a quarter ended.

BEFORE YOU CUT THE NEXT BASELINE

Which named decision will be measured against this baseline? One sentence.

What is `Changes since` on the current one, and is the next diff going to be readable?

Are you cutting it because a decision needs a fixed reference, or because a quarter ended?

Step 11.9 — Read the three health reports as a standing item, with targets

Produce. Three numbers on the Board paper every month, with a target beside each.

You need. `RPT12 Integrity Check`, `RPT13 Version Control`, `RPT15 Catalogue Completeness`.

Look it up. Volume 1, chapter 18, *Your first ninety days*, *Telling within a month whether the estate is healthy* — the signals an outsider reads, which are not these three.

Where it lands. `RPT12 Integrity Check` — read `Sheet`, `Object type`, `Records`, `As source`, `As target`, `Total links`, `Links per record`, `Status`. `Status` is derived: `Linked`, `Thinly linked` or `ISOLATED`.

Report	Number to read	Target	A bad one means
`RPT12`	`Links per record`, per type	above 1.0	below 1.0 is `Thinly linked`; the type is recorded but not wired into anything
`RPT12`	count of `Status` = `ISOLATED`	0	the type has no relationships at all and no report can traverse it
`RPT12`	total `Links per record`	above 2.0	Meridian's is 2.41 across 1,727 records, which is healthy
`RPT13`	`% changed`, per type	5–40% over a year	0% is `Unchanged since baseline`: stable, or unmaintained, and you must say which
`RPT13`	`Pending approval`	0	a change was made and never approved; Meridian's total is 4
`RPT13`	total `Changes logged`	rising steadily	Meridian's is 132 changes across 1,727 records, 4.9%
`RPT15`	`Links` as `n/6`	6/6	the `Where it breaks` column names the first missing link in the chain

How.

1. Read `RPT12`'s `ISOLATED` list first. Meridian has eighteen isolated object types, and three of them are the governance types this volume's two Phase G and H chapters depend on: `057 Governance Body`, `058 Governance Gate` and `062 Delivery Risk` [3].
2. For each isolated type, decide: add the relationship sheet, or accept the type as a register rather than a model and say so in writing. `CR-015` in the register did the former — *Close isolated entity types found by integrity check*, raised off `RPT12`, two relationship types added.

3. Read `RPT13`'s `Unchanged since baseline` rows. Nine of Meridian's object types read 0%, including `O01 Driver`, `O02 Goal`, `O03 Objective` and `O04 Measure` — the whole motivation spine, unchanged in fourteen months while the architecture around it moved substantially.
4. Read `RPT15`'s `Where it breaks` column, not its tick marks. `Contested source` against `DEN-01`, `DEN-02` and `DEN-03` is one problem appearing three times.
5. Put three numbers on the paper: isolated types, pending approvals, and concepts below 6/6. Nothing else.

Signed by. Chief Enterprise Architect.

Done when. Each of the three numbers has a target and a direction of travel beside it.

Shows up in. `RPT01 Dashboard`, which aggregates across the repository.

Goes wrong when. A `% changed` of 0% is reported as stability. `RPT13`'s own assessment wording refuses to make that call for you: *a type with no changes since baseline is either stable or unmaintained*. The motivation spine reading 0% while the application portfolio churns is the second of those, and it is the subject of chapter 12.

Step 11.10 — Keep volume 1's cross-maps alive when a process changes

Produce. A dependent-sheet list that fires whenever a process, capability or application moves.

You need. Volume 1, chapter 19, appendix A, *The cross-maps* — the seven maps and the four rules that close it.

Look it up. Volume 1, chapter 19, appendix A, *The cross-maps*, *Process to data, as CRUD* — the map that breaks first when a process moves.

Volume 1's seven maps are capability to process, process to role as RACI, process to data as CRUD, process to application, application to data with the system of record, application to role, and capability to application as the coverage map. In this repository they are relationship sheets, and a change to one hub object obliges a change to several.

What changed	Sheets that must change with it	Report that shows the break
A business process (`O16`)	`R09 ROL-BPR`, process-to-data and process-to-application relations, `O14 Value Stream`	`RPT23 Process Flow`
A business capability (`O13`)	`R38 CAP-CAP` decomposition, `R37 APP-CAP` support	`RPT03 Capability Heatmap`, `RPT07 Capability Coverage`
An application (`O23`)	`R37 APP-CAP`, `R41 APP-VST`, `R36 PSV-APP`, `R32 REQ-APP`	`RPT02 Application Portfolio`, `RPT05 App-Data Matrix`
A data entity (`O20`)	`O36 Data Entity Relationship`, `O37 Data Attribute`, `RPT04` CRUD	`RPT15 Catalogue Completeness`
A role (`O11`)	`R08 ACT-ROL`, `R09 ROL-BPR`	`RPT12` link counts for `O11`

Drawn in ArchiMate 4 this is a layered view around the hub: the changed element, the Capability it realises, the Process it is assigned to and the Business Object it accesses. The dependent sheets are the relationships on that view, which is the quickest way to find one you have missed.

How.

1. Apply volume 1's first rule — one owner per dimension, named. In the repository that means `Owner` filled on every `O13`, `O16`, `O20`, `O23` and `O11` record, and a person behind it. The repository fails this today: twenty-three of twenty-four data entities are owned by a role with no actor performing it.
2. Apply the second rule — change the hub last. Add `CAP-99` rather than renumbering, and retire IDs rather than reusing them. The `Record Status` column gives you `Retired` for this.
3. Apply the third rule — review on a trigger. The triggers are: before each Architecture Board, before each work package evaluation, and after each reorganisation. All three are already in the cadence from step 11.7.
4. Apply the fourth rule — publish the gaps separately. `RPT09 Gaps and Roadmap` is that page.
5. When a change request's `Scope` names one of the five hub types above, copy the dependent-sheet row into the request before it is decided. The assessment is then a list, not a conversation.

Signed by. The dimension owner from rule one; `GB-01` for the change request.

Done when. No change request with a hub type in `Scope` is approved without its dependent sheets listed.

Shows up in. `RPT12 Integrity Check`, where a hub change made without its dependents shows as a falling `Links per record` for the affected type.

Goes wrong when. The maps are maintained as a separate document. Volume 1 appendix A is the starting content; the relationship sheets are the live version. Keeping both means keeping neither.

Insurance triggers

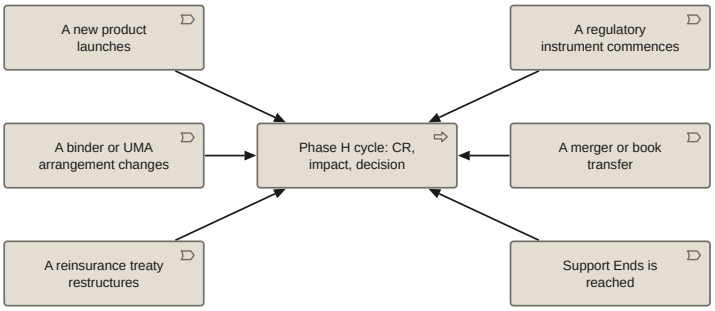

Figure 11.3 The six insurance events that force a Phase H cycle, each triggering it directly, whether or not anyone thinks the architecture is affected. Six triggers, no discretion.

Step 11.11 — Run a Phase H cycle on each of the six insurance triggers, without discretion

Produce. A trigger-to-action table posted where the business can see it.

You need. `O18 Product`, `O19 Contract`, `O27 Physical Technology Compone`, `O63 Control Objective`, and volume 1, chapter 15 for the regulatory instruments.

Look it up. Volume 1, chapter 2, *The lines of business, Sasria, and why every South African policy system has a field for it* — what a statutory product change looks like when it arrives.

Trigger	First action	Sheets that change	Decision needed from
A new product launches	Raise a `CR` typed `Addition`, `Scope` = `Product, Contract, Channel`	`O18 Product`, `O19 Contract`, `O13`, `R37 APP-CAP`	`GB-01`, and `GB-02` if a platform change follows
A binder or UMA arrangement starts or ends	Raise a `CR` typed `Addition` or `Reassessment` against `O19`	`O19 Contract`, `O11 Role`, `O09 Organization Unit`, `O63`	`GB-01` plus the compliance function
A reinsurance treaty restructures	Reassess the data the treaty needs	`O19 Contract`, `O20 Data Entity`, `RPT04 CRUD Matrix`	`GB-01`, with the Chief Financial Officer
A regulatory instrument commences	Add the obligation and its control objective	`O01 Driver`, `O63 Control Objective`, `O06 Requirement`, `R60 O63-REQ`	`GB-01` plus Chief Risk Officer
A merger or book transfer	Open a re-architecting request before any system decision	`O09`, `O23`, `O20`, `O56 Project`, `04 Baselines`	`GB-02`, then `GB-01`
A system reaches `Support Ends`	See step 11.12	`O27`, `O32 Standard`, `RPT10 Standards Conformance`	`GB-01`

How.

1. Post the table. Not in the repository — on whatever the business actually reads. The trigger has to be recognised by someone who has never opened the workbook.
2. For the product trigger, start from `O18 Product` and work outward. Meridian's `CR-009`, *Add embedded distribution pilots*, is the worked example: `Scope = Product, Contract, Channel`, four log rows, retail and telco pilots plus the Simple Funeral product.
3. For the regulatory trigger, `CR-013` is the worked example: *Add climate exposure requirement*, `Scope = Requirement, Gap, Work Package`, raised off PA Guidance Notice `G3/2025`, three log rows, decided in seven days.
4. For the binder and UMA trigger, note the gap honestly. `O19 Contract`'s `Type` column holds `Distribution`, `Reinsurance`, `Technology` and `Supplier` — there is no binder or UMA row in the repository, and no column records delegated underwriting or claims authority. Volume 1, chapter 15 treats binder limits as a Policyholder Protection Rules matter; the repository cannot currently answer a question about them. Add the `Type` value and an authority column, or record that delegated authority is held outside the repository.
5. For the merger trigger, cut a baseline first. A book transfer with no fixed reference point produces a diff nobody can read for two years.

Signed by. `GB-01 Architecture Board` for all six; `GB-02` where the trigger moves money.

Done when. Each of the six triggers has a named person in the business who raises the request, not an architect who notices.

Shows up in. `03 Change Requests`, where the `Raised By` column should show business roles and not only the Chief Enterprise Architect. Meridian's does, across seven different executives.

Goes wrong when. The architecture function finds out about the product launch at the launch. Every one of these six triggers is known to someone three months before it reaches the repository. The work of this step is not the table; it is the six conversations that make the table fire.

THE SIX CONVERSATIONS THAT MAKE THE TABLE FIRE

For each trigger, who in the business raises the request — a named role, not "the architect who notices"?

Which of the six would you currently find out about on the day it happens?

Which trigger does the repository have nowhere to record — and are you adding a column or recording that it lives outside?

Working table — trigger owners and how you hear

One row per trigger. The second column is a role in the business, and the third is the mechanism — a standing agenda item, a distribution list, a committee paper you are copied on. The last column is the date you had the conversation, not the date you intend to.

Trigger	Who raises it	How you hear	Conversation held
New product launches	Head of Product	Product committee paper	2027-11-04

Step 11.12 — Respond to a support expiry as a dated change, not a renewal conversation

Produce. A change request per component whose support ends inside twelve months.

You need. `O27 Physical Technology Compone`, columns `Lifecycle`, `Support Ends`, `Standard Status`, `Annual Cost (Rm)`.

Look it up. Volume 1, chapter 14, *Technology providers*, *What rots first* — the order things go, which tells you which expiry to believe.

Where it lands. `O27 Physical Technology Compone` — read `Support Ends`. The column holds a year and month, or `rolling` for a service with no fixed end, or empty where no product has been selected. All three values mean different things and only the first is a date you can plan against.

How.

1. Filter `O27` to `Support Ends` within twelve months. Meridian's returns three: `PTC-15 MFT Server` at `2027-06`, `PTC-03 WebSphere Cluster` at `2027-09` and `PTC-07 Legacy Windows Estate` at `2027-10`. All three carry `Lifecycle` = `Declining` and `Standard Status` = `Contained` [8].
2. Check whether a dispensation depends on the component. `PTC-15` is the reason `CR-G01` exists — the MFT server must be replaced before the gateway exists, so the replacement will be point-to-point.
3. Note the components with an empty `Support Ends`. `PTC-16 API Gateway Platform` and `PTC-17 Event Streaming Cluster` both read `To be selected`, and the empty support date is correct until a product is chosen. Do not fill it with a guess.
4. Raise one request per component, typed `Reassessment`, and cross-check `RPT10 Standards Conformance`, which carries `Support Ends` and `Standard Status` side by side.
5. Report the three together, not separately. Three contained declining components expiring within five months of each other is a capacity statement about 2027, not three renewals.

Signed by. `GB-01 Architecture Board`; Chief Information Officer owns the components.

Done when. No component has a `Support Ends` date in the past without an `O61` row against it.

Shows up in. `RPT10 Standards Conformance` and `RPT20 Portfolio Assessment`.

Goes wrong when. The expiry is handled by the vendor manager as a contract renewal. The contract can be extended; the architecture position cannot. `CR-007` in the register is the right shape: *Update technology support end dates*, five log rows, raised when the vendor roadmap confirmed three components out of support in 2027.

Step 11.13 — Put four numbers and two decisions in front of the board once a year

Produce. An annual architecture review pack that asks for something.

You need. `RPT01 Dashboard`, `RPT09 Gaps and Roadmap`, `RPT30 Compliance Position`, `RPT31 Governance Activity`, `04 Baselines`, and the year's `60 Summary` copies.

How.

1. Open with the compliance position, not the roadmap. `RPT30`'s three cells — assessed, non-compliant, compliant — are the only numbers in the pack that say whether the practice works. Meridian's are 13, 13 and 0.
2. Give the governance activity next: decisions taken, decisions outstanding, dispensations live, dispensations expired without remediation. From `RPT31` and `O61`.
3. Give repository health as three numbers from step 11.9, with the ungoverned-change trend from the year's sweeps beneath them.
4. Give the roadmap position last, from `RPT09` and `RPT27 Programme Sequencing`. The board has seen the roadmap. It has not seen whether the roadmap is being governed.
5. Close with the decisions. Not recommendations — decisions, each with a body and a date. Meridian's two are already named: `DEC-G06`, refuse `WP-02` at G2 or re-baseline it, and `DEC-G07`, fund `PRJ-12` or accept the POPIA exposure in writing with a named owner.
6. Record the review itself as an `O60` row with `Affects = All projects`.

Signed by. Group Executive Committee receives it; `GB-01` prepares it; `GB-02` owns the funding decisions in it.

Done when. The pack contains at least one decision with a named body and a date, and the minute records what was decided.

Shows up in. `O60 Governance Decision`, with `Date` filled. A review that produces no dated `O60` row did not happen, whatever the attendance list says.

Goes wrong when. The annual review is a presentation of progress. `RPT31`'s closing note is the test to apply to your own pack: every escalation trigger in the governance model is live and none has been escalated. A review that reports progress against a position like that is not a review.

PHASE H — A REVIEWER'S QUESTIONS

What is the one decision this year's review is asking the board for, and which body takes it?

Which change in the repository this year has no change request behind it?

Is the ungoverned-change count rising or falling, and what did you do about the direction?

Which of these triggers fired this year without the architecture function hearing about it first?

THE CHAPTER IN TEN LINES

- Every field change gets a 02 Change Log row with a Change Request, or the comparison tool classifies it as UNGOVERNED CHANGE.
- All 172 log rows in the repository carry a CR, and the tool still finds five values that moved without one.
- Three classes of change, decided by questions about the repository: simplification, incremental, re-architecting.
- Comparison mode on 01 Config changes the meaning of every result. Read it before you read anything else.
- Sweep monthly for the ungoverned count, quarterly for the full summary, and after every major release.
- A rising ungoverned count is a practice failure, not an estate failure. Report it that way.
- Cut a baseline when a named decision needs a fixed reference. Monthly hides drift; never makes diffs unreadable.
- RPT12 isolated types, RPT13 pending approvals, RPT15 broken chains — three numbers, every month.
- 0% changed in RPT13 is stable or unmaintained, and you have to say which. The whole motivation spine reads 0%.
- Six insurance triggers force a cycle. The work is not the table; it is the six conversations that make it fire.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheet 02 Change Log
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets 03 Change Requests and 01 Version Control
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT12 Integrity Check, RPT13 Version Control and RPT15 Catalogue Completeness
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheet 04 Baselines
- **5.** Meridian_Architecture_Comparison_Tool.xlsx, sheets 01 Config, 32 Diff Attributes and 60 Summary
- **6.** Meridian_Architecture_Comparison_Tool.xlsx, sheet 50 Review
- **7.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Phase H: Architecture Change Management https://pubs.opengroup.org/togaf-standard/adm/
- **8.** Meridian_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Compone and O19 Contract

CHAPTER 12

Requirements Management

A requirement exists to be tested at a gate. Anything that cannot be is a preference in requirement's clothing.

Requirements Management is not a phase you do and finish. It runs underneath every other phase, and the quality of it determines whether anything the other phases produced can be tested. That is the whole argument of this chapter: a requirement exists to be tested at a gate, and anything that cannot be tested at a gate is a preference wearing a requirement's clothes.

The repository holds seventy-six requirements in `O06 Requirement` [1]. Twenty of them carry a source driver, an objective and a named owner, and have no acceptance criteria, no building block and no work package. Fifty-six carry acceptance criteria, a building block and a work package, and have no source driver, no objective and no owner. The register is in two halves that do not meet [1]. `RPT08 Traceability` puts the consequence in one cell: sixteen requirements of seventy-six have a complete trace [6]. `RPT32 Requirement Traceability` puts the other consequence in another: ever tested, zero [7].

Meridian Assurance Group is invented. This defect is the most common one in real repositories, because the two halves are produced by different people at different times for different reasons, and nobody owns the join.

This chapter builds the join.

> **BEFORE YOU START THIS PHASE**
>
> Volume 1 does most of the groundwork here, because most of an insurer's requirements come from outside it.
>
> - **Chapter 15, *Regulation, risk and what the regulator inspects*** — *Modelling a regulatory requirement properly*: how an instrument becomes something testable rather than a memo.
> - **Chapter 15** — *Treating Customers Fairly: six outcomes, six design consequences*: six obligations, each with the design consequence that makes an acceptance criterion writable.
> - **Chapter 16, *Modelling insurance in ArchiMate*** — *Motivation: from a rule to the system that satisfies it*: the spine step 12.3 traces, drawn.
> - **Chapter 6, *How insurers make money*** — *Reading the ratios*: where a benefit measure comes from, and which ones somebody already reports.
> - **Chapter 10, *The information model*** — *Master data and the single view of customer*: the requirement everyone writes and nobody can test.

What you are building, continuously

Step	Produce	Lands in
12.1	One register, every requirement in it	`O06 Requirement`
12.2	An acceptance criterion per requirement	`O06 Acceptance Criteria`
12.3	A motivation trace upward	`O01` – `O04`, `R01` – `R04`
12.4	A governing principle per requirement	`R05 PRN-REQ`
12.5	Principles that do work, or are retired	`RPT11 Principle Impact`
12.6	Obligations turned into requirements	`O63`, `R60 O63-REQ`
12.7	Requirements bound to blocks and packages	`O06`, `R32 REQ-APP`
12.8	The honesty score read correctly	`RPT08`, `RPT32`
12.9	The two halves of the register joined	`O06`, `R04 REQ-OBJ`
12.10	An impact assessment for a mid-phase arrival	`03 Change Requests`, `O06`
12.11	A refusal, written down	`O61 Governance Change Request`
12.12	A retirement, so the register stays usable	`O06 Record Status`

The register

Step 12.1 — Hold every requirement in one sheet, whatever phase produced it

Produce. A single register with a `Source` column that says where each requirement came from.

You need. Everything anyone has written down and called a requirement: the Phase A strategic set, the architecture requirements specification derived in Phases B to D, and whatever is in project documents.

Where it lands. `O06 Requirement` — fill `ID`, `Name`, `Description`, `Type`, `Priority`, `Source Driver`, `Objective`, `Owner`, `Status`, `Source`, `Building Block`, `Work Package`, `Acceptance Criteria`.

`Type` is `Architecture`, `Functional` or `Non-functional`. Meridian's split is 56, 10 and 10. `Priority` is `Must` or `Should` — the register has no `Could`, which is a defensible choice. 61 `Must` and 15 `Should`. `Source` says which phase produced it: `Phase A strategic`, `Derived in Phase C`, `Derived in Phase D`. `Status` is `Proposed`, `Approved` or `Baselined`, and records approval, not conformance.

How.

1. Load the Phase A strategic requirements first, with the `REQ-` prefix. These are the ones the business recognises.
2. Load the derived architecture requirements with a prefix that says which domain derived them. The repository uses `ARS-B-`, `ARS-A-`, `ARS-D-` and `ARS-T-` for business, application, data and technology.
3. Set `Source` on every row. Without it you cannot tell a strategic requirement from a derived one, and they need different treatment at every later step.
4. Set `Priority` honestly. Eighty per cent `Must` is what the repository has, and it is what most registers have; it means the priority column is carrying no information and the sequencing decision has been pushed to the roadmap instead.
5. Record the consolidation as a change request. Meridian's is `CR-061`, *Hold all seventy-six requirements in one register*, approved 2027-11-20.

Signed by. Chief Enterprise Architect for the register; the `Owner` per requirement.

Done when. No document outside `O06` contains a sentence that anyone treats as a requirement.

Shows up in. `RPT13 Version Control`, where `O06 Requirement` shows 76 records; and `RPT12 Integrity Check`, which rates it `Thinly linked` at 0.99 links per record [8].

Goes wrong when. `Owner` is left empty because the derived requirements have no obvious owner. Fifty-six of Meridian's seventy-six have no `Owner`. An unowned requirement cannot be refused, amended or retired by anyone, so it stays in the register for ever.

Step 12.2 — Write the acceptance criterion, or do not call it a requirement

Produce. A testable criterion per requirement, in the `Acceptance Criteria` column.

You need. The requirement, the building block that will realise it, and someone prepared to be measured.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *Modelling a regulatory requirement properly* — a worked obligation turned into something a gate can test.

Where it lands. `O06 Requirement` — fill `Acceptance Criteria`. This column is the join to gate `G2` in chapter 10. The `O59 Compliance Assessment` rows a gate produces are one per acceptance criterion; a requirement with an empty cell here generates no row, is assessed by nobody, and passes every gate by default.

The repository's good ones share a shape: a number, a bound and a prohibition.

Requirement	Acceptance criterion as recorded
`ARS-B-01 Party Identity Resolution`	Resolves 95% of presented parties in under 2 seconds; never auto-merges below a confidence threshold
`ARS-B-03 Data Ownership and Stewardship`	No concept without a named owner who exists; no concept with a contested source
`ARS-B-04 Managed Integration`	Every new interface routed through it; no direct database reads; every call attributable
`ARS-T-12 Standards Currency`	No standard past its review date; every standard either `Current` or withdrawn
`ARS-T-13 Environment Register`	Every environment recorded with its data classification; no unmasked personal information outside production

How.

1. Write the criterion as a state of the world, not an activity. *No concept without a named owner who exists* is a state. *Implement data ownership* is an activity and cannot fail.
2. Include at least one prohibition. The prohibitions are what a gate refuses on.
3. Put a number in where a number exists. 95%, 2 seconds, zero generic accounts, 48 hours.
4. Inherit, never duplicate. The criterion belongs to the building block in `O55 Building Block`; `O06` carries the same string. Chapter 10, step 10.3 is the rule that enforces this.
5. Refuse to baseline a `Must` requirement with an empty `Acceptance Criteria`. Twenty of Meridian's are in exactly that state, and all twenty are `Priority = Must`.

Signed by. `GB-01 Architecture Board`, as part of approving the building block.

Done when. Every `Must` requirement has a criterion containing either a number or the word *no*.

Shows up in. `RPT19 Building Block Coverage`, which carries an `Acceptance criteria` column for the fourteen data building blocks, and in `RPT30 Compliance Position`, where the criterion reappears as the thing a verdict was taken against.

Goes wrong when. The criterion is written by the architect to be achievable. It should be written to be refusable. `CA-07`'s criterion in `O59` — *a gateway product is selected and deployed* — is the model: it either happened or it did not, and the repository records that it did not.

THE CRITERION YOU CANNOT YET WRITE

Take your hardest requirement. Why can you not yet write a criterion for it — is the requirement vague, or is nobody prepared to be measured?

Rewrite it as a state of the world rather than an activity. What is the prohibition in it?

What number belongs in it, and who owns the system that would produce that number?

The motivation spine

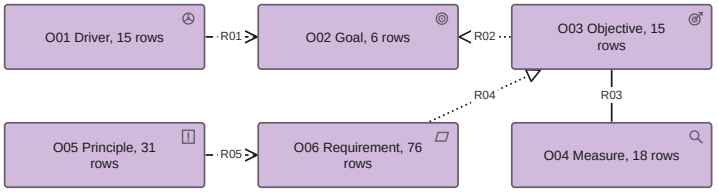

Figure 12.1 The motivation spine as ArchiMate 4 draws it: the driver influences the goal, the outcome realises the goal, the requirement realises the outcome, and the principle influences the requirement. Each join names its relationship sheet and its qualifier column. All four motivation sheets read 0% changed since baseline.

Step 12.3 — Trace the requirement up the spine to an objective, or admit it is a preference

Produce. A chain from driver to measure, with the requirement hanging off the objective.

You need. `O01 Driver`, `O02 Goal`, `O03 Objective`, `O04 Measure`, and the four relationship sheets that join them.

Look it up. Volume 1, chapter 16, *Modelling insurance in ArchiMate, Motivation: from a rule to the system that satisfies it* — the same spine drawn, faster to check than a filtered sheet.

Where it lands. `R04 REQ-OBJ` — fill `Source (REQ)`, `Target (OBJ)`, `Necessity`, `Note`, `Key`.

`Necessity` is the column that does the work. It records how badly the objective needs this requirement. A requirement whose `Necessity` cannot be stated is one the objective would survive without [2].

Draw the spine as an ArchiMate 4 goal realisation view: the Driver with an Influence relationship to the Goal, the Outcome realising the Goal, the Requirement realising the Outcome, and the measure as an Assessment beside it. `O03 Objective` is an Outcome, not a Goal — that is the one mapping people get wrong.

The spine has four joins and each carries a qualifier worth filling:

Join	Sheet	Qualifier column	What it records
Driver to Goal	`R01 DRV-GOL`	`Strength`	how hard the driver pushes the goal
Goal to Objective	`R02 GOL-OBJ`	`Contribution`	how much of the goal this objective delivers
Objective to Measure	`R03 OBJ-MSR`	`Type`, `Reporting`	what kind of measure and how often it is read
Requirement to Objective	`R04 REQ-OBJ`	`Necessity`	whether the objective fails without it

How.

1. Find the objective in `O03 Objective`. Read its `Baseline`, `Target` and `Target Date`. An objective with no `Target` is a wish and the requirement beneath it inherits that.
2. Create the `R04 REQ-OBJ` row and write `Necessity`.

3. Walk up: the objective's `Goal` column points at `O02`, and `R01 DRV-GOL` says which driver pushed the goal. Check the chain exists before you claim it.
4. Check the measure. `O04 Measure` carries `Baseline`, `Current`, `Target`, `Frequency` and `Source System`. If `Source System` is empty, nobody can read the measure and the requirement cannot be shown to have worked.
5. Where no objective exists, stop. Do not create one to justify the requirement. Take the requirement to the business and ask which objective it serves; if the answer is none, the requirement is somebody's preference and the honest action is step 12.11.

Signed by. The objective's `Owner` in `O03`.

Done when. Every `Must` requirement has an `R04 REQ-OBJ` row with a non-empty `Necessity`.

Shows up in. `RPT08 Traceability`, where the `Chain` column reads `Complete` or `BROKEN`. `REQ-03 Contract-level measurement data` reads `BROKEN` because its `Work Packages` count is zero while its driver, goal, objective, principle and application counts are all one [6].

Goes wrong when. The spine is never maintained after the first load. `RPT13` reports `O01 Driver`, `O02 Goal`, `O03 Objective` and `O04 Measure` all at `0` changes logged and `0%` changed since baseline, assessed as `Unchanged since baseline`, while the application and technology layers around them churned through sixty-three change requests [8]. Fifteen drivers that have not moved in fourteen months, in an insurer facing the COFI transition and an IFRS 17 reporting cycle, is not stability.

Step 12.4 — Attach the governing principle, and record the relationship

Produce. A `R05 PRN-REQ` row per requirement, with the nature of the governance stated.

You need. `O05 Principle` (31 rows across Business, Data, Application, Technology, Security & Risk and Governance categories, tiered 1 to 3) and the requirement.

Where it lands. `R05 PRN-REQ` — fill `Source (PRN)`, `Target (REQ)`, `Relationship`, `Note`, `Key`.

`Relationship` says how the principle bears on the requirement — whether the requirement exists because of the principle, is constrained by it, or tests it.

How.

1. Find the principle whose `Statement (condensed)` the requirement enacts. `AP-02 Capability is Exposed Through Managed Interfaces` governs `ARS-B-04 Managed Integration` almost word for word.
2. Create the `R05` row. One requirement may have more than one governing principle; `RPT08` counts them in `Governing Principles`.
3. Check the `Tier`. `GP-03 Principles are Applied as a Set` gives the conflict rule: where two conflict, the higher tier prevails, and where tiers match, the Board decides.
4. Note where the principle and the requirement disagree. That is a Board item, not a modelling problem.
5. Leave it empty rather than attaching a plausible principle. A wrong link is worse than no link because `RPT11` will count it.

Signed by. The principle's `Owner` in `O05` — which ranges from the Architecture Board to the Chief Compliance Officer to the LOB Managing Directors.

Done when. Every requirement with `Type` = `Architecture` has at least one `R05` row.

Shows up in. `RPT11 Principle Impact`, column `Requirements governed`.

Goes wrong when. The principles are all `Status` = `Draft`, as twenty-six of Meridian's thirty-one are, and the links are made anyway. A draft principle governs nothing. Get the set approved or stop citing it [3].

Step 12.5 — Read RPT11 and retire the principles that are decoration

Produce. A shortened, approved principle set where every member governs something.

You need. `RPT11 Principle Impact`.

Where it lands. `RPT11 Principle Impact` — read `Principle`, `Name`, `Category`, `Tier`, `Requirements governed`, `Gaps engaging it`, `Direct violations`, `Assessment`. `Assessment` is derived and takes three values: `Applied`, `VIOLATED`, `Not yet applied`.

How.

1. Read the three summary cells first. Meridian's: principles with a direct violation 14, principles not yet applied anywhere 9, Tier 1 principles violated 7 [4].
2. Filter to `Requirements governed` = 0 *and* `Gaps engaging it` = 0. That is the decoration set. Nine of Meridian's thirty-one qualify, and all five information principles added under `CR-035` — `IP-01` to `IP-05` — are among them. They were approved in September 2027 and have governed nothing since.
3. For each, take one of three actions and record which: link it to the requirements it should govern; demote its `Tier`; or retire it. Do not leave it.
4. Filter to `Assessment` = `VIOLATED` at `Tier` 1. Seven of Meridian's are. A violated Tier 1 principle is either a principle nobody intends to follow or an estate in breach, and `RPT11` does not distinguish — you must.

5. Note the one that matters most. `BP-01 Primacy of Principles` has one governed requirement, one gap engaging it and one direct violation. The principle that says principles apply to everything is itself violated.

Signed by. `GB-01 Architecture Board`, which owns `GP-01`, `GP-02` and `GP-03`.

Done when. No principle reads `Not yet applied` without a dated decision to link, demote or retire it.

Shows up in. `RPT11` itself, where the `Not yet applied` count should fall each quarter.

Goes wrong when. A long principle set is treated as a sign of a mature practice. Thirty-one principles of which nine govern nothing and fourteen are violated is weaker than twelve principles that all bite. Say so to the Board in those words: a principle with zero governed requirements is decoration, and decoration costs credibility at every gate where somebody cites it and is ignored.

Regulation as requirements

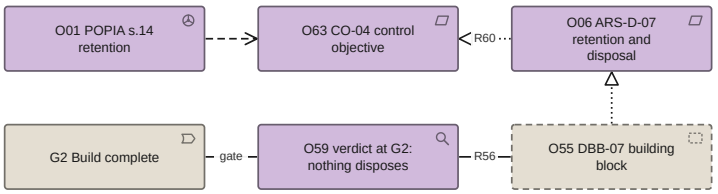

Figure 12.2 One POPIA section, end to end. The obligation is a driver; the control objective and the architecture requirement are both requirements, the second realising the first through `R60`; the building block realises the requirement; the verdict is taken at `G2`. Seven of twelve control objectives have no evidence at all today. This is what stops a regulatory obligation becoming a compliance memo.

THE PRINCIPLE THAT GOVERNS NOTHING

Which principle in your set governs no requirement at all, and which decision should it have governed?

Are you linking it, demoting it, or retiring it? One of the three, with a date.

Which principle is directly violated and governs nothing — the pairing an auditor reads first?

Step 12.6 — Turn a regulatory instrument into a requirement, not a compliance memo

Produce. A control objective per obligation, linked to the requirement that meets it.

You need. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, for the instruments themselves — the Insurance Act and its prudential standards, the Policyholder Protection Rules, FAIS, FICA, POPIA, IFRS 17 and Joint Standard 1 of 2024 on material outsourcing [9][10][11]. Read the regulation there. This step turns it into rows.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *Treating Customers Fairly: six outcomes, six design consequences* — six obligations already carried down to design consequence.

Where it lands. `O63 Control Objective` — fill `ID`, `Obligation`, `Regulatory Source`, `Control Objective`, `Control`, `Evidence Today`, `Requirement`.

`Obligation` is what the law requires, in the law's terms. `Regulatory Source` is the instrument and, where it matters, the section — the repository uses `POPIA s.4`, `POPIA s.11`, `POPIA s.14`, `POPIA s.72`. `Control Objective` is what must be true in the architecture for the obligation to be met. `Control` is the mechanism. `Evidence Today` is the honest state, and it is the column a regulator reads.

The repository's twelve rows are the worked examples. These are the ones to copy the shape of:

Instrument	Obligation	Control objective	Evidence today
Insurance Act	Maintain prudential reporting continuity	No reporting capability is taken offline without proven continuity	Not yet due. Eight capabilities depend on LegacyReporting
FAIS	Distinguish advice from execution-only	Every quotation records whether advice was given and by whom	None. 1.19m quotations a year have no needs analysis behind them
FICA	Screen parties against sanctions and PEP lists	Every party is screened and the result recorded against the party	Partial. The four party masters mean a party can be screened under one identity and transact under another
POPIA s.14	Retain and dispose on a lawful schedule	Every data entity has a retention rule that executes	None. Twenty-four obligations, nothing disposes
IFRS 17	Report contract-level measurement	Measurement data is available at contract level and reconcilable	Partial. It works, and reads FinERP tables directly rather than through an interface
Joint Standard 1 of 2023	Log and monitor material system activity	Every interface and application is authenticated, logged and correlated	None. 44 of 50 interfaces log nothing

How.

1. Write the `Obligation` from the instrument, not from a summary of it. One line, in the law's language.
2. Write the `Control Objective` as a testable state. It becomes an acceptance criterion in step 12.2, so write it as one.
3. Name the `Control`. Where none exists, write the control that should exist — `Consent and purpose registry`, `Retention and disposal engine` — and leave `Evidence Today` reading `None`.
4. Create the `R60 O63-REQ` link to the requirement that meets the objective, carrying `Evidence Today`.
5. Where no requirement exists, write that in the `Requirement` column in words rather than leaving it empty. The repository does this well: `CO-02` and `CO-06` both read *No requirement covers this*, which is why `R60` has ten rows against `O63`'s twelve [5].

Signed by. Chief Risk Officer or Chief Compliance Officer per obligation; `GB-01` for the requirement.

Done when. Every obligation in the register has a control objective, and every control objective either has an `R60` link or a stated reason it does not.

Shows up in. `RPT12 Integrity Check`, where `O63 Control Objective` currently reads `Thinly linked` at 0.83 links per record, which is exactly the ten-of-twelve shortfall.

Goes wrong when. Two things, both present in the repository and both worth fixing in your copy.

- **The instrument has no row at all.** `O63`'s `Regulatory Source` column holds ten distinct values and neither the Policyholder Protection Rules nor Joint Standard 1 of 2024 is among them. Volume 1, chapter 15 covers both — the PPR's data governance, complaints and binder limits, and the outsourcing standard's thirty-day notification before a material outsourcing arrangement and seven days after termination [10][11]. Neither is in the repository. The result is that no architecture requirement anywhere traces to either, and a question about binder limits or material outsourcing has no answer in these sheets.
- **The gap is left unnamed.** `CO-02` on FAIS and `CO-06` on cross-border transfer have no requirement, which means the architecture has no plan to meet them. Naming that in the `Requirement` column is what turns a compliance memo into a backlog item.

Step 12.7 — Bind the requirement to a building block and a work package

Produce. A requirement that somebody is funded to deliver.

You need. `O55 Building Block`, `O30 Work Package`, and `R32 REQ-APP` for the application that will realise it.

Where it lands. `O06 Requirement` — fill `Building Block` and `Work Package`. The building block is which thing realises the requirement; the work package is who is paid to build that thing. Both, or neither is useful.

Draw it as an ArchiMate 4 requirements realisation view: the Requirement, the Grouping that stands for its building block, and the Work Package that realises the Grouping. A requirement with nothing to its left on that view is the `Reaching nothing` count made visible.

How.

1. Set `Building Block` from `O55`. The repository's derived requirements map one to one — `ARS-B-01` to `ABB-01`, `ARS-D-07` to `DBB-07`, `ARS-T-10` to `TEC-BB-10`.
2. Set `Work Package` from the block's own `Work Package` column, not independently. If they disagree, one of them is wrong and the block is the authority.
3. Create the `R32 REQ-APP` row for the application that realises it, and fill `State` — whether the realising application is a baseline component or a target one.
4. Where the block has no work package, leave `Work Package` empty rather than guessing. Twelve of the fifty-six blocks in `RPT24 Building Block Coverage` read `No work package`, and the requirements above them read empty, correctly.
5. Count the empties. Twenty-three of seventy-six requirements reach nothing — that is `RPT32`'s `Reaching nothing` cell — made up of the twenty strategic requirements plus three derived ones whose blocks are unfunded.

Signed by. The work package `Owner` in `O30`.

Done when. Every `Must` requirement either has a `Work Package` or an `O61` row explaining why not.

Shows up in. `RPT32`'s `Building block` and `Work package` columns, and `RPT08`'s `Work Packages` count.

Goes wrong when. `RPT08` and `RPT32` disagree and nobody notices. They disagree today. `REQ-01` reads `Chain = Complete` in `RPT08` with two work packages, and `Work package = none` with `Traced = no` in `RPT32`. Both are right: `RPT08` follows the relationship sheets from requirement to application to work package, while `RPT32` reads `O06`'s own `Work Package` column, which for `REQ-01` is empty. When the two disagree, the relationship sheets are the architecture and the column is the admin — fix the column.

Step 12.8 — Read the gap between Traced and Ever tested as the practice's honesty score

Produce. Two numbers, reported together, every quarter.

You need. `RPT08 Traceability` and `RPT32 Requirement Traceability`.

Where it lands. `RPT32 Requirement Traceability` — read `Ref`, `Requirement`, `Source`, `Priority`, `Building block`, `Work package`, `Traced`, `Ever tested`. The summary block below gives requirements, reaching nothing, and ever tested.

Number	Where	Meridian today	What it means
Requirements	`RPT32`	76	the size of the register
Reaching nothing	`RPT32`	23	requirements no package will deliver
`Traced = yes`	`RPT32`	53	requirements bound to a block and a package
`Ever tested = yes`	`RPT32`	0	requirements a gate has actually assessed
Complete chain	`RPT08`	16 of 76	requirements traceable end to end both ways

How.

1. Report `Traced` and `Ever tested` side by side and never separately. Fifty-three traced and zero tested is the honest statement of a practice that has designed and not yet governed.
2. Read `RPT32`'s closing note before you explain the zero: the `Status` column records approval, not conformance, and no gate existed to test any of them until Phase G [7]. The zero is a consequence of chapter 10's finding, not an independent failure.
3. Set the target. Every `Must` requirement attached to a package that has passed its `End` date should read `Ever tested = yes`. Four packages in `O30` are past their end date; their requirements are the first batch.
4. Read `RPT08`'s `BROKEN` rows and fix them in two groups: the fifty-six with `** no driver **` need an `R04` row (step 12.9); the handful like `REQ-03` with everything but a work package need the binding from step 12.7.
5. Track the two numbers as a pair over quarters. The gap closing is the only evidence that Requirements Management is running.

Signed by. Chief Enterprise Architect.

Done when. The two numbers appear on the same line of the same paper.

Shows up in. `RPT30 Compliance Position` as a rising assessed count, because a tested requirement is an `O59` row.

Goes wrong when. `Traced` is reported alone, as coverage. Fifty-three of seventy-six is 70% and reads as progress. Zero tested is the number that says what 70% coverage is currently worth.

TRACED AGAINST EVER TESTED

What are your two numbers, on one line? Traced, and ever tested.

Which requirements belong to a package already past its `End` date — the first batch a gate should assess?

Which requirement reaches nothing at all, and is that because nobody designed for it or nobody asked for it?

Step 12.9 — Join the two halves of the register

Produce. A register where every requirement has both an upward trace and a downward binding.

You need. `O06` filtered by `Source`, `R04 REQ-OBJ`, and the derived requirements' building blocks.

Look it up. Volume 1, chapter 19, appendix A, *The cross-maps*, *What these are, and what they are not* — why a pre-drafted register is a starting position and not an answer.

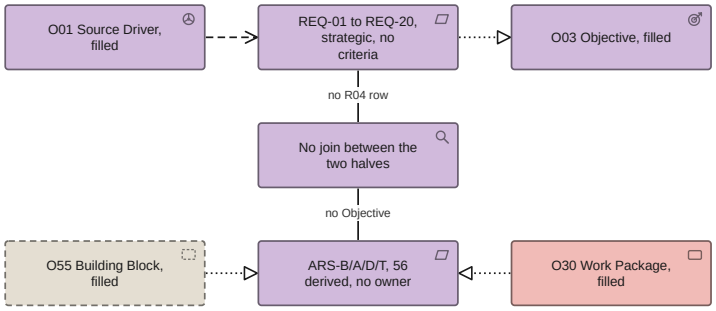

Figure 12.3 The structural defect in `O06 Requirement`, one register in two halves. Twenty strategic requirements trace up to a driver and an objective and bind to nothing below. Fifty-six derived requirements are realised by a block and a package and trace to nothing above. The assessment between them is what step 12.9 closes.

Where it lands. `O06 Requirement` — for the twenty `Phase A strategic` rows, fill `Building Block`, `Work Package` and `Acceptance Criteria`. For the fifty-six derived rows, fill `Source Driver`, `Objective` and `Owner`, and create the `R04 REQ-OBJ` rows.

How.

1. Take the twenty strategic requirements. For each, find the derived requirement that implements it. `REQ-01 Single enterprise customer identifier` is implemented by `ARS-B-01 Party Identity Resolution`; `REQ-02 End-to-end customer timeline` by `ARS-B-02 Party Timeline`.
2. Where the match is clean, copy the derived requirement's `Building Block`, `Work Package` and `Acceptance Criteria` up to the strategic row. The strategic requirement becomes testable without being rewritten.
3. Copy the strategic requirement's `Source Driver`, `Objective` and `Owner` down to the derived row, and create the `R04 REQ-OBJ` link with a `Necessity` value.
4. Where there is no match, you have found something real: either a strategic requirement nobody designed for, or a derived requirement nobody asked for. Record both as `O61` rows of `Type` = `Compliance` against `Repository`.
5. Do not merge the two rows. The strategic requirement is what the business committed to; the derived one is how the architecture meets it. Collapsing them loses the distinction that makes the register readable.

Signed by. `GB-01 Architecture Board`, as a single change request covering the whole join.

Done when. No row in `O06` has an empty `Objective` and an empty `Acceptance Criteria` at the same time — one or the other must be present, and for a `Must` requirement both must be.

Shows up in. `RPT08`'s complete-chain count, which should move from 16 towards 76, and `RPT12`'s `Links per record` for `O06`, which should rise above 1.0 and off the `Thinly linked` assessment.

Goes wrong when. The join is attempted as a bulk exercise and abandoned at row thirty. Do it per work package, at the gate, when the requirements for that package are in front of you anyway. Twenty strategic requirements across the four Wave 1 packages is an afternoon; seventy-six in one sitting is a project nobody finishes.

Requirements that arrive, and requirements that leave

WHERE THE TWO HALVES DO NOT MEET

Which strategic requirement has no derived requirement implementing it — and which of the two is wrong?

Which derived requirement nobody asked for, and who would miss it if you retired it?

Which strategic requirement became testable by inheriting a criterion rather than being rewritten?

Working table — joining the two halves, one work package at a time

One row per strategic requirement in the package in front of you. Where the match is clean, the criterion and the block come up and the driver, objective and owner go down. Where there is no match, that row is the finding — record it as an `O61` row of `Type = Compliance`. Signed by `GB-01` as one change request covering the join.

REQ- ID	Derived ARS- ID	Building block	Work package	Match?
REQ-01	ARS-B-01	ABB-04	WP-04	Clean

Step 12.10 — Assess a requirement that arrives mid-phase before you accept it

Produce. An impact assessment attached to a change request, before the requirement enters `O06`.

You need. The new requirement, the phase you are in, and the work packages already committed.

Look it up. Volume 1, chapter 12, *The application landscape*, 2. *Product configuration and the product factory* — how to tell a configuration change from an architecture one.

Where it lands. `03 Change Requests` — fill `Type = Addition`, and `Scope` with every object type the assessment finds. Meridian's `CR-013`, *Add climate exposure requirement*, raised off PA Guidance Notice `G3/2025`, has `Scope = Requirement, Gap, Work Package` and produced three log rows in seven days.

The assessment is six questions, in this order, against these sheets:

Question	Sheet to read	If the answer is bad
Which objective does it serve?	`003 Objective`	no objective: refuse, step 12.11
Which principle governs it?	`005 Principle`	a Tier 1 principle it violates: Board decision
Which building block realises it?	`055 Building Block`	none: a new block, and a new gap in `029`
Which work package pays for it?	`030 Work Package`	none: it is unfunded and must be said so
What does it do to the sequence?	`RPT27 Programme Sequencing`	slip months rise: `GB-02` decides
Does it change an acceptance criterion already at a gate?	`059`, `055`	yes: re-architecting, not addition

How.

1. Run the six questions and write the answers into the change request before it is tabled.
2. Add the requirement to `O06` with `Status = Proposed`, not `Approved`. Thirteen of Meridian's seventy-six sit at `Proposed` and that is the correct resting state for a requirement under assessment.
3. If it needs a new building block, raise the gap in `029 Gap` in the same request. A requirement with a block that has no gap record is invisible to every roadmap report.
4. If it changes a criterion already assessed at a gate, stop and reclassify the change as re-architecting per chapter 11, step 11.4. A criterion that moves after a verdict invalidates the verdict.
5. Decide it at the body that owns the consequence: `GB-01` for the architecture, `GB-02` where cost or sequence moves.

Signed by. `GB-01`; `GB-02` where the answer to question five is non-zero.

Done when. The request carries all six answers and a decision date.

Shows up in. `RPT13`'s change request block, with a `Changes` count that matches the log rows.

Goes wrong when. The requirement is accepted into `O06` while the assessment is pending, because accepting it looks cooperative. It then appears in every traceability report as an untraced `Must`, and the practice's honesty score falls for a reason that is nobody's fault.

Step 12.11 — Refuse a requirement in writing, with the reason in a column

Produce. A recorded refusal that survives the next reorganisation.

You need. The requirement, the failed test from step 12.10, and `O61`.

Where it lands. `O61 Governance Change Request` — fill `Title`, `Against` = `Repository`, `Type` = `Compliance`, `Why`, `Status`, `Consequence`. Leave `Expires` empty: a refusal is not time-bound, which is the one case where an empty `Expires` is correct.

Refuse on one of four grounds, and name which:

- **No objective.** It traces to nothing in `O03`. It is a preference, however senior the person holding it.
- **No criterion.** Nobody can write a testable `Acceptance Criteria` for it, so no gate can assess it.
- **Principle conflict.** It violates a Tier 1 principle in `O05` and the Board has not granted a dispensation.
- **Unfunded with no gap.** It needs a building block nobody will pay for and no `O29 Gap` record carries it.

How.

1. Write the `O61` row. The refusal is a record, not a conversation.
2. Write `Consequence` from the requester's point of view: what they will not get, and under what condition the refusal would be revisited.
3. Create the `O60 Governance Decision` row and link it in `R57 O60-O61`.
4. Tell the requester the ground, by name. *No objective* is a different conversation from *no criterion*, and the second is often fixable in an afternoon.
5. Do not delete the requirement from `O06` if it was already loaded. Set `Record Status` to `Retired` per step 12.12 so the refusal remains discoverable.

Signed by. `GB-01 Architecture Board`.

Done when. The refusal has an `O61` row and an `O60` decision with a date.

Shows up in. `RPT31 Governance Activity`, as a `Change request` row. `O57` records that Meridian's Board *has never refused anything*; a refusal row is how that changes.

Goes wrong when. The refusal is delivered verbally and the requirement comes back in six months with a different name. The `O61` row is what makes the second arrival a re-raise rather than a new idea.

Step 12.12 — Retire requirements, so the register stays worth reading

Produce. A register where every live row is live.

You need. `O06`'s `Record Status` and `Status` columns, and the reason the requirement is done or dead.

Where it lands. `O06 Requirement` — set `Record Status` to `Retired`, and log the change in `02 Change Log` with `Field` = `Record Status`, `Previous Value` = `Baseline`, `New Value` = `Retired` and a `Reason`.

Do not delete the row. Do not reuse the `ID`. Volume 1 appendix A's second rule applies: retire IDs rather than reusing them, as you would product versions.

Retire on four grounds, each with its own evidence:

Ground	Evidence	Where the evidence lives
Met	A `Compliant` verdict at `G2` or `G3`	`O59 Compliance Assessment`
Superseded	A later requirement covers it	the superseding `ID` in the log `Reason`
Refused	An `O61` row and an `O60` decision	`O61`, `R57 O60-O61`
Obsolete	The driver behind it is no longer `Active`	`O01 Driver Status`

How.

1. Run the four grounds across the register once a quarter, after the gate cycle.
2. Retire `Met` requirements only from an `O59` verdict. A delivery report is not evidence; see chapter 10, step 10.10.
3. Retire `Obsolete` requirements when the driver's `Status` in `O01` changes from `Active`. Check this at the annual review, since the drivers are the thing least likely to have been maintained.
4. Log every retirement. A retirement is a field change like any other and the comparison tool will classify it as `UNGOVERNED CHANGE` if it is not logged.
5. Keep the retired rows in the sheet. `RPT13`'s `Records` counts and `RPT12`'s link counts both read the whole sheet, so a retired row stays visible as history and stops the ID being reused.

Signed by. The requirement's `Owner`; `GB-01` for anything `Baselined`.

Done when. No requirement in the register is `Baselined` against a driver whose `Status` is not `Active`, and no `Met` requirement is still live.

Shows up in. `RPT32`'s requirement count, which should fall as well as rise. A register that only grows is a register nobody is using to make decisions.

Goes wrong when. Nothing is ever retired, on the reasonable-sounding ground that the record is cheap to keep. It is not. Seventy-six requirements with twenty untestable and twenty-three reaching nothing is already a register a project manager will skim rather than read, and

skimming is how a `Must` requirement gets missed. A repository that never retires anything is as useless as one that records nothing, and it takes longer to discover.

The one test for this discipline

Pick any requirement in your register at random, and time yourself.

REQUIREMENTS MANAGEMENT — A REVIEWER'S QUESTIONS

Which objective does this requirement serve, and how badly does that objective need it? `O03` and `R04 REQ-OBJ Necessity`.

Which principle governs it, and is that link recorded in `R05 PRN-REQ`?

What would make it pass a gate? Quote the `Acceptance Criteria` cell.

Has a gate ever assessed it, and what did the verdict say?

In the repository as delivered, question four has the same answer for all seventy-six rows, and question one has no answer for fifty-six of them. If you can answer all four for a requirement you picked blind, in under a minute, Requirements Management is running. If you cannot, the other eleven chapters produced documents.

THE CHAPTER IN TEN LINES

- A requirement without an Acceptance Criteria value cannot be tested at G2, so it is not a requirement.
- The register is in two halves: 20 strategic rows with no criteria, 56 derived rows with no objective. Join them per package.
- R04 REQ-OBJ Necessity is the test of whether an objective would survive without the requirement.
- The whole motivation spine — O01 to O04 — reads 0% changed since baseline while everything around it churned.
- RPT11: nine principles govern nothing and seven Tier 1 principles are violated. A principle with zero governed requirements is decoration.
- O63 turns an obligation into a control objective, a control and an Evidence Today value a regulator reads.
- Neither the Policyholder Protection Rules nor Joint Standard 1 of 2024 has an O63 row. Volume 1, chapter 15 has both.
- RPT08 and RPT32 disagree on REQ-01 because one follows relationship sheets and one reads a column. Fix the column.
- Traced 53, ever tested 0. Report the two numbers on one line or the first one misleads.
- Retire on four grounds — met, superseded, refused, obsolete — and log every retirement, or the tool calls it ungoverned.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheet O06 Requirement
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets O01 Driver, O02 Goal, O03 Objective, O04 Measure, and R01–R04
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets O05 Principle and R05 PRN-REQ
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT11 Principle Impact
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheet O63 Control Objective, and R60 O63-REQ
- **6.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT08 Traceability
- **7.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT32 Requirement Traceability
- **8.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT12 Integrity Check and RPT13 Version Control
- **9.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Requirements Management https://pubs.opengroup.org/togaf-standard/adm/
- **10.** National Treasury, "Release of final insurance conduct regulations and policy" (15 December 2017), on the Policyholder Protection Rules https://www.treasury.gov.za/comm_media/press/2017/2017121501%20Media%20statement%20release%20of%20final%20insurance%20conduct%20regulations%20and%20policy.pdf
- **11.** Bowmans, "South Africa: new requirements for outsourcing by insurers published in Joint Standard 1 of 2024" https://bowmanslaw.com/insights/south-africa-new-requirements-for-outsourcing-by-insurers-published-in-joint-standard-1-of-2024/

APPENDIX A

The sheet-by-sheet reference

One hundred and sixty-two sheets, what each holds, which columns carry judgement, and the four relationship sheets that do not exist.

You are holding 162 sheets. Fifteen of them matter every week, forty matter in the phase you are in, and the rest you open once and never again. This appendix tells you which is which, what each one holds, what to type in it, and when.

Meridian Assurance Group is an invented insurer. Its data is real within this workbook and nowhere else.

The shape of the repository in one page

Five families of sheet, in this order along the top of the workbook [1].

Family	Sheets	Count	What it is for	Do you type in it?
Governance	`00 Index`, `01 Version Control`, `02 Change Log`, `03 Change Requests`, `04 Baselines`	5	The repository's record of itself: contents, versioning policy, every field change, the change-request register, and named approved snapshots	Yes — `02` and `03` every time you change anything
Objects	`O01 Driver` to `O66 Architecture Increment`	66	One sheet per metamodel entity. ID in column A, Name in column B, always	Yes — this is where the architecture lives
Relationships	`R01 DRV-GOL` to `R60 O63-REQ`	56	One sheet per relationship type. Source ID, target ID, and one column of judgement	Yes — and it is the half people skip
Export	`05 Export Attributes`, `06 Export Records`, `07 Export Change Log`	3	Live formula extracts shaped for the comparison tool's three inputs	No — copy out, never type in
Reports	`RPT01 Dashboard` to `RPT32 Requirement Traceability`	32	Every cell a formula over the object and relationship sheets	No — read only; a typed cell is a broken report

Five plus sixty-six plus fifty-six plus three plus thirty-two is 162.

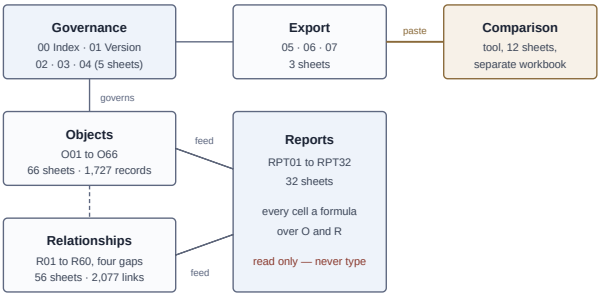

Figure 13.1 The five families of sheet. Objects and relationships are typed; reports and exports are formulas over them; the comparison tool is a separate workbook fed by paste.

> **THE R-SHEET NUMBERING HAS FOUR HOLES**
>
> The relationship sheets run `R01` to `R60` but there are **fifty-six of them**, not sixty. `R45`, `R51`, `R52` and `R53` do not exist. The numbers were allocated when the Phase E and F extensions were drafted under change requests CR-047 to CR-054 and four of the intended sheets were never cut. Nothing is missing from the model — the supertype relationship `R42 O55-O55` absorbed what `R51` to `R53` were meant to carry — but a macro that walks `R01` to `R60` in a loop will throw four times. Walk the sheet names, not the numbers [2].

Three counts in the repository's own documentation are wrong, and you should know before someone quotes them at you. `00 Index` describes itself as "33 object types, 36 relationship types, 13 reports". The real figures, which `RPT01 Dashboard` computes correctly, are 66, 56 and 32 — the header text was written against the first load and never updated as change requests CR-025 to CR-061 added forty sheets. `RPT12 Integrity Check` says it "counts every appearance of each object type across all 34 relationship sheets"; it counts across all 56 [3].

The object catalogue

Six tables, by domain. `ID prefix` is taken from the data in column A, not from the `Code` column on `00 Index` — the two disagree in several places and the data wins. `ArchiMate 4 element` is the element to use when the sheet's rows are drawn rather than listed; where nothing in the notation fits, the cell says so rather than forcing one. **These columns are a summary; appendix D, *The repository in ArchiMate 4*, carries the reasoning and is the authority where the two appear to differ.** `Phase · year one` gives the ADM phase that fills the sheet, then a judgement about an insurer standing a

practice up from nothing: **yes** means the sheet earns its keep in the first twelve months, **if relevant** means fill it when the phase or the business makes it necessary, **later** means a mature practice needs it and a new one does not.

Motivation — why the architecture exists

Sheet	What it holds	ID prefix	ArchiMate 4 element	The columns that matter	Phase · year one
`001 Driver`	External and internal conditions that force the enterprise to set goals — COFI, IFRS 17, Joint Standard 1 of 2023	`DRV-`	Driver	`Category`, `Origin`, `Horizon`	Preliminary, A · yes
`002 Goal`	High-level intent the group sets for itself	`GOL-`	Goal	`Pillar`, `Owner`, `Target Date`	A · yes
`003 Objective`	Time-bounded measurable intent realising a goal	`OBJ-`	Outcome	`Baseline`, `Target`, `Target Date`	A · yes
`004 Measure`	The indicator that says whether an objective is met	`MSR-`	no direct equivalent — the values sit on the Outcome	`Baseline`, `Current`, `Target`, `Source System`	A · yes
`005 Principle`	Thirty-one principles in seven families across six categories	`BP- DP- AP- IP- TP- SP- GP-`	Principle	`Tier`, `Statement (condensed)`, `Category`	Preliminary · yes
`006 Requirement`	Twenty strategic requirements plus fifty-six derived in Phases C and D	`REQ-`, `ARS-A-`, `ARS-B-`, `ARS-D-`, `ARS-T-`	Requirement	`Priority`, `Source Driver`, `Building Block`, `Acceptance Criteria`	A, then C and D · yes
`007 Constraint`	What prevents particular approaches, with an expiry	`CON-`	no direct equivalent — carry as a Requirement	`Type`, `Applies Until`	A · yes
`008 Assumption`	What is taken as true pending validation	`ASM-`	no direct equivalent — repository only	`Confidence`, `Validate By`	A · yes

`005` is the one most often filled badly. `Tier` is the conflict-resolution rule, not a priority score: nineteen of the thirty-one are Tier 1, ten are Tier 2, two are Tier 3, and `GP-03 Principles are Applied as a Set` states that where two principles conflict the higher tier prevails. If you tier everything 1 you have no tie-break. All thirty-one are still `Status` = Draft, which is itself the Preliminary Phase finding [4].

Organisation and business — who does what, and what the business does

Sheet	What it holds	ID prefix	ArchiMate 4 element	The columns that matter	Phase · year one
`009 Organization Unit`	Sixteen units with parent, head, headcount, location	`ORG-`	Business Actor	`Parent`, `Head`, `Headcount`	Preliminary · yes
`010 Actor`	Twenty actors, internal and external, with population	`ACT-`	Business Actor	`Internal/External`, `Population`	B · yes
`011 Role`	Twenty-seven roles with the authority each carries	`ROL-`	Role	`Authority Level`, `Domain`	B · yes
`012 Location`	Ten sites with criticality	`LOC-`	Location	`Type`, `Criticality`	B · if relevant
`013 Business Capability`	556 capabilities on four levels; assessment at Level 3 only	`C01` to `C12.05.03.03`	Capability	`Level`, `Criticality`, `Current Maturity`, `Target Maturity`, `Disposition`, `Differentiation`	B · yes
`014 Value Stream`	Seven streams with triggering stakeholder and stages	`VST-`	Value Stream	`Triggering Stakeholder`, `Value Proposition`, `Stages`	B · yes
`015 Business Service`	Twenty services with the capability each realises	`BSV-`	Service	`Capability`, `Consumer`, `Channel Agnostic`	B · yes
`016 Business Process`	Twenty-two processes with volume and cycle time	`BPR-`	Process	`Owner Role`, `Automation Level`, `Volume p.a.`, `Cycle Time`	B · yes
`017 Business Function`	Twelve functions, the organisational cut of behaviour	`FUN-`	Function	`Organization Unit`, `Type`	B · if relevant
`018 Product`	Eighteen products with policies in force and premium	`PRD-`	Product	`Line of Business`, `Policies In Force`, `GWP (Rm)`, `Config or Code`	B · yes
`019 Contract`	Ten supplier and outsourcing agreements	`CTR-`	Business Object	`Counterparty`, `Expiry`, `Notice Period`	B · if relevant
`035 Business Building Block`	Fourteen packages of required capability, defined by what they must do	`ABB-`	Grouping	`What It Must Do`, `Acceptance Criteria`, `Closes Gap`, `Work Package`	B, confirmed in E · yes

`O13` is the largest sheet in the repository and the one that will consume most of your first quarter. Twelve Level 1 areas, forty-six Level 2, 134 Level 3, 364 Level 4. Only the 134 Level 3 rows carry `Current Maturity`, `Target Maturity`, `Criticality`, `Disposition` and `Differentiation` — the other 422 rows are deliberately blank in those columns and `RPT03 Capability Heatmap` rolls Levels 1 and 2 up by formula. Filling maturity at Level 4 does not improve the heatmap; it breaks the averages.

Volume 1, chapter 19, appendix A, *The cross-maps*, already drafts most of this half: `P-01` – `P-12` processes, `C-01` – `C-20` capabilities, `R-01` – `R-33` roles. Start from it, then argue with it — and **map by name, never by number.** Twelve drafted processes are not 22 `BPR-` rows, twenty capabilities are not 556 four-level `C01.02.03` codes, thirty-three roles are not 27 `ROL-` rows. The schemes only look alike.

Data — the domain where insurers fail

Sheet	What it holds	ID prefix	ArchiMate 4 element	The columns that matter	Phase · year one
`O20 Data Entity`	Twenty-four entities a business expert would recognise	`DEN-`	Data Object; Business Object on conceptual views	`Authoritative Source`, `Owner Role`, `Tier`, `Classification`, `Quality`, `Retention Period`, `Retention Basis`, `Mastering Style`	C (data) · yes
`O21 Logical Data Component`	Twelve logical stores, each holding named entities	`LDC-`	Grouping	`Entities Held`, `State`, `Owner Role`	C · yes
`O22 Physical Data Component`	Eighteen physical stores with the technology under them	`PDC-`	Artifact	`Realises`, `Technology`, `Hosted On`, `State`	C · yes
`O36 Data Entity Relationship`	Thirty-one entity-to-entity relationships with cardinality	`DER-`	no direct equivalent — an Association between two Data Objects	`Relationship`, `Cardinality`	C · yes
`O37 Data Attribute`	161 key attributes: identity, classification, regulatory consequence	`ATT-`	no direct equivalent — a property of the Data Object	`Key`, `Personal Information`	C · if relevant
`O38 Data Set`	Nine assembled collections with their lawful basis	`DS-`	Data Object	`Classification`, `Lawful Basis`, `Refresh`, `Consumer`	C · yes
`O39 Data Migration`	Six movements with approach and a quality gate	`MIG-`	no direct equivalent — properties on the Work Package the row names	`Approach`, `Quality Gate`, `Gate Status`, `Wave`	F · if relevant
`O40 Source To Target Mapping`	Fourteen entity-level mappings	`STM-`	no direct equivalent — nearest is a Flow between the two stores	`Transformation Rule`, `Quality Issue To Resolve First`	F · if relevant
`O42 Metadata Element`	Twenty-two metadata elements in four categories	`MDE-`	no direct equivalent — repository only	`What It Records`, `Held In`, `Coverage`	C · later
`O43 Master Data Rule`	Twelve golden-record and survivorship rules	`MDR-`	no direct equivalent — a Requirement per entity where it must be traced	`Golden Record Definition`, `Survivorship Rule`, `Mastering Status`, `Implemented`	C · yes
`O44 Data Building Block`	Fourteen data mechanisms beneath the business blocks	`DBB-`	Grouping	`What It Must Do`, `Serves`, `Acceptance Criteria`, `Closes Gap`	C, confirmed in E · yes

`Authoritative Source` on `O20` takes three non-application values and they are the point of the column. `APP-nn` means one named master. `SPLIT` means authority is legitimately divided by segment or line and the split is recorded. `CONTESTED` means two or more applications claim the same authority and nobody has decided — seven entities are `SPLIT` and three are `CONTESTED` at Meridian, and `RPT05 App-Data Matrix` surfaces every one. `NONE` on `DEN-nn` means no application masters it at all. Never leave the cell blank; blank reads as "not assessed" and `CONTESTED` reads as "assessed, and the answer is ugly" [5].

`Owner Role` is the column where good intentions go to die. Twenty-three of the twenty-four entities point at `ROL-09 Data Owner`, which is a role with no actor behind it. `RPT29 Transition Readiness` holds the exit criterion in plain words: *every data entity has a named individual as owner, not a role*. It fails today.

Volume 1, chapter 19, appendix A, *Conceptual data entities*, drafts `D-01` – `D-20`; chapter 9, *The conceptual data model*, argues each one in. The repository holds 24 `DEN-` rows. Match on name; the numbers do not correspond.

Application and technology — what runs

Sheet	What it holds	ID prefix	ArchiMate 4 element	The columns that matter	Phase · year one
`023 Application Component`	Twenty-six applications with cost, value and fitness scores	`APP-`	Application Component	`Criticality`, `Lifecycle`, `Disposition`, `Annual Run Cost (Rm)`, `Business Value`, `Technical Fitness`, `Portfolio Quadrant`, `Disposition Agrees`	C (application) · yes
`024 Information System Service`	Twenty automated services with availability targets	`ISS-`	Service	`Realises Business Service`, `Interface Style`, `Availability Target`	C · yes
`045 Application Building Block`	Fourteen application mechanisms required at target	`APP-BB-`	Grouping	`What It Must Do`, `Serves`, `Acceptance Criteria`, `Closes Gap`	C, confirmed in E · yes
`025 Technology Service`	Fourteen technical capabilities with availability targets	`TSV-`	Service	`Category`, `Availability Target`, `Consumers`	D · yes
`026 Logical Technology Component`	Fifteen vendor-independent infrastructure encapsulations	`LTC-`	Node	`Provides Service`, `Standard Status`	D · if relevant
`027 Physical Technology Component`	Eighteen deployed products with support end dates	`PTC-`	Node, Device, System Software or Communication Network by `Category`	`Vendor / Product`, `Lifecycle`, `Support Ends`, `Standard Status`, `Annual Cost (Rm)`	D · yes
`028 Platform Service`	Eight standard platform offerings	`PSV-`	Service	`Technology Service`, `Consumption Model`, `State`	D · if relevant
`046 Technology Building Block`	Fourteen technology mechanisms required at target	`TEC-BB-`	Grouping	`What It Must Do`, `Serves`, `Acceptance Criteria`, `Closes Gap`	D, confirmed in E · yes
`048 Environment`	Four environments and the data each holds	`ENV-`	Grouping	`Data It Holds`, `Where It Runs`, `Provenance`	D · yes
`049 Resilience Tier`	Twelve recovery objectives per Level 1 capability	`RES-`	no direct equivalent — properties on the capability	`RTO`, `RPO`, `Test Cadence`, `Last Tested`	D · yes
`050 Network Link`	Ten inter-site and external links	`LNK-`	Path, realised by Communication Network	`Type`, `What It Carries`, `Provenance`	D · if relevant

`023` carries a computed column and a check on it. `Portfolio Quadrant` is derived from `Business Value` and `Technical Fitness`; `Disposition Agrees` compares the computed quadrant against the `Disposition` someone typed. Six of twenty-six disagree. That column is the most useful single cell on the sheet, because it finds the applications where the portfolio story and the portfolio data have come apart.

`027` has one column worth a standing calendar reminder. `Support Ends` holds real vendor dates: three platforms go out of support in 2027 and they carry sixteen application deployments between them, which `RPT21 Software Distribution` totals for you [6].

Note the prefix collision: `023 Application Component` uses `APP-nn` and `045 Application Building Block` uses `APP-BB-nn`. They are different things — a component exists, a building block must exist — and `R48 O51-O55` will resolve either, so a mistyped `APP-07` where `APP-BB-07` was meant will not error. It will just be wrong.

Volume 1, chapter 19, appendix A, *Conceptual applications*, drafts `A-01` – `A-25`; chapter 11, *The conceptual application model*, says what belongs in each box. The repository holds 26 `APP-` rows. The counts are close and the sets are not: match on name, and expect a handful of the twenty-five to have no row here.

Realisation and delivery — what gets done about it

Sheet	What it holds	ID prefix	ArchiMate 4 element	The columns that matter	Phase · year one
`029 Gap`	Eighteen gaps, each with a severity and a domain	`GAP-`	no direct equivalent — carry as Assessment	`Domain`, `Gap Type`, `Severity`, `Affected Object`, `Addressed By`	B, C, D · yes
`030 Work Package`	Sixteen packages with wave, cost, benefit and delivery state	`WP-`	Work Package	`Wave`, `Start`, `End`, `Cost (Rm)`, `Benefit (Rm p.a.)`, `Transition State`, `Delivery Status`	E · yes
`031 Deliverable`	Twenty contractual outputs of work packages	`DLV-`	Deliverable	`Type`, `ADM Phase`, `Work Package`, `Due`, `Status`	E, F · yes
`032 Standard`	Sixteen governed specifications with a review date	`STD-`	Requirement, with the specification as an Artifact	`Status`, `Applies To`, `Review Due`	Preliminary, D · yes
`033 Architecture Contract`	Seven contracts; compliance derived, not asserted	`ACO-`	no direct equivalent — Business Object only where a view needs it	`Parties`, `Signed`, `Expiry`, `Compliance Status`, `Status Basis`	G · if relevant
`051 Solution Building Block`	Twenty-nine buy, build or configure decisions	`SBB-`	Grouping while a candidate, the real element once chosen	`Sourcing`, `Realises`, `Work Package`, `Provenance`	E · yes
`052 Implementation Factor`	Twenty-six factors, each with the deduction that follows	`IF-`	no direct equivalent — repository only	`Factor`, `Deduction`, `Affects`, `Changes Sequence`	E · yes
`053 Transition Architecture`	Three states the enterprise can stop at safely	`TA-1` to `TA-3`	Plateau	`Window`, `Exit Criteria`, `Still Broken Here`, `Standing`	E, F · yes
`054 Architecture Alternative`	Five options with cost, benefit and when to choose each	`A` to `E`	Course of Action	`Committed Cost (Rm)`, `Fits 2030`, `When To Choose It`, `Recommended`	E · if relevant
`055 Building Block`	Fifty-six blocks: a composed view over `035`, `044`, `045`, `046`	`ABB-` `DBB-` `APP-BB-` `TEC-BB-`	no element of its own — a composed view	`Domain`, `Held In` — everything else is read through	E · read only
`056 Project`	Fifteen chartered, fundable units of delivery	`PRJ-`	Work Package	`Work Packages`, `Team`, `Lead`, `Depends On`, `Override`, `Why This Shape`	F · yes
`062 Delivery Risk`	Eight risks with likelihood, impact, mitigation, owner	`RSK-`	no direct equivalent — repository only	`Likelihood`, `Impact`, `Mitigation`, `Owner`, `Status`	F, G · yes
`066 Architecture Increment`	Six increments: what is added at each step	`INC-1` to `INC-6`	no direct equivalent — a Grouping over what it adds	`Lands`, `What Is Added`, `What Becomes Possible`	F · if relevant

`055` is the trap on this table. It is a **composed view, not a store**: its rows are read from `035`, `044`, `045` and `046` at load and nothing is entered directly. It exists because three relationship sheets needed a supertype to point at, which is why `R42`, `R48` and `R56` all target `055` rather than four separate sheets. Type a new row into `055` and it will survive until the next reload and then vanish, taking the relationship rows that pointed at it [7].

`030` has a column added late and for a good reason. `Status` records an approval state; `Delivery Status` records whether the thing landed. Before `Delivery Status` existed, a package that had not been built looked identical to one that had. Nine packages are In flight, four are Past end date, not evidenced, three are Not started, and none is recorded as delivered.

Governance and security — the half that makes the rest count

Sheet	What it holds	ID prefix	ArchiMate 4 element	The columns that matter	Phase · year one
`034 Architecture Deliverable`	Twenty architecture work products under version control	`DLV-`	Deliverable	`Artifact Type`, `ADM Phase`, `Deliverable Version`, `Lifecycle Status`, `Approver`	Preliminary, continuous · yes
`041 Security Control`	Eight controls on data, with the gap where absent	`SEC-C`	no direct equivalent — carry as Requirement	`Covers`, `State`, `Gap`	C, D · yes
`047 Reference Model Entry`	Thirty-four purposes with the approved answer for each	`REF-A-`, `REF-T-`	no direct equivalent — a property on the designated component	`The Need`, `Standard Today`, `At Target`, `When To Use It`, `When Not To`	Preliminary, D · yes
`057 Governance Body`	Five bodies: mandate, limits, and whether each functions	`GB-`	Collaboration; Business Actor where one unit	`Cadence`, `May Decide`, `May Not Decide`, `Standing`	Preliminary · yes
`058 Governance Gate`	Four gates: G0 charter, G1 design, G2 build, G3 production	`G0 to G3`	Event	`When`, `What It Tests`, `Held By`	Preliminary, G · yes
`059 Compliance Assessment`	Thirteen verdicts against one criterion at one gate	`CA-`	Assessment	`Acceptance Criterion`, `Gate`, `Verdict`, `Evidence`	G · yes
`060 Governance Decision`	Seven decisions taken or outstanding at a body	`DEC-G`	no direct equivalent — repository only	`Date`, `Body`, `Status`, `Why`, `Affects`	G, H · yes
`061 Governance Change Request`	Five requests raised against a project or package	`CR-G`	no direct equivalent — repository only	`Against`, `Type`, `Status`, `Expires`, `Consequence`	G, H · yes
`063 Control Objective`	Twelve regulatory obligations and the evidence for each	`CO-`	Requirement	`Regulatory Source`, `Control`, `Evidence Today`	Preliminary, A · yes
`064 Security Policy`	Four board-language commitments	`SEC-P`	Principle	`Principles`, `Standards`, `Enforced At`, `Position`	Preliminary · yes
`065 Enforcement Point`	Five places a control actually acts	`EP-`	no element of its own — the row names an existing one	`Acts On`, `Enforces`, `Position`	D, G · yes

Two prefix pairs on this table need care. `041 Security Control` uses `SEC-C01` and `064 Security Policy` uses `SEC-P01` — the single letter is the whole distinction. And `061 Governance Change Request` uses `CR-G01` while the repository's own metamodel change requests on `03 Change Requests` use `CR-001`. The first is a request to change the architecture; the second is a request to change the repository's structure. Mixing them is the most common citation error in a governance pack.

`057` carries the column that makes the sheet honest. `Standing` is not a status field, it is a sentence about whether the body works: the Architecture Board is "constituted on paper and has never refused anything", the Data Governance Council "has never reached quorum", and the Executive Committee has never been used because nothing has escalated — which, against thirteen non-compliant criteria, is itself the finding.

The relationship catalogue

A relationship sheet has the same shape every time: `Source (XXX)`, `Source Name`, `Target (YYY)`, `Target Name`, then one or two payload columns, then `Key` and the seven governance columns. The two name columns are formulas. **A lookup error in a name column means the ID in the adjacent column does not exist in the referenced object sheet** — that is your referential integrity check, and `RPT12 Integrity Check` aggregates it.

The payload column is the whole reason the sheet exists. Without it you have a link; with it you have an architectural statement. Seven of the fifty-six sheets have no payload beyond `Note` — `R13`, `R15`, `R16`, `R18`, `R22`, `R29` and `R50` — and those are the ones you can fill fast.

`ArchiMate 4 relationship` names what to draw between the two elements; appendix D argues each one. Where ArchiMate 4 points the relationship the other way round from the sheet's `Source` to `Target` — a service realises a capability, not the reverse — the cell reads *reversed*. Fifteen of the fifty-six do. Six read `Association`, the notation's way of saying the two records are related and it has nothing stronger to offer: `R28`, `R34`, `R39`, `R40`, `R42` and `R56`. That is the honest answer, not a failure. Six cannot be drawn at either end — `R03`, `R35`, `R43`, `R46`, `R57`, `R59` — and three work only by substitution: `R06`, `R33`, `R44`.

Motivation and organisation

Sheet	From → To	What the link means	ArchiMate 4 relationship	Payload column	Fill when
`R01 DRV-GOL`	Driver → Goal	This condition is why we set that goal	Influence	`Strength`	Phase A, with the vision
`R02 GOL-OBJ`	Goal → Objective	The goal is realised through this objective	Realization (reversed)	`Contribution`	Phase A
`R03 OBJ-MSR`	Objective → Measure	The objective is tracked by this indicator	not expressible — Measure is not an element	`Type` , `Reporting`	Phase A
`R04 REQ-OBJ`	Requirement → Objective	The requirement exists to realise this objective	Realization	`Necessity`	Phase A, then each phase
`R05 PRN-REQ`	Principle → Requirement	The principle governs this requirement	Influence	`Relationship`	Phase A
`R06 CON-WPK`	Constraint → Work Package	The constraint bites on this package	Influence, from the Requirement standing for the constraint	`Impact` , `Mitigation`	Phase E
`R34 PRN-GAP`	Principle → Gap	The gap engages or violates this principle	Association	`Engagement`	Phase B to D, per gap
`R35 ASM-WPK`	Assumption → Work Package	The package rests on this assumption	not expressible — no Assumption element	`If Invalidated`	Phase E
`R60 O63-REQ`	Control Objective → Requirement	The obligation is met by this requirement	Realization (reversed)	`Evidence Today`	Preliminary, Phase A
`R07 ORG-ACT`	Organization Unit → Actor	The unit employs this actor type	Aggregation	`Count`	Phase B
`R08 ACT-ROL`	Actor → Role	The actor performs this role	Assignment	`Basis`	Phase B
`R27 ORG-LOC`	Organization Unit → Location	The unit sits at this site	Assignment (reversed)	`Type` , `Headcount`	Phase B

`R34` is the one architects skip and auditors read first. `Engagement` says whether a gap merely touches a principle or breaches it, and `RPT11 Principle Impact` turns that into a `VIOLATED` verdict per principle. A principle with no gaps engaging it is either perfectly observed or doing no work; nine of the thirty-one read "Not yet applied", eight read "Applied" and fourteen read `VIOLATED` .

Business

Sheet	From → To	What the link means	ArchiMate 4 relationship	Payload column	Fill when
`R10 ORG-CAP`	Organization Unit → Business Capability	The unit owns this capability	Assignment	`Ownership`	Phase B
`R11 VST-CAP`	Value Stream → Business Capability	The stream is enabled by this capability	Serving (reversed)	`Criticality to stream`	Phase B
`R38 CAP-CAP`	Business Capability → Business Capability	Parent decomposes to child	Composition	`Decomposition`	Phase B, with the map
`R12 CAP-BSV`	Business Capability → Business Service	The capability is realised by this service	Realization (reversed)	`Coverage`	Phase B
`R13 FUN-BPR`	Business Function → Business Process	The function contains this process	Aggregation	`Note` only	Phase B
`R09 ROL-BPR`	Role → Business Process	The role participates in this process	Assignment on `Accountable` rows, Association otherwise	`Participation`	Phase B
`R14 BPR-BSV`	Business Process → Business Service	The process orchestrates this service	Realization	`Sequence`	Phase B
`R15 BSV-PRD`	Business Service → Product	The service supports this product	Aggregation (reversed)	`Note` only	Phase B
`R16 PRD-CTR`	Product → Contract	The product is governed by this agreement	Composition	`Note` only	Phase B

`R11` carries the payload that should drive sequencing. `Criticality to stream` plus `Stage` says *where* in the stream the capability sits and *how much the stream depends on it* — fifty links across seven streams. A weak capability that is critical to two streams outranks a weaker one that is critical to none, and no other sheet tells you that. Be warned that the column is not yet doing its job: all fifty links read `High` , which is the same as saying nothing. Re-score it in Phase B and the capability roadmap changes shape.

`R38` has 544 instances, more than any other sheet in the repository, because it carries the whole four-level decomposition of `013`. Build it by formula from the dotted `ID` structure rather than by hand.

Data

Sheet	From → To	What the link means	ArchiMate 4 relationship	Payload column	Fill when
`R17 BPR-DEN`	Business Process → Data Entity	The process touches this entity	Access	`CRUD`	Phase C (data)
`R18 LDC-DEN`	Logical Data Component → Data Entity	The logical store encapsulates this entity	Aggregation	`Note` only	Phase C
`R19 PDC-LDC`	Physical Data Component → Logical Data Component	The physical store realises the logical one	Realization	`State`	Phase C
`R21 APP-DEN`	Application Component → Data Entity	The application holds this entity	Access	`Role`, `Authoritative`	Phase C

`R17`'s `CRUD` column is four letters that produce `RPT04 CRUD Matrix` and one of the most quotable findings in the whole repository: twenty-four entities, every one carrying a retention obligation, and not one `D` anywhere on the matrix. Nothing in the estate deletes anything, which `RPT16 Retention and Disposal` restates as a POPIA exposure.

`R21`'s pair of payload columns do different jobs. `Role` is M for master or C for copy — a statement of fact about the application. `Authoritative` is a governance decision: of the several applications holding an entity, is *this* the designated authority? Five applications record `M` against `DEN-01 Party`; `Authoritative` is where somebody has to say which one wins, and `RPT05 App-Data Matrix` prints `CONTESTED` until they do.

Application and technology

Sheet	From → To	What the link means	ArchiMate 4 relationship	Payload column	Fill when
R20 APP-BSV	Application Component → Business Service	The application implements this service	Realization	Coverage , State	Phase C (application)
R22 APP-ISS	Application Component → Information System Service	The application provides this service	Realization	Note only	Phase C
R23 APP-APP	Application Component → Application Component	The two exchange data	Flow	Managed (plus Style , Direction , Frequency , Interface ID , Data Entity)	Phase C
R24 APP-TSV	Application Component → Technology Service	The application rests on this technology service	Serving (reversed)	Dependency	Phase C, D
R36 PSV-APP	Platform Service → Application Component	The platform service serves this application	Serving	Consumption , State	Phase D
R37 APP-CAP	Application Component → Business Capability	The application supports this capability	Realization	Support	Phase C
R41 APP-VST	Application Component → Value Stream	The application enables this stream	Serving	Basis	Phase C
R25 PTC-LTC	Physical Technology Component → Logical Technology Component	The product realises the logical component	Realization	Standard Status	Phase D
R26 PTC-LOC	Physical Technology Component → Location	The node runs here	Assignment (reversed)	Role	Phase D
R31 STD-PTC	Standard → Physical Technology Component	The standard governs this product	Realization where conformant, Association otherwise (reversed)	Conformance	Phase D
R43 047-APP	Reference Model Entry → Application Component	This application is the designated answer	not expressible — the entry is not an element	Status , Domain	Preliminary, Phase C
R46 047-PTC	Reference Model Entry → Physical Technology Component	This product is the designated answer	not expressible — as R43	Status , Purpose	Preliminary, Phase D
R44 048-LOC	Environment → Location	The environment runs at this site	Assignment to the Nodes in the environment (reversed)	Provenance	Phase D

R23 is the richest relationship sheet in the repository, and Managed is the column that matters. Fifty interfaces are recorded and forty-four of them are No — unmanaged point-to-point links, four of which are direct database reads into another application's tables. RPT06 Interface Register prints the lot. If you fill only one payload column in Phase C, fill this one.

R37 carries 181 links, more than any sheet except R38 , and RPT07 Capability Coverage counts them to decide whether a capability is fully or only partly supported and whether several applications support the same thing. Partial only against a Tier 1 capability is the shape of most insurance estates. The Support column itself reads Supports on all 181 rows — a second payload column that has been filled uniformly rather than judged. Grading it Full, Partial or Peripheral is a day's work and it changes every coverage figure in the workbook.

Realisation, delivery and governance

Sheet	From → To	What the link means	ArchiMate 4 relationship	Payload column	Fill when
R28 GAP-WPK	Gap → Work Package	The package closes this gap	Association	Closure	Phase E
R29 WPK-DLV	Work Package → Deliverable	The package produces this output	Realization	Note only	Phase E
R30 WPK-APP	Work Package → Application Component	The package does something to this application	Realization	Action	Phase E
R32 REQ-APP	Requirement → Application Component	The requirement is realised by this application	Realization (reversed)	State	Phase C, E
R47 WPK-WPK	Work Package → Work Package	The first depends on the second	Triggering (reversed)	Verdict, Derived via	Phase E
R42 O55-O55	Building Block → Building Block	The first depends on the second	Association	Domain	Phase E
R48 O51-O55	Solution Building Block → Building Block	The solution realises this architecture block	Realization	Sourcing, Work Package	Phase E
R49 WPK-O51	Work Package → Solution Building Block	The package delivers this solution	Realization	Sourcing	Phase E
R50 O53-WPK	Transition Architecture → Work Package	The state contains this package	Realization (reversed)	Note only	Phase E, F
R54 O56-WPK	Project → Work Package	The project delivers this package	Composition	Project Type	Phase F
R55 O56-O56	Project → Project	The first depends on the second	Triggering (reversed)	Override	Phase F
R59 O66-O56	Architecture Increment → Project	The increment is delivered by this project	not expressible — no Increment element	Lands	Phase F
R33 ACO-WPK	Architecture Contract → Work Package	The contract governs this package	not expressible as drawn — no Contract element	Review Cadence	Phase G
R56 O59-O55	Compliance Assessment → Building Block	The assessment tests this block	Association	Verdict, Gate	Phase G
R57 O60-O61	Governance Decision → Governance Change Request	The decision resolves this request	not expressible — neither end is an element	Status	Phase G, H
R58 O64-O65	Security Policy → Enforcement Point	The policy is enforced here	Realization where it enforces, Association otherwise (reversed)	Position	Phase D, G
R39 ORG-DLV	Organization Unit → Architecture Deliverable	The unit owns this deliverable	Association	Role	Preliminary, continuous
R40 DLV-DLV	Architecture Deliverable → Architecture Deliverable	The first depends on the second	Association	Nature	Preliminary, continuous

R47 carries the single most valuable payload column in the delivery half of the repository. Verdict is not a dependency type — it is a judgement about whether the dependency is survivable. Three dependencies read Impossible: the dependent package finishes before its prerequisite does, in each case because the dates were set by funding cycles rather than by the model. WP-07 needs WP-08, WP-09 needs WP-07, WP-12 needs WP-06. RPT27 Programme Sequencing prices this: a 62-month critical path against 50 months remaining, with ten of sixteen packages moving if the dependencies are honoured.

R55's Override column exists because O56 Project is allowed to disagree with O30 Work Package. Two package pairs were combined into one project each to remove an impossible dependency, and Override is where that decision is recorded rather than hidden.

The seven columns every object sheet carries

Every object sheet and every relationship sheet ends with the same seven columns. They are the most ignored part of any repository and the first thing an auditor asks about, because they are the difference between an architecture document and an architecture record.

Column	What it is for	The rule
`Version`	How many approved changes this record has had since baseline	Derived from `02 Change Log` by formula. **Nobody types a version number.** `1.0` is the baseline load; the minor number increments once per approved change, so `1.3` means three changes. If a change is not in the log, the record still reads `1.0` and the change did not happen
`Record Status`	Whether this row is settled	`Baseline` for the original load, `Approved` once a change request carrying it has been decided, `Draft` while a change request against it is open. A row affected by an open request must read `Draft`
`In Repository Since`	The date the record first existed	Set once, at creation, and never touched again. It is not the date of the last edit
`Last Changed`	The date of the most recent approved change	Equal to `In Repository Since` for an untouched record. If the two differ and `Version` still reads `1.0`, something was edited outside the change log
`Changed By`	Who made the change	`Repository load` for baseline rows, otherwise the named executive or architect role — `Chief Data Officer`, `Business Domain Architect`. A person, not a system
`Change Ref`	The change request that authorised it	`CR-001` for the baseline load, otherwise the `CR` on `03 Change Requests`. A populated `Changed By` with a blank `Change Ref` is an ungoverned change
`Approved By`	Who signed it off	`Architecture Board` for Tier 1 principle-engaging changes, `Chief Enterprise Architect` otherwise. Approval is an act, not a field — if the Board has not met, do not type its name

FOUR RULES THAT KEEP THESE COLUMNS HONEST

1. **Derivation, not entry.** `Version`, `Last Changed` and `Changed By` come from `02 Change Log`. Change the log and the record follows. Type into the record and the two disagree, which `RPT13 Version Control` reports as a defect.
2. **Amendment by replacement.** Principle `GP-01` forbids a silent edit. `02 Change Log` holds `Previous Value` and `New Value` for every field changed, so the state of the repository at any past date can be reconstructed. That reconstruction is the evidence base for Joint Standard 1 of 2023.
3. **The log is never truncated.** Not at year end, not at baseline, not when it gets long. 172 changes sit in it today and all 172 stay.
4. **Four pending is normal; forty is not.** `01 Version Control` reports changes awaiting approval. Four open against sixty-three raised is a functioning process. A number that only grows means the Board is not meeting, which is `GAP-14`.

One documentation mismatch to know: `01 Version Control` describes the seven columns as "Version, Status, Effective From, Last Changed, Changed By, Change Ref and Approved By". The sheets themselves carry `Record Status` and `In Repository Since`. Same columns, older names. Use the column headers, not the policy text [3].

Naming and ID rules

These are the repository's own conventions, read off the data. Follow them and every lookup, every report and the comparison tool all keep working.

Rule	Why
ID in column A, Name in column B. Never reorder.	Every lookup in all 162 sheets resolves source and target names by position from these two columns. Reordering them breaks the entire workbook silently
Prefix, hyphen, zero-padded number: `DEN-01`, `WP-04`, `ATT-161`	Pad to the width of the largest expected set: two digits for sheets under a hundred rows, three for `037 Data Attribute`. `DEN-1` and `DEN-01` are different strings to a lookup
One prefix per sheet, except where the sheet holds families	`005 Principle` runs seven families (`BP-`, `DP-`, `AP-`, `IP-`, `TP-`, `SP-`, `GP-`) because the prefix carries the domain. `006 Requirement` separates `REQ-` strategic from `ARS-A/B/D/T-` derived, so you can tell at a glance which phase produced a requirement
Hierarchy goes in the ID, dotted: `C01`, `C01.02`, `C01.02.03`, `C01.02.03.01`	`013` is the only sheet that does this, and the dots let `Level`, `L1`, `L2` and the whole of `R38 CAP-CAP` be derived rather than typed
A suffixed prefix means a subtype: `APP-BB-`, `TEC-BB-`, `SEC-C`, `SEC-P`, `CR-G`, `DEC-G`, `REF-A-`, `REF-T-`	Four prefix pairs collide at the first three characters. The suffix letter is load-bearing: `SEC-C01` is a control, `SEC-P01` is a policy
Names are nouns, title case, no trailing full stop	`Produce Quotation`, not `producing quotations.` Names appear in 32 reports and in the `Source Name` column of every relationship sheet
Never reuse a retired ID	`02 Change Log` and the comparison tool both key on a composite of sheet, ID and attribute name. Reusing `APP-27` after decommissioning makes two different applications one record across the history
Codes on `00 Index` are not ID prefixes	The `Code` column gives a three-letter code for `001` to `034` and repeats the sheet number for `035` to `066`. The IDs do not follow it: `013`'s code is `CAP` and its IDs start `C01`, and `035`'s code reads `035` while its IDs are `ABB-01`. Read the data

The last rule has already bitten once. `50 Review` in the comparison tool records a finding against "Business Capability `CAP-15`". No record with that ID exists in `013`, whose IDs are dotted `Cnn` codes. The classification is sound — a fraud-detection maturity score was changed with no recorded assessment — but the record it points at cannot be found by lookup [8].

What the repository does not hold, and where that content lives

A repository is a register of facts about the architecture. It is not the architecture, and treating it as such is how it stops being maintained.

Not in the repository	Where it lives	The link between them
Diagrams. No sheet holds a picture, and no report draws one	`Meridian_ArchiMate4_Views.pptx` — the worked views you were handed with the workbook	A view is a selection over the repository. Every element on a view should resolve to an `ID` on an `O` sheet; if it does not, either the view is wrong or the sheet is incomplete
Narrative. No sheet holds an argument, a rationale paragraph, or a recommendation to an executive	The Architecture Definition Document and the phase deliverables listed on `034 Architecture Deliverable`	`034` registers the document, its `Deliverable Version`, `Lifecycle Status` and `Approver`. The repository holds the *record of the document*, not its text
Decisions, in full. `060 Governance Decision` holds a one-line decision and a `Why`, not the options considered	Architecture decision records, referenced as `ADR-nnn` from `03 Change Requests` column `Decision Record`	`GP-01` requires context, options, rationale and consequences. That is an ADR's job. The repository carries the pointer
Code, configuration and schemas	The estate itself. `022 Physical Data Component` names the store; `037 Data Attribute` holds 161 key attributes and stops there	Key attributes only: identity, classification, and those carrying a regulatory consequence. A full attribute model belongs in the physical design
Project plans. `056 Project` holds fifteen rows of shape, cost and dependency, not tasks	The programme's own planning tool	`RPT27 Programme Sequencing` is an architectural test of a plan, not a plan
A historical state. The sheets hold now	`02 Change Log` plus `04 Baselines`. Two baselines exist; `BL-002 Governance Establishment Baseline` is current	Reconstruct any past date from `Previous Value` and `New Value` in the log. Do not keep a second copy of the workbook as "the baseline"

Two cross-workbook rules follow from this. First, the comparison tool takes **pasted values, never links**: copy from `05`, `06` and `07`, then Paste Special, Values Only into `10`, `11` and `12`. External workbook references break on rename, need both files at fixed paths, and depend on users enabling link updates. Second, if a thing you need is not in the repository, the honest move is to say so on the sheet and record what you did instead. Four object sheets — `048 Environment`, `049 Resilience Tier`, `050 Network Link` and `051 Solution Building Block` — carry the word AUTHORED in their header and a `Provenance` column precisely because somebody did that.

THE CHAPTER IN TEN LINES

- Five governance sheets, 66 object sheets, 56 relationship sheets, three export sheets, 32 reports. 162 in all.
- There is no R45, R51, R52 or R53. Walk sheet names, not numbers.
- 00 Index says 33 object types, 36 relationship types and 13 reports. It is wrong on all three; RPT01 is right.
- ID in column A, Name in column B. Reordering them breaks every lookup in the workbook.
- O55 Building Block is a composed view over O35, O44, O45 and O46. Never type in it.
- O13 carries maturity at Level 3 only. Filling Level 4 breaks the heatmap averages.
- The payload column is why a relationship sheet exists: CRUD on R17, Authoritative on R21, Managed on R23, Verdict on R47.
- Version, Last Changed and Changed By are derived from 02 Change Log. Nobody types a version number.
- A populated Changed By with a blank Change Ref is an ungoverned change.
- Diagrams live in the views pack, narrative in the definition document, rationale in the ADRs. The repository holds the pointers.

SOURCES FOR THIS CHAPTER

1. Meridian_TOGAF_Metamodel.xlsx, sheet 00 Index
2. Meridian_TOGAF_Metamodel.xlsx, sheet names R01 DRV-GOL to R60 O63-REQ
3. Meridian_TOGAF_Metamodel.xlsx, sheets 01 Version Control, RPT01 Dashboard and RPT12 Integrity Check
4. Meridian_TOGAF_Metamodel.xlsx, sheet O05 Principle
5. Meridian_TOGAF_Metamodel.xlsx, sheets O20 Data Entity and RPT05 App-Data Matrix
6. Meridian_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Component and RPT21 Software Distribution
7. Meridian_TOGAF_Metamodel.xlsx, sheet O55 Building Block, header note under CR-053
8. Meridian_Architecture_Comparison_Tool.xlsx, sheet 50 Review
9. The Open Group, TOGAF Standard, Architecture Content Framework https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX B

The report catalogue — what each RPT sheet answers

Thirty-two reports, the one question each answers, and what the number means when it is bad.

Thirty-two report sheets. Every cell on every one of them is a formula over the object and relationship sheets, so a report is never wrong about the repository — it is only ever wrong about the business, which is a different and more interesting problem [1]. Nothing on an `RPT` sheet is typed. If you find a typed cell, somebody has broken a report.

This appendix gives each report five things: the question it answers, the ArchiMate 4 viewpoint its content belongs on if you are going to draw it rather than tabulate it, when to read it, what good looks like, and what a bad number actually means. The viewpoint names are the specification's own, spelling included. Two of the thirty-two are about the repository rather than the architecture and belong on no view; the column says so. Use it to stop reading reports in the order they appear in the tab bar, which is the order they were built, not the order they are useful. Meridian Assurance Group is invented; its numbers are real within this workbook.

Portfolio and estate

The first thing anyone asks is how big the estate is and what it costs. These seven answer that and nothing else, which is their virtue.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT01 Dashboard`	How big is the repository, what is in the estate, and what does it cost?	Layered	Preliminary, then monthly. The first sheet you open and the last one you quote from	Relationships per object record above 1.2; legacy share of run cost under a fifth; interfaces recorded roughly equal to interfaces managed	1.20 relationships per record, as here, means the objects are catalogued and barely linked. 44 of 50 interfaces unmanaged and a 33% legacy run-cost share are not repository defects; they are the estate
`RPT02 Application Portfolio`	What applications exist, what do they cost, and what is meant to happen to them?	Application Structure	Phase C (application), then quarterly for cost	Every application has a `Disposition` and an `Owner Org`; `% of Run` concentrated in applications marked Sustain or Extend	A high `% of Run` against `Replace` is money spent on something you are leaving. `Unmanaged` close to `Interfaces` on a Tier 1 row means that application cannot be changed safely
`RPT20 Portfolio Assessment`	Where does each application sit on value against fitness, and does the recorded disposition agree?	Application Structure	Phase C (application), and before any portfolio decision goes to the Investment Committee	`Agrees` reads Yes on every row. Invest and Tolerate hold most of the estate; Eliminate holds very little cost	Six `No` rows here mean six applications where the portfolio story and the portfolio data have come apart. Fix the data or fix the disposition — do not take either to a board
`RPT21 Software Distribution`	Where does each application physically run, and what technology is out of support?	Implementation and Deployment	Phase D, then at each vendor roadmap review — at least twice a year	No `Support ends` date inside eighteen months on a node carrying a Tier 1 application	Three platforms out of support in 2027 carrying sixteen application deployments is a dated, costed, unarguable problem. This is the report that funds technology currency work
`RPT06 Interface Register`	What integrates with what, how, and is any of it managed?	Application Cooperation	Phase C (application), then at every design gate	`Managed` = Yes on every interface; no `Direct DB read` anywhere	44 unmanaged of 50, with four direct database reads, means any change to one application can break three others without warning. This is the number that justifies an integration platform
`RPT23 Process Flow`	How do the processes flow by value stream, with what volume, cycle time and automation?	Business Process Cooperation	Phase B, then annually	Manual processes at low volume; automated processes at high volume; `Effort hours p.a.` concentrated where the effort buys something	910,000 manual needs analyses at 32 minutes each is 485,000 hours a year. Volume times cycle time is where the business case for automation is found, not in the automation level column
`RPT26 Resilience Test Status`	Which recovery objectives have been tested, and when?	Technology	Phase D, then at each test cycle	A `Last tested` date inside the stated `Test cadence` on every Tier 1 row	Twelve objectives, every one "Never tested", is the finding the Prudential Authority asks about first. An untested RTO is an aspiration

`RPT02` and `RPT20` look like the same report and are not. `RPT02` is the register with cost analysis — it tells you what you have. `RPT20` is the assessment with the agreement check — it tells you whether what you have been saying about it is true. Read `RPT02` to brief yourself and `RPT20` before you open your mouth in a committee.

These seven report on the estate volume 1, chapter 12, *The shape of the estate*, describes. Read it before `RPT02`, so a count of twenty-six applications means something.

Capability and coverage

Three reports, and they are the ones that connect the architecture to the business. An executive who will read nothing else will read these.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT03 Capability Heatmap`	Where is business capability weak relative to how critical it is?	Capability Map	Phase B, then each time the capability assessment is refreshed — annually at most	`Gap` under 1.0 on Critical capabilities; `Build` count low; `Apps` present wherever `Critical` is high	A `Gap` of 2.33 on Customer Management with seven critical sub-capabilities, as here, means the thing the group most depends on is the thing it is worst at. Levels 1 and 2 roll up by formula, so a bad number at L1 is always an L3 problem
`RPT07 Capability Coverage`	Where is capability unsupported by any application, and where is it duplicated?	Application Usage	Phase C (application), and before any consolidation decision	`Coverage` reads Supported; `Duplication` blank or "Two apps" with a stated reason	`Partial only` plus `DUPLICATED` on the same row — four applications part-covering one business service — is the classic insurance shape: nobody owns it and everybody has built a bit of it
`RPT25 Reference Model Conformance`	For each purpose, is the designated answer usable today?	Technology	Preliminary and Phase D, then quarterly as products move	Every purpose has a `Standard today` and an `At target`, and neither is "No standard" or "Undecided"	Five purposes with no approved answer means five decisions nobody is taking. `Undecided` at target on "administer a life policy" means the scope excluded the question, which is honest but still a gap

`RPT07` is the report people misuse. It is organised by **business service**, not by capability, so a capability supported through two services appears twice. It answers "is this service implemented, and by how many things" — the capability column is context. For capability strength, read `RPT03`.

Volume 1 teaches this content twice: chapter 13, *How to use a capability model properly*, for what a maturity score is worth, and chapter 19, appendix A, *Capability to application — the coverage map*, for the drafted version of what `RPT07` computes.

Data

Seven reports, and together they make the strongest case in the repository. Insurers lose more money to unresolved data authority than to any application decision, and these are the sheets that prove it.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT04 CRUD Matrix`	Which processes create, read, update or delete which data?	Information Structure	Phase C (data), then when any process changes	Every entity has at least one `C` and one `D` somewhere in its column; `U` concentrated in one process per entity	No `D` anywhere on the matrix — the position here — means nothing in the estate disposes of anything. Multiple `C` on one entity means multiple processes create the same record
`RPT05 App-Data Matrix`	Which applications master which entities, and where is authority contested?	Information Structure	Phase C (data), then at every data-ownership session	`Status` reads Single on most rows; `Masters (all)` equal to 1 where `Recorded authority` names an application	`CONTESTED` with five masters against `DEN-01 Party` means five applications each believe they own the customer. That is not a data problem, it is an unmade decision
`RPT15 Catalogue Completeness`	Does each information concept trace from a business definition through to a data set, and where does the chain break?	Information Structure	Phase C (data), monthly while the catalogue is being built	`Links` reads 6/6; `Where it breaks` blank	5/6 with "Contested source" on every row means the chain breaks at the same link every time. One decision fixes twenty-four rows
`RPT16 Retention and Disposal`	Does anything in the estate dispose of the data it is obliged to dispose of?	Information Structure	Phase C (data), and before any POPIA assessment	`Disposed by any process` reads yes wherever a retention period has expired	"Held indefinitely" against a stated POPIA s14 obligation is a regulatory exposure with a named section number attached. Twenty-four of twenty-four is a finding, not a backlog
`RPT17 Master Data`	Which entities are mastered, by what style, under what rule, and is any of it implemented?	Information Structure	Phase C (data), then when a mastering decision is taken	`Status` reads Designated; `Quality` Good or Fair; the golden-record rule exists and is implemented	Twelve rules authored and none implemented in a system means the rules are a document, not a control. `Contested` plus `Poor` quality on a Tier 1 entity is where the reconciliation failures come from
`RPT18 Metadata Coverage`	What metadata does the group hold about its data, and which categories are absent?	Information Structure	Phase C (data), annually	No category entirely `Absent`; lineage present at least partially	Seven of twenty-two elements absent, lineage among them, means no reported figure can be traced to its authority without manual reconstruction — which breaches `DP-05` directly
`RPT22 Portfolio Data Risk`	Which entities have no surviving master once the recorded portfolio dispositions are executed?	Migration	Phase E, and again before any decommissioning gate	Every entity has at least one surviving master whose disposition is Sustain, Extend or Build	"Authority unrecorded" with `LIFE/400` listed as `At risk from` means the application being replaced is the only one anyone ever agreed held the data. Four entities with no surviving master is a programme-stopping finding

`RPT22` is the most under-read report in the workbook and the one that will save you a career-defining mistake. It crosses `O20 Data Entity` against `O23 Application Component` dispositions and asks the question nobody asks in a decommissioning business case: when this application goes, who holds the data? Read it in Phase E, before the project charter, not in Phase F after the contract is signed.

Volume 1, chapter 10, *The information model*, is the substance behind these five; chapter 15, *POPIA, and the two sections that catch insurers*, is why `RPT16 Retention and Disposal` comes first.

Traceability and compliance

Six reports that answer the only question an auditor ever really asks: can you show me that this was decided, and by whom, and against what?

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT08 Traceability`	Does every requirement trace from a driver through to a delivering work package?	Goal Realization	Phase A, then at every requirements review	`Chain` reads Complete on every row; no zero in `Goals served`, `Realising Apps` or `Work Packages`	`BROKEN` means a zero somewhere on the row. A requirement with a driver and no work package is a promise nobody is keeping; one with a work package and no driver is scope that arrived from somewhere else
`RPT32 Requirement Traceability`	Does every requirement reach a work package, and has any been tested?	Requirements Realization	Phase G, and at every gate	`Traced` yes and `Ever tested` yes on everything delivered	Seventy-six requirements, fifty-three traced, none ever tested. The twenty strategic `REQ-` requirements reach no work package at all, which means the vision and the plan were written by different people
`RPT10 Standards Conformance`	Where does the estate breach its own standards?	Technology	Phase D, then quarterly	`Non-conformant` rows carry a `Note` naming the containment and a dated exit	Non-conformant plus `Lifecycle` Legacy plus a `Support Ends` date is three problems on one row. "Contained" with no expiry date is a permanent exception pretending to be a temporary one
`RPT11 Principle Impact`	Which principles are doing work, and which are being violated by a recorded gap?	Motivation	Preliminary, then at each board meeting	Most rows read Applied; `Direct violations` zero; few rows read "Not yet applied"	`VIOLATED` on `BP-01 Primacy of Principles` is the worst single cell in the repository — it means the rule that the principles apply at all is itself being breached. Nine of thirty-one read "Not yet applied" and fourteen read `VIOLATED` — only eight are quietly working
`RPT30 Compliance Position`	What has been assessed, what passed, and what do the contracts claim?	Requirements Realization	Phase G, at every G2 gate	`Verdict` Compliant with named `Evidence`; `Contract status` agreeing with the verdict	Thirteen assessed, thirteen non-compliant, is bad. Four contracts previously claiming Compliant against those same criteria was worse — asserted compliance is how a repository starts lying
`RPT31 Governance Activity`	What has the governance regime actually decided, changed and escalated?	Implementation and Migration	Phase G and H, monthly	Decisions with dates; dispensations with expiry dates; at least one escalation on record	Two decisions outstanding and no escalation ever raised, against thirteen non-compliant criteria, means the governance process is recording rather than governing. A decision with no date has not been taken

`RPT08` and `RPT32` are two views of the same chain and you need both. `RPT08` walks it forwards from the driver and shows where the chain breaks. `RPT32` walks it from the requirement to the building block and asks the harder question — has it ever been tested? Nothing in the repository has. `RPT08` is the Phase A report; `RPT32` is the Phase G report.

Volume 1, chapter 15, *Modelling a regulatory requirement properly*, is where a traceable requirement comes from; *What the regulator actually inspects*, same chapter, is the test these reports rehearse for.

Delivery and sequencing

Seven reports. The repository is at its most useful here, because a roadmap is the one architectural artefact that can be proved wrong arithmetically.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT09 Gaps and Roadmap`	What are the gaps, what closes them, and what does it cost?	Migration	Phase E and F, then monthly through delivery	Every gap `Addressed`; benefit comparable to cost across the set; severity spread, not all Critical	A gap with no work package is a gap nobody owns. Cost far above benefit on a Critical gap is still worth doing; cost above benefit on a Medium one is not
`RPT19 Building Block Coverage`	Which **data** building blocks are funded, which serve a business block, and which do neither?	Requirements Realization	Phase C (data) and Phase E	All fourteen `Funded` yes, each naming a business block it serves	Five with no work package and three named by no business block means eight of fourteen are orphans. A data block nothing asks for is a data platform looking for a purpose
`RPT24 Building Block Coverage`	Across all four domains, which building blocks have a work package and which do not?	Implementation and Migration	Phase E, before the roadmap goes to the Investment Committee	Every block `Funded` yes and assigned to a `Wave`	Twelve unfunded of fifty-six, and every one of the twelve a governance mechanism, is a pattern not an accident: governance is what gets cut when a programme is costed
`RPT27 Programme Sequencing`	Can the programme be delivered in dependency order, and what moves if it is?	Implementation and Migration	Phase F, and again every time a date changes	`Slip months` blank or a dash on every row; critical path inside the time available	A 62-month critical path against 50 months remaining, with ten of sixteen packages slipping, means the plan is arithmetically impossible. A 40-month slip on one package is not a risk, it is a fact
`RPT28 Solution Sourcing`	What will be bought, built or configured, and what has nothing to buy it with?	Project	Phase E, before procurement starts	Every solution block has a `Sourcing` decision, a work package and `Funded` yes	"No product selected" on a block seven packages are waiting for is the cheapest blocker in the programme to clear. Six of twenty-nine with no work package means six solutions nobody is paying for
`RPT29 Transition Readiness`	Which transition-state exit criteria are met today?	Migration	Phase F, then monthly at the programme board	`Standing` reads met on the criteria for the state you are in	All six `TA-1` criteria failing while `TA-2` work is already running means the programme has started the second state before finishing the first. That is the most common and most expensive sequencing error there is
`RPT14 Deliverable Register`	What does the repository hold about itself — every deliverable, version, status and dependency?	Project	Preliminary, then monthly	Most deliverables `Baselined`; nothing at version 0.x that something else depends on	A `Draft` at version 0.1 with two dependants means two pieces of work are built on something that has not been agreed. `Depends on` counts are where circularity hides

> **TWO TRAPS THAT HAVE ALREADY CAUGHT PEOPLE**
>
> **`RPT19` and `RPT24` are both named "Building Block Coverage" and they are different sheets.** `RPT19` covers the fourteen **data** building blocks only — its own title row reads *Data Building Block Coverage* — and carries an `Acceptance criteria` column that `RPT24` does not. `RPT24` covers all fifty-six blocks across business, data, application and technology, with a `Domain` column. Cite them by number and by scope, never by name alone. In the tab bar they sit five sheets apart, and `RPT19` has a header row at row 2 where `RPT24` has one at row 5, so a formula written against one will not read the other [2].
>
> **The gap report is `RPT09 Gaps and Roadmap`.** There is no "RPT07 Gap Analysis". `RPT07` is `Capability Coverage`, which answers a different question entirely — service implementation and duplication, not baseline-to-target difference. A gap-analysis pack that cites `RPT07` has been written from memory [3].

Volume 1, chapter 14, *What a core replacement actually costs*, is the check on every cost column here; chapter 18, *The ninety-day plan*, is what `RPT29 Transition Readiness` measures you against.

Repository health

Two reports about the repository rather than about the architecture. Read them before anyone else does.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT12 Integrity Check`	Which object types have no relationships, and do all references resolve?	none — model hygiene	Monthly, and after every bulk edit	`Status` reads Linked on every row; `Links per record` above 1 everywhere; no lookup errors in any name column	A type with zero links is either unused or unmodelled, and you have to say which. `Links per record` of exactly 1.0 often means a one-to-one sheet that should have been a column
`RPT13 Version Control`	What has changed since baseline, and what is awaiting approval?	none — model hygiene	Monthly, and before any baseline is cut	Change activity in the sheets the current phase touches; `Pending approval` small and falling; `Latest change` recent	"Unchanged since baseline" across a whole domain means either genuine stability or an abandoned sheet. 172 changes against sixty-six object sheets means a handful of sheets carry all the churn — look at which

One caveat on `RPT12`. Its own header says it counts across "all 34 relationship sheets"; it counts across all 56. The text predates the Phase E to H extensions. The arithmetic is right and the sentence is stale [4].

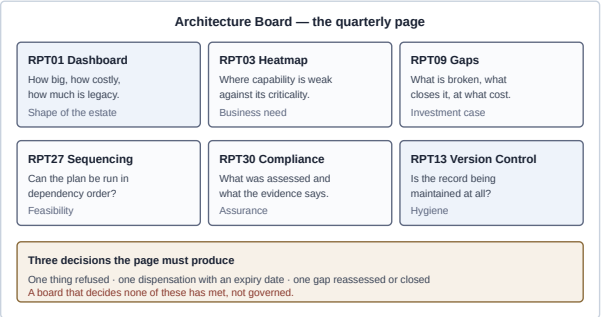

Figure 14.1 Six reports, one page, three decisions. Each panel maps to something only the board can do: fund, prioritise, sequence, assure, unblock.

The six reports to put on one page for a quarterly architecture board

Thirty-two reports will not get read. Six will, if each one answers a question the board can act on and no two answer the same question. These six, in this order.

#	Report	The question it puts to the board	Why this one and not a neighbour
1	`RPT01 Dashboard`	Is the estate getting bigger, more expensive or more legacy-weighted than last quarter?	It is the only report with every headline on one screen, and every figure is a formula, so nobody can argue the number without arguing the data behind it
2	`RPT03 Capability Heatmap`	Where is the business weakest at the things it most depends on?	The board is not an IT committee. This is the only report framed in capability rather than in systems, and it is the one a line executive will read without translation
3	`RPT09 Gaps and Roadmap`	What is broken, what closes it, and at what cost against what benefit?	It carries gap, severity, work package, cost and benefit on one row. `RPT24` is better on building-block funding but means nothing to a board; `RPT09` is the investment conversation
4	`RPT27 Programme Sequencing`	Can the plan we have approved actually be run?	Sequencing is the one architectural judgement a board cannot overrule by deciding harder. A critical path longer than the time available is arithmetic, and it belongs in front of the people who set the dates
5	`RPT30 Compliance Position`	Of the work we have governed, what met its criteria and what did not?	It is the assurance report with `Evidence` on the row. `RPT31` tells the board what it decided, which it already knows; `RPT30` tells it what happened afterwards, which it usually does not
6	`RPT13 Version Control`	Is the record being maintained, and is anything stuck waiting for us?	The only one of the six about the practice rather than the architecture. `Pending approval` is the board's own backlog. A board that never sees this number will not notice when the repository quietly stops being true

Each of the six maps to one thing the board is uniquely able to do: fund it, prioritise it, sequence it, assure it, or unblock it. Nothing on the page is there to inform. If a report cannot be acted on by the people in the room, it belongs in the pack, not on the page.

Two notes on using the page. Put the prior quarter's figure beside each current one — `RPT01` and `RPT13` are only interesting as a trend, and a dashboard with no previous column invites the response that the numbers have always been like that. And take `RPT20 Portfolio Assessment` as the standing seventh, held in reserve: when somebody disputes a disposition, the `Agrees` column settles it in one cell, and six rows currently read No.

THE CHAPTER IN TEN LINES

- Every cell on every RPT sheet is a formula. A typed cell is a broken report.
- RPT19 covers the fourteen data building blocks only; RPT24 covers all fifty-six across four domains.
- The gap report is RPT09 Gaps and Roadmap. RPT07 is Capability Coverage. There is no RPT07 Gap Analysis.
- RPT02 tells you what you have; RPT20 tells you whether what you said about it is true.
- RPT22 Portfolio Data Risk is the most under-read sheet: who holds the data when the application goes?
- RPT08 walks the chain forwards for Phase A; RPT32 walks it to the building block for Phase G.
- RPT27 proves a roadmap wrong arithmetically: 62 months of critical path against 50 remaining.
- RPT29 shows TA-2 work running while all six TA-1 criteria fail. That is the expensive error.
- RPT12 and RPT13 are about the repository, not the architecture. Read them before anyone else does.
- Six for the board: RPT01, RPT03, RPT09, RPT27, RPT30, RPT13 — shape, need, investment, feasibility, assurance, hygiene.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT01 Dashboard to RPT32 Requirement Traceability
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT19 Building Block Coverage and RPT24 Building Block Coverage
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT07 Capability Coverage and RPT09 Gaps and Roadmap
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheet RPT12 Integrity Check, header note
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheet 00 Index, report index
- **6.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX C

Workshop packs, checklists and the ninety-day calendar

Five workshops that fill real cells, ten checklists you can tick, and thirteen weeks that end in something a programme director cannot argue with.

Volume 1 chapter 18, *Your first ninety days*, covers the learning side: what to read, who to ask, which twelve questions tell you most, and how to earn the right to be believed. This appendix is the other half — the repository side. It assumes you have been handed a populated workbook on day one and have to make it true. Read both. Where volume 1 says *build the slice*, this says which sheets the slice lands in.

Meridian Assurance Group is invented. Role names are taken from volume 1 chapter 5 where they describe a business role, and from `O11 Role` and `O57 Governance Body` where the repository names them [1].

Five workshop packs

Each pack runs once per domain and then annually. A workshop that fills no cells was a meeting.

Volume 1, chapter 5, *Running the discovery session*, covers the room itself: who to put in it, what to ask, and how to stop the most senior voice settling a question of fact. These five packs say which cells the room has to leave filled.

Pack 1 — The capability maturity workshop

Fills. `O13 Business Capability`, Level 3 rows only: `Criticality`, `Current Maturity`, `Target Maturity`, `Disposition`, `Differentiation`. Thirty to forty rows per session.

Who must be in the room. The line executive who owns the capability area and can be held to the score. For a claims area: the **claims manager** and a **claims handler** who still works a queue. For underwriting: the **chief underwriter** and a **pricing actuary**. For distribution: the **brokerage principal** or a **broker consultant**, because distribution capability is half outside the company. Plus the **Domain Architect** (`ROL-13`) to hold the pen and one person from finance who can say what the capability costs. Never more than nine people. A tenth person turns an assessment into a presentation.

How long. Three hours for one Level 1 area, which is eleven to thirteen Level 3 capabilities. Do not attempt two areas in a day; the scoring drifts after lunch and the afternoon area always scores higher.

What to send beforehand, three working days out. The Level 3 rows for that area with `Name` and `Description` only, maturity columns blank. The maturity scale, five lines, one per level. `RPT07 Capability Coverage` filtered to the services in that area, so the room can see which capabilities already have applications behind them. Volume 1, chapter 19, appendix A, *Business capabilities*, already drafts the capability names — start from there, then argue with it, and match on name rather than number: its `C-01`–`C-20` are twenty headline capabilities, and `O13` holds 556 rows on four levels.

Running order	Minutes	What happens
1. Boundaries	25	Read each capability name and description aloud. The room either recognises it or renames it. Renaming is allowed; adding is not
2. Criticality first	30	Score `Criticality` before anything else: Critical, High, Medium. This is about dependence, not performance
3. Current maturity	60	One to five, with evidence for each score. No score without a named example
4. Target maturity	35	Where it needs to be by the horizon on `O02 Goal`, not where it would be nice
5. Differentiation	20	Advantage, Differentiating, Parity or Commodity. This is what stops everything being targeted at five
6. Disposition	20	Sustain, Invest, Build, Optimise or Outsource. Falls out of the previous two columns

Facilitation moves that work. Score `Criticality` before `Current Maturity` — once a room has admitted a capability is weak it will not then admit it is critical, because the two together sound like an accusation. Ask for the evidence, not the score: "what would I see if I watched this being done for a week?" and let the score follow. When two people disagree by two points, write both down and move on; the disagreement is the finding, and `RPT03 Capability Heatmap` will surface it as an average that satisfies nobody. Use `Differentiation` as the brake: a Commodity capability targeted at five is a procurement decision, not an architecture one.

Three ways it derails. First, **everything becomes critical**. Cap it: no more than a third of the Level 3 rows in an area may be Critical, and make the room choose. Second, **the room scores Level 4**. The maturity columns are deliberately blank on all 364 Level 4 rows, and filling them breaks the formula roll-up in `RPT03`. Say this in the first five minutes. Third, **somebody brings a system**. The moment a vendor name enters the conversation you are assessing software, not capability. Park it in a list and promise it goes to Pack 3.

Done when. Every Level 3 row in the area has all five judgement columns filled, and `RPT03 Capability Heatmap` shows a `Gap` figure for that area that the executive who owns it will repeat in their own words.

Pack 2 — The data ownership workshop

Fills. `O20 Data Entity`: `Authoritative Source`, `Owner Role`, `Tier`, `Classification`, `Quality`, `Retention Period`, `Retention Basis`. Then `R21 APP-DEN` column `Authoritative` for every application holding the entity.

Who must be in the room. This one lives or dies on the attendance list. The **head of data** chairs as the Data Governance Council (`GB-03`). Each candidate master needs its **application owner** (`ROL-11`) present, because the question "do you master Party?" has to be answered by someone who can be wrong in public. The **information officer (POPIA)** for `Classification` , `Retention Period` and `Retention Basis` — statutory role, not optional. A **policy servicing clerk** and a **claims handler**, because they know which record people actually trust. The **chief financial officer** or a delegate when the entity is `DEN-21 Financial Transaction` .

How long. Four hours, six to eight entities. Twenty-four entities is three sessions. Do the master entities first — `DEN-01 Party` , `DEN-02 Customer` , `DEN-03 Contact Detail` , `DEN-06 Policy` , `DEN-10 Claim` — because every later session is easier once Party is settled.

What to send beforehand, five working days out. `RPT05 App-Data Matrix` showing which applications hold an `M` against each entity. `RPT17 Master Data` with the `Mastering style` and the golden-record rule as drafted. `RPT16 Retention and Disposal` showing that nothing in the estate disposes of anything. And one sentence per entity: the proposed definition. Half the session is spent agreeing what the word means.

Running order	Minutes	What happens
1. One word, one meaning	40	Agree the definition. `IP-02 A Word Means One Thing` is the authority. Record disagreements verbatim
2. Who holds it	35	Walk the `M` columns on `RPT05` . Each owner confirms or denies. Five claimants on Party is normal
3. Who is authoritative	60	The decision. One of: a named `APP-nn` , `SPLIT` with the split stated, `CONTESTED` , or `NONE`
4. Who owns it	30	A **named individual**, recorded against `Owner Role` . Not a role, not a team
5. Classification and retention	45	`Classification` , `Retention Period` , `Retention Basis` with a section number
6. Quality	20	Good, Fair or Poor, with the reconciliation that proves it

Facilitation moves that work. Put `CONTESTED` on the board as a legitimate outcome in the first ten minutes. A room that believes it must produce an answer will produce a wrong one; a room that knows `CONTESTED` is allowed will be honest, and `CONTESTED` recorded is worth more than a master invented. Ask the clerk, not the owner, which record people trust — the answer is often a spreadsheet, and that spreadsheet is the real authoritative source. For retention, make the information officer read the obligation aloud: "POPIA s14" said out loud lands differently to "POPIA s14" typed in a cell.

Three ways it derails. First, **ownership is assigned to a role**. Twenty-three of the twenty-four entities currently point at `ROL-09 Data Owner` , a role with no actor, and `RPT29 Transition Readiness` fails `TA-1` on exactly this. Demand a human name and write it in the `Note` if the column will not take it. Second, **the session becomes a tooling conversation**. The question is who decides, not what MDM product. Third, **the Council does not reach quorum**, which is `GB-03` 's recorded standing today. Fix it before the invitation goes out, not on the day: get the quorum rule written down and the chair to confirm the attendance list in writing.

YOUR DATA OWNERSHIP SESSIONS

The five entities you will settle first, and the one person who must be in the room for each:

The quorum rule the Data Governance Council will work to, in one sentence:

Pack 3 — The application portfolio scoring workshop

Fills. `O23 Application Component` : `Business Value` , `Technical Fitness` , `Criticality` , `Lifecycle` , `Disposition` . `Portfolio Quadrant` and `Disposition Agrees` are computed — do not type in them.

Who must be in the room. Two halves, and both must be present at once. For value: the **product manager** and the business executive whose process the application serves. For fitness: the **application owner** (`ROL-11`), the **CIO** delegate, and whoever runs the platform underneath it. Scoring value without the business in the room gives you an IT opinion; scoring fitness without technology gives you a wish list. The **chief financial officer** or a delegate, because `Annual Run Cost (Rm)` will be disputed.

How long. Four hours for twenty-six applications, which is nine minutes each. That is enough. A longer session produces more precise scores, not better ones.

What to send beforehand, three working days out. `RPT02 Application Portfolio` with cost, age and interface counts. `RPT06 Interface Register` filtered per application, so `Unmanaged` is visible when fitness is scored. `RPT21 Software Distribution` for the support-end dates. The two scoring scales, each one page: `Business Value` out of twelve, `Technical Fitness` out of twelve, with what each band means.

Running order	Minutes	What happens
1. The two scales	20	Read both aloud. Agree that cost is not value and that age is not fitness
2. Value pass	70	Score all twenty-six on `Business Value` only, business voices leading
3. Break	15	Deliberate. The break stops the two scores being correlated by the same conversation
4. Fitness pass	70	Score all twenty-six on `Technical Fitness` only, technology voices leading
5. The quadrant reveal	30	Show the computed `Portfolio Quadrant` against the recorded `Disposition`
6. The disagreements	35	Work only the rows where `Disposition Agrees` reads No. Six of twenty-six today

Facilitation moves that work. Score the two dimensions in separate passes with a break between. If you score value and fitness on the same row in the same breath, the room anchors and you get a diagonal line instead of a scatter. Keep the reveal until step five — a room that can see the quadrant will reverse-engineer scores to reach the disposition it already wanted. Then spend the last half hour only on the `No` rows: those six are the whole value of the session, and each one ends either with a changed score or a changed disposition and a reason recorded in `02 Change Log`.

Three ways it derails. First, **the run cost becomes the agenda**. Cost belongs in `Annual Run Cost (Rm)` and in the business case, not in either score. Rule it out loud. Second, **a Tier 1 application scores low on value** and the room panics. Tier 1 criticality and low business value is a real and common position: it means the thing is load-bearing and nobody likes it, which is exactly what `Migrate` means. Third, **somebody scores the roadmap**. Score what runs today. `045 Application Building Block` is where the target lives.

Done when. All twenty-six rows have both scores, `Disposition Agrees` reads Yes on every row or carries a dated change request, and `RPT20 Portfolio Assessment` is ready to go to the Investment Committee (`GB-02`) without a footnote.

Pack 4 — The gap classification session

Fills. `50 Review` in `Meridian_Architecture_Comparison_Tool.xlsx`: `Your classification`, `Rationale`, `Owner`, `Action`. Rows classified Gap flow to `70 Gap Candidates` and from there into `029 Gap`.

Who must be in the room. Small and senior. The **Chief Enterprise Architect** chairs. The four **Domain Architects** (`ROL-13`), because every difference belongs to a domain. One **Architecture Board Member** (`ROL-14`) who can take an Accepted decision on the spot. Nobody else. This is the one session on the list where a wider room makes it worse, because the work is judgement about the record, not discovery about the business.

How long. Two hours, forty to sixty differences. You will not clear 2,906 unreviewed differences in a session and should not try.

What to send beforehand, two working days out. `60 Summary` showing the classification split: at Meridian, 31 governed drift, one governed drift with further change, five ungoverned changes, 2,739 added in B, 138 removed in B. And the instruction to read `00 How to Use` on the three categories the tool separates — design delta, governed drift, ungoverned change — because a room that has not read it will argue about the tool instead of the findings.

Running order	Minutes	What happens
1. The five ungoverned changes first	35	Highest value, smallest number. Each is a `GP-01` defect, and each needs a retrospective CR or a reversal
2. Record removals	20	138 rows. Most are decommissioning and Accepted; the exceptions are omissions
3. Design deltas, by domain	45	Filter `32 Diff Attributes` on Classification, take one domain at a time
4. The drift rows	15	Governed drift is not a gap. The question is whether the target is now stale
5. Owner and action on every row	5	No row leaves the session without a named owner

Facilitation moves that work. Filter `32 Diff Attributes` on `Classification` and work one classification at a time — never walk the sheet in row order, because the tool's own ordering mixes governed drift and genuine deltas and the room loses the thread. Make the distinction explicit before the first row: an ungoverned change is a governance defect, not an architectural gap, and it gets a retrospective change request rather than a gap record. Insist on a `Rationale` sentence for every row, even the Accepted ones; a classification with no rationale will be reopened within a quarter by somebody who was not there.

Three ways it derails. First, **the room treats tool output as findings**. The tool applies no judgement about significance and says so. Nothing reaches `70 Gap Candidates` until a person has classified it. Second, **somebody pastes formulas into the input sheets**. Sheets `10`, `11` and `12` take Paste Special, Values Only from `05`, `06` and `07`. Pale yellow marks a cell you fill in; everything else is calculated, and overwriting a formula breaks the tool. Third, **the identifiers do not resolve**. `50 Review` already carries a finding against Business Capability `CAP-15`, and no such ID exists in `013`, whose IDs are dotted `C01.01.01` codes. Check every ID against its object sheet before the row becomes a gap [2].

Done when. Every row on `50 Review` has a classification, a rationale, an owner and an action, and `70 Gap Candidates` holds only rows a person has agreed are gaps, shaped to paste into `029 Gap`.

Pack 5 — The roadmap sequencing session

Fills. `030 Work Package`: `Wave`, `Start`, `End`, `Transition State`. `053 Transition Architecture`: `Exit Criteria`, `Still Broken Here`, `Standing`. `R47 WPK-WPK`: `Verdict`. `052 Implementation Factor`: `Changes Sequence`.

Who must be in the room. The **Chief Enterprise Architect**, the programme director who owns the dates, the **chief financial officer** or Investment Committee delegate who owns the funding cycle, and the four **Domain Architects**. The **Chief Risk Officer** for `062 Delivery Risk`. Crucially, somebody who can change a date — otherwise the session produces a finding and no plan.

How long. Half a day. Sequencing is the one session worth the time, because it is the only architectural judgement that can be settled arithmetically.

What to send beforehand, five working days out. `RPT27 Programme Sequencing` as it stands: sixteen packages, ten would slip, a 62-month critical path against 50 months remaining, and three dependencies in `R47` marked Impossible — the dependent package finishes before its prerequisite does. `RPT29 Transition Readiness` showing all six `TA-1` criteria failing while `TA-2` work runs. `052 Implementation Factor`, all twenty-six rows, because a factor with no deduction is an observation and the room has to supply the deductions.

Running order	Minutes	What happens
1. The arithmetic	30	Read the critical path against the time available. Do not negotiate it
2. The three Impossible dependencies	50	`WP-07` needs `WP-08`; `WP-09` needs `WP-07`; `WP-12` needs `WP-06`. Each ends in a split, a combine or a dropped scope
3. Transition states	60	Which packages belong in `TA-1`, `TA-2`, `TA-3`. A package in two states is in neither
4. Exit criteria	50	Per state, what must be true to stop here safely. Testable, not aspirational
5. Still broken here	30	Per state, what remains wrong. The honesty column
6. Alternatives	30	If the arithmetic will not close, `054 Architecture Alternative` is where the options go

Facilitation moves that work. Open with the arithmetic and refuse to negotiate it: 62 months against 50 is not an opinion, and a room allowed to argue about it will spend the session there. Work the three Impossible dependencies individually, because each has a different resolution — `WP-07` and `WP-08` combine into one project, which is why `056 Project` resolves sixteen packages into fifteen projects and records the reason in `Override`. Write the exit criteria as things somebody could test on a Tuesday: *the Board has met, has a quorum rule, and has refused at least one thing* is a criterion; *governance is embedded* is not. Make the room fill `Still Broken Here` for every state — a transition state with nothing broken is a target architecture in disguise.

Three ways it derails. First, **the dates are defended rather than the sequence**. The dates were set by funding cycles; the sequence is set by dependencies, and one of them has to move. Second, **a transition state becomes a phase of the programme**. `053` holds states the enterprise can stop at safely, not milestones. If you cannot stop there, it is not a state. Third, **the session ends with a finding and no decision**. Close by naming which of the five rows on `054 Architecture Alternative` the Investment Committee will be asked to choose, and when.

YOUR SEQUENCING SESSION

The date you will hold it, and the one person attending who can change a date:

The three dependencies you expect to be Impossible, and your proposed resolution for each:

Quality checklists, one per ADM phase

Tick these against the repository, not against your memory of it. Each one is an assertion of the form *every X has a Y* and each is checkable in under two minutes.

Preliminary Phase

Tick	Assertion	Check against
[_]	Every principle has a `Tier`, and not everything is Tier 1	`005 Principle`
[_]	Every governance body has a `Cadence`, a `May Decide` and a `May Not Decide`	`057 Governance Body`
[_]	Every body's `Standing` says honestly whether it functions	`057`, column `Standing`
[_]	Every gate has a `What It Tests` and a named `Held By`	`058 Governance Gate`
[_]	Every control objective names its `Regulatory Source`	`063 Control Objective`
[_]	A baseline exists, is dated, and names its approver	`04 Baselines`
[_]	Every architecture deliverable has an `Approver` and a `Lifecycle Status`	`034`, `RPT14 Deliverable Register`

Phase A — Architecture Vision

Tick	Assertion	Check against
[_]	Every driver reaches at least one goal	`R01 DRV-GOL`, `RPT12 Integrity Check`
[_]	Every goal has at least one objective and every objective at least one measure	`R02`, `R03`
[_]	Every measure has a `Baseline`, a `Target` and a `Source System`	`004 Measure`
[_]	Every requirement traces to an objective and is governed by a principle	`R04`, `R05`, `RPT08 Traceability`
[_]	No row on `RPT08` reads `BROKEN`	`RPT08 Traceability`
[_]	Every assumption has a `Validate By` date	`008 Assumption`
[_]	Every constraint has an `Applies Until`	`007 Constraint`

Phase B — Business Architecture

Tick	Assertion	Check against
[_]	Every Level 3 capability has all five judgement columns; no Level 4 row has any	`013 Business Capability`
[_]	No more than a third of Level 3 rows in an area read `Critical`	`013`, `RPT03 Capability Heatmap`
[_]	Every capability has exactly one owning organisation unit	`R10 ORG-CAP`
[_]	Every value stream reaches at least four capabilities with a `Criticality to stream`	`R11 VST-CAP`
[_]	Every business process has an `Owner Role`, a `Volume p.a.` and a `Cycle Time`	`016 Business Process`, `RPT23`
[_]	Every business service names the capability it realises	`015`, `R12 CAP-BSV`
[_]	Every product has a `Line of Business` and `Config or Code` stated	`018 Product`
[_]	Every Phase B gap has a severity and an affected object	`029 Gap`

Phase C — Data Architecture

Tick	Assertion	Check against
[_]	Every data entity has an `Authoritative Source` — a name, `SPLIT`, `CONTESTED` or `NONE`, never blank	`020 Data Entity`
[_]	Every entity's `Owner Role` resolves to a named individual	`020`, `011 Role`, `RPT29`
[_]	Every entity has a `Retention Period` and a `Retention Basis` with a section number	`020`, `RPT16`
[_]	Every mastered entity has a golden-record rule and a survivorship rule	`043 Master Data Rule`, `RPT17`
[_]	Every entity appears at least once in the CRUD matrix	`R17 BPR-DEN`, `RPT04`
[_]	Every data set names its `Lawful Basis`	`038 Data Set`
[_]	Every `M` on `R21` has `Authoritative` answered Yes or No, never blank	`R21 APP-DEN`, `RPT05`

Phase C — Application Architecture

Tick	Assertion	Check against
[_]	Every application has a `Criticality`, a `Lifecycle` and a `Disposition`	`O23 Application Component`
[_]	Every application has both scores and `Disposition Agrees` reads Yes	`O23`, `RPT20`
[_]	Every interface has a `Style`, a `Direction`, a `Frequency` and `Managed` answered	`R23 APP-APP`, `RPT06`
[_]	Every Tier 1 application supports at least one capability	`R37 APP-CAP`, `RPT07`
[_]	Every application building block has an `Acceptance Criteria`	`O45`
[_]	Every information system service has an `Availability Target`	`O24`

Phase D — Technology Architecture

Tick	Assertion	Check against
[_]	Every physical component has a `Support Ends` date or a stated reason it has none	`O27`, `RPT21`
[_]	Every component governed by a standard has a `Conformance` verdict	`R31 STD-PTC`, `RPT10`
[_]	Every non-conformant component has a containment note and an exit date	`RPT10 Standards Conformance`
[_]	Every Level 1 capability has an `RTO`, an `RPO` and a `Test Cadence`	`O49 Resilience Tier`, `RPT26`
[_]	Every environment states the data it holds	`O48 Environment`
[_]	Every security control has a `State` and, where absent, a `Gap`	`O41`, `O65 Enforcement Point`

Phase E — Opportunities and Solutions

Tick	Assertion	Check against
[_]	Every gap is addressed by at least one work package	`R28 GAP-WPK`, `RPT09`
[_]	Every building block across all four domains has a work package and a wave	`O55`, `RPT24`
[_]	Every building block has acceptance criteria written once, not per project	`O35`, `O44`, `O45`, `O46`
[_]	Every solution building block has a `Sourcing` decision	`O51`, `RPT28`
[_]	Every implementation factor has a `Deduction`	`O52 Implementation Factor`
[_]	Every work package has a `Cost (Rm)`, a `Benefit (Rm p.a.)` and a `Transition State`	`O30 Work Package`

Phase F — Migration Planning

Tick	Assertion	Check against
[_]	No dependency in `R47` reads Impossible	`R47 WPK-WPK`, `RPT27`
[_]	Every transition state has testable exit criteria and a `Still Broken Here`	`O53`, `RPT29`
[_]	Every work package belongs to exactly one transition state	`R50 O53-WPK`
[_]	Every project has a `Lead`, a `Team` and a `Why This Shape`	`O56 Project`
[_]	Every entity has a surviving master once dispositions are executed	`RPT22 Portfolio Data Risk`
[_]	Every migration has a `Quality Gate` and a `Gate Status`	`O39`, `O40`
[_]	Every delivery risk has a `Mitigation` and a named `Owner`	`O62 Delivery Risk`

Phase G — Implementation Governance

Tick	Assertion	Check against
[_]	Every work package in flight has an architecture contract	033 , R33 ACO-WPK
[_]	Every contract's Compliance Status is derived from assessments, not asserted	033 , column Status Basis
[_]	Every acceptance criterion assessed has Evidence in words, not a tick	059 , RPT30
[_]	Every dispensation has an Expires date	061 Governance Change Request
[_]	Every decision has a Date — a decision with no date has not been taken	060 , RPT31
[_]	No interface has reached production without passing G1	058 , RPT06

Phase H and Requirements Management

Tick	Assertion	Check against
[_]	Every change to any record has a row in the change log with previous and new value	02 Change Log
[_]	Every logged change names a Change Request and an Approved By	02 , 03 Change Requests
[_]	No record has a Changed By with a blank Change Ref	every O and R sheet
[_]	Pending approval is falling, not growing	01 Version Control , RPT13
[_]	No object type reads unlinked on the integrity check	RPT12 Integrity Check
[_]	Every requirement that reached delivery has Ever tested answered	RPT32 Requirement Traceability

The ninety-day calendar

Thirteen weeks against the repository. Volume 1 chapter 18 runs the learning track in parallel; this is the sheet-by-sheet track, and the two are designed to be worked in the same weeks. Where volume 1 says *read the record*, weeks one and two below say which RPT sheets to read it through.

The tracks line up section by section with volume 1, chapter 18. Weeks one and two are *Week one: eight things to read, and where they live* and *Twelve questions that tell you most*; weeks three and four, *What you ask for, and what you will actually get*; weeks five to eight, *The first model, and why it must be small*, whose warning applies to the repository too; week nine onwards, *Telling within a month whether the estate is healthy*. Read *Four mistakes newcomers make* before week one and again in week six.

One rule before week one: **weeks one and two touch no sheet.** Everything after week two changes the record, and everything that changes the record needs a row in 02 Change Log and a CR on 03 Change Requests . Raise CR number one in week three, not week eight.

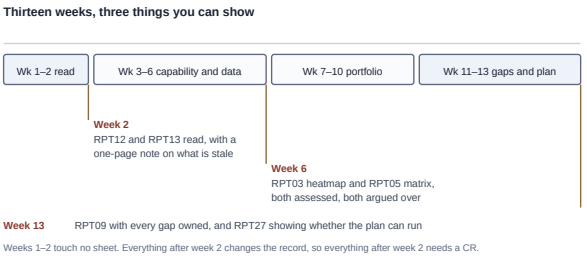

Figure 15.1 The repository track across thirteen weeks. Weeks one and two touch no sheet; everything after week two needs a change request.

Week	What you do in the repository	Sheets you touch
1	Read `00 Index`, `01 Version Control`, `04 Baselines` end to end. Then `RPT01 Dashboard` and `RPT12 Integrity Check`. Write down every number that surprises you	None — read only
2	Read `RPT13 Version Control` and work out which sheets are stale. Read `02 Change Log` from the top; 172 rows tells you more about the practice than any document. Reconcile `00 Index`'s stated counts against `RPT01`	None — read only
3	Raise your first change request. Fix what you found in week two: stale header counts, unowned deliverables. Learn the change discipline on something that does not matter	`03 Change Requests`, `02 Change Log`, `O34`
4	Audit the seven governance columns across six object sheets. Find every row with a `Changed By` and no `Change Ref`. This is your ungoverned-change baseline	`02 Change Log`, `RPT13`
5	Run Pack 1 for one Level 1 capability area. Fill thirty to forty Level 3 rows	`O13`, `R10`, `R11`, `R38`
6	Run Pack 1 for a second area. Read `RPT03 Capability Heatmap` and take it to the executive who owns the worst number	`O13`, `RPT03`
7	Run Pack 2 for the five master entities. Settle Party or record it `CONTESTED`	`O20`, `R21`, `O43`
8	Finish Pack 2 for the remaining entities. Fill `Retention Period` and `Retention Basis` with the information officer	`O20`, `O38`, `RPT16`, `RPT17`
9	Fill `R17 BPR-DEN` with `CRUD` for every process. The absence of any `D` is your first unarguable regulatory finding	`R17`, `RPT04`
10	Run Pack 3. Score all twenty-six applications, then work only the `Disposition Agrees` = No rows	`O23`, `RPT20`
11	Fill `R23 APP-APP` properly, `Managed` column included. Then read `RPT06` and `RPT21` together and build the technology currency case	`R23`, `O27`, `RPT06`, `RPT21`
12	Export `05`, `06`, `07` to the comparison tool. Run Pack 4 on the first sixty differences. Paste the agreed gaps into `O29 Gap`	`05 - 07`, `50 Review`, `70`, `O29`
13	Run Pack 5. Fill `O53` exit criteria and `R47` verdicts. Read `RPT09` and `RPT27` together, and cut a baseline	`O30`, `O53`, `R47`, `04 Baselines`

What you can show, and when

Point	What you put in front of people	Why it lands
End of week 2	One page: the repository's own numbers against its own documentation. `00 Index` claims 33 object types, 36 relationship types and 13 reports; there are 66, 56 and 32. `RPT13` showing which sheets have not moved since baseline. Four change requests awaiting approval	It costs nothing, needs nobody's cooperation, and establishes that you read the record before forming an opinion. It also gets the Board to approve something in your second week, which is a habit worth starting
End of week 6	`RPT03 Capability Heatmap` for two Level 1 areas, assessed by the people who own them. Alongside it, `RPT05 App-Data Matrix` showing contested authority on the master entities	Two executives have scored their own capability and will defend the number. The contested-authority finding is the one that opens the data conversation, and it arrives with their fingerprints on it, not yours
End of week 13	`RPT09 Gaps and Roadmap` with every gap carrying a severity, an owner and a work package. `RPT27 Programme Sequencing` showing whether the plan can be run in dependency order. A baseline cut on `04 Baselines` and approved	A gap register with named owners is the artefact that makes the practice real. `RPT27` is the one that changes a decision: a critical path longer than the time available is arithmetic, and it is the first thing you will have produced that a programme director cannot argue with

YOUR NINETY DAYS

Your start date, week 2 show date, week 6 show date, week 13 show date:

The three sheets you have decided not to touch in year one, and why:

The person who will approve your first change request, by name:

If you only do five things

You will not get the time to do all of it. Nobody does. These five, in this order, in the first six months, and the repository stays true enough to be worth keeping.

#	Do this	Sheets	Why this one
1	**Make every change go through the log.** One change request, one log row with previous and new value, one approver, every time. No exceptions, including your own corrections	`02 Change Log`, `03 Change Requests`	Everything else in the repository can be caught up later. This cannot. A repository without a change history is a document, and the version columns on all 122 object and relationship sheets derive from this log. It is also the evidence base for Joint Standard 1 of 2023
2	**Assess capability at Level 3, with the people who own it.** `Criticality` before `Current Maturity`, `Differentiation` as the brake	`O13`, `RPT03`	It is the only artefact in the workbook a line executive will read without translation, and it is the input to every prioritisation argument you will have for three years
3	**Settle data authority, or record it as contested.** A named master, `SPLIT` with the split stated, or `CONTESTED`. Never blank	`O20`, `R21`, `RPT05`	Unresolved data authority is the most expensive thing in an insurance estate and the hardest to retrofit. `CONTESTED` recorded is worth more than a master invented, because it names a decision somebody has to take
4	**Fill the `Managed` column on every interface.** Style, direction, frequency, managed, and the entity it carries	`R23`, `RPT06`	Forty-four unmanaged interfaces of fifty is the number that funds an integration platform, and it is the number that tells you which applications cannot be changed safely. It takes a day and a half
5	**Test the sequence arithmetically once.** `R47` verdicts, then read `RPT27`	`R47`, `O30`, `RPT27`	It is the one architectural judgement that can be proved rather than argued. A 62-month critical path against 50 months available is the finding that gets a roadmap reopened before money is spent against it

What is deliberately not on that list: `O37 Data Attribute`, `O42 Metadata Element`, `O50 Network Link`, `O54 Architecture Alternative` and the whole `O47 Reference Model Entry` set. All five are useful and none of them is urgent. A new practice that tries to fill 162 sheets fills none of them properly, and a half-filled sheet is worse than an empty one, because an empty sheet is honest about what nobody has done yet.

THE CHAPTER IN TEN LINES

- A workshop that fills no cells was a meeting. Every pack here names the cells.
- Score Criticality before Current Maturity. A room that admits weakness will not then admit dependence.
- Maturity goes on Level 3 rows only. Filling Level 4 breaks the RPT03 roll-up.
- Put CONTESTED on the board as a legitimate outcome in the first ten minutes of the ownership workshop.
- Score business value and technical fitness in separate passes with a break between, or you get a diagonal line.
- Keep the quadrant hidden until step five, then work only the Disposition Agrees = No rows.
- The comparison tool applies no judgement. Nothing reaches 70 Gap Candidates until a person classifies it.
- Open the sequencing session with the arithmetic and refuse to negotiate it.
- Weeks one and two touch no sheet. Raise CR number one in week three.
- If you do five things: log every change, assess capability at L3, settle data authority, fill Managed on every interface, test the sequence once.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx, sheets O11 Role and O57 Governance Body
- **2.** Meridian_Architecture_Comparison_Tool.xlsx, sheets 00 How to Use, 50 Review and 60 Summary
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets RPT27 Programme Sequencing and RPT29 Transition Readiness
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheet 01 Version Control, versioning policy
- **5.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX D

The repository in ArchiMate 4

One hundred and twenty-two sheets of rows, forty-two elements and eleven relationships — and the rule that when they disagree, the rows are right.

Why this mapping exists

You have been handed two descriptions of one architecture that agree nowhere on the page. `Meridian_TOGAF_Metamodel.xlsx` is a TOGAF content metamodel written out as rows: 66 object sheets, 56 relationship sheets, 32 report sheets, every record carrying a `Version`, a `Record Status`, a `Change Ref` and an `Approved By` [1]. `Meridian_ArchiMate4_Views.pptx` is a pack of pictures. Nothing in either says how one becomes the other. This appendix is that bridge. Meridian Assurance Group is an invented South African composite insurer, and its data is real only inside this workbook.

The repository is the **record**. It answers by filter and formula: how many Tier 1 capabilities have no tested recovery objective, which requirements reach no work package, which entity has four masters. A spreadsheet answers those in seconds; a diagram cannot answer them at all.

An ArchiMate 4 view is the **argument**. It takes eight rows out of nine thousand and shows the shape: this driver pushes this goal, this outcome realises it, this requirement realises the outcome, and nothing realises the requirement. That is a sentence you can say to a Chief Risk Officer in thirty seconds, and no filter on `RPT08 Traceability` says it as fast. Neither artefact replaces the other, and authority runs one way.

> **THE RULE**
>
> The repository is authoritative. A view that disagrees with it is wrong — not a variant, not a simplification, not "the version we drew for the board". Where a view shows an element the repository does not hold, either the repository is missing a record and you raise a change request on `03 Change Requests`, or the view is inventing architecture and you delete it. There is no third case.

That rule exists because of a failure this volume has seen. An architect draws a target view, invents a component called "Customer API" to make the picture hang together, and the picture is printed, approved and quoted. Eighteen months later the component does not exist, no work package delivers it, and three designs assume it. The repository catches that at the moment of drawing: no row in `O23 Application Component`, none in `O45`, nothing in `R30 WPK-APP`.

Three working consequences.

Every element on a view carries its repository ID. `APP-01 LIFE/400`, not "LIFE/400". The ID is what makes a view checkable; a box with a name only cannot be reconciled against anything.

Views are generated, not drawn. Every view below is a filter over one object sheet and one or two relationship sheets. Where your tool imports `05 Export Attributes` and `06 Export Records`, import them; where it cannot, lay out by hand once and regenerate the content.

The loss runs one way. Everything on a view exists in the repository. Much of the repository cannot be drawn: `Current Maturity`, `Retention Basis`, `Authoritative`, `Conformance`, `Severity`, `Verdict`. That is not a defect in the notation; it is why the repository exists.

Volume 1, chapter 16, *Modelling insurance in ArchiMate*, teaches the notation. This appendix assumes you have read it, and maps *these sheets* onto it.

The object mapping — all 66 object sheets

The repository's domain labels are not ArchiMate 4 layers, and two differences matter. `O13 Business Capability` and `O14 Value Stream` are labelled Business and belong to the strategy layer. And nine ArchiMate 4 elements carry no layer at all — Role, Collaboration, Process, Function, Service, Event, Path, Grouping and Location — so four service sheets collapse onto one element and the layer column reads "none" for all four.

Twenty of the 66 sheets have no faithful element. Each says so, and says how to carry the row instead. A near fit is worse than nothing, because the near fit gets believed.

Motivation

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`001 Driver`	Driver	Motivation	A condition that motivates goals	`Category` and `Origin` are properties; an external driver needs a Stakeholder beside it
`002 Goal`	Goal	Motivation	Intent with no end date in the notation	`Target Date` cannot be drawn, and all six Meridian goals carry one
`003 Objective`	Outcome	Motivation	Time-bounded and measurable	Not a Goal. Collapse the two and the realisation chain disappears
`004 Measure`	No direct equivalent	—	No metric element exists, and Value is not one	Carry `Baseline`, `Current` and `Target` on the Outcome; movement lives in `RPT01`
`005 Principle`	Principle	Motivation	A rule informing how the enterprise acts	`Tier` is a property; `Statement (condensed)` is documentation, not the label
`006 Requirement`	Requirement	Motivation	A statement a system satisfies or fails	80 rows. Show only those traced to the view's subject
`007 Constraint`	No direct equivalent	—	The 42 elements hold no Constraint	A Requirement with `Type` set to the constraint kind, said plainly in the notes
`008 Assumption`	No direct equivalent	—	Not an analysis outcome, so Assessment misstates it	Repository only; `If Invalidated` becomes a note on the package `R35` names

Strategy

Three sheets, two of them labelled Business in the repository.

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`013 Business Capability`	Capability	Strategy	An ability with an owner and a maturity	560 rows, four levels. One level per view; maturity is a property
`014 Value Stream`	Value Stream	Strategy	Activity ending in a result for a stakeholder	`Stages` is a count, not rows. Draw stages and you are inventing them
`054 Architecture Alternative`	Course of Action	Strategy	An approach chosen to reach a goal	Publish the chosen one; keep the rejected four in the repository, rejected

Organisation and business

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`009 Organization Unit`	Business Actor	Business	A structure that performs behaviour	`Parent` becomes Aggregation. Not an excuse to model the org chart
`010 Actor`	Business Actor	Business	Identity, not position	Some rows are positions: `ACT-03 Policyholder` and `ACT-04 Beneficiary` are Roles
`011 Role`	Role	none	A position an actor occupies	`Authority Level` cannot be drawn, and it decides who may bind
`012 Location`	Location	none	A place	`Criticality` is a property, carried into the resilience view
`015 Business Service`	Service	none	Behaviour visible outside, with a consumer	Four sheets map to Service; without the ID on the label `BSV-04` reads as `ISS-04`
`016 Business Process`	Process	none	A sequence producing a defined outcome	The `Value Stream` column is a relationship held in a column, not on a sheet
`017 Business Function`	Function	none	Behaviour with no fixed order	Not departments, though several share a name with an `009` unit
`018 Product`	Product	Business	What is sold, bundled and contracted	`Config or Code` is a property driving Phase E, not the drawing
`019 Contract`	Business Object	Business	The 42 hold no Contract element	`Counterparty` and `Notice Period` are properties. Composed by the Product
`035 Business Building Block`	Grouping	none	A package, not a thing that exists yet	Realization to `Capabilities Realised`; replace once `051` names a solution

Data

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`020 Data Entity`	Data Object; Business Object on conceptual views	Application	An encapsulation of data	`Authoritative Source` reads `CONTESTED` on three Tier 1 entities, undrawably
`021 Logical Data Component`	Grouping	none	A boundary zone over entities, not a system	Aggregation to `Entities Held` and `R18`. Not an Application Component
`022 Physical Data Component`	Artifact	Technology	A deployed store is a deployable	Where `Technology` names a product, that is System Software under the Artifact
`036 Data Entity Relationship`	No element — a relationship	—	Relationships living on an object sheet	Each row is an Association between Data Objects. `Cardinality` is lost
`037 Data Attribute`	No direct equivalent	—	Attributes are properties	165 rows, never drawn. `Personal Information` is the filter behind the POPIA view
`038 Data Set`	Data Object	Application	A named collection with a consumer	`Lawful Basis` cannot be drawn, and a set with none is a finding
`039 Data Migration`	No direct equivalent	—	The row already names its `Work Package`	Properties on that package, or a Grouping over source and target stores
`040 Source To Target Mapping`	No direct equivalent	—	A transformation rule is not a construct	Nearest is a Flow between stores; `Quality Issue To Resolve First` decides it
`042 Metadata Element`	No direct equivalent	—	Metadata about the model is not in it	Repository only. Group by `Category` if coverage must be shown
`043 Master Data Rule`	No direct equivalent	—	A survivorship rule is policy, not structure	A Requirement per entity if it must be traced; otherwise repository only
`044 Data Building Block`	Grouping	none	As `035`, one layer down	`Closes Gap` becomes an Association to the Assessment standing for the gap

Application

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`023 Application Component`	Application Component	Application	Encapsulated functionality	`APP-01 LIFE/400` is one row and a dozen components — volume 1, chapter 16
`024 Information System Service`	Service	none	Automated behaviour with a consumer	`Realises Business Service` becomes Realization to an `015` Service. Show both IDs
`045 Application Building Block`	Grouping	none	What must exist before anything is chosen	`Wave` is derived from `030`, never stated here; do not show it as fact

Technology

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`025 Technology Service`	Service	none	A technical capability with a consumer	`Availability Target` is a property; the notation carries no service level
`026 Logical Technology Component`	Node	Technology	Vendor-independent infrastructure	Node, not Device, at logical level; `Category` says where a network is truer
`027 Physical Technology Component`	Node, Device, System Software or Communication Network by `Category`	Technology	The row names a product, so name its kind	`Support Ends` drives `RPT10 Standards Conformance`, not the picture
`028 Platform Service`	Service	none	A grouped capability consumed by applications	`Consumption Model` is a property. Serving to the component, per `R36`
`046 Technology Building Block`	Grouping	none	As `035` and `045`	Replace with the Node once `051` names a product
`048 Environment`	Grouping	none	A named set of deployed things, not a thing	Aggregation over its Nodes and Artifacts, Location from `R44`. Marked AUTHORED
`049 Resilience Tier`	No direct equivalent	—	RTO and RPO are properties	Properties on the `013` capability named in `Capability`. `GAP-16`: none has been tested
`050 Network Link`	Path, realised by Communication Network	Technology	Path is the connection, the network realises it	Draw the counterparty on an external link or the view hides the dependency

Implementation and migration

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`029 Gap`	No direct equivalent — carry as Assessment	Motivation	The implementation aspect holds only Work Package, Deliverable and Plateau	Assessment is the honest carrier: a gap is an analysis outcome
`030 Work Package`	Work Package	Implementation	A set of actions	`Delivery Status` is a property, and no package is recorded as delivered
`031 Deliverable`	Deliverable	Implementation	A specified, signed-off output	Shares the element with `034`. Never both on one view
`032 Standard`	Requirement, with the specification as an Artifact	Motivation	A standard is testable against a component	Conformance comes from `R31`, never from the element
`033 Architecture Contract`	No direct equivalent	—	No Contract element, and the useful fact is the verdict	Business Object if a view needs it; otherwise show `059` instead
`051 Solution Building Block`	Grouping while a candidate, the real element once chosen	varies	It becomes a component or node when `Sourcing` is decided	Left a Grouping after the decision, it hides what was bought
`052 Implementation Factor`	No direct equivalent	—	Planning rationale is not architecture	Repository only; `Deduction` belongs in the view notes
`053 Transition Architecture`	Plateau	Implementation	A state the enterprise can stop at	`Still Broken Here` is the column that matters and cannot be drawn
`055 Building Block`	No element of its own	—	A composed view over `035`, `044`, `045`, `046`	Modelled separately, every block appears twice. Filter with it, never draw it
`056 Project`	Work Package	Implementation	A chartered unit of delivery is the same element	Sixteen packages, fifteen projects. One level per view or cost is counted twice
`062 Delivery Risk`	No direct equivalent	—	A delivery risk is not an architecture statement	Repository only, unless it changes a decision. Four read `Certain` and are issues
`066 Architecture Increment`	No direct equivalent	—	The notation models states, not deltas	A Grouping over what the increment adds, on the plateau view

Governance

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`034 Architecture Deliverable`	Deliverable	Implementation	The function's own work products are deliverables	Shares the element with `031`. `Formats` and `Lifecycle Status` are properties
`041 Security Control`	No direct equivalent — carry as Requirement	Motivation	No control element exists	A control recorded `Absent` is a Requirement with no Realization edge; that is the finding
`047 Reference Model Entry`	No direct equivalent	—	It records what should be chosen, not what exists	`Status` becomes a property on the component designated by `R43` or `R46`
`057 Governance Body`	Collaboration; Business Actor where one unit	Business	Four of five draw members from several units	`May Not Decide` is the column that matters and cannot be drawn
`058 Governance Gate`	Event	none	A point at which work is assessed	The assessing behaviour is a Process, assigned to the body in `Held By`
`059 Compliance Assessment`	Assessment	Motivation	A verdict reached by analysis	`Verdict` stays in the repository; all thirteen 2027 criteria are non-compliant
`060 Governance Decision`	No direct equivalent	—	The effect is in the model; the decision is not	Repository only, reachable from the `Change Ref` every element row carries
`061 Governance Change Request`	No direct equivalent	—	It is about the model, not in it	Repository only. A dispensation with no `Expires` date is permanent
`063 Control Objective`	Requirement	Motivation	What must be true is a need statement	The obligation itself is already a Driver in `001`. Seven of twelve have no evidence
`064 Security Policy`	Principle	Motivation	A board-language commitment	`Position` records breach, and the notation cannot say "breached". Three of four are
`065 Enforcement Point`	No element of its own — the row names an existing one	varies	A gateway, identity provider, platform or gate is already modelled	`EP-03 API gateway` names no product, so there is nothing to map it to

The relationship mapping — the 56 relationship sheets

The relationship sheets run `R01` – `R44`, `R46` – `R50` and `R54` – `R60`. There is no `R45`, `R51`, `R52` or `R53`; those numbers were taken and released during the repository's own build and the gaps are real, not an export error [1].

Each sheet carries `Source`, `Target`, a payload column or two and a `Key`. The sheet's direction is the author's sentence, not an ArchiMate direction. Fifteen of the 56 reverse when drawn, because the notation's arrow runs the other way: a service realises a capability, a capability serves a value stream, a location is assigned to what sits in it. Draw a reversed one as the sheet reads and you get a model that validates and means the opposite.

Sheet	From → To	ArchiMate 4 relationship	Direction in the model	Note
`R01 DRV-GOL`	Driver → Goal	Influence	As sheet	`Strength` becomes a label; the notation has no weighting
`R02 GOL-OBJ`	Goal → Objective	Realization	**Reversed** — Outcome realises Goal	The sheet reads "is realised through"; drawn that way the chain inverts
`R03 OBJ-MSR`	Objective → Measure	**Not expressible**	—	Measure is not an element; the values sit on the Outcome
`R04 REQ-OBJ`	Requirement → Objective	Realization	As sheet	The edge `RPT08 Traceability` walks
`R05 PRN-REQ`	Principle → Requirement	Influence	As sheet	Not Realization: a principle shapes a requirement, it does not make it true
`R06 CON-WPK`	Constraint → Work Package	Influence, from the Requirement standing for it	As sheet	Expressible only by substitution, because `007` has no element
`R07 ORG-ACT`	Org Unit → Actor	Aggregation	As sheet	Not Composition: `ACT-02 Independent Broker` belongs to no unit
`R08 ACT-ROL`	Actor → Role	Assignment	As sheet	One actor, many roles — the pattern volume 1, chapter 16 insists on
`R09 ROL-BPR`	Role → Business Process	Assignment	As sheet	Assignment for `Accountable` rows only; the rest are Association
`R10 ORG-CAP`	Org Unit → Capability	Assignment	As sheet	138 rows of ownership. Never draw the whole sheet
`R11 VST-CAP`	Value Stream → Capability	Serving	**Reversed** — Capability serves Value Stream	The sheet reads "is enabled by"
`R12 CAP-BSV`	Capability → Business Service	Realization	**Reversed** — Service realises Capability	`Coverage` is the duplication signal in `RPT07 Capability Coverage`
`R13 FUN-BPR`	Business Function → Process	Aggregation	As sheet	26 rows over 22 processes, so a process sits in two functions
`R14 BPR-BSV`	Process → Business Service	Realization	As sheet	`Sequence` orders processes; express order with Triggering, not here
`R15 BSV-PRD`	Business Service → Product	Aggregation	**Reversed** — Product aggregates Service	Aggregation, not Composition: the same cover sells under several products
`R16 PRD-CTR`	Product → Contract	Composition	As sheet	The contract exists only as part of the product
`R17 BPR-DEN`	Process → Data Entity	Access	As sheet	`CRUD` sets the type: C and U write, R reads. This sheet is `RPT04`
`R18 LDC-DEN`	Logical Data Component → Entity	Aggregation	As sheet	Grouping over Data Objects; an entity may sit in two components
`R19 PDC-LDC`	Physical → Logical Data Component	Realization	As sheet	`State` separates Baseline from Target. One state per view
`R20 APP-BSV`	Application → Business Service	Realization	As sheet	`Coverage` is `Partial` on most rows; four partials on `BSV-01` is the finding
`R21 APP-DEN`	Application → Data Entity	Access	As sheet	`Role` is Master or Copy; `Authoritative` is Yes, No, Contested or Target authority
`R22 APP-ISS`	Application → Information System Service	Realization	As sheet	The cleanest edge in the layer, and the one most often left out
`R23 APP-APP`	Application → Application	Flow	As sheet	`Data Entity` labels it; `Bidirectional` means two Flows, not a two-headed arrow
`R24 APP-TSV`	Application → Technology Service	Serving	**Reversed** — Service serves Application	The sheet reads "is realised by"; the edge to a consumer is Serving
`R25 PTC-LTC`	Physical → Logical Technology Component	Realization	As sheet	`Standard Status` carries into `RPT10 Standards Conformance`
`R26 PTC-LOC`	Component → Location	Assignment	**Reversed** — Location assigned to Node	The notation assigns the place to the thing
`R27 ORG-LOC`	Org Unit → Location	Assignment	**Reversed** — Location assigned to Actor	`Headcount` belongs to the pairing and has nowhere to live
`R28 GAP-WPK`	Gap → Work Package	Association	As sheet	Not Realization: a package does not realise a gap. `Closure` is a property

`R29` `WPK-DLV`	Work Package → Deliverable	Realization	As sheet	The standard implementation edge; nothing lost
`R30` `WPK-APP`	Work Package → Application	Realization	As sheet	`Action` includes Retire, which has no edge; show it as absence on a later Plateau
`R31` `STD-PTC`	Standard → Component	Realization where conformant, otherwise Association	**Reversed** — the component realises the standard	Only the ten `Conformant` rows earn a Realization
`R32` `REQ-APP`	Requirement → Application	Realization	**Reversed** — Component realises Requirement	The sheet reads "is realised by". This is the edge an auditor asks for
`R33` `ACO-WPK`	Architecture Contract → Work Package	**Not expressible** as drawn	—	No Contract element. Show `059` verdicts instead
`R34` `PRN-GAP`	Principle → Gap	Association	As sheet	Not Influence: the gap engages the principle, it does not change it
`R35` `ASM-WPK`	Assumption → Work Package	**Not expressible**	—	No Assumption element. `If Invalidated` becomes a note on the package
`R36` `PSV-APP`	Platform Service → Application	Serving	As sheet	`State` separates Baseline from Target. One state per view
`R37` `APP-CAP`	Application → Capability	Realization	As sheet	185 rows, all reading `Supports`. Filter to one capability branch per view
`R38` `CAP-CAP`	Capability → Capability	Composition	As sheet	548 rows across four levels, from the `Decomposition` column
`R39` `ORG-DLV`	Org Unit → Architecture Deliverable	Association	As sheet	No ownership relationship exists, and Assignment to a passive element is barred
`R40` `DLV-DLV`	Deliverable → Deliverable	Association	As sheet	"Depends on" has no edge between passive elements. Label with `Nature`
`R41` `APP-VST`	Application → Value Stream	Serving	As sheet	81 rows. Useful at stream level, unreadable at application level
`R42` `055-055`	Building Block → Building Block	Association	As sheet	Groupings depending on Groupings. Label with `Domain`
`R43` `047-APP`	Reference Model Entry → Application	**Not expressible**	—	The entry is not an element. `Status` is a property on the component
`R44` `048-LOC`	Environment → Location	Assignment to the Nodes in the environment	**Reversed**	Location cannot be assigned to a Grouping
`R46` `047-PTC`	Reference Model Entry → Component	**Not expressible**	—	As `R43`: the designation is a property, not an edge
`R47` `WPK-WPK`	Work Package → Work Package	Triggering	**Reversed** — the prerequisite triggers the dependent	`Verdict` reads Impossible on three rows, and those three are what `054` answers
`R48` `051-055`	Solution Block → Building Block	Realization	As sheet	The edge turning "what must exist" into "what we are buying"
`R49` `WPK-051`	Work Package → Solution Block	Realization	As sheet	`Sourcing` is a property. Mirrors `RPT28 Solution Sourcing`
`R50` `053-WPK`	Transition Architecture → Work Package	Realization	**Reversed** — Work Package realises Plateau	A plateau is a state; the package brings it about
`R54` `056-WPK`	Project → Work Package	Composition	As sheet	Both ends map to Work Package. One level per view
`R55` `056-056`	Project → Project	Triggering	**Reversed** — the prerequisite project triggers the dependent	`Override` records where sequence was forced. Keep it in the label
`R56` `059-055`	Compliance Assessment → Building Block	Association	As sheet	The Assessment carries no verdict; `Verdict` and `Gate` stay in the sheet
`R57` `060-061`	Decision → Change Request	**Not expressible**	—	Neither end is an element. Repository only
`R58` `064-065`	Security Policy → Enforcement Point	Realization where it enforces, Association where it does not	**Reversed** — the point realises the policy	Four of five enforce nothing, so four of five edges are Association
`R59` `066-056`	Increment → Project	**Not expressible** as drawn	—	No Increment element. Use a Grouping over what the project adds
`R60` `063-REQ`	Control Objective → Requirement	Realization	**Reversed** — the requirement realises the control objective	Both ends are Requirements. Fix on Realization or Specialization and never mix

SIX SHEETS YOU CANNOT DRAW

R03, R35, R43, R46, R57 and R59 have no ArchiMate 4 expression at either end, and R06, R33 and R44 have one only by substitution. Nine of 56. Each is a place where a reader of the views alone reaches a wrong conclusion — that no measure tracks an objective, that no reference model governs the estate. Name the missing sheets in every view's notes.

The view set — what to produce in each ADM phase

For each phase: the views, what they contain, the sheets that populate them, and who asks. Where a standard ArchiMate 4 viewpoint fits, it is named. Where none fits, the view is marked *named here* — yours to define and yours to defend.

Preliminary

View	Elements and relationships	Populated from	For
Organisation (Organization viewpoint)	Actors, Roles, Aggregation, Assignment	O09, O10, O11, R07, R08	The sponsor, to agree whose architecture this is
Principles (Motivation viewpoint)	Principles in four Groupings by Category	O05	GB-01 Architecture Board
Governance (*named here*)	Collaborations, gate Events, the assessment Process	O57, O58	GB-04 Executive Committee
Repository scope (*named here*)	A Grouping per domain over the sheets in use	00 Index, RPT12	The architecture function

Phase A — Architecture Vision

View	Elements and relationships	Populated from	For
Motivation (Motivation viewpoint)	Drivers, Goals, Outcomes, Principles, Requirements	O01 – O06, R01, R02, R04, R05	The sponsor and GB-02 Investment Committee
Goal realisation (Goal Realization viewpoint)	One Goal, its Outcomes, the Requirements realising them	O02, O03, O06, R02, R04	The executive owning that goal
Stakeholder (Stakeholder viewpoint)	Stakeholders, Drivers, Assessments	O11, O01, O29, R34	The Chief Enterprise Architect
Capability map, level 1 (Capability Map viewpoint)	Capabilities at Level 1, shaded by Current Maturity	O13, R38, R10	GB-02 Investment Committee
Value streams (Value Stream viewpoint)	Seven Value Streams and the Capabilities serving each	O14, O13, R11	The Chief Operating Officer
Solution concept (Layered viewpoint)	One line of business, motivation to technology, twenty elements	Everything, filtered hard	The board

Phase B — Business Architecture

View	Elements and relationships	Populated from	For
Business process co-operation	Processes, Roles, Assignment, Triggering, Services realised	O16, O11, O15, R09, R14	Operations, binder oversight
Business function (Organization viewpoint)	Functions aggregating Processes	O17, O16, R13	The Chief Operating Officer
Product (Product viewpoint)	Product aggregating Services, composing the Contract	O18, O15, O19, R15, R16	Product managers, conduct compliance
Capability map, levels 1–3	Capabilities with owners and realising Services	O13, R38, R10, R12	GB-02 Investment Committee
Service coverage (*named here*)	Services realising Capabilities, duplicates visible	R12, R20, mirrors RPT07	The Chief Enterprise Architect

Phase C — Data Architecture

View	Elements and relationships	Populated from	For
Information structure	Data Objects and the Associations between them	O20, O36	The Chief Data Officer
Data realisation (*named here*)	Entity, its Logical Grouping, the Artifact realising it	O20, O21, O22, R18, R19	Data architects, the migration lead
Process-data access (*named here*)	Processes with Access edges typed from CRUD	O16, R17, mirrors RPT04	Business data owners
Authority (*named here*)	Data Objects with every Access edge marked Master	O20, R21, mirrors RPT05	GB-03 Data Governance Council
Personal information (*named here*)	Data Objects flagged personal, with their controls	O37, O38, O41, O20	The Information Officer

Phase C — Application Architecture

View	Elements and relationships	Populated from	For
Application co-operation	Components with Flows labelled by `Data Entity`	`023`, `R23`, mirrors `RPT06`	The integration lead
Application usage	Components realising Services, serving Processes and streams	`R20`, `R22`, `R41`	The Chief Information Officer
Application structure	One component decomposed into its real parts	`023`, `045`	The modernisation business case
Application-data (*named here*)	Components with Access edges to Data Objects	`R21`, mirrors `RPT05`	Data and application architects together
Portfolio (*named here*)	Four Groupings by `Portfolio Quadrant`	`023`, mirrors `RPT02`, `RPT20`	`GB-02 Investment Committee`

Phase D — Technology Architecture

View	Elements and relationships	Populated from	For
Technology (Technology viewpoint)	Nodes, System Software, Services, Realization	`025`, `026`, `027`, `R25`	The infrastructure lead
Technology usage	Technology and Platform Services serving Components	`R24`, `R36`	The Chief Information Officer
Deployment (Implementation and Deployment viewpoint)	Nodes at Locations, Paths between sites, Environments	`027`, `012`, `048`, `050`, `R26`	Resilience and outsourcing compliance
Standards conformance (*named here*)	Requirements for Standards, Influence to Nodes	`032`, `R31`, mirrors `RPT10`	`GB-01 Architecture Board`
Resilience (*named here*)	Level 1 Capabilities carrying RTO, RPO, last test	`049`, `013`, mirrors `RPT26`	The Chief Risk Officer

Phase E — Opportunities and Solutions

View	Elements and relationships	Populated from	For
Gaps and building blocks (*named here*)	Assessments for gaps, Groupings for blocks	`029`, `055`, `R28`, `R42`, mirrors `RPT09`	The programme board
Solution sourcing (*named here*)	Solution Groupings realising blocks, by `Sourcing`	`051`, `R48`, `R49`, mirrors `RPT28`	`GB-02 Investment Committee`
Architecture alternatives (Strategy viewpoint)	Courses of Action against the Goals each reaches	`054`, `002`	The board, once
Transition overview (Migration viewpoint)	Three Plateaus with Triggering between them	`053`, mirrors `RPT29`	The sponsor

Phase F — Migration Planning

View	Elements and relationships	Populated from	For
Implementation and migration	Work Packages realising Plateaus and Deliverables	`030`, `031`, `053`, `R29`, `R50`	The programme board
Project (Project viewpoint)	Projects composing Work Packages, Triggering between	`056`, `R54`, `R55`	The portfolio office
Sequencing (*named here*)	Work Packages with Triggering, Impossible edges marked	`R47`, mirrors `RPT27`	The programme director
Increment (*named here*)	A Grouping per increment over what each Plateau adds	`066`, `R59`	The architecture function

Phase G — Implementation Governance

View	Elements and relationships	Populated from	For
Compliance (*named here*)	Assessments against block Groupings at gate Events	`059`, `058`, `055`, `R56`, mirrors `RPT30`	`GB-01 Architecture Board`
Gate and body (*named here*)	Collaborations, the Events they hold, the Processes they run	`057`, `058`	`GB-04`
Enforcement (*named here*)	Principles for policies, Realization to points that enforce	`064`, `065`, `R58`	The Chief Information Security Officer

Phase H — Architecture Change Management

View	Elements and relationships	Populated from	For
Change impact (*named here*)	The element a request touches, plus first-degree neighbours	`03 Change Requests`, `02 Change Log`	`GB-01 Architecture Board`
Baseline against target (*named here*)	Two Plateaus with the classified differences between them	`70 Gap Candidates` [2], then `O29`	The Chief Enterprise Architect
Requirements realisation	Requirements with their realising Components and Processes	`O06`, `R32`, `R04`, mirrors `RPT08`, `RPT32`	Internal audit, the Chief Risk Officer

> **HOW MANY VIEWS IS ENOUGH**
>
> Forty-four views across ten phase tables, and you will not produce them all. Start with the Phase A motivation view, the capability map, the application co-operation view, the authority view and the transition overview. Those five answer most of what gets asked in the first six months, and each is a filter over sheets that are already populated.

Four figures from the repository

Each is built from rows that exist, with their IDs. Nothing in them is invented.

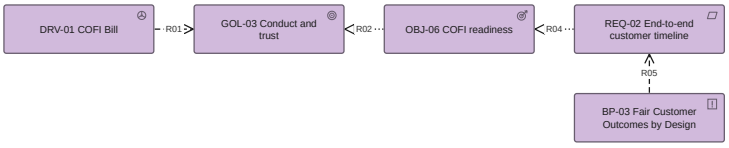

Figure 16.1 Motivation chain for Meridian, built from O01, O02, O03, O05 and O06 with R01, R02, R04 and R05. The Objective row becomes an Outcome, and the Measure that tracks it has no element.

The chain reads left to right in intent and right to left in realisation. `DRV-01` influences `GOL-03`; `OBJ-06` realises that goal; `REQ-02` realises the outcome; `BP-03` influences the requirement. Five boxes, four relationship sheets. Note what is absent: `O04 Measure` holds the indicators that say whether `OBJ-06` is moving, and no element here carries them. `OBJ-06` stands at 12% against a target of 100%, and the picture cannot say so.

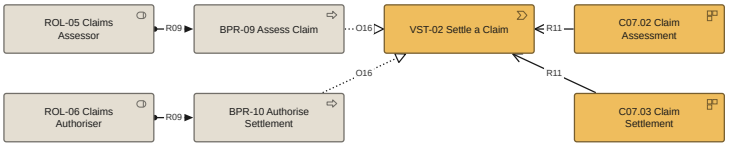

Figure 16.2 VST-02 Settle a Claim with the capabilities serving it and the roles accountable for its processes, from O11, O13, O14, O16, R09 and R11.

Roles are assigned to processes, processes realise the value stream, capabilities serve it. The role edges come from `R09`, but only the rows marked `Accountable`: `ROL-05 Claims Assessor` is accountable for `BPR-09` and responsible on two others, and drawing all three would say she runs the stream. The process-to-stream edges come from the `Value Stream` column of `O16`, not from a relationship sheet — a relationship held in a column, which is what a generated view drops.

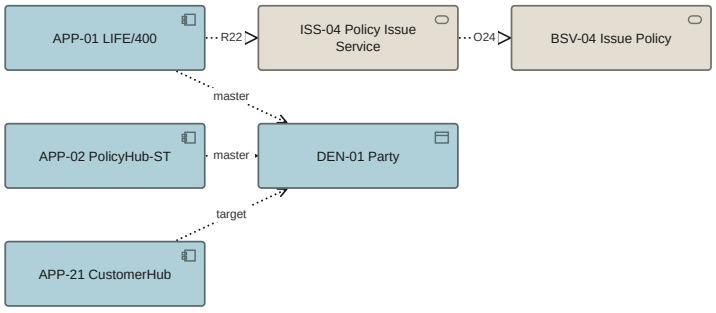

Figure 16.3 Three application components holding DEN-01 Party as master, and the service chain above them. The duplicate authority is countable, not drawable.

APP-01 LIFE/400 realises ISS-04, which realises BSV-04. Both are Service in ArchiMate 4, so the IDs on the labels do the work colour used to do. Then the point of the figure: three components hold DEN-01 Party as master. R21 records APP-01, APP-02, APP-09 and APP-18 as Master with Authoritative reading Contested, and APP-21 CustomerHub as the designated target authority. O20 carries CONTESTED and a Master Count of 4. The notation cannot express contested authority — every Access edge looks alike. What the view does is make the count visible, and the count is the whole of GAP-01.

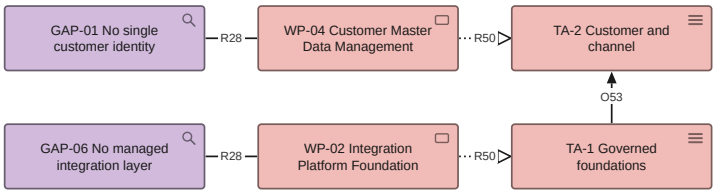

Figure 16.4 Two gaps carried as Assessments, the work packages addressing them, and the two plateaus those packages realise, from O29, O30, O53, R28 and R50.

Two gaps carried as Assessments, two work packages, two plateaus. R28 gives the gap edges, R50 gives package-to-plateau once reversed, and the Triggering between TA-1 and TA-2 comes from the Transition column of O53. The view cannot say that the two states overlap by twelve months, that TA-2 costs R303.6m, or that WP-02 has a criterion assessed non-compliant. All three are in the repository; none is drawable.

Keeping views and repository in step

Four rules, short because they are either kept or the arrangement collapses inside two quarters.

Regenerate, never hand-maintain. A view is a query result with a layout. When a row changes, re-run the query. Hand-editing a view to match a changed sheet works twice and fails the third time, and nobody notices until a committee reads a stale picture.

The view carries the ID, not just the name. WP-04 Customer Master Data Management, not "Customer MDM". Names are edited in O30 without a change request; IDs are not. A view that cannot be reconciled is not evidence.

An element the repository does not hold is a change request waiting to be raised. Not an error to fix quietly — a change request on 03 Change Requests, with a Type and a Scope, so the decision to add it is made by someone entitled to make it. The usual cause is a workshop that agreed something true and never wrote it down. Worth capturing; not worth capturing by drawing.

And the reverse: a record on no view is a question, not a problem. RPT12 Integrity Check counts links per record, and the low scorers are often records nobody needs. But the twenty strategic requirements in O06 that reach no work package are not unnecessary — they are unrealised, which is the finding. Establish which of the two it is before deleting anything.

The test that catches most of this runs in a minute. Take a published view, list its element IDs, look each one up. Every ID resolves, every relationship appears on its sheet with the same two endpoints, every reversal above has been applied. If that holds, the view is a true statement about the repository. If it does not, you have two architectures.

THE CHAPTER IN TEN LINES

- The repository is the record and an ArchiMate 4 view is the argument; where they disagree the repository wins and the view is wrong, not a variant.
- Every element on a view carries its repository ID, because a box with a name only cannot be reconciled against anything.
- Twenty of the 66 object sheets have no faithful ArchiMate 4 element, and the honest answer for each is a property, a Grouping or the repository alone.
- Nine ArchiMate 4 elements carry no layer, so the four service sheets O15, O24, O25 and O28 all become Service and the ID on the label is what tells them apart.
- O03 Objective is an Outcome, not a Goal; collapse the two and the realisation chain that the whole motivation view rests on disappears.
- Eleven of the 56 relationship sheets reverse when drawn: R02, R11, R12, R15, R24, R26, R27, R32, R44, R50 and R58.
- Six relationship sheets cannot be drawn at all — R03, R35, R43, R46, R57 and R59 — and every view should name in its notes what it cannot show.
- Forty-four views cover the phases, but five of them answer most of what gets asked in the first six months.
- Gaps have no element in ArchiMate 4's implementation aspect; carry them as Assessments and keep Severity and Gap Type in O29.
- Regenerate views rather than hand-maintaining them, and treat an element with no repository record as a change request, not a correction.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx — 66 object sheets (O01–O66), 56 relationship sheets (R01–R44, R46–R50, R54–R60) and 32 report sheets
- **2.** Meridian_Architecture_Comparison_Tool.xlsx, sheets 32 Diff Attributes, 50 Review and 70 Gap Candidates
- **3.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/
- **4.** The Open Group, ArchiMate Forum — language overview https://www.opengroup.org/archimate-forum/archimate-overview

APPENDIX E

Where to look in volume 1

The volume read backwards: start from the step you are stuck on, and find the chapter and section where volume 1 already worked the judgement through.

This appendix runs the other way round from the rest of the volume. Everywhere else you start from a phase and work forwards. Here you start from the step you are stuck on and work back to the place in volume 1 where the judgement behind it has already been argued through.

The two volumes were written to be used together and are almost never read that way. Volume 1 is the why; this volume is the what and the where. Work the steps without the teaching and you fill cells correctly and defend them badly.

In the step index the third column gives the volume 1 **chapter number** and then, in italics, the **section heading inside it**. A dash means volume 1 has nothing to say about that step, which is honest and common: the comparison tool, the governance columns and the change-request pipeline are this volume's subject and nobody else's.

Where volume 1 carries pre-drafted content — chapter 19, appendix A — its IDs are its own. `P-01` – `P-12`, `C-01` – `C-20`, `R-01` – `R-33`, `D-01` – `D-20` and `A-01` – `A-25` are five teaching sets. The repository holds 22 `BPR-` rows, 556 four-level `C01.02.03` codes, 27 `ROL-`, 24 `DEN-` and 26 `APP-`. **Match on name. Never on number.**

Meridian Assurance Group, the insurer in the repository, and Cape Mutual, volume 1's, are both inventions and their data is not interchangeable.

The step index

One hundred and fifty-seven steps, eleven chapters, in the order you would meet them.

Chapter 2 · Standing the practice up (Preliminary)

Step	What it asks you to decide	Volume 1 chapter and section
2.1	Which licences and legal entities are in	15 · *The Prudential Authority: licence classes are a data attribute*
2.2	Whether the record is fit to be a baseline	—
2.3	Which sheets you will not fill this year	16 · *Is it an element at all?*
2.4	Which of the 31 principles survive	—
2.5	Whether each principle governs anything	—
2.6	Who may decide what, and who may not	5 · *Risk, compliance and legal*
2.7	What each gate tests, and who holds it	15 · *What the regulator actually inspects*
2.8	Where the architecture function sits	5 · *Technology, and where you sit*
2.9	How good the practice is today	18 · *Telling within a month whether the estate is healthy*
2.10	What is forcing the enterprise to change	15 · *A calendar worth carrying*
2.11	Which measures the architecture answers to	6 · *Reading the ratios*
2.12	What you will have produced by when	18 · *The ninety-day plan*
2.13	Which referential errors to fix first	—
2.14	Whether the Preliminary Phase is closed	—

Chapter 3 · Architecture Vision (Phase A)

Step	What it asks you to decide	Volume 1 chapter and section
3.1	Who is asking, and for what	18 · *What you ask for, and what you will actually get*
3.2	Whose view of the architecture counts	5 · *The org chart will not help you*
3.3	What each stakeholder needs to see	5 · *One process, every role*
3.4	How the business actually makes money	4 · *The canvas for a composite insurer*
3.5	Which outcomes the enterprise delivers	4 · *Customer journeys, done honestly*
3.6	Breadth, depth, time and entity scope	2 · *The table to keep*
3.7	What you cannot change, and what you are guessing	15 · *Twin Peaks, and why South Africa has two regulators*
3.8	The twenty requirements that shape everything	15 · *Modelling a regulatory requirement properly*
3.9	The one-page argument for changing	17 · *The motivation view: why the group is changing*
3.10	What the architecture work will cost	—
3.11	Whether Phase A earned its mandate	—

Chapter 4 · Business Architecture (Phase B)

Step	What it asks you to decide	Volume 1 chapter and section
4.1	Which lines and units Phase B covers	2 · *The question that decides everything else*
4.2	The organisation as fact, not architecture	5 · *The org chart will not help you*
4.3	Which rows are actors and which are roles	16 · *Party and role, and why getting it wrong poisons a model*
4.4	Where each role's authority stops	5 · *Underwriting*
4.5	Where value begins and where it lands	4 · *The moments that decide the relationship*
4.6	Which capabilities each stream depends on	19 · *Capability to process*
4.7	Whether the inherited map is yours	13 · *How to use a capability model properly*
4.8	That the four levels decompose cleanly	19 · *Business capabilities*
4.9	One named owner per assessed capability	19 · *Keeping these alive*
4.10	How mature each capability really is	13 · *Mapping ACORD capabilities onto an estate*
4.11	Target, criticality, differentiation, disposition	13 · *Calibrate: what other industries have*
4.12	Which capabilities make the shortlist	—
4.13	What is a function and what is a process	8 · *L0, L1, L2, L3 — and the warning*
4.14	Volumes and cycle times per process	8 · *The L1 process map*
4.15	Who is responsible as well as accountable	19 · *Process to role, as RACI*
4.16	Whether a service is channel independent	7 · *2. Distribution, and why it shapes the architecture more than anything else*
4.17	Which products are configured and which coded	12 · *2. Product configuration and the product factory*
4.18	What Phase C inherits	—

Chapter 5 · Data Architecture (Phase C)

Step	What it asks you to decide	Volume 1 chapter and section
5.1	How many entities are in scope	9 · *How many*
5.2	Which entities are missing	19 · *Conceptual data entities*
5.3	Which application masters each entity	10 · *Master data and the single view of customer*
5.4	A person, not a role, behind each entity	19 · *Keeping these alive*
5.5	Cardinality on every relationship	9 · *The cardinality that carries meaning*
5.6	Which logical store holds each entity	9 · *Where this sits relative to the logical model and to ACORD*
5.7	The physical estate as it is	12 · *14. Warehouse, lakehouse and regulatory reporting*
5.8	Which attributes are worth recording	10 · *Policy, product, version, endorsement, schedule*
5.9	Which entities carry personal information	15 · *POPIA, and the two sections that catch insurers*
5.10	The lawful basis for each data set	15 · *FICA, and its argument with POPIA*
5.11	What a golden record is, per entity	10 · *Master data and the single view of customer*
5.12	What metadata exists and what does not	10 · *Do not draw this yourself: the industry data models*
5.13	Which process creates, reads, updates, deletes	19 · *Process to data, as CRUD*
5.14	Which data has to move, and how	10 · *Bitemporality, and why this is the hardest temporal problem you will meet*
5.15	Whether the data layer can be handed on	—

Chapter 6 · Application Architecture (Phase C)

Step	What it asks you to decide	Volume 1 chapter and section
6.1	Which conceptual boxes the estate fills	11 · *The conceptual application model*
6.2	Every application anyone can name	12 · *The shape of the estate*
6.3	Criticality, lifecycle and run cost	12 · *1. Policy administration — the core*
6.4	The applications nobody declared	12 · *The mainframe reality in life insurance*
6.5	What each application is worth	11 · *As a package-fit assessment*
6.6	How sound each application is	14 · *What rots first*
6.7	Invest, tolerate, migrate or eliminate	14 · *Build versus buy, per capability — and the forgotten third option*
6.8	Which automated services exist	11 · *What goes in a box, and what does not*
6.9	Which application realises which service	16 · *Is it a service or a process?*
6.10	Which capabilities each application supports	19 · *Capability to application — the coverage map*
6.11	Where coverage is thin and support duplicated	19 · *Process to application*
6.12	Which application is the system of record	19 · *Application to data, with the system of record*
6.13	Which interfaces are managed	12 · *Integration patterns that actually show up*
6.14	Which requirement each application answers	16 · *Motivation: from a rule to the system that satisfies it*
6.15	Whether the estate matches the reference model	13 · *The Reference Architecture: seven facets*
6.16	Where decommissioning will orphan data	12 · *System of record versus system of engagement*
6.17	What the portfolio catalogue says publicly	11 · *As the basis for the application portfolio*

Chapter 7 · Technology Architecture (Phase D)

Step	What it asks you to decide	Volume 1 chapter and section
7.1	Which technical capabilities are offered	11 · *Finance, risk, data and the common layer*
7.2	What each application rests on	12 · *What replaced the service bus*
7.3	The vendor-independent components	14 · *Data platforms and the hyperscalers*
7.4	Which product realises which component	12 · *The mainframe reality in life insurance*
7.5	Which specifications are governed	21 · *Industry models and messaging standards*
7.6	Conformance, component by component	13 · *What is free, and what is not*
7.7	What goes out of support, and when	14 · *What rots first*
7.8	Which platform services are standard	14 · *Data platforms and the hyperscalers*
7.9	Where the estate physically sits	—
7.10	What each environment holds	—
7.11	What each link carries	13 · *AL3, and why insurance integration looks the way it does*
7.12	RTO and RPO per capability	15 · *What the regulator actually inspects*
7.13	Which controls exist and which are absent	15 · *POPIA, and the two sections that catch insurers*
7.14	Whether Phase D can be signed off	21 · *What will date fastest*

Chapter 8 · Opportunities and Solutions (Phase E)

Step	What it asks you to decide	Volume 1 chapter and section
8.1	The three inputs the tool needs	—
8.2	Which comparison mode answers the question	—
8.3	Which records appear, vanish or move	—
8.4	What the tool thinks each change is	—
8.5	Whether each difference was governed	—
8.6	How severe each candidate gap is	—
8.7	Which capability gaps the tool cannot see	13 · *How to use a capability model properly*
8.8	The five capabilities usually missing entirely	11 · *Where this goes wrong*
8.9	Which principle each gap engages	—
8.10	What must exist, before who supplies it	11 · *As a target architecture skeleton*
8.11	Which block depends on which	—
8.12	Buy, build or configure, per block	14 · *Build versus buy, per capability — and the forgotten third option*
8.13	What gets funded together	14 · *What a core replacement actually costs*
8.14	Which states you can stop at safely	17 · *The change view*
8.15	Which three options the board chooses between	14 · *Why a shortlist is an architecture decision*
8.16	What the roadmap says and shows	18 · *The ninety-day plan*

Chapter 9 · Migration Planning (Phase F)

Step	What it asks you to decide	Volume 1 chapter and section
9.1	What becomes a chartered project	—
9.2	What will shape delivery whatever you plan	12 · *The mainframe reality in life insurance*
9.3	How far the plan is from the model	—
9.4	Accept, resequence, descope or refuse	14 · *The one-platform trap*
9.5	Where each cost number came from	14 · *What a core replacement actually costs*
9.6	Whether a benefit traces to a measure	6 · *Reading the ratios*
9.7	What each step makes possible	17 · *The change view*
9.8	The consequence of each delivery risk	14 · *The one-platform trap*
9.9	Which packages an assumption holds up	—
9.10	Approach and quality gate per movement	10 · *Bitemporality, and why this is the hardest temporal problem you will meet*
9.11	Which quality issue to fix first	9 · *The spine: a party, a contract, a thing*
9.12	Which weeks the business will actually give you	7 · *10. The reporting calendar*
9.13	Whether the plan is funded	—

Chapter 10 · Implementation Governance (Phase G)

Step	What it asks you to decide	Volume 1 chapter and section
10.1	Who can refuse, at which gate	—
10.2	What the delivery organisation signed up to	15 · *Joint Standard 1 of 2024: vendor selection becomes a regulatory filing*
10.3	Whether the criteria were inherited or invented	—
10.4	Whether the design will meet the criteria	12 · *Integration patterns that actually show up*
10.5	A verdict per criterion, with evidence	—
10.6	Whether a contract may claim compliance	—
10.7	What the governance position is this quarter	—
10.8	What is excused, by whom, until when	15 · *Treating Customers Fairly: six outcomes, six design consequences*
10.9	Whether the decision was the body's to take	5 · *Risk, compliance and legal*
10.10	Whether the benefit arrived	6 · *Two results, not one*
10.11	Whether every block in scope is funded	—
10.12	Which requirements the gate cannot test	16 · *Motivation: from a rule to the system that satisfies it*
10.13	How the gates meet the control functions	15 · *What the regulator actually inspects*
10.14	What the gate calendar holds this month	15 · *A calendar worth carrying*

Chapter 11 · Architecture Change Management (Phase H)

Step	What it asks you to decide	Volume 1 chapter and section
11.1	What is changing, and how narrowly	—
11.2	The before and after of every field	—
11.3	Which body decides, and on what papers	5 · *Risk, compliance and legal*
11.4	Simplification, incremental or re-architecting	13 · *What a reference architecture is for, and what it is not*
11.5	What changed with no change request	—
11.6	Who owns each unexplained difference	—
11.7	When the sweeps run, and off what trigger	19 · *Keeping these alive*
11.8	Whether a decision needs a fixed reference	—
11.9	The three health numbers and their targets	18 · *Telling within a month whether the estate is healthy*
11.10	Which maps a process change invalidates	19 · *Process to data, as CRUD*
11.11	Which business events force a cycle	2 · *Sasria, and why every South African policy system has a field for it*
11.12	What to do about an expiry inside a year	14 · *What rots first*
11.13	What the board is being asked for	6 · *Reading the ratios*

Chapter 12 · Requirements Management (continuous)

Step	What it asks you to decide	Volume 1 chapter and section
12.1	Where a requirement lives, whatever made it	—
12.2	Whether it can be tested at a gate	15 · *Modelling a regulatory requirement properly*
12.3	Which objective the requirement serves	16 · *Motivation: from a rule to the system that satisfies it*
12.4	Which principle governs it, and how	—
12.5	Which principles govern nothing	—
12.6	What the obligation actually requires	15 · *Treating Customers Fairly: six outcomes, six design consequences*
12.7	Who is funded to deliver it	—
12.8	Traced against ever tested	—
12.9	Whether the register is one register	19 · *What these are, and what they are not*
12.10	Whether to accept it into this phase	12 · *2. Product configuration and the product factory*
12.11	The reason the refusal is defensible	—
12.12	Which rows are no longer live	—

The reverse map

The same thing from the other end: each of volume 1's twenty-two chapters, where this volume draws on it, and the one thing to carry away if you read nothing else in it. Chapter numbers in the middle column are **workbook** chapters.

Volume 1 chapter	Workbook chapters that use it	The one thing to take from it
1 · *The industry in one chapter*	1, 2	The seven things that decide every later judgement. Read it again the week before Phase A, not only on day one.
2 · *The lines of business*	3, 4, 11	Short-term and long-term are two businesses. Scope that does not say which is not scope.
3 · *The vocabulary*	4, 5, 12	The words that have been redefined. Half of all catalogue arguments are vocabulary arguments.
4 · *How the business model works*	3, 4	The canvas and the four customer journeys, which is where value streams come from.
5 · *Who does what*	2, 3, 4, 10, 11	Who may decide what. The org chart will not tell you and the repository cannot.
6 · *How insurers make money*	2, 9, 10, 11	Which numbers an insurer already reports, which is the only place a benefit can land.
7 · *The operating model*	4, 9	The reporting calendar. It decides which weeks your plan is allowed to have.
8 · *The process model*	4	L1 and L2 are different artefacts. Loading L3 into `016` is the commonest Phase B error.
9 · *The conceptual data model*	5, 9	Party, contract, thing — and the cardinality between them, which carries the business rule.
10 · *The information model*	5, 9	Bitemporality. No migration plan written without it has ever held.
11 · *The conceptual application model*	6, 7, 8	What must exist before any vendor is named. The skeleton of every target architecture.
12 · *The application landscape*	4, 5, 6, 7, 10, 12	Sixteen system types and the integration patterns between them. The estate you are cataloguing.
13 · *The reference architecture*	4, 6, 7, 8, 11	What a capability model is for, and what ACORD does and does not give you free.
14 · *Technology providers*	6, 7, 8, 9, 11	What rots first, what a core replacement costs, and why a shortlist is an architecture decision.
15 · *Regulation, risk and what the regulator inspects*	2, 3, 5, 7, 10, 12	The obligations in substance. This workbook records them; it does not teach them.
16 · *Modelling insurance in ArchiMate*	4, 6, 10, 12, 16	Five decisions that settle most of a model. Appendix D maps the sheets; this teaches the notation.
17 · *Cape Mutual, end to end*	3, 8, 9, 16	What a complete, consistent model looks like before you try to produce one.
18 · *Your first ninety days*	2, 11, 15	The learning track that runs beside appendix C's repository track, week for week.
19 · Appendix A, *The cross-maps*	4, 5, 6, 11, 12	Pre-drafted catalogues and the three rules for keeping them alive. Match on name, never on number.
20 · Appendix B, *Glossary*	all	The arbiter when two people in a workshop mean different things by the same word.
21 · Appendix C, *Standards, bodies and where to find them*	7	Where the standards in `032` come from, and which of them will date fastest.
22 · Appendix D, *References*	all	The sources. Cite from here rather than from memory when a number is challenged.

Two patterns in that middle column. Chapters 12 and 15 are each used in six workbook chapters, which is the right shape: the estate and the regulation are the two things an insurance architect is never done with. And chapters 20 to 22 read *all*, because a glossary is consulted under pressure rather than read.

What each volume deliberately leaves out

Neither volume is a shortened version of the other, and the boundary saves you looking in the wrong place.

Volume 1 covers, and this volume does not:

- **The industry itself.** What insurance is, how the risk pool works, why a promise to pay later behaves unlike any other product. This volume assumes you know.
- **The economics.** Combined ratio, the float, new business strain, persistency, reserving. The repository has a `Benefit (Rm p.a.)` column and no opinion about where a credible number comes from. Chapter 6 has the opinion.
- **The vendor landscape.** Who sells what, which platforms have landed in South Africa, what a core replacement costs. `027 Physical Technology Component` names eighteen products and says nothing about the market around them.
- **The regulation in substance.** Twin Peaks, SAM, TCF, FAIS, FICA, POPIA, IFRS 17, the joint standards. `063 Control Objective` holds twelve obligations as rows; what each obliges you to do is chapter 15, and no row can carry it.
- **A complete worked model.** Chapter 17 draws Cape Mutual end to end across ten views. This volume draws fragments, because its subject is the record.

This volume covers, and volume 1 does not:

- **The repository.** One hundred and sixty-two sheets, which columns carry judgement, which are derived, and the naming rules that keep every lookup resolving. Appendix A is the whole of it.
- **The comparison tool.** Twelve sheets, three extracts, the composite `Key` and the gap candidates that fall out. Chapter 8 teaches it, chapter 11 sets the cadence. Volume 1 does not mention it.

- **The ADM as a sequence of acts.** Not the phases in the abstract but 157 steps, each with a sheet, a signatory, a testable completion condition and a report. This volume is where somebody can check that you did the work.
- **Refusal.** Gates, dispensations with expiry dates, architecture contracts and decision rights. Chapter 10 exists because a practice that cannot refuse has not implemented the method.
- **The ninety-day calendar against the record.** Chapter 18 says what to read and who to ask. Appendix C says which sheets you touch in week three.

One closing instruction, and it is the only thing here worth memorising. Where a step and volume 1 disagree about the business, volume 1 is probably right and your sheet is stale. Where they disagree about the repository — a count, a column name, an ID prefix — the repository is right and both books are describing it. Check the sheet before you argue with either.

THE CHAPTER IN TEN LINES

- Start from the step, not the phase: the index runs step to teaching, not teaching to step.
- The third column gives a volume 1 chapter number, then the section heading inside it.
- A dash is honest. Roughly a third of the 157 steps have no volume 1 counterpart.
- Volume 1 appendix A's IDs are a teaching set. Match catalogues on name, never on number.
- Chapter 15 and chapter 12 are each used in six workbook chapters. Regulation and the estate never close.
- Volume 1 owns the industry, the economics, the vendors and the regulation in substance.
- This volume owns the repository, the comparison tool, the gates and the 157 steps.
- Meridian is this volume's insurer; Cape Mutual is volume 1's. Do not mix their data.
- Disagreement about the business: believe volume 1. Disagreement about the repository: believe the sheet.
- Read the reverse map once a quarter. It is the fastest check on which teaching you have stopped using.

APPENDIX F

The forms

36 printable worksheets, 12 sets of the questions you must answer before you can fill one honestly, and the discipline that gets the writing off the paper and into the sheet.

This appendix is paper. Everything before it tells you which sheet and which column. This one gives you the sheet of paper you carry into the room and bring back with writing on it.

Meridian Assurance Group is an invented insurer. Its data is real within this workbook and nowhere else. Every example row below is marked as an example and is there to be crossed out.

How to use these forms

Print the forms for the phase you are in, and only those. There are 36 of them and nobody needs all of them in a week. The index two pages on gives the form number, its title, the sheet it transcribes into and the ADM phase it belongs to. Find your phase, print those pages, and put them in the folder with the agenda.

Fill them by hand, in the room, while the people who know the answer are still in front of you. Transcribe into the repository the same day — not the same week. Keep the paper until the record is approved and the `Approved By` column on the sheet has a name in it. Then retire the paper the way the last section of this appendix describes.

Why paper beats a laptop in a workshop

A laptop does three things to a room, all of them bad. It puts a screen between you and the person you are asking, so you stop watching their face and miss the hesitation that is the real finding. It turns an argument into a typing queue: the room slows to the speed of the typist, and the typist starts steering towards the cells that are easy to fill. And it tempts you to type into the repository live, which puts a half-agreed judgement into the sheet, into `02 Change Log`, and in front of a report before anybody has signed it.

Paper is also faster at the thing workshops are for. Two people can fill two forms at once. A form can be slid across a table to the executive who owns the capability, and they will initial a row they would never have agreed to by email. A crossing-out stays visible, which matters: on Form B-3 the score somebody changed under pressure is more interesting than the score they settled on. And when the room disagrees you write both numbers in the cell, which no spreadsheet lets you do.

The cost of paper is transcription, and transcription is where the discipline has to live. That is the last section of this appendix and it is the part to read twice.

A form that does not transcribe into a real sheet is useless

Every column on every form below is a real column of a real sheet of `Meridian_TOGAF_Metamodel.xlsx`, spelled the way the sheet spells it, in the order the sheet holds it [1]. That is not tidiness. It is the property that makes a form worth filling: the person transcribing reads left to right across the paper and left to right across the sheet, and nothing has to be interpreted on the way.

Which means the forms carry some things that look like faults and are not. `Source (ROL)` is an ugly column heading, and it is what `R09 ROL-BPR` calls that column, so that is what the form says. Form CA-2 prints a column the sheet computes, because the room's expectation and the formula's answer disagreeing is worth catching. Form D-1 warns you that `O27 Physical Technology Component` carries two columns headed `Version` — the product version and the record version — because a transcriber who does not know that will put the right number in the wrong place.

Where a sheet has more than six columns, the form splits. Six is what fits the 170mm text column and still leaves a cell wide enough to write a date in. So `O13 Business Capability` becomes Form B-2, identify, and Form B-3, assess. `O20 Data Entity` becomes C-1 and C-2, and C-2 splits again into a tiering pass and a retention pass, because the Data Governance Council signs one and the information officer signs the other. `O23`, `O56`, `O06`, `R23` and `03 Change Requests` all run as two passes over the same identifiers for the same reason. Each form says which columns it covers and which are typed straight into Excel; the ones left to Excel are, with few exceptions, sentences, formulas and audit fields — `Description`, `Key`, `Version`, `Record Status`, `In Repository Since`, `Last Changed`, `Changed By`, `Change Ref`, `Approved By`. Nobody should be writing a formula out by hand.

One form has no sheet behind it at all. Form A-1, the stakeholder register, feeds nothing, because the repository has sixty-six object sheets and not one of them is a stakeholder sheet. The form says so on its face, and keys each line into `O10 Actor` or `O11 Role` so that at least the people are in the model even though their concerns are not. That honesty is better than inventing a sheet.

The 12 question sets

Between the forms are 12 `ask` sets. Each sits before the form it prepares and holds three to five questions with ruled space for the answers. They are not warm-ups. They are the questions that decide whether the form can be filled honestly — whether you know who can rule an entity out of scope, which capability the business would pay to be better at than its competitors, which record the clerks actually trust. Answer them on paper before the session. If you cannot answer them, the session will produce rows that look like data and are not.

The index of forms

Form	Title	Feeds	Phase
P-1	Scope, legal entities and sites	`009 Organization Unit`, `012 Location`	Preliminary
P-2	Principles — adopt, edit, retire	`005 Principle`	Preliminary
P-3	Governance bodies and gates	`057 Governance Body`, `058 Governance Gate`	Preliminary
P-4	Drivers, goals and objectives	`001 Driver`, `002 Goal`, `003 Objective`	Preliminary
P-5	Measures and the deliverable plan	`004 Measure`, `031 Deliverable`	Preliminary
A-1	Stakeholder register	no sheet — keys into `010 Actor` and `011 Role`	A
A-2	Value streams	`014 Value Stream`	A
A-3	Constraints and assumptions	`007 Constraint`, `008 Assumption`	A
B-1	Actors and roles	`010 Actor`, `011 Role`	B
B-2	Capability identify	`013 Business Capability` — structure columns	B
B-3	Capability assess	`013 Business Capability` — judgement columns	B
B-4	Functions, processes and participation	`017 Business Function`, `016 Business Process`, `R09 ROL-BPR`	B
B-5	Business services, products and contracts	`015 Business Service`, `018 Product`, `019 Contract`	B
C-1	Data entity identify	`020 Data Entity` — identity columns	C data
C-2	Data entity assess	`020 Data Entity` — assessment columns	C data
C-3	Entity relationships, data sets and master data rules	`036 Data Entity Relationship`, `038 Data Set`, `043 Master Data Rule`	C data
C-4	Process-to-data CRUD	`R17 BPR-DEN`	C data
CA-1	Application identify	`023 Application Component` — identity columns	C application
CA-2	Application assess	`023 Application Component` — scoring columns	C application
CA-3	Application to capability and to data	`R37 APP-CAP`, `R21 APP-DEN`	C application
CA-4	Interface register	`R23 APP-APP`	C application
D-1	Technology services and components	`025 Technology Service`, `026 Logical Technology Component`, `027 Physical Technology Component`	D
D-2	Standards and conformance	`032 Standard`, `R31 STD-PTC`	D
D-3	Resilience tiers and environments	`049 Resilience Tier`, `048 Environment`	D
E-1	Gap register	`029 Gap`	E
E-2	Building blocks, acceptance criteria and sourcing	`035`, `044`, `045`, `046`, `051 Solution Building Block`	E
E-3	Work packages and transition architectures	`030 Work Package`, `053 Transition Architecture`	E
F-1	Projects	`056 Project`	F
F-2	Implementation factors	`052 Implementation Factor`	F
F-3	Delivery risks and data migration	`062 Delivery Risk`, `039 Data Migration`	F
G-1	Architecture contract and dispensation	`033 Architecture Contract`, `061 Governance Change Request`	G
G-2	Compliance assessment	`059 Compliance Assessment`	G
H-1	Change request	`03 Change Requests`	H
H-2	Change log line and baseline record	`02 Change Log`, `04 Baselines`	H
R-1	Requirement capture with acceptance criteria	`006 Requirement`	Requirements
R-2	Control objective to requirement	`063 Control Objective`, `R60 063-REQ`	Requirements

Three things you might expect on that list are not there. The Level 4 rows of `013` have no form, because all 364 are deliberately unscored and a form would invite somebody to score them. `037 Data Attribute` has no form: 165 rows of attribute, type and key, which is a data-modelling tool's work and not a workshop's. And none of the thirty-two `RPT` sheets has a form, because every cell in them is a formula — a form that fed a report would be a form that broke it.

Preliminary Phase

Five forms, filled before you have the authority to fill anything else. These are about scope, power and dates rather than about architecture, which is why a sponsor reads them more carefully than anything in Phase B.

BEFORE YOU DRAW THE SCOPE BOUNDARY

Which legal entities does this architecture bind, and which does it merely describe? State the difference in one sentence.

Who, by name, can rule an entity out of scope — and would they say so in front of the group executive?

Name one organisation unit nobody will claim. What happens to its capabilities on Form B-2?

Which of the sites would stop the business if it stopped, and what is the evidence rather than the assumption?

Form P-1 — Scope, legal entities and sites

Feeds `O09 Organization Unit` — `ID`, `Name`, `Type`, `Parent`, `Head`, `Headcount` — and `O12 Location` — `ID`, `Name`, `Type`, `City`, `Country`, `Criticality`. **Typed into Excel** `O09 Description` and `Location`. **First pass** sixteen units and ten sites at Meridian; continue on a second copy. **Signed by** the Group Chief Executive, or `GB-04 Executive Committee` sitting as a body. Nobody below that may rule a legal entity out of scope.

ID	Name	Type	Parent	Head	Headcount
ORG-01	*example — delete*	Legal entity	—	Group CE	2,140

`O12 Location`. `Criticality` is about what stops if the site stops, not about how many people sit in it.

ID	Name	Type	City	Country	Criticality
LOC-01	*example — delete*	Head office	Johannesburg	South Africa	Critical

BEFORE YOU TOUCH THE PRINCIPLE SET

Which principle has been quoted in a real decision in the last year? Give the decision.

Which one has never been quoted? Say why you would keep it rather than retire it.

Name two principles that will conflict on a live question, and say which `Tier` resolves it.

If everything is Tier 1, what is your tie-break? Write the sentence you will say to the Board.

Form P-2 — Principles — adopt, edit, retire

Feeds `O05 Principle` — `ID`, `Name`, `Category`, `Tier`, `Status`, `Owner` — and `Statement (condensed)` on the second table, for any principle the room re-words. **The decision lives in `Status`**: Adopted, Draft or Retired. There is no separate decision column and you should not add one. **First pass** thirty-one principles, four copies. **Signed by** `GB-01 Architecture Board`. A principle nobody has signed is a preference.

ID	Name	Category	Tier	Status	Owner
BP-01	*example — delete*	Business	1	Adopted	COO

Re-worded statements. Write the condensed statement in full, not an instruction to the typist. If it needs two lines it is two principles.

ID	Statement (condensed)
BP-01	*example — delete*

BEFORE YOU WRITE THE GOVERNANCE BODIES DOWN

Which of these bodies has met in the last quarter, with a minute and a quorum?

For each that has not met: what decision is waiting on it, and who is taking that decision instead?

Name a decision the architecture function has taken that `May Not Decide` says it should not have.

At which gate would a project that ignored the architecture actually be stopped, and has that ever happened?

Form P-3 — Governance bodies and gates

Feeds `O57 Governance Body` — `ID`, `Body`, `Cadence`, `May Decide`, `May Not Decide`, `Standing` — and `O58 Governance Gate` — `ID`, `Name`, `When`, `What It Tests`, `Why It Exists`, `Held By`. **Typed into Excel** `O57 Membership`, a list of names that will not fit a printed cell. **First pass** five bodies and four gates, the whole of both sheets. **Signed by** `GB-04 Executive Committee`. A governance body cannot constitute itself, and `Standing` is where you record that it has not met.

ID	Body	Cadence	May Decide	May Not Decide	Standing
GB-01	*example — delete*	Monthly	Design conformance	Funding	Constituted

`O58 Governance Gate`. Meridian runs four: `G0` charter, `G1` design, `G2` build, `G3` production. Add a fifth only if you can say what it tests that no other gate tests.

ID	Name	When	What It Tests	Why It Exists	Held By
G0	*example — delete*	Before funding	Charter fit	Stops unmandated work	GB-02

Form P-4 — Drivers, goals and objectives

Feeds three sheets. `001 Driver` — `ID`, `Name`, `Category`, `Origin`, `Horizon`, `Owner`. `002 Goal` — `ID`, `Name`, `Pillar`, `Owner`, `Target Date`, `Status`. `003 Objective` — `ID`, `Name`, `Goal`, `Baseline`, `Target`, `Target Date`. **Typed into Excel** the `Description` on all three. **First pass** fifteen drivers, six goals, fifteen objectives — three copies. **Signed by** `GB-04 Executive Committee` for goals, the named executive for each driver and objective. `Origin` names the instrument, not the regulator.

ID	Name	Category	Origin	Horizon	Owner
DRV-01	*example — delete*	Regulatory	COFI Bill	2027	CRO

`002 Goal`. Drivers outnumber goals, always. A driver with no goal behind it is the finding.

ID	Name	Pillar	Owner	Target Date	Status
GOL-01	*example — delete*	Growth	Group CE	2028-12	Approved

`003 Objective`. `Baseline` and `Target` take the same unit, or the objective cannot be measured.

ID	Name	Goal	Baseline	Target	Target Date
OBJ-01	*example — delete*	GOL-01	18 days	5 days	2027-06

Form P-5 — Measures and the deliverable plan

Feeds `004 Measure` — `ID`, `Name`, `Objective`, `Baseline`, `Target`, `Source System` — and `031 Deliverable` — `ID`, `Name`, `Type`, `ADM Phase`, `Owner`, `Due`. **Typed into Excel** `004 Unit`, `Current`, `Frequency` and `Owner`; `031 Work Package`, empty until Phase E, `Status` and `Repository Location`. **Signed by** the measure's owner, who is promising that a number can be produced monthly without a spreadsheet; `GB-01 Architecture Board` signs the deliverable plan as the thing it will hold you to.

ID	Name	Objective	Baseline	Target	Source System
MSR-01	*example — delete*	OBJ-01	18	5	APP-04

`031 Deliverable`. Twenty rows: every deliverable the first cycle will produce, named and dated before you start. The one Preliminary table a sponsor reads line by line, because it is the only one with dates on it.

ID	Name	Type	ADM Phase	Owner	Due
DLV-01	*example — delete*	Catalogue	Phase B	Domain Architect	2027-03-31

Phase A — Architecture Vision

Three forms. The repository barely holds Phase A: the stakeholder register has no sheet at all, which Form A-1 says on its face.

Form A-1 — Stakeholder register

Feeds nothing directly. The repository has no stakeholder sheet. Sixty-six object sheets and not one of them is `Stakeholder`. The register lives in the Statement of Architecture Work as a document, and each line keys into the repository through an `ACT-` identifier in `O10 Actor` or a `ROL-` identifier in `O11 Role`. **Transcribe** the `Key` column into whichever of the two sheets the row belongs to; the concern and the plan stay in the document. **First pass** twelve to twenty names. **Signed by** the sponsor. If a name on this form has not agreed to be on it, say so before the sponsor signs.

Name and post	Key	Concern, one line	Can stop	Class	Plan
J. Pillay, CRO	ROL-04	*example — delete*	G1	Keep satisfied	Monthly 1:1

Form A-2 — Value streams

Feeds `O14 Value Stream` — `ID`, `Name`, `Triggering Stakeholder`, `Value Proposition`, `Stages`, `Owner Org`. **Typed into Excel** `Description`. The stage names go into `R11 VST-CAP` column `Stage` when you map capabilities to the stream, so write them in order on the back of this sheet and number them. **First pass** seven streams. More than ten and you are writing processes, not streams. **Signed by** the Chief Operating Officer, with each stream's `Owner Org` head agreeing their own stream in the room.

ID	Name	Triggering Stakeholder	Value Proposition	Stages	Owner Org
VST-01	*example — delete*	Broker	Cover in force same day	6	ORG-04

Form A-3 — Constraints and assumptions

Feeds `007 Constraint` — `ID`, `Name`, `Type`, `Applies Until`, `Owner` — and `008 Assumption` — `ID`, `Name`, `Confidence`, `Validate By`, `Owner`, `Status`. **Typed into Excel** the `Description` on both. **Signed by** the Chief Enterprise Architect, with the sponsor accepting the assumption list, because an assumption is a risk the sponsor is carrying without having said so. `Applies Until` is compulsory: a constraint with no expiry becomes a principle by neglect.

ID	Name	Type	Applies Until	Owner
CON-01	*example — delete*	Contractual	2029-03	CIO

`008 Assumption`. `Validate By` is a date and a method, not a hope. A Low-confidence assumption under a funded work package belongs in `R35 ASM-WPK` the same week.

ID	Name	Confidence	Validate By	Owner	Status
ASM-01	*example — delete*	Medium	2027-02-28	CFO	Open

Phase B — Business Architecture

Five forms, and the two capability forms are the ones you will print most. `013 Business Capability` has 556 rows and twenty columns, so it splits: B-2 is structure and takes an hour at a desk, B-3 is judgement and takes three hours in a room with an executive in it.

Form B-1 — Actors and roles

Feeds `O10 Actor` — `ID`, `Name`, `Type`, `Internal/External`, `Organization Unit`, `Population` — and `O11 Role` — `ID`, `Name`, `Domain`, `Authority Level`, `Organization Unit`. **Typed into Excel** the `Description` on both. The units these point at come from Form P-1 and must already exist: an actor pointing at a unit that is not in `O09` is the commonest broken reference in the repository. **Signed by** the Group HR Director for roles, the Chief Operating Officer for actors. `Authority Level` is what the role may decide alone, and it is the column a room will skip.

ID	Name	Type	Internal/External	Organization Unit	Population
ACT-01	*example — delete*	Individual	Internal	ORG-04	310

`O11 Role`. One row per role, not per person. `Domain` is Business, Data, Application, Technology or Governance.

ID	Name	Domain	Authority Level	Organization Unit
ROL-01	*example — delete*	Business	Approves up to R2m	ORG-05

Form B-2 — Capability identify

Feeds `O13 Business Capability`, structure columns only — `ID`, `Name`, `Level`, `Parent`, `Owner Org`. **Typed into Excel** `Description`, and never the `L1` and `L2` columns, which are formulas that read `Parent`. The six judgement columns are Form B-3 and a different room. **First pass** the Level 3 rows of one Level 1 area, eleven to thirteen rows; `O13` holds 556 rows on four levels, so this form runs twelve times. **Signed by** the `Owner Org` head for their own area. Volume 1, chapter 19, appendix A, *The cross-maps*, already drafts twenty headline capabilities — start there, then argue with it, matching on name rather than number.

ID	Name	Level	Parent	Owner Org
C04.02.01	*example — delete*	3	C04.02	ORG-06

BEFORE YOU SCORE A SINGLE CAPABILITY

Which of these capabilities would the business pay to be better at than its competitors, and who says so? Name the executive, not the strategy document.

Which capability is critical and weak at the same time? Say it now, because the room will not admit both after lunch.

What would you see if you watched this capability being done for a week? That observation is what justifies the `Current Maturity` score.

How many of this area's Level 3 rows are you prepared to call Critical? Write the number before the session and hold the room to it.

Which rows are Commodity in `Differentiation`? Those are procurement decisions — who owns them instead?

Form B-3 — Capability assess

Feeds `013 Business Capability`, judgement columns only — `ID`, `Criticality`, `Current Maturity`, `Target Maturity`, `Disposition`, `Differentiation`. Nothing else; `Name` is printed so the room knows which row it is scoring and is not retyped. **Leave Level 4 rows blank** — all 364 are blank by design, and filling them breaks the roll-up in `RPT03 Capability Heatmap`. **First pass** one Level 1 area, three hours. **Signed by** the `Owner Org` head, in the room, at the end of the session, because the scores drift by Friday. Capture form for workshop pack 1 in appendix C.

ID	Criticality	Current Maturity	Target Maturity	Disposition	Differentiation
C04.02.01	Critical	2	4	Invest	Differentiating

BEFORE YOU FILL IN PARTICIPATION

For each process, name the one person who would be called at midnight. Is that the same person as the `Owner Role`?

Where two roles both believe they are Accountable, which one signs the customer outcome? Record both and say what you did.

Name a process with an Accountable role that has no actor behind it in `010 Actor`. What does that make the accountability?

Form B-4 — Functions, processes and participation

Feeds `O17 Business Function` — `ID`, `Name`, `Organization Unit`, `Type`. `O16 Business Process` — `ID`, `Name`, `Value Stream`, `Owner Role`, `Automation Level`, `Volume p.a.`. `R09 ROL-BPR` — `Source (ROL)`, `Source Name`, `Target (BPR)`, `Target Name`, `Participation`, `Note`. **Typed into Excel** `Description`, `O16 Cycle Time` from the operations report, and `R09 Key`, a formula. **`Participation` is why the third table exists**: Accountable, Responsible, Consulted or Informed, with exactly one Accountable per process. **Signed by** the Chief Operating Officer; volumes countersigned by whoever owns the report they came from.

ID	Name	Organization Unit	Type
FUN-01	*example — delete*	ORG-06	Core

`O16 Business Process`. `Automation Level` is Manual, Partial or Automated, and is the column Phase C will come back and argue with.

ID	Name	Value Stream	Owner Role	Automation Level	Volume p.a.
BPR-01	*example — delete*	VST-03	ROL-07	Partial	41,000

`R09 ROL-BPR`. Two Accountable rows against one `BPR-` is the finding, not a data-entry error.

Source (ROL)	Source Name	Target (BPR)	Target Name	Participation	Note
ROL-07	*example — delete*	BPR-01	Claim intake	Accountable	—

Form B-5 — Business services, products and contracts

Feeds `O15 Business Service` — `ID`, `Name`, `Capability`, `Consumer`, `Channel Agnostic`, `Criticality`. `O18 Product` — `ID`, `Name`, `Line of Business`, `Policies In Force`, `GWP (Rm)`, `Config or Code`. `O19 Contract` — `ID`, `Name`, `Type`, `Counterparty`, `Expiry`, `Notice Period`. **Typed into Excel** `O15` `Description` and `Owner Org`; `O18` `Status`; `O19` `Start`, `Value (Rm p.a.)` and `Owner`. **First pass** twenty services, eighteen products, ten contracts — four copies. **Signed by** the `Line of Business` managing director for products, the Chief Financial Officer for premium and contract value, procurement for `Notice Period`.

ID	Name	Capability	Consumer	Channel Agnostic	Criticality
BSV-01	*example — delete*	C04.02	Policyholder	Yes	Critical

`O18 Product`. `Config or Code` says whether a new variant needs a release. The most useful column on the sheet and the one most often guessed.

ID	Name	Line of Business	Policies In Force	GWP (Rm)	Config or Code
PRD-01	*example — delete*	Short-term personal	142,000	890	Config

`O19 Contract`. `Notice Period` is what makes an exit real. Copy it from the signed agreement, not from the relationship manager.

ID	Name	Type	Counterparty	Expiry	Notice Period
CTR-01	*example — delete*	Outsourcing	Named vendor	2029-06-30	12 months

Phase C — Data Architecture

Four forms. `O20 Data Entity` splits the way the capability sheet does, except that its assessment half splits again, because tiering and retention are answered by different people and signed by different ones.

BEFORE YOU DESIGNATE AN AUTHORITATIVE SOURCE

Which record do the people who do the work actually trust? Ask the clerk, not the owner, and write what they said.

If that answer is a spreadsheet, is the spreadsheet the authoritative source? Defend it in one sentence.

Which entity has five claimants and no master? Name it, and name who will be told the answer is `CONTESTED`.

For each master you are about to designate: what breaks tomorrow if that system is wrong, and who finds out first?

Who is the human being — a name, not a role — behind `Owner Role` for your most contested entity?

Form C-1 — Data entity identify

Feeds `O20 Data Entity`, identity columns only — `ID`, `Name`, `Subject Area`, `Authoritative Source`, `Owner Role`. **Typed into Excel** `Description`, the one agreed sentence, which you write out in the session minute and transcribe from there rather than cramming into a printed cell. The seven assessment columns are Form C-2. **`Authoritative Source` takes one of four values**: a named `APP-nn`, `SPLIT` with the split stated in the minute, `CONTESTED`, or `NONE`. Put `CONTESTED` on the whiteboard as a legitimate answer in the first ten minutes. **Signed by** `GB-03 Data Governance Council`, the only body with the stated power to designate an authoritative source.

ID	Name	Subject Area	Authoritative Source	Owner Role
DEN-01	*example — delete*	Party	APP-03	ROL-09

Form C-2 — Data entity assess

Feeds `O20 Data Entity`, assessment columns only. Seven columns do not fit 170mm, so this form runs two passes over the same rows. **Typed into Excel** `Master Count` and `Mastering Style`, which come from `O43 Master Data Rule` and not from this room. **Signed by** `GB-03 Data Governance Council` for `Tier` and `Quality`; the information officer signs `Classification`, `Retention Period` and `Retention Basis`, which are statutory and not an architecture opinion. Capture form for workshop pack 2 in appendix C.

Pass one — what it is worth, and whether it can be trusted.

ID	Classification	Tier	Quality
DEN-01	Confidential	1	Fair

Pass two — how long it may be kept, under what authority. `Retention Basis` takes a section number, not a law. Make the information officer read it aloud.

ID	Retention Period	Retention Basis	Data Category
DEN-01	5 years	POPIA s14	Personal information

Form C-3 — Entity relationships, data sets and master data rules

Feeds `036 Data Entity Relationship` — `ID`, `From Entity`, `To Entity`, `Relationship`, `Cardinality`. `038 Data Set` — `ID`, `Name`, `Entities`, `Refresh`, `Consumer`, `Lawful Basis`. `043 Master Data Rule` — `ID`, `Entity`, `Mastering Style`, `Golden Record Definition`, `Survivorship Rule`, `Mastering Status`. **Typed into Excel** `038 Purpose`, `Classification` and `State`; `043 Category` and `Implemented`. **Signed by** the Chief Enterprise Architect for the relationship model, the information officer for every `Lawful Basis`, `GB-03 Data Governance Council` for every master rule.

ID	From Entity	To Entity	Relationship	Cardinality
DER-01	DEN-01	DEN-06	*example — delete*	1:n

`038 Data Set`. A data set with no lawful basis is a finding you report, not a blank you leave.

ID	Name	Entities	Refresh	Consumer	Lawful Basis
DS-01	*example — delete*	DEN-01, DEN-06	Daily	Pricing	Contract

`043 Master Data Rule`. One rule per entity that has a master — twelve of twenty-four at Meridian. `Survivorship Rule` must be executable: "latest verified contact detail wins" is a rule, "the best data wins" is not.

ID	Entity	Mastering Style	Golden Record Definition	Survivorship Rule	Mastering Status
MDR-01	DEN-01	Registry	*example — delete*	Latest verified wins	Designed

Form C-4 — Process-to-data CRUD

Feeds `R17 BPR-DEN` — `Source (BPR)`, `Source Name`, `Target (DEN)`, `Target Name`, `CRUD`, `Note`. **Typed into Excel** `Key`, a formula. **`CRUD` is any combination of C, R, U and D with no spaces**, written the same way every time, because `RPT04 CRUD Matrix` reads the string. One row per process-and-entity pair the process touches; absence is meaningful, so do not write rows with an empty `CRUD`. **First pass** fifty rows. **Signed by** the process `Owner Role` from `O16`. **Watch for** an entity nothing creates and an entity everything updates: both are gaps, and both come straight off this form.

Source (BPR)	Source Name	Target (DEN)	Target Name	CRUD	Note
BPR-01	*example — delete*	DEN-10	Claim	CRU	—

Phase C — Application Architecture

Four forms. The two relationship forms are where most of the Phase C work actually is: `R37 APP-CAP` alone runs to 181 rows at Meridian.

Form CA-1 — Application identify

Feeds `O23 Application Component`, identity columns only — `ID`, `Name`, `Type`, `Owner Org`, `Lifecycle`, `Vendor`. **Typed into Excel** `Description`, `Owner Role`, `Year Live`, `Users` and `Hosting`, all of which come from the CMDB or the contract rather than from a workshop. The scoring columns are Form CA-2. **`Lifecycle` is Emerging, Current, Contain or Sunset**, and it states the product's support position, not what you intend to do about it. **First pass** twenty-six applications, three copies. **Signed by** the Chief Information Officer. Anything holding a `DEN-` entity is in scope even if IT does not run it.

ID	Name	Type	Owner Org	Lifecycle	Vendor
APP-01	*example — delete*	Package	ORG-08	Current	Named vendor

BEFORE YOU SCORE THE PORTFOLIO

Which application has high business value and low technical fitness, and who in the business does not yet know that?

For the three most expensive: what would the business lose in the first week without each? Be specific about the week.

Which application does the business rate highly because it is familiar rather than because it is good? How will you tell the difference in the room?

Who is in the room who can be wrong in public about run cost?

Form CA-2 — Application assess

Feeds `O23 Application Component`, scoring columns only — `ID`, `Business Value`, `Technical Fitness`, `Disposition`, `Annual Run Cost (Rm)`. **`Portfolio Quadrant` is printed here but is a formula.** Do not type in it: write the quadrant the room expects, then check it against what the sheet computes after transcription. Where they differ, one of the two scores is wrong and you have found something. `Disposition Agrees` is computed the same way. **First pass** twenty-six applications in four hours, nine minutes each, which is enough. **Signed by** three people: the business executive for value, the Chief Information Officer for fitness, the Chief Financial Officer for run cost. Capture form for workshop pack 3 in appendix C.

ID	Business Value	Technical Fitness	Disposition	Annual Run Cost (Rm)	Portfolio Quadrant
APP-01	4	2	Replace	18.4	Reinvest

Form CA-3 — Application to capability and to data

Feeds `R37 APP-CAP` — `Source (APP)`, `Source Name`, `Target (CAP)`, `Target Name`, `Support` — and `R21 APP-DEN` — `Source (APP)`, `Target (DEN)`, `Target Name`, `Role`, `Authoritative`. **Typed into Excel** `Key` and `Note` on both. `Support` is Full, Partial or Peripheral. `Role` is Master, Replica, Consumer or Producer, and `Authoritative` is Yes or No with **at most one Yes per entity across the whole sheet**. **First pass** `R37` runs to 181 rows at Meridian and `R21` to 49; expect six copies. **Signed by** the application owner for their own rows; `GB-03 Data Governance Council` for every `Authoritative` = Yes, which is a designation and not an observation.

Source (APP)	Source Name	Target (CAP)	Target Name	Support
APP-01	*example — delete*	C04.02.01	Claim intake	Partial

`R21 APP-DEN`. Fill `Role` for every application that holds the entity, then come back and put a single Yes in `Authoritative`.

Source (APP)	Target (DEN)	Target Name	Role	Authoritative
APP-01	DEN-10	Claim	Master	Yes

Form CA-4 — Interface register

Feeds `R23 APP-APP`, which carries nineteen columns and so runs as two passes over the same `Interface ID`. **Typed into Excel** `Source Name`, `Target Name` and `Key`, all formulas or lookups. **`Managed` is the column that earns the form**: Yes means somebody owns the contract, monitors it, and would know if it stopped. `RPT06 Interface Register` is where the Nos become visible. **First pass** fifty rows, five copies. **Signed by** the integration lead, with both application owners named on the row agreeing `Managed`.

Source (APP)	Target (APP)	Interface ID	Style	Direction	Frequency
APP-01	APP-03	IF-001	File	One-way	Nightly

Second pass over the same identifiers. A blank `Managed` is not an answer. Write No and let `RPT06` count it.

Interface ID	Managed	Data Entity	Note
IF-001	No	DEN-01	*example — delete*

Phase D — Technology Architecture

Three forms. Phase D is the phase where a form is often unnecessary, because most of the data sits in a CMDB and should be exported rather than written out. Print these for the columns a CMDB does not hold: `Standard Status`, `Availability Target`, `Last Tested`.

Form D-1 — Technology services and components

Feeds `025 Technology Service` — `ID`, `Name`, `Category`, `Availability Target`, `Consumers`, `Owner Org`. `026 Logical Technology Component` — `ID`, `Name`, `Category`, `Provides Service`, `Standard Status`. `027 Physical Technology Component` — `ID`, `Realises`, `Vendor / Product`, `Lifecycle`, `Support Ends`, `Annual Cost (Rm)`. **A trap on `027`**: it carries two columns headed `Version` — the deployed product version, and the record version over with the audit columns. Transcribe into the first. **Signed by** the Chief Information Officer; procurement countersigns `Support Ends` from the contract.

ID	Name	Category	Availability Target	Consumers	Owner Org
TSV-01	*example — delete*	Integration	99.5%	APP-01, APP-03	ORG-08

`026`. `Standard Status` is Standard, Permitted, Contained or Prohibited, and governs what Form D-2 will test.

ID	Name	Category	Provides Service	Standard Status
LTC-01	*example — delete*	Middleware	TSV-01	Standard

`027`. `Support Ends` is the column the whole of Phase D turns on. A date in the past is a gap, and it goes on Form E-1 the same day.

ID	Realises	Vendor / Product	Lifecycle	Support Ends	Annual Cost (Rm)
PTC-01	LTC-01	*example — delete*	Contain	2026-10-14	6.2

BEFORE YOU SET A STANDARD

For each standard you are about to write: name one product in the estate that fails it today. If none fails, the standard describes what you already have.

What happens to a project that ignores this standard — at which gate, decided by whom, with what consequence?

Which standard will you grant a dispensation against in the next six months? Write the expiry date now.

Who reviews these at `Review Due`, and what happens if nobody does?

Form D-2 — Standards and conformance

Feeds `O32 Standard` — `ID`, `Name`, `Domain`, `Status`, `Applies To`, `Review Due` — and `R31 STD-PTC` — `Source (STD)`, `Target (PTC)`, `Target Name`, `Conformance`, `Note`. **Typed into Excel** `O32` `Description` and `Owner`; `R31` `Source Name` and `Key`. `Conformance` is Conformant, Partial, Non-conformant or Dispensation, and a Dispensation row must name the `CR-G` identifier from Form G-1 in `Note`. **Signed by** `GB-01 Architecture Board`, the only body that may set a standard. **Shows up in** `RPT10 Standards Conformance`: if every row reads Conformant you have written the standards to match the estate.

ID	Name	Domain	Status	Applies To	Review Due
STD-01	*example — delete*	Integration	Active	All new interfaces	2027-09-30

`R31 STD-PTC`. One row per standard-and-product pair you have actually tested. An untested pair is a blank row, not a Conformant one.

Source (STD)	Target (PTC)	Target Name	Conformance	Note
STD-01	PTC-01	*example — delete*	Partial	Legacy adapter

Form D-3 — Resilience tiers and environments

Feeds `049 Resilience Tier` — `ID`, `Capability`, `Tier`, `RTO`, `RPO`, `Test Cadence` — and `048 Environment` — `ID`, `Name`, `Purpose`, `Where It Runs`, `Data It Holds`. **Typed into Excel** `049` `Last Tested` and `Applications`; `048` `Evidence In The Repository` and `Provenance`. **One `049` row per Level 1 capability**, which is twelve rows, not one per application. **`Last Tested` is where this sheet tells the truth**: an `RTO` of four hours with a `Last Tested` of never is a gap, and `RPT26 Resilience Test Status` prints it. **Signed by** the Chief Risk Officer for tiers, the Chief Information Officer for environments.

ID	Capability	Tier	RTO	RPO	Test Cadence
RES-01	C04	Tier 1	4 hours	15 min	Twice yearly

`048 Environment`. Four rows at Meridian. `Data It Holds` is the POPIA question: a non-production environment holding live personal information is a finding for Form E-1. Write what it holds today, not what policy says.

ID	Name	Purpose	Where It Runs	Data It Holds
ENV-01	*example — delete*	Pre-production	LOC-03	Masked copy

Phase E — Opportunities and Solutions

Three forms. Phase E is where paper wins hardest, because the work is classification and argument rather than data entry, and a laptop in the room turns an argument into a typing queue.

BEFORE YOU CLASSIFY A DIFFERENCE AS A GAP

Is this absent, inadequate, duplicated or non-conformant? Say which, and say what evidence would change your mind.

Which domain owns it? If two domains claim it, the gap is probably two gaps — split it here.

What is the consequence of closing nothing? Write the sentence a sponsor would repeat to the executive committee.

Which three of these would you take to the Board if you could only take three, and why these three?

Form E-1 — Gap register

Feeds `O29 Gap` — `ID`, `Name`, `Domain`, `Gap Type`, `Severity`, `Affected Object`. **Typed into Excel** `Description`, `Traces To` and `Addressed By`, the last of which stays blank until work packages exist on Form E-3. **`Gap Type` is Absent, Inadequate, Duplicated or Non-conformant** — and choosing between Absent and Inadequate is the judgement the session exists to make. **`Affected Object` names a single identifier**, not a sentence. **First pass** eighteen gaps, two hours, forty to sixty differences reviewed. **Signed by** the Chief Enterprise Architect, with one Architecture Board member able to take an Accepted decision in the room. Capture form for workshop pack 4 in appendix C; `Meridian_Architecture_Comparison_Tool.xlsx` sheet `70 Gap Candidates` is its input [2].

ID	Name	Domain	Gap Type	Severity	Affected Object
GAP-01	*example — delete*	Data	Absent	High	DEN-01

Form E-2 — Building blocks, acceptance criteria and sourcing

Feeds one of four building-block sheets, which share a shape. Ring the one this copy is for: `035 Business Building Block`, `044 Data Building Block`, `045 Application Building Block` or `046 Technology Building Block`. Columns in all four: `ID`, `Name`, `What It Must Do`, `Acceptance Criteria`, `Closes Gap`, `Work Package`. Then `051 Solution Building Block` — `ID`, `Name`, `What It Is`, `Sourcing`, `Realises`, `Work Package`. **`Acceptance Criteria` is the column that makes governance possible**: write it so Form G-2 can return a one-word verdict. 'Fast enough' cannot be assessed; 'quote returned in under four seconds at the ninety-fifth percentile' can. One criterion per row. **Signed by** `GB-01 Architecture Board`.

ID	Name	What It Must Do	Acceptance Criteria	Closes Gap	Work Package
DBB-01	*example — delete*	Master party once	One Yes in R21 per entity	GAP-01	WP-03

`051 Solution Building Block`. `Sourcing` is Buy, Build, Configure, Reuse or Retire. A solution block that realises nothing is a purchase looking for a justification, and `RPT28 Solution Sourcing` will show which of these nobody has funded.

ID	Name	What It Is	Sourcing	Realises	Work Package
SBB-01	*example — delete*	Party registry	Buy	DBB-01	WP-03

BEFORE YOU SEQUENCE ANYTHING

What is the earliest a work package could start given who is available, rather than given the plan? Name the constraint and the person.

Which package must finish before another can start for a reason that is architectural rather than administrative?

What is still broken when the first transition state is reached, and who will be disappointed by it? Tell them before the state is declared.

Who in the room can change a date? If nobody can, the session produces a finding and not a plan — say so at the start.

Form E-3 — Work packages and transition architectures

Feeds `O30 Work Package` — `ID`, `Name`, `Wave`, `Start`, `End`, `Cost (Rm)` — and `O53 Transition Architecture` — `ID`, `Name`, `Window`, `Exit Criteria`, `Still Broken Here`. **Typed into Excel** `O30 Description`, `Benefit (Rm p.a.)`, `Owner`, `Status`, `Transition State` and `Delivery Status`; `O53 Transition`, `Work Packages`, `Cost (Rm)` and `Standing`. **`Still Broken Here` is the honest column** and the reason `O53` is worth filling: a transition state that fixes everything is a target architecture with a different name. **Signed by** `GB-02 Investment Committee` for cost and wave, `GB-01 Architecture Board` for the exit criteria. Capture form for workshop pack 5 in appendix C.

ID	Name	Wave	Start	End	Cost (Rm)
WP-03	*example — delete*	1	2027-04	2028-03	42.0

`O53 Transition Architecture`. Three rows is right. More than four and nobody will believe any of them.

ID	Name	Window	Exit Criteria	Still Broken Here
TA-1	*example — delete*	2027-H2	One party master live	Claims still on paper

Phase F — Migration Planning

Three forms. Everything in Phase F has a date on it, which means every Phase F session needs somebody in the room who can change a date.

Form F-1 — Projects

Feeds `056 Project`, twenty-one columns, so two passes over the same identifiers. Pass one: `ID`, `Name`, `Type`, `Work Packages`, `Start`, `End`. Pass two: `ID`, `Cost (Rm)`, `Benefit (Rm p.a.)`, `Lead`, `Depends On`. **Typed into Excel** `Months`, which is arithmetic, `Team`, `Override` and `Why This Shape`. **Write** `Why This Shape` **out in full in the session minute** and transcribe it: it is the one field that stops the same argument being held twice. **Signed by** `GB-02 Investment Committee`, which charters a project, with the programme director accepting the dates. A project with no `Lead` named is not chartered, whatever the committee minute says.

ID	Name	Type	Work Packages	Start	End
PRJ-01	*example — delete*	Platform	WP-03, WP-04	2027-04	2028-06

Second pass, same identifiers, with the Investment Committee delegate in the room for the money columns.

ID	Cost (Rm)	Benefit (Rm p.a.)	Lead	Depends On
PRJ-01	58.0	21.0	Named individual	PRJ-02

Form F-2 — Implementation factors

Feeds `052 Implementation Factor` — `ID`, `Category`, `Factor`, `Source`, `Deduction`, `Changes Sequence`. **Nothing is typed into Excel**: the sheet is six columns wide and the form covers all six. **`Deduction` is what makes the row useful.** A factor without a deduction is a complaint. 'Actuarial capacity is committed to IFRS 17 until March 2028' is a factor; 'therefore no pricing work package may start before April 2028' is the deduction, and `Changes Sequence` is Yes. **First pass** twenty-six rows, collected across the sequencing session rather than in one sitting. **Signed by** the Chief Enterprise Architect; `Source` must be a named person or a named document, never 'the business'.

ID	Category	Factor	Source	Deduction	Changes Sequence
IF-01	Resource	*example — delete*	Chief Actuary	No pricing work before 2028-04	Yes

Form F-3 — Delivery risks and data migration

Feeds `O62 Delivery Risk` — `ID`, `Risk`, `Project`, `Likelihood`, `Impact`, `Owner` — and `O39 Data Migration` — `ID`, `Name`, `Target`, `Entities`, `Quality Gate`, `Gate Status`. **Typed into Excel** `O62 Consequence`, `Mitigation` and `Status`; `O39 Sources`, `Volume`, `Approach`, `Wave`, `Work Package` and `Risk`. **`Quality Gate` must be a test with a number in it**, and `Gate Status` is Passed, Failed or Not run — never blank, because Not run is itself the answer at the moment somebody asks. **Signed by** the Chief Risk Officer for risks, `GB-03 Data Governance Council` for every quality gate: a migration that passes a gate nobody set has not passed anything.

ID	Risk	Project	Likelihood	Impact	Owner
RSK-01	*example — delete*	PRJ-01	Medium	High	Named individual

`O39 Data Migration`. One row per movement, not per source system.

ID	Name	Target	Entities	Quality Gate	Gate Status
MIG-01	*example — delete*	APP-03	DEN-01, DEN-02	99.5% match on ID	Not run

Phase G — Implementation Governance

Two forms. These are the only forms somebody outside the architecture function may ask to see as evidence, so fill them in ink and keep the originals longer.

Form G-1 — Architecture contract and dispensation

Feeds `033 Architecture Contract` — `ID`, `Name`, `Work Package`, `Parties`, `Signed`, `Expiry` — and `061 Governance Change Request` — `ID`, `Title`, `Against`, `Type`, `Why`, `Expires`. **Typed into Excel** `033 Compliance Status`, `Last Review`, `Owner`, `Project` and `Status Basis`: compliance on `033` is **derived** from `059 Compliance Assessment` and must never be asserted by hand. **There is no dispensation sheet.** A dispensation is an `061` row with `Type` = Dispensation and an `Expires` date, and the decision granting it is an `060 Governance Decision` row. An expiry is compulsory: a dispensation with no end date is a silent change to the standard. **Signed by** `GB-01 Architecture Board` and the delivery lead accepting the contract.

ID	Name	Work Package	Parties	Signed	Expiry
ACO-01	*example — delete*	WP-03	Board / PRJ-01	2027-04-18	2028-06-30

`061 Governance Change Request`. `Against` names the standard, principle or building block being departed from. `Why` is one sentence and will be read aloud at the Board.

ID	Title	Against	Type	Why	Expires
CR-G01	*example — delete*	STD-01	Dispensation	Adapter not yet certified	2027-12-31

BEFORE YOU GIVE A VERDICT AT A GATE

For this acceptance criterion: what artefact would a third party open to check it, and does that artefact exist today?

Is Partially compliant an honest verdict here, or a Non-compliant you do not want to defend? Write the harder answer.

If you refuse this gate, what stops? Name the cost, so the Board decides with it in front of them.

Which criterion on Form E-2 turned out to be unassessable? That is a Phase E defect — record it and send it back.

Form G-2 — Compliance assessment

Feeds `O59 Compliance Assessment` — `ID`, `Work Package`, `Building Block`, `Acceptance Criterion`, `Verdict`, `Evidence`. **Typed into Excel** `Gate`, `Recorded In` and `Assessed`. **One row per acceptance criterion per gate.** Not one per work package and not one per project, which is why `O59` has thirteen rows at Meridian against seven contracts. `Verdict` is Compliant, Partially compliant, Non-compliant or Irrelevant. **`Evidence` names an artefact a third party could open** — a test report, a reconciliation, a signed note. 'Confirmed by the team' is not evidence. **Signed by** the gate owner from `O58`: `GB-02 Investment Committee` at `G0`, `GB-01 Architecture Board` at `G1` and `G2`.

ID	Work Package	Building Block	Acceptance Criterion	Verdict	Evidence
CA-01	WP-03	DBB-01	*example — delete*	Partially compliant	Test report 2027-11

Phase H — Architecture Change Management

Two forms, both short, neither for a workshop. You carry these so that a decision taken away from your desk still reaches the repository.

Form H-1 — Change request

Feeds `03 Change Requests`, eleven columns, so two passes over the same `CR` identifier: raised, then decided. **Typed into Excel** `Changes`, which lists the `02 Change Log` line numbers the request produced and can only be filled afterwards. **`Type` and `Scope` decide who sees it**: a `Scope` of one sheet is the architect's own call, a `Scope` touching a baselined object goes to the Board. This form is used one request at a time, which is why it carries fewer rows than the rest. **Signed by** the body named in `Decision Record`.

CR	Title	Raised By	Raised	Type	Scope
CR-062	*example — delete*	Domain Architect	2027-05-04	Addition	O23, R23

Decision half. Fill it when the decision is taken, not when it is expected. `Decision Record` names the minute or the `DEC-G` row that holds it.

CR	Status	Decided	Decision	Decision Record
CR-062	Approved	2027-05-21	Accept with scope limit	DEC-G08

Form H-2 — Change log line and baseline record

Feeds `02 Change Log` — `Change`, `Sheet`, `Record`, `Field`, `Previous Value`, `New Value` — and `04 Baselines` — `Baseline`, `Name`, `Date`, `Workbook Version`, `Approved By`, `Status`. **Typed into Excel** `02 Date`, `Reason`, `Change Request`, `Changed By` and `Approved By`; `04 Note` and `Changes since`. **One line per field changed** — not one per record and not one per sitting. Six columns changed on nine rows is fifty-four lines, and `RPT13 Version Control` shows the omission if you shortcut it. **This form is for changes made away from the keyboard**: a decision taken in a corridor, a correction a signatory dictates. At a desk, type into the sheet.

Change	Sheet	Record	Field	Previous Value	New Value
CHG-412	O20	DEN-01	Tier	2	1

`04 Baselines`. Six rows exist at Meridian. A baseline is a named snapshot somebody approved, not a copy of the file with a date in the name. Signed by `GB-01 Architecture Board`.

Baseline	Name	Date	Workbook Version	Approved By	Status
BL-007	*example — delete*	2027-06-30	v2.4	GB-01	Approved

Requirements Management

Two forms, used in every phase. Requirements arrive between the phases, which is why these live at the back where you can find them without knowing which phase you are in.

BEFORE YOU WRITE A REQUIREMENT DOWN

Which driver or objective does this requirement serve? If you cannot name one, whose preference is it?

Write the test that closes it. If you cannot write a test, what are you actually asking for?

Who owns it — a name — and what have they agreed to be held to?

Which obligation on Form R-2 has no requirement behind it yet?

Form R-1 — Requirement capture with acceptance criteria

Feeds `O06 Requirement`, twenty columns, so two passes over the same `ID`. Pass one: `ID`, `Name`, `Type`, `Priority`, `Source Driver`, `Objective`. Pass two: `ID`, `Owner`, `Acceptance Criteria`. **Typed into Excel** `Description`, `Status`, `Source`, `Building Block` and `Work Package`, the last two blank until Phase E. **A requirement with no `Source Driver` and no `Objective` is somebody's preference**, and `RPT08 Traceability` will print it as a broken chain. **Signed by** the named `Owner` per requirement, with `GB-01 Architecture Board` accepting the set.

ID	Name	Type	Priority	Source Driver	Objective
REQ-01	*example — delete*	Functional	Must	DRV-01	OBJ-01

Acceptance criteria, in full, one per line, in wide rows because this is the field that decides whether the requirement can ever be closed. If you cannot write a test you have written a wish, and the Board should refuse it.

ID	Owner	Acceptance Criteria
REQ-01	Named individual	*example — delete*

Form R-2 — Control objective to requirement

Feeds `O63 Control Objective` — `ID`, `Obligation`, `Regulatory Source`, `Control Objective`, `Control`, `Evidence Today` — and `R60 O63-REQ` — `Source (O63)`, `Target (REQ)`, `Target Name`, `Evidence Today`, `Note`. **Typed into Excel** `R60 Key`, a formula. **`Regulatory Source` takes a section number**: 'POPIA s19', or 'Joint Standard 1 of 2023, para 12' — never 'POPIA'. **`Evidence Today` appears on both sheets and means different things**: on `O63` what the enterprise can show a regulator now, on `R60` what shows this particular requirement addresses the obligation. **Signed by** the Chief Risk Officer and the compliance officer, jointly.

ID	Obligation	Regulatory Source	Control Objective	Control	Evidence Today
CO-01	*example — delete*	POPIA s19	Personal data secured	Access review	Quarterly log

`R60 O63-REQ`. One row per obligation-and-requirement pair. A `Target (REQ)` left blank is a control objective nothing is being built to meet, and it is the most expensive blank in the repository.

Source (O63)	Target (REQ)	Target Name	Evidence Today	Note
CO-01	REQ-14	*example — delete*	Access review report	—

Transcription — the part that makes the paper worth filling

A filled form is not a record. It is a promise to make a record, and the promise has a shelf life of about a day.

Form to sheet the same day

Transcribe before you go home. Not out of diligence but because of memory: the cells that are hard to read are the cells where the room argued, and you are the only person who remembers what the argument was. By Thursday you will be guessing at your own handwriting, and a guess transcribed looks exactly like a decision.

Book the hour with the session. A three-hour capability workshop needs an hour after it, in the diary, before the workshop is booked. If you cannot find the hour, run a shorter session: twelve rows transcribed beat twenty rows in a folder.

Who types

The person who held the pen types. Not an assistant, not the domain architect who was not in the room, not the graduate. The pen-holder is the only person who can tell a 3 from a crossed-out 4, and the only one who knows which cell carries a caveat the paper does not show. Where volume makes that impossible — `R37 APP-CAP` at 181 rows, `R17 BPR-DEN` at fifty — the pen-holder sits beside whoever types and reads the rows aloud. It is slower, and it is still faster than fixing it twice.

Type into the sheet, not into a staging spreadsheet. A staging file is a second repository with no version control, and it will develop opinions of its own.

The `Change Ref` on every row

Every row you add or change carries a `Change Ref`. One change request per session, raised on Form H-1 before the session rather than after it, with `Scope` naming the sheets the session may touch. Then every row from that session carries the same `CR` identifier in `Change Ref`, and every field change gets a line in `02 Change Log` with that `Change Request` against it.

This is the step people skip, and `RPT13 Version Control` exists to catch them skipping it: it compares records changed against changes logged, sheet by sheet. A sheet showing nine records changed and no changes logged is not a reporting problem. It means nine judgements are in the repository with nobody's name against them, and the first person to ask where a number came from will not get an answer.

The audit columns are not optional either. `Record Status` goes to Draft on transcription and to Approved only when a signatory says so. `Changed By` is you. `Approved By` stays empty until it is true — an `Approved By` filled in hope is the one entry in the repository that is actively dishonest.

When the room disagrees

Record both and mark the row `Draft`. This is the most useful habit in this appendix.

On the paper, write both values in the cell — '2 / 4', with the initials of whoever argued each. In the repository, put the one the owner signed, set `Record Status` to Draft, and put the other in the `Note` column where the sheet has one, or in the session minute where it does not. Then name the disagreement in the `02 Change Log` `Reason` field: claims manager 2, operations head 4, owner signed 2, unresolved.

A disagreement recorded is worth more than an agreement invented. On `O20 Data Entity` it has a name, because `CONTESTED` is a legitimate value for `Authoritative Source`. On `O13 Business Capability` it shows up in `RPT03 Capability Heatmap` as an average that satisfies nobody, which is the correct outcome and will bring the right people back to the table. What you must not do is split the difference. A `Current Maturity` of 3 that nobody in the room said is a number you invented, and you will be asked to defend it.

Put a date on every Draft row. A row that has been Draft for two quarters is a decision somebody is avoiding, and `RPT13 Version Control` counts those too, in its `Pending approval` column.

Retiring a filled form

Keep the paper until three things are true: the rows are in the sheet, the `02 Change Log` lines exist with the `Change Request` on them, and `Approved By` has a name in it. Until then the paper is the only evidence of what the room decided, and losing it means running the session again.

When all three are true, do this. Write the `CR` identifier and the transcription date across the top of the form. Scan it. File the scan where the deliverable lives — `O31 Deliverable` has a `Repository Location` column and that is what it is for — and name the file after the form number and the session date, not after the sheet. Then destroy the paper, because a filled Form C-2 carries classification and retention decisions about personal information, and a filled Form A-1 carries named people and what they are worried about. Neither belongs in a drawer.

One exception. The forms signed as evidence at a gate, G-1 and G-2, keep their originals in ink for as long as the architecture contract runs. Those are the two somebody outside the architecture function may one day ask to see.

Then print the next phase's forms, and go and ask somebody something they will have to think about.

THE CHAPTER IN TEN LINES

- Print the forms for the phase you are in, not all of them.
- Fill them by hand in the room; a laptop steers the conversation.
- Every column is a real column of a real sheet, in sheet order.
- Six columns maximum per table — a wider sheet splits into identify and assess.
- Form A-1 feeds nothing: the repository has no stakeholder sheet.
- Answer the ask set before the session, not during it.
- Transcribe the same day, and the person who held the pen types.
- One change request per session, and its `CR` on every row.
- When the room disagrees, record both values and mark the row `Draft`.
- Keep the paper until `Approved By` has a name, then scan it and destroy it.

SOURCES FOR THIS CHAPTER

- **1.** Meridian_TOGAF_Metamodel.xlsx — sheet and column headings as read from the workbook
- **2.** Meridian_Architecture_Comparison_Tool.xlsx, sheets 50 Review and 70 Gap Candidates

APPENDIX G

The principles and standards starter set

Forty-five principles and thirty-one standards somebody has already argued with, a tier distribution that lets the conflict rule work, and the ninety minutes that turn an inherited set into an adopted one.

Why you start from somebody else's set

Drafting principles from nothing is the most reliably wasted week in the Preliminary Phase. Put five executives in a room with a blank page and they will write forty statements that nobody can argue with, in no order, with no test, and the catalogue will be quoted in a steering committee once and never again. The failure is structural rather than a failure of effort: a principle written to be agreed is written to be unarguable, and an unarguable statement cannot break a tie.

The useful work is the other thing — adopting, editing and rejecting a set somebody else has already argued with. Argument is cheaper against a position than against a blank page. "Reuse, then buy, then build" takes thirty seconds to read and provokes an immediate objection from the person who has a statutory deadline and nothing to reuse, and that objection is the beginning of a tier decision. The same person given a blank page writes "we will make sound sourcing decisions", and nothing follows from it.

So this appendix gives you a set: forty-five principles and thirty-one standards, each marked with where it came from. Nine principles are kept from the repository unchanged, twenty-one are edited, one is recommended for retirement, and fourteen are new — added for things an insurer needs and the inherited set does not cover [1]. Thirty-one of the principles and sixteen of the standards come from a populated repository of the kind described in appendix A; Meridian Assurance Group, the insurer in that repository, is invented, and so is Cape Mutual in volume 1, but the shape of what they ship is not.

> **ADOPTING THE SET UNCHANGED IS THE SECOND FAILURE, NOT THE SAFE OPTION**
>
> Running this set through a workshop and emerging with forty-five adoptions and no edits means you tested nothing against your own insurer. It looks like efficiency and it is the same outcome as the blank page, reached faster. The adoption workshop later in this appendix sets a floor: one rejection or edit per category, or the session does not close.

Two things this appendix does not do. It does not reproduce the full rationale, implications, evidence and test columns for every row — those are in `Insurance_EA_Principles_and_Standards.xlsx` and they run to a hundred pages of prose. And it does not tell you what your principles should be. It tells you what forty-five defensible ones look like, where each would be contested, and how to run the argument.

What makes a principle real

A principle is real when three things are true of it. **It can be broken** — there is a decision somebody could take tomorrow that would breach it, and you can describe that decision. **You can point at the evidence** — a report, a sheet, a demonstration that shows whether it is being followed. **Something happens when it is broken** — a refusal at a gate, a dispensation with an expiry, a gap record with a severity.

Most inherited principles fail the second test. They carry a rationale, which makes them a policy statement, and they carry implications, which makes them an intention. What turns a policy statement into a control is a named test: the thing you would ask to be shown. Every row in the spreadsheet carries one, in a column headed `The test — what you point at`, and that column is the point of the instrument [1].

The three-question motherhood test

#	Question	Pass	Fail
1	Would any sane executive argue the opposite?	Somebody does argue it, regularly, and you can name when	Nobody argues against it. Add the arguable clause, or demote it
2	Does it say what to do when two good things conflict?	It names what it yields to, or what yields to it	It states a good outcome with no priority. Add the tier and the exception
3	Can you name a decision it would have changed in the last year?	You can name the decision, the date and what changed	You cannot. It is decoration, or nobody consulted it

Question three is answered mechanically by `RPT11 Principle Impact` : `Requirements governed` greater than zero is the whole test [3].

Two worked sharpenings

Weak. *Data is a shared enterprise asset.* Nobody argues against it, so it fails question one and cannot break a tie — and at Tier 1, where the repository put it, it beat purpose limitation, which is the wrong answer and a POPIA problem.

Sharpened. *Data belongs to the enterprise, not to the system or project that captured it; the holder of a record may not refuse access to the designated owner.* The second clause is argued against every time a line of business declines to release a file, so the principle now has an opponent, and it moves to Tier 3 where it loses predictably to `DP-04` — which since 6 March 2026 carries a separate gate on health information [9] — and to `IP-05` . **The test:** one

access request refused by a system owner and overturned, and one refused on purpose-limitation grounds and upheld. A principle that has only ever produced the first outcome is being read at the wrong tier.

Weak. *Operations continue despite loss of any single system, site or supplier.* Agreed by everyone, and in the repository it governs no requirement and is engaged by no gap, so `RPT11` reads `Not yet applied` [3]. As written there is nothing to fail.

Sharpened. …*demonstrated by a test inside its stated cadence rather than by a plan.* **The test:** the last continuity test for one Tier 1 capability, its date, what it exercised, and the gap record raised by anything it failed. `049 Resilience Tier` already carries `Test Cadence` and `Last Tested` and `RPT26 Resilience Test Status` already reads them, so the test costs nothing to run.

The set

Six categories, forty-five rows. `ID`, `Name`, `Tier` and `Statement (condensed)` transcribe into `O05 Principle` columns A to E, with `Category` taken from the heading above each table and `Owner` and `Status` from the spreadsheet [1][3]. `Source` is not an `O05` column — it is working information for the adoption pass, and you drop it when you paste. `IP-01`, `IP-02`, `IP-04` and `IP-05` carry `Category` = Data and `IP-03` carries `Category` = Security & Risk, which is what the repository does and is not a mistake in the prefix.

Business

ID	Name	Tier	Statement (condensed)	Source
BP-01	Primacy of Principles	1	Applies to every part of the group and every initiative, including partner-delivered and SaaS	repository
BP-02	Maximise Benefit to the Enterprise	2	Enterprise benefit outranks the fastest option for one line, and the cost of the slower path is stated	edited
BP-03	Fair Customer Outcomes by Design	1	Design assumptions and delivered experience are held at the same grain and compared	edited
BP-04	Business Continuity	2	Operations survive loss of any system, site, supplier or utility, demonstrated by a test inside cadence	edited
BP-05	Compliance is Designed, Not Retrofitted	1	Obligations are architecture requirements from the outset, with evidence designed in	repository
BP-06	Change is Incremental and Reversible	3	Sequenced increments leaving a stable state, except where a statutory date fixes the sequence	edited
BP-07	Every Product Carries Its Licence Class	1	Licence class and sub-class is a product attribute, derivable for every contract in force	new
BP-08	Delegated Authority Does Not Delegate the Record	2	Business written under delegated authority is held in the insurer's own records at policy and claim grain	new
BP-09	Advice and Sale are Recorded as Evidence	2	The record of advice is an entity linked to the analysis, the options, the product sold and the adviser	new
BP-10	Product Design Assumptions are Data	2	Target market, expected experience and value proposition stored as data at product and cohort grain	new

Data

ID	Name	Tier	Statement (condensed)	Source
DP-01	Data is a Shared Enterprise Asset	3	Data belongs to the enterprise, not the capturing system; the holder may not refuse the designated owner	edited
DP-02	One Authoritative Source per Data Entity	1	Each entity has exactly one designated authority; everything else is a marked copy	repository
DP-03	Data is Owned and Stewarded	2	Every entity has a named individual accountable for definition, quality and access	edited
DP-04	Data is Protected and Purpose-Limited	1	Collected for a stated lawful purpose, protected by classification, gated differently where special	edited
DP-05	Data Lineage is Traceable	2	Any reported figure traces back to its authority without manual reconstruction	repository
DP-06	Measure From the Event, Not From the Report	2	The event is recorded once at the finest grain any consumer needs; every basis derives from it	new
DP-07	History is Immutable and Bitemporal	1	What was true and what was known are both held; a past state is reconstructible without a build	new
DP-08	Identifiers Leave Only Under Authorisation	2	A unique identifier leaves the group only under recorded authorisation, through a switchable flow	new
DP-09	One Number, One Derivation	3	One definition and one derivation path; a required second basis is derived, not separately sourced	new

Four more principles carry `Category` = Data under the `IP-` prefix.

ID	Name	Tier	Statement (condensed)	Source
IP-01	Information is Owned by the Business	3	Every information concept has a named accountable owner who is a person	retire
IP-02	A Word Means One Thing	2	One agreed definition per concept; where two parts of the business disagree, the disagreement is visible	edited
IP-04	Information is Kept Only as Long as it Must Be	2	Every concept carries a retention obligation and a disposal rule, per purpose, and the disposal runs	edited
IP-05	Purpose is Stated Before Information is Used	2	A data set names its lawful basis and cannot be reused without a recorded decision	repository

Application

ID	Name	Tier	Statement (condensed)	Source
AP-01	Reuse, then Buy, then Build	3	Reuse, then acquire, then justify a build — except where a statutory date or licence condition cannot wait	edited
AP-02	Capability is Exposed Through Managed Interfaces	2	Defined, versioned, documented interfaces. No direct access to another system's data store	repository
AP-03	Products are Configured, Not Coded	3	Cover, rating, rules and wordings as business-maintained configuration — except on a recorded frozen book	edited
AP-04	Services are Channel-Agnostic	3	Implemented once, consumed by every channel, except where a counterparty mandates its own interface	edited
AP-05	Usability and Accessibility	3	Designed for real users, including low connectivity, low data budgets and disability	repository
AP-06	A Mandate is Mastered, Not Retried	2	One named component holds the collection mandate as a long-lived object; instructions reference it	new
AP-07	Translate at the Boundary, Never Inward	3	External message standards translate at the boundary; internal data is never restructured to match	new

Technology

ID	Name	Tier	Statement (condensed)	Source
TP-01	Technology Independence	3	Capability and data are not permanently bound to a product or supplier	repository
TP-02	Controlled Diversity and Standards Conformance	2	The estate is constrained to a managed set of standards	repository
TP-03	Reduce the Cost of Run	3	Every decision is assessed for its effect on run cost, and the assessment is recorded even where cost loses	edited
TP-04	Resilience Proportionate to Criticality	2	Criticality tiers defined once, applied consistently, objectives tested inside a stated cadence	edited
TP-05	Data Residency is Designed, Not Discovered	2	Workload and data placement is a recorded decision with a transfer basis per flow, taken before deployment	new

Security & Risk

ID	Name	Tier	Statement (condensed)	Source
SP-01	Security is Built In	2	Controls designed in from inception, proportionate to classification and criticality, proportionality recorded	edited
SP-02	Least Privilege and Accountable Access	2	Minimum access for a stated role, attributable to an individual, reviewed on a cycle	edited
SP-03	Models and Artificial Intelligence are Governed	2	Any model influencing a customer outcome is registered, explainable, monitored and owned by a person	edited
SP-04	Automated Decisions are Declared and Reconstructible	2	Every customer decision records whether it was wholly automated, on what inputs, under which version	new
IP-03	Classification Travels With the Information	2	Handling follows the information, not the document or the system it happens to sit in	edited

Governance

ID	Name	Tier	Statement (condensed)	Source
GP-01	Architecture Decisions are Recorded and Traceable	2	Context, options, rationale and consequences recorded, with the principles the decision engaged	edited
GP-02	Compliance is Assessed and Dispensations are Time-Bound	2	Departures are explicit, dated, remediated, owned by an executive, and the expiry has a consequence	edited
GP-03	Principles are Applied as a Set	1	Higher tier prevails; at equal tier the more specific prevails; otherwise the Board decides and sets precedent	edited
GP-04	Outsourcing is Assessed Before It Is Chosen	2	Due diligence, concentration assessment and an exit plan precede selection; material arrangements notified first	new
GP-05	A Gate Must Be Able To Say No	2	A gate that has never refused anything is not a gate, and its mandate is reviewed	new

The eleven that carry the weight

For each of these the spreadsheet carries a rationale, implications, an evidence list and a test. Condensed here, rationale and implications first, then **the test** — the thing you ask to be shown [1].

BP-01 Primacy of Principles (Tier 1, kept). An insurer's estate is substantially built and operated by other people: UMAs, binder holders, third-party administrators, SaaS providers. A set that applies only to internal build governs a shrinking minority of it. Implication: every procurement, binder and outsourcing agreement carries the principles as a schedule, and a partner that cannot meet one needs a dispensation with an expiry in the insurer's own register. **The test:** one contract or binder agreement whose schedule carries the principles, and one partner dispensation with an expiry date. Where neither exists, the set governs internal build only, whatever it says.

BP-03 Fair Customer Outcomes by Design (Tier 1, edited). TCF outcomes 2 and 5 form a loop — products designed for identified customer groups, products performing as customers were led to expect. Almost no insurer can evidence outcome 5, because the outcome 2 assumptions live in a pricing document rather than in data. That is a modelling decision, not a reporting one. Implication: target market, expected claims experience and value proposition stored as machine-readable data at product and cohort grain, and every customer communication reproducible as issued [8]. **The test:** the design assumptions for one product as data, the experience for the same product at the same cohort grain, and the comparison between them. A product approval pack is not the assumptions; a management report is not the comparison.

BP-07 Every Product Carries Its Licence Class (Tier 1, new). The Insurance Act restructured licensing into classes and sub-classes in Schedule 2, and returns are reported by class [5]. A product that drifts outside its sub-class is unlicensed business rather than mis-reported business. A configurable product platform — the thing everyone wants, for good reasons — lets a business user launch a variant in a fortnight with no class decision attached. The demarcation boundary makes it concrete: gap cover and hospital cash plans are permitted, primary healthcare insurance is not. Implication: class and sub-class are mandatory product attributes with a controlled vocabulary taken from Schedule 2, not from marketing, and product configuration cannot save without them. **The test:** the count of in-force contracts by licence sub-class, reconciled to the product catalogue, and the count sitting in a miscellaneous or unmapped bucket.

BP-08 Delegated Authority Does Not Delegate the Record (Tier 2, new). In a binder-heavy market the policy administration system is often not where policies are administered, and data arrives as periodic bordereaux in somebody else's format. An insurer outsourcing a critical function remains ultimately responsible, so the record cannot be outsourced with the activity [8]. Implication: a defined inbound schema at policy and claim grain, a submission schedule, a reconciliation, and the arrangement in the operator inventory with a POPIA operator contract and a transfer basis. **The test:** one bordereau and the reconciliation that followed it, then the policy count on that book as at a date between two submissions. Where the insurer cannot state its own count between submissions, the record has been delegated.

DP-02 One Authoritative Source per Data Entity (Tier 1, kept). Tier 1 because the breach is not a control failure: without one authority you cannot reproduce a regulated number, and reproducibility is the property every regulator's question reduces to. Implication: `O20 Data Entity Authoritative Source` filled for every Tier 1 entity, exactly one application marked `Authoritative` per entity in `R21 APP-DEN`, and `RPT05 App-Data Matrix` reading `Single` rather than `CONTESTED`. **The test:** read the application-to-data matrix and check the master count and status per entity. Against the party entity in the repository it reads several masters and a contested status — one cell, and the finding is the repository's rather than the architect's opinion [3].

DP-06 Measure From the Event, Not From the Report (Tier 2, new). The two regulators want data at different grains — money at a valuation date, process over a period — and both are derivations from the same events. Implication: the event is recorded once at the finest grain any consumer needs, and the IFRS 17 measurement, the pre-IFRS 17 premium definition the prudential template still uses and the economic valuation all derive from it rather than from each other. **The test:** take one premium instalment, ask which store holds it at full grain, then ask which of the three bases is a derivation of a derivation.

DP-07 History is Immutable and Bitemporal (Tier 1, new). Both axes — what was true and what was known. IFRS 17 sets the level of aggregation at initial recognition and does not reassess it, which is a permanent retrospective obligation; SAM needs a re-runnable assumption set per valuation date; conduct reporting covers a period. All three break on an estate holding current state. Implication: effective-from and effective-to on policy, cover, premium, reserve and assumption records, plus a knowledge-time axis, and no overwrite in place. **The test:** ask for one policy reconstructed as at an arbitrary past date and watch the ten minutes that follow. The ten minutes are the finding.

AP-03 Products are Configured, Not Coded (Tier 3, edited). Configuration over code is right almost always and impossible on a closed book whose rules are frozen — and causal event charges on pre-2009 legacy life policies are version-dated product logic on exactly such a book [8]. Tier 3 makes the principle lose predictably, and the exception requires the freeze to be recorded rather than assumed. **The test:** `O18 Product Config or Code` weighted by `Policies In Force` and `GWP (Rm)` — the share of the in-force book on coded products, not the product count. Then the last three launches with decision and go-live dates.

AP-06 A Mandate is Mastered, Not Retried (Tier 2, new). A DebiCheck mandate is a bank-held, long-lived object with its own lifecycle — registration, amendment, suspension, cancellation — and something must master it. Where nobody can name the component holding mandate state, collection failure is being managed by retrying blindly, and in the mass market that failure is a leading cause of lapse. **The test:** name the component holding mandate state, then ask whether suspense can be aged at policy level. Unallocated cash is normal; unallocated cash nobody can age means cover-in-force status is an opinion.

SP-04 Automated Decisions are Declared and Reconstructible (Tier 2, new). POPIA section 71 protects a data subject from a decision based solely on automated processing with legal or significant effect. The published worked example concerns lending, and I found no authoritative insurance-specific reading, so treat the application to automated underwriting declines, algorithmic pricing and fraud-score repudiation as inference rather than settled law — the spreadsheet carries the same flag against this row [1][7]. The design implication is unambiguous either way: record whether a decision was wholly automated, and preserve the inputs and the rule or model version. **The test:** for one declined application and one repudiated claim, the automation flag, the inputs and the version. Ask also what happens when the engine cannot decide at all — accept, refer and no decision is the real three-way outcome.

GP-04 Outsourcing is Assessed Before It Is Chosen (Tier 2, new). Joint Standard 1 of 2024 took effect on 1 December 2024: due diligence before outsourcing any activity, explicit consideration of concentration risk, notification to both regulators at least thirty days before entering a material arrangement, and compliance for existing arrangements by 30 November 2026 [6]. If a SaaS core is a material arrangement, the shortlist is a regulatory filing and the exit plan is a selection deliverable. Whether a SaaS core, a cloud platform or a binder arrangement is "material" is not settled in the practitioner material — the spreadsheet flags it against this principle and against `STD-29`, and so does volume 1, chapter 15. Get it in writing from the standard's own text and from legal advice, and record the answer. **The test:** the last platform selection, its concentration assessment, its exit plan, and the date the notification was filed.

> **THE ONE ROW TO ARGUE HARDEST ABOUT**
>
> `BP-10 Product Design Assumptions are Data` is the operational half of `BP-03`, separated deliberately. `BP-03` states the outcome and will be adopted without argument; `BP-10` states the cost and is the one that gets edited in the room. If the workshop adopts `BP-03` and rejects `BP-10`, the conduct loop stays open and the catalogue now says otherwise. Watch for it.

The tier problem, and the tie-break ladder

`GP-03` says that where two principles conflict, the higher tier prevails. The repository ships thirty-one principles with nineteen at Tier 1 [3]. The rule then resolves almost nothing, because almost every pair is Tier 1 against Tier 1. What happens next is predictable: the rule gets abandoned, and after that principles are applied selectively by whoever is in the room.

Tier is not a measure of importance. It is a tie-break mechanism, and a tie-break mechanism with no discrimination is not one.

The proposed distribution

Band	Count	Share	What qualifies
Tier 1 — the apex	8	18%	Breach is a licence matter, a legal breach, or the loss of reproducibility on a regulated number
Tier 2 — the working set	26	58%	The body of the practice: ownership, lineage, interfaces, resilience, security, governance records
Tier 3 — yields	11	24%	Principles that should predictably lose, and say so

The eight at Tier 1: `BP-01` (the set applies to work you do not control), `BP-03` (conduct), `BP-05` (obligations are requirements), `BP-07` (licence class), `DP-02` (one authority), `DP-04` (purpose and special categories), `DP-07` (bitemporal history), and `GP-03` (the conflict rule itself).

The admission test for Tier 1: would you stop a release for it? Not "is it important" — everything in the catalogue is important, which is why importance cannot be the criterion. Eight is small enough that an executive can recall the band, which is the property that makes rung one work without a meeting.

Tier 3 is the band people get wrong. It is not a bin for weak statements. It is the explicit band of principles that should lose: `AP-01`, `AP-03`, `AP-04`, `AP-05`, `AP-07`, `TP-01`, `TP-03`, `BP-06`, `DP-01`, `DP-09`, and `IP-01` while its retirement is pending. Each yields to something named — sourcing to a statutory date, configuration to a frozen closed book, channel-agnostic services to a counterparty's mandated interface, sharing to purpose limitation. Saying so in the tier is what makes them usable, because a principle overridden quietly stops being consulted at all.

The ladder

Rung	Rule	What it resolves	How it is recorded
1	The higher tier prevails	Tier 1 against anything; Tier 2 against Tier 3	Mechanical. No judgement, no meeting
2	At equal tier, the more specific prevails	Most Tier 2 against Tier 2 conflicts	The architect applies it; the decision record in `060 Governance Decision` names the rung
3	Otherwise the Board decides, and the decision is a precedent	Genuine ties at equal specificity	`060 Governance Decision` with both principles named, cited next time rather than re-argued

Rung two does the heaviest lifting, because general-against-specific is the common shape of a real conflict: a principle about health data beats a principle about data generally; a principle about this product line beats a principle about products.

> **TIER SETTLES ABOUT A THIRD OF THE CONFLICTS YOU WILL ACTUALLY MEET**
>
> Be honest about this with the Board, because somebody will test the arithmetic. Forty-five principles make 990 possible pairs. Of those, 582 cross a tier boundary and rung one settles them mechanically, which sounds decisive. It is not, because conflicts do not arrive evenly: the working body of the practice is Tier 2, and 325 of the 408 same-tier pairs are Tier 2 against Tier 2. Those are the arguments you have in real life — ownership against delivery, security against usability, lineage against cost — and rung one is silent on every one of them. Count the conflicts you actually meet rather than the pairs you could draw, and tier settles roughly a third. That is a property of any set this size, and it is why rungs two and three are written down rather than improvised.

Re-tiering happens by recorded decision, never by preference. The repository's own example is worth pointing at: `TP-01 Technology Independence` was re-tiered from 1 to 3 under `CR-012`, with the Board's reasoning recorded as independence yielding to other concerns [3]. That is a principle doing work — it now loses, predictably, to something else.

Reviewing the distribution is annual work, run alongside the maturity assessment in appendix H. Three checks. Can an executive name the Tier 1 principles without the sheet? Has any Tier 3 principle won a conflict — if so it is mis-tiered. Has any Tier 1 principle never been cited — if so it is either not a principle or nobody is consulting the set.

Settle Tier 1 **before** the workshop, with the Chief Risk Officer and the Chief Compliance Officer. That band is where the regulatory argument sits, and it is the only band that should not be decided by a show of hands.

The standards catalogue

Thirty-one standards, nineteen of them insurance-specific. Sixteen come from the repository, fifteen are new [1]. `ID`, `Name`, `Description`, `Domain`, `Status`, `Owner`, `Applies To` and `Review Due` are `O32 Standard` columns A to H, in sheet order [4]. The tables below group by `Domain` and drop `Description`, which is a sentence and is in the spreadsheet; the `Review Due` dates in the spreadsheet run from 2027-01 to 2027-09 and you reset them to your own review cycle on the way in. `Owner` values are `O11 Role` IDs — `ROL-09 Data Owner`, `ROL-13 Domain Architect`, `ROL-14 Architecture Board Member`, `ROL-16 Information Officer`, `ROL-17 Model Owner` and others — and they must be re-pointed at your own role records, because an owner that is a vacant role is not an owner.

Application and governance

ID	Name	Status	Owner	Applies To	Ins.
STD-01	REST API Design Standard	Emerging	ROL-13	All new interfaces	—
STD-02	Interface Versioning and Deprecation	Emerging	ROL-13	All managed interfaces	—
STD-03	No New Point-to-Point Integration	Emerging	ROL-14	All initiatives	—
STD-13	Accessibility Standard	Emerging	ROL-13	Customer journeys	—
STD-20	Event and Asynchronous Messaging Standard	Emerging	ROL-13	All event-based integration	—
STD-22	DebiCheck Mandate Handling Standard	Emerging	ROL-08	All premium collection	yes
STD-27	Automated Decision Record Standard	Emerging	ROL-17	Underwriting, pricing, claims, fraud	yes
STD-15	Architecture Decision Record Format	Emerging	ROL-14	All decisions	—
STD-29	Operator, Outsourcing and Transfer Register Standard	Emerging	ROL-16	All operators and outsourcing	yes

Data

ID	Name	Status	Owner	Applies To	Ins.
STD-07	Data Classification Standard	Current	ROL-09	All data entities	yes
STD-08	Data Retention Standard	Emerging	ROL-16	All data stores	yes
STD-17	ACORD Message Family Selection Standard	Emerging	ROL-13	Counterparty messaging	yes
STD-18	Southern Africa Short-Term Interchange Standard	Emerging	ROL-13	Broker and UMA exchange	yes
STD-19	Delegated Authority and Bordereau Data Standard	Emerging	ROL-09	Binder and UMA arrangements	yes
STD-23	Effective Dating and Bitemporal History Standard	Emerging	ROL-09	Policy, claims, actuarial, finance	yes
STD-24	IFRS 17 Group Key Standard	Emerging	ROL-27	All insurance contract cash flows	yes
STD-25	Licence Class and Sub-Class Mapping Standard	Emerging	ROL-24	All products and contracts in force	yes
STD-26	Record of Advice Standard	Emerging	ROL-15	All advised sales	yes
STD-30	Reinsurance Cession and Recovery Interchange Standard	Emerging	ROL-22	All reinsurance arrangements	yes

Technology, security and risk

ID	Name	Status	Owner	Applies To	Ins.
STD-04	Relational DBMS Standard	Current	ROL-13	All persistence	—
STD-05	Container Platform Standard	Emerging	ROL-13	New workloads	—
STD-10	Resilience Tiering Standard	Emerging	ROL-13	All capabilities	—
STD-11	Logging and Monitoring Standard	Current	ROL-13	All applications	—
STD-14	Exit Assessment Standard	Emerging	ROL-14	Platform selection	yes
STD-16	Cloud Placement Standard	Current	ROL-13	Cloud workloads	yes
STD-21	Payment and Disbursement Message Standard	Emerging	ROL-27	Payment and disbursement paths	yes
STD-31	Environment and Test Data Standard	Emerging	ROL-13	All non-production environments	yes
STD-06	Identity and Access Standard	Current	ROL-16	All applications	—
STD-09	Encryption Standard	Current	ROL-16	All data stores	—
STD-12	Model Documentation Standard	Emerging	ROL-17	All production models	yes
STD-28	Special Personal Information Handling Standard	Emerging	ROL-16	All special personal information	yes

> **TWO STANDARDS CARRY AN HONEST GAP, AND IT STAYS THERE**
>
> `STD-18` names the ACORD Southern Africa Short-Term Insurance standards for broker and UMA exchange. The current release version and its local adoption level are not published, and a separate Exposure Reporting standard exists and is named without published detail. `STD-17` depends on ACORD family licensing, which is members-only with unpublished terms — so the subscription belongs on the budget and in the critical path, not in a later phase. Both carry the flag in the spreadsheet and neither is resolved here. Do not let a programme plan assume a version number nobody has published.

The lifecycle vocabulary

Two vocabularies, routinely confused. A standard has a `Status`. A component has a `Lifecycle`. The assessment between them is a conformance verdict [2][4].

Value	Used on	What it obliges	What it is not
Current	`032 Status` ; `026 Standard Status`	Refuse a non-conforming new design; raise a gap against existing non-conformance	Not a claim that the estate conforms. A long non-conformant list means enforcement
Emerging	`032 Status` ; `026 Standard Status`	Refuse non-conforming new work; leave existing components unless they change anyway	Not a draft, not optional. One you will not enforce on new work should be withdrawn
Contained	`026 Standard Status` ; `047 Status`	Leave in place to the stated date; report a dated exposure	Not permanent. Contained with no expiry is the commonest dishonesty in a catalogue
Declining	`027 Lifecycle`	Keep running, refuse new workloads, fund replacement before support ends	Not Legacy. Declining still has support, which decides whether the risk is operational or existential
Legacy	`027 Lifecycle`	Contain it, date it, count what runs on it	Not a synonym for old. Legacy is a support position, not an age
Retired	`032 Status`	Keep the row for history; point references at the replacement	Not deleted. Deleting breaks every assessment that cited it

The conformance vocabulary

Verdict	Recorded on	What it obliges	What it is not
Conformant	`R31 STD-PTC Conformance`	Nothing, until the next assessment or change to either side	Not permanent. An undated verdict is an assertion
Partial	`R31 STD-PTC Conformance`	A note naming exactly what is unmet, and a dated remediation	Not a pass. Partial at a gate is a refusal unless a dispensation carries an expiry
Non-conformant	`R31 STD-PTC Conformance`	A dated containment with an executive owner, or a gap record with a severity. One of the two	Not automatically a defect. Some non-conformance is the right economic answer; undecided never is
Not assessed	`R31 STD-PTC` row absent, component in scope	Count it. The position is meaningless without the denominator	Not conformance. An unassessed component is not compliant by default

THE DENOMINATOR RULE

Report conformance as assessed-and-conformant over **in-scope**, never over assessed. A catalogue that assesses only what it expects to pass produces a rising conformance rate and a falling estate quality, and the report cannot tell the two apart. The same arithmetic applies to the review dates: a review either reaffirms, edits or retires, and cannot defer — a deferred review is how Emerging becomes permanent, and a standard past its `Review Due` is cited at gates on the authority of nobody.

The adoption workshop

Ninety minutes, five people, one change request out the other end [2]. Steps 2.4 and 2.5 of this volume set the same pass against the thirty-one principles the repository ships, and TOGAF puts it in the Preliminary Phase for the same reason: nothing downstream can be refused on a rule nobody has adopted [10]. Appendix C holds the facilitation craft — keeping the most senior voice from settling a question of fact, time-boxing an argument, what to send as a pre-read — and this section does not repeat it. What follows is the running order and the recording discipline specific to a principles pass.

In the room. The Chief Enterprise Architect chairs and does not advocate: a chair arguing for a principle guarantees consent rather than adoption. The Chief Information Officer settles whether the sourcing order, the interface rule and the configuration principle survive contact with the delivery portfolio. The Chief Data Officer settles whether one authoritative source per entity is a decision this insurer will take, and who takes it. The Chief Risk Officer settles the Tier 1 band and whether `GP-04` matches the position already notified to the regulators. The Chief Information Security Officer settles whether `SP-01` can leave Tier 1 without the security position weakening — and the answer should be yes, because the specific principles do the work. The Chief Compliance Officer settles whether the conduct and POPIA principles carry the obligations, and is the person most likely to add one in the room.

Not in the room. Solution architects, project managers and vendors. They are affected and they are not the owners. Bring their objections as a written, attributed pre-read. A principles workshop with twenty people in it produces a vocabulary, not a decision.

Segment	Minutes	What happens
Frame it	10	Two slides: what a principle is for, and the three-rung ladder. Then the session rule — silence is adoption, anyone may reject anything, and a rejection must name the decision the principle would have got wrong
The eight Tier 1 principles	20	One at a time, two minutes each. No re-tiering here; re-tiering is a recorded decision, not a show of hands
Everything else, by category	35	ID and statement read aloud, nothing else. Objections get a two-minute box, then edit, reject or defer. Six categories, six minutes each
Bind Tier 1 to requirements	15	One requirement per Tier 1 principle. Where none exists, the requirement is missing, not the link
Close	10	Read back the counts. Name the change request, the date `Status` changes from Draft, and who re-points links for any retirement

Step 1 — Pre-mark the whole set and settle Tier 1 before the room sits

Produce. A marked copy of the forty-five rows with Adopt, Edit, Reject or Defer against each, circulated — plus an agreed Tier 1 band.

You need. One day of your own reading. Ninety minutes each with the Chief Risk Officer and the Chief Compliance Officer.

How.

1. Read all forty-five in one sitting, about an hour.
2. Mark every row. An unmarked row is one you will argue about in the room instead of before it.
3. Take the eight proposed Tier 1 principles to the CRO and the Chief Compliance Officer; agree that band or a different one.
4. Circulate the marked sheet three working days out, with the ladder on one page.
5. Collect written objections from solution architects and project managers, attributed.

Signed by. Nobody yet. This is preparation, and pretending otherwise turns a pre-read into a decision nobody made.

Done when. Every row carries your mark and the Tier 1 band has two executive names behind it.

Goes wrong when. You walk in with an unmarked sheet. The room starts from nothing, and ninety minutes becomes three sessions, of which two never happen.

Step 2 — Run the Tier 1 band, two minutes a principle

Produce. Eight decisions, each with reasoning where it is anything other than Adopt.

You need. Step 1. Twenty minutes of the ninety.

How.

1. Read the ID, the name and the statement. Not the rationale — that was the pre-read.
2. Ask the role in the `Owner` column whether they accept the ownership. That question alone reshapes two or three rows.
3. Take a rejection only where the objector names the decision the principle would have got wrong.
4. Refuse a re-tier in the room; record it as a proposed re-tier for the Board.
5. Write the decision down as you go, not afterwards.

Signed by. The Architecture Board (`GB-01`), which approves dispensations with an expiry and therefore owns the rules it dispenses from.

Done when. All eight carry a decision and every rejection names a decision it would have changed.

Goes wrong when. The band grows in the room. Every addition weakens rung one for every other pair, and the room cannot see that cost from inside the discussion.

Step 3 — Run the remaining thirty-seven by category, silence as adoption

Produce. Thirty-seven decisions, with at least one edit or rejection in every category.

You need. Step 2. Thirty-five minutes, and the pre-read objections in front of you.

How.

1. Business, Data, Application, Technology, Security & Risk, Governance, in that order. Six minutes each.
2. Read the ID and the statement. Silence is adoption; say so again at the start of the segment.
3. An objection gets two minutes and then one of three outcomes: edit in the room, Reject with a reason, or Defer. A fourth is not available.
4. Where a category survives unchanged, make somebody state which decision in the last year each of its principles would have changed.
5. Accept no new principle in the session. It goes on Form PS-2 and comes to the next Board meeting with a test written.

Signed by. The Architecture Board (`GB-01`).

Done when. All thirty-seven carry a decision, and no category reads as unanimously adopted without point 4 having been answered out loud.

Goes wrong when. It becomes a drafting session — forty minutes on one wording, three categories unread, a follow-up that never happens. Wording that cannot be settled in two minutes goes to the owner to redraft and returns to the Board, not to the room.

Step 4 — Bind every Tier 1 principle to a requirement while the owners are present

Produce. At least one `R05 PRN-REQ` row per Tier 1 principle, agreed by the person who owns the requirement.

You need. Step 3. Fifteen minutes. The strategic requirements in `006 Requirement` .

Where it lands.

> **`R05 PRN-REQ`**
>
> - `Source (PRN)` the principle ID, `Target (REQ)` the requirement ID. `Source Name` and `Target Name` resolve by formula; a lookup error means the ID does not exist.
> - `Relationship` — `Directly mandates` where the principle makes the requirement non-negotiable, `Supports` where it adds weight without creating the obligation. Using the verbs loosely is how a principle set becomes a mood.
> - `Key` — the form `DP-02|REQ-01` .

How.

1. Take the eight Tier 1 principles in order.
2. Ask the room which requirement each governs, and write the ID down.
3. Where no requirement exists, record that the requirement is missing. Do not invent one.
4. Add `R34 PRN-GAP` rows for any gap the room says engages a principle, with `Engagement = Directly violated` where it is a negation rather than a weakness.
5. Leave a principle unbound rather than fabricating a link.

Signed by. The Chief Enterprise Architect. The Board sees the result in `RPT11` , not the row.

Done when. No Tier 1 principle has zero rows in `R05 PRN-REQ` , or the empty ones are recorded as missing requirements with an owner.

Goes wrong when. This is the segment people cut, and cutting it is why principles govern nothing: the function then writes the links alone in an afternoon, and they are defensible and nobody has agreed them.

Step 5 — Record the whole pass as one change request

Produce. One row in `03 Change Requests` carrying the entire pass, and `Status` set on all forty-five rows.

You need. Step 4. Half a day, the same week.

Where it lands.

03 Change Requests

- `Type` — `Reassessment`. Use `Addition` only for a separate request covering principles written from nothing.
- `Scope` — `Principle (all records)`. One per principle produces a register nobody reads.
- `Raised By` — Chief Enterprise Architect. The function that owns the catalogue raises it; the Board decides it.
- `Decision` — the counts, then the named exceptions. A `Decision` reading "approved" records nothing.
- `Decision Record` — an ADR reference. The ADR carries the reasoning; the request carries the change.

How.

1. Raise the one request, with the counts in `Decision`.
2. Set `Status` on every `005 Principle` row from Draft to its adopted value. Forty-five cells, ten minutes.
3. Retire `IP-01` by setting `Status`, never by deleting the row — it is referenced from `R05 PRN-REQ` and `R34 PRN-GAP`, and deleting it breaks both.
4. Re-point the links for anything retired, in the same request, or `RPT11` loses the history.
5. Write the `Change Ref` back onto every principle row.

Signed by. The Architecture Board (`GB-01`), with the counts reported to the Executive Committee (`GB-04`).

Done when. All forty-five rows carry a `Change Ref` from this pass and none still reads `Status` = Draft.

Goes wrong when. The decisions live in your notebook. A pass with no change request did not happen, and next year's reviewer cannot tell an adoption from a load.

Step 6 — Run the standards catalogue as a second session

Produce. Thirty-one standards adopted, edited or rejected, with `Status` and `Review Due` set.

You need. Step 5. A second ninety minutes, only if the first landed. The CIO, the technology lead and the CISO; the CRO for `STD-29`.

Where it lands. `032 Standard`, columns `ID`, `Name`, `Description`, `Domain`, `Status`, `Owner`, `Applies To`, `Review Due`.

How.

1. Take the nineteen insurance-specific standards first. Nobody else will write those for you.
2. Set `Status` to Current or Emerging, and be clear which you will enforce on new work.
3. Re-point every `Owner` at a real `011 Role` row with an actor behind it in `R08 ACT-ROL`.
4. Set `Review Due` to your own cycle, not the spreadsheet's 2027 dates.
5. Leave the `STD-17`, `STD-18` and `STD-29` flags in place, recorded as open questions with owners and dates.

Signed by. The Architecture Board (`GB-01`). `STD-29` additionally needs the Chief Risk Officer, because materiality is a regulatory question.

Done when. Every standard has a `Status`, an `Owner` with an actor behind it and a `Review Due` inside your horizon — and the three unresolved questions are written down as questions.

Goes wrong when. The catalogue is adopted and conformance is never assessed, so `R31 STD-PTC` stays empty and `RPT10 Standards Conformance` has nothing to report. A catalogue with no assessment constrains nothing.

Making the principles do work

A principle with zero governed requirements is decoration. `RPT11 Principle Impact` says so mechanically [2], in four columns: `Requirements governed`, `Gaps engaging it`, `Direct violations`, and an `Assessment` of Applied, VIOLATED or Not yet applied [3].

Read the report in two minutes, in this order. **Principles with a direct violation** — that is the work, and it should be owned; a violation with a gap record behind it is the healthy shape, because the principle detected a breach and the breach has an ID, a severity and a work package. **Principles reading `Not yet applied`** — zero governed, zero engaged, zero violated; that is the catalogue's own defect and it reads as the harmless verdict, which is why it survives. **Tier 1 principles violated** — that is the board conversation, because by this appendix's own admission test a violated Tier 1 principle is a licence matter, a legal breach or a loss of reproducibility.

The worst combination is `Direct violations` above zero with `Requirements governed` at zero: broken, and mandating nothing, so it has no mechanism to stop the next breach. The worst single cell is `VIOLATED` against `BP-01`, the principle that makes the set apply at all — it means the rule making the catalogue binding is itself being breached, which invalidates every other verdict on the report. Where that appears, stop reading and go and find out whether any gate has ever refused anything.

A WORKED TRACE, END TO END

`DP-02`, Tier 1. `R05 PRN-REQ` holds `DP-02 → REQ-01`, `Directly mandates`, noting that one authoritative source per entity is the principle behind the single enterprise customer identifier; `BP-02` also reaches `REQ-01` with `Supports` — weight, not obligation. `R34 PRN-GAP` holds `DP-02 → GAP-01` (four customer masters, no common identifier, around 640,000 records needing manual matching) and `DP-02 → GAP-02` (no customer timeline: advice, policy and claim records share no key), both `Directly violated`. `RPT11` therefore reads 1 governed, 2 engaging, 2 violations, `VIOLATED` [3].

Read that as the principle working. Two different gaps engaging one principle means it is describing a real structural property of the estate rather than a coincidence of wording. What it adds to gaps already marked Critical is the argument for sequencing them first, and the reason a decommissioning gate cannot pass while an entity has no surviving master. Resist softening the statement so the report looks better.

Before any of that means anything: every Tier 1 principle needs at least one `R05 PRN-REQ` row, and every recorded gap needs its engaged principles linked. Both are done in the workshop, while the owners are in the room.

The forms

Three forms and two question sets. The forms transcribe into `005 Principle`, `032 Standard` and `03 Change Requests`; the question sets are the thinking that decides whether the forms can be filled honestly.

BEFORE THE WORKSHOP — ANSWER THESE ON PAPER

How many principles are at Tier 1 today, and what breaks a tie between two of them? Write the sentence you will say to the Board.

Which three principles would you reject outright, and for each, name the decision it would have got wrong.

Which principle does your insurer break routinely, and who benefits from it being broken?

For each of the eight proposed Tier 1 principles: would you stop a release for it? Name the ones where the answer is no.

Who, by name, owns `BP-07` licence class and `GP-04` outsourcing — and have they agreed to own it?

Form PS-1 — Principle decision sheet

Feeds `O05 Principle` — `ID`, `Name`, `Category`, `Tier`, `Status`, `Owner`. **The decision lives in** `Status`: Adopted, Draft or Retired. Record Adopt / Edit / Reject / Defer in the `Decision` column of the spreadsheet, not as a new repository column. **Typed into Excel** `Statement (condensed)` for any row the room re-words — use the second table below and write it out in full, not as an instruction to the typist. **First pass** forty-five principles, five copies. **Signed by** `GB-01 Architecture Board`. A principle nobody has signed is a preference.

ID	Name	Category	Tier	Status	Owner
BP-01	*example — delete*	Business	1	Adopted	GB-01

Re-worded statements. If it needs two lines it is two principles.

ID	Statement (condensed)	Rejected because — name the decision it would have got wrong
DP-01	*example — delete*	*example — delete*

Form PS-2 — A new principle the insurer adds

Feeds `005 Principle`, one row per principle, but not until the rationale, implications and test are written. **No new principle is accepted in the adoption session** — this form captures the proposal, and the proposal goes to the next Board meeting. **First pass** two or three; if the room proposes eight, the set was not the problem. **Signed by** `GB-01 Architecture Board`, after the owner named below has written the three paragraphs.

Field	Fill
`ID`	
`Name`	
`Category`	
`Tier` — and would you stop a release for it?	
`Owner` — a named executive role, not a committee	
Proposed by, and date	

`Statement (condensed)` — one sentence of consequence.

Rationale — why this insurer needs it, and who argues the opposite.

Implications — what somebody has to do differently on Monday.

The test — what you would ask to be shown, naming the sheet or report.

Which existing principle does it overlap, and which of the two yields?

Form PS-3 — Standards decision sheet

Feeds `032 Standard — ID, Name, Domain, Status, Owner, Review Due`. **Typed into Excel** `Description` and `Applies To`, both sentences. **`Status` values** Current, Emerging, Contained or Retired — and Contained with no expiry date is the commonest dishonesty in a catalogue, so there is a column for the date. **First pass** thirty-one standards, three copies. **Signed by** `GB-01 Architecture Board; STD-29` also needs the Chief Risk Officer.

ID	Name	Domain	Status	Owner	Review Due
STD-24	*example — delete*	Data	Emerging	ROL-27	2027-03

Containment. One row per standard where existing non-conformance is permitted for a period. No row without a date.

ID	What is non-conformant	Contained by	Exit date	Executive owner
STD-09	*example — delete*	*example — delete*		

SIX MONTHS ON — IS THE SET DOING WORK?

How many rows in `RPT11 Principle Impact` read `Not yet applied`? Which ones, and what will you bind each to?

Name one design refused on a principle in the last six months, and the `060 Governance Decision` row that records it.

Has any Tier 3 principle won a conflict? If so, it is mis-tiered — which way?

Can your Chief Information Officer name the Tier 1 principles without the sheet? Ask them, and write what they say.

How many dispensations in `061 Governance Change Request` name the principle they depart from, and how many expired without a consequence?

Which standards are past their `Review Due`, and who is citing them at gates anyway?

Where the spreadsheet adds something

`Insurance_EA_Principles_and_Standards.xlsx` carries what will not fit a printed page and should not be summarised. Seven sheets.

Sheet	What it adds beyond this appendix
`00 Start Here`	How to run the pass, who to involve, how long it takes, the three things it could not source
`01 Principles`	A to G paste into `005 Principle`. H onward is the principles document: what changed and why, rationale, implications, **the test**, the evidence list, the regulatory anchor
`02 Tier rule`	The distribution, the reasoning per band, the ladder and the honest limitation
`03 Standards`	A to H paste into `032 Standard`, then what the `Status` means for that standard and how conformance is evidenced
`04 Lifecycle and conformance`	Both vocabularies in full, with what each value is not
`05 Adoption workshop`	The running order, the room, the motherhood test, the three failure modes
`06 Principle impact`	`DP-02` traced end to end, and the three verdicts compared

The paste-ready ranges are the practical reason to open it. `01 Principles` A to G and `03 Standards` A to H go in without translation, because `Status` already reads Draft — the value `005 Principle` uses — and the IDs already carry the prefixes the repository expects.

The thing it carries that this appendix cannot is the evidence column: every row names the sheets and reports that evidence it, `049 Resilience Tier` with `Test Cadence` and `Last Tested` plus `RPT26 Resilience Test Status` against `BP-04`, and so on down the set. That column is how a test stops being a sentence and becomes a cell somebody reads on a Tuesday.

THE CHAPTER IN TEN LINES

- Drafting principles from nothing produces forty unarguable statements; argue with a set instead.
- A principle is real when it can be broken, the evidence can be pointed at, and something happens.
- Forty-five principles: nine kept, twenty-one edited, one recommended for retirement, fourteen new.
- Nineteen Tier 1 principles out of thirty-one makes GP-03 useless; eight out of forty-five makes it work.
- Tier 1 admission test: would you stop a release for it? Not whether it is important.
- Tier alone settles about a third of possible pairs; specificity and then the Board carry the rest.
- Thirty-one standards, nineteen insurance-specific; a status says what you may enforce, not what conforms.
- Report conformance over in-scope components, never over assessed ones, or the number rises as the estate falls.
- A principle with zero governed requirements is decoration; bind the Tier 1 band in the room.
- Three things stay unresolved on purpose: the ACORD Southern Africa release, SaaS materiality, POPIA section 71.

SOURCES FOR THIS CHAPTER

- **1.** Insurance_EA_Principles_and_Standards.xlsx, sheets 00 Start Here, 01 Principles, 02 Tier rule and 03 Standards
- **2.** Insurance_EA_Principles_and_Standards.xlsx, sheets 04 Lifecycle and conformance, 05 Adoption workshop and 06 Principle impact
- **3.** Meridian_TOGAF_Metamodel.xlsx, sheets O05 Principle, R05 PRN-REQ, R34 PRN-GAP and RPT11 Principle Impact
- **4.** Meridian_TOGAF_Metamodel.xlsx, sheets O32 Standard, O26 Logical Technology Component, O27 Physical Technology Component, R31 STD-PTC and RPT10 Standards Conformance
- **5.** Insurance Act 18 of 2017, Schedule 2 — classes and sub-classes of life and non-life insurance business https://www.acts.co.za/insurance-act-2017/table_2__classes_and_sub-class
- **6.** Prudential Authority and FSCA, Joint Standard 1 of 2024: Outsourcing by Insurers (effective 1 December 2024) https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-public-awareness/covid-19-response/2024/joint-comms-1-of-2024/Joint%20Standard%201%20of%202024%20Outsourcing%20by%20Insurers.pdf
- **7.** Webber Wentzel, "Artificial intelligence has POPIA implications" (section 71 and wholly automated decisions) https://www.webberwentzel.com/News/Pages/artificial-intelligence-has-popia-implications.aspx
- **8.** National Treasury, "Release of final insurance conduct regulations and policy" (15 December 2017), on the Policyholder Protection Rules https://www.treasury.gov.za/comm_media/press/2017/2017121501%20Media%20statement%20release%20of%20final%20insurance%20conduct%20regulations%20and%20policy.pdf
- **9.** Moonstone, "New POPIA regulations on health information now in force" (effective 6 March 2026; named parties include insurers) https://www.moonstone.co.za/new-popia-regulations-on-health-information-now-in-force/
- **10.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method, Preliminary Phase https://pubs.opengroup.org/architecture/togaf10-doc/adm/

APPENDIX H

Running the maturity assessment

Twenty-one dimensions, two scores that stay apart, and the rule that makes both defensible: no score without the evidence that supports it.

What the assessment is for

You run this twice: in your first month, and then once a year. The first run tells you what you have walked into. The second run, and every run after it, tells you whether anything moved — and that is the only reason the first one is worth the effort.

It is not a score for its own sake. A maturity number on a slide persuades nobody and ages badly. What the instrument produces that is worth having is three things: a set of dimension scores each attached to a named piece of evidence, a gap record for every dimension scoring 1 or 2, and a weighted mean written into `O04 Measure` with a target and a date so next year has something to compare against. If no gap severity changed as a result and no work package was released, you scored yourself for nothing [1].

Timing. First run: about ten working days of elapsed time inside the first thirty days, of which three to four days are yours. Eight conversations of forty-five minutes each, plus the evidence requests. The re-run is two days, because the evidence requests are already known and you are comparing against last year's sheet.

Who scores, and who challenges. Scoring is yours, because you are the one who has to defend each number. Challenge is not. Have every score challenged by someone outside the architecture function before it goes anywhere — a self-assessed maturity score is worth nothing to a board. Volume 1, chapter 18 (*Your first ninety days*) puts this in the first fortnight, and step 2.9 of this volume closes the Preliminary Phase with it — which is where TOGAF puts it, because a phase that cannot say where it started cannot show it moved [6].

> **THE ASSESSMENT TELLS YOU WHERE TO LOOK. IT DOES NOT TELL YOU WHAT IS THERE**
>
> It is not a benchmark. The levels describe observable states, not an industry position, and there is no external data behind them. And it is not a substitute for the ninety-day work: build one small slice of the estate end to end and have the people who do the work correct it. The assessment points; the slice tells you what is actually there.

Two assessments, kept separate

There are two scores and they stay apart. **(a) EA practice maturity**, ten dimensions: is the architecture function any good. **(b) Insurance architecture health**, eleven dimensions: is this insurer's architecture any good.

The separation is the design decision that matters most in the instrument. A competent function can inherit a poor estate — that is the common case for anyone starting a new job — and a poor function can inherit a good one, which happens after a well-run core replacement and a reorganisation. Averaging the two produces a middling number that describes neither, and the first person to disagree with it will be right.

It also changes what you do next. A low (a) and a reasonable (b) means fix the practice: gates, repository, traceability. A reasonable (a) and a low (b) means the practice is working and the estate is the problem, which is a funding conversation rather than a governance one. A single score cannot tell those apart, and they call for opposite first moves.

> **WHERE THIS SITS AGAINST THE REPOSITORY'S OWN MEASURE**
>
> The repository ships `DLV-04 Architecture Maturity Assessment` as a nine-element ACMM score, with `OBJ-12 Architecture maturity` at baseline 2.0 and target 3.0 by 2028, and `MSR-13 ACMM weighted mean` as the measure, `Frequency` Annual, `Source System` Manual [2]. This instrument is wider: ten practice dimensions weighted to 100, plus eleven estate dimensions weighted to 100 separately. If you are keeping the repository's measure, write the practice weighted mean into `MSR-13` and raise a second measure for the estate — do not write both into one cell and do not quietly redefine `MSR-13`, because the comparison with 2.0 is the only history you have.

The scoring rule

Five levels per dimension, each with an observable descriptor — what you would see, not what you would be told. Score the level whose descriptor you can point at. Not the one that sounds closest, and not the average of several. If you cannot evidence level 3, you are at level 2, even where level 3 is plainly intended [1].

Then the hard rule, which is what separates this from a workshop with a flipchart.

> **NO SCORE IS RECORDED WITHOUT THE EVIDENCE THAT SUPPORTS IT**
>
> `Evidence seen` takes one of three things and nothing else: a named repository sheet or report with the reading you took from it, a document you were given, or a person who showed you something and the date they showed you. A score with an empty `Evidence seen` is an opinion with a number attached, and the first person who disagrees with it will say so. There is a `Confidence` column for the cases where you saw little — use it. Where you cannot evidence anything at all, score it low, set `Confidence` = Low and write the reason, because **absence of evidence is the finding**. A blank hides it.

The weighted mean reads on the same 1 to 5 scale as the dimension scores, so it is a level rather than a percentage. Five bands apply to either assessment.

Band	Range	What it means, and what to do
1 Absent	below 1.5	It does not exist in a form anyone could inspect. Do not write a target architecture — build the record: one repository, one capability view, one defect register with named owners
2 Initial	1.5–2.49	It exists on paper and has not been exercised. Most inherited practices and most estates sit here. Pick the two highest-weighted dimensions at 2 and move one level each. Nothing else
3 Defined	2.5–3.49	Defined, owned and running, and not yet measured. Add the measure before adding the next capability, or next year's score is another opinion
4 Managed	3.5–4.49	Enforced, with consequences somebody can point at. Find the dimension carrying the others and check it is not one person
5 Measured	4.5+	Measured, reported and improving, with the movement attributable to something that was done. Re-weight: at this level the judgement lives in the weights

(a) The EA practice dimensions

Ten dimensions, weights totalling 100. The weights are editable and the edit is a judgement you own; what you may not do is edit them after you have seen the scores.

Ref	Dimension	Wt	What it measures	Evidence
EA-01	Architecture governance	14	Whether gates exist, are held by a body with a mandate, and whether anything has ever failed one	`O57` , `O58` , `O59` , `O60` , `O61` , `RPT30 Compliance Position` , `RPT31 Governance Activity`
EA-02	The architecture repository	10	Whether one repository exists, is current, and is trusted enough that someone else quotes it	`RPT12 Integrity Check` , `RPT13 Version Control` , `02 Change Log` , `03 Change Requests` , `04 Baselines`
EA-03	Principles	8	Whether principles are adopted rather than inherited, and whether they govern anything	`O05` , `R05 PRN-REQ` , `R34 PRN-GAP` , `RPT11 Principle Impact`
EA-04	Requirements traceability	12	Whether a requirement runs from a driver to something that delivers it, and whether it was tested	`O06` , `RPT08 Traceability` , `RPT32 Requirement Traceability` , `R60 O63-REQ`
EA-05	Business architecture	12	Whether the business is described in capability and value-stream terms that connect to the estate	`O13` , `O14` , `R10 ORG-CAP` , `R37 APP-CAP` , `RPT03 Capability Heatmap` , `RPT07 Capability Coverage`
EA-06	Data governance	14	Whether data has one authority, a named owner, a mastering rule that runs and a retention rule that executes	`O20` , `O43` , `RPT05 App-Data Matrix` , `RPT15` , `RPT16` , `RPT17 Master Data` , `RPT18` , `RPT22`
EA-07	Application portfolio	10	Whether the estate is known, costed, dispositioned, and whether the disposition agrees with the assessment	`O23` , `RPT02 Application Portfolio` , `RPT06 Interface Register` , `RPT20 Portfolio Assessment`
EA-08	Technology standards position	8	Whether a catalogue exists, conformance is assessed, and non-conformance carries a dated exit	`O32` , `O27` , `R31 STD-PTC` , `RPT10 Standards Conformance` , `RPT25 Reference Model Conformance`
EA-09	Delivery engagement	8	Whether architecture is engaged before the expensive decisions, and whether the roadmap is deliverable	`O30` , `O33` , `RPT24 Building Block Coverage` , `RPT27 Programme Sequencing` , `RPT29 Transition Readiness`
EA-10	Skills and capacity	4	Whether the function has the roles, authority and days it has committed to, and survives one departure	`O11` , `R08 ACT-ROL` , `O31` , `RPT14 Deliverable Register`

The span, pole to pole

The spreadsheet carries all five descriptors per dimension. Here are the two poles, so you can see how far the scale actually travels [1].

Ref	Level 1 — what you would see	Level 5 — what you would see
EA-01	No body can refuse a design. `058` is empty, `059` has no rows, designs reach production unseen	Gate pass rate, time in gate and dispensation ageing are measures in `004` with a source other than Manual, and the board's own backlog is falling
EA-02	Content lives in project sites, decks and personal drives. There is no record, so nothing can be out of date	The repository is the source for committee papers and regulatory evidence requests; the integrity check runs on a schedule and a failure raises a change request
EA-03	No principles record, or principles on a slide with no owner, no tier and no reference from anywhere	Each principle carries a stated test, the test runs on a cycle, re-tiering happens by recorded decision, and no row reads `Not yet applied`
EA-04	Requirements live in project documents and vendor responses. `006` is empty or holds only this project's	`RPT32` `Ever tested` reads yes on everything delivered, with evidence in `059`, and a requirement cannot close without a recorded test
EA-05	The business is described by reporting lines and a project list. No capability model, so nothing can be said about weakness	Capability maturity moves between assessments and the movement is attributed to delivered work packages; the heatmap is quoted in the investment decision, not after it
EA-06	No entity catalogue. Ownership is asserted by whoever runs the capturing system, and contested the moment two reports disagree	Data quality is measured per entity against a target in `004`; lineage is present for every return line; every entity has a surviving master before a decommissioning gate opens
EA-07	The inventory is a spreadsheet somebody typed, naming decommissioned systems and missing live ones	Run cost moves in the direction the dispositions predict, and decommissioning is funded inside the replacement work package
EA-08	No catalogue. Technology choices are made and defended per project, and two concurrent projects choose differently	The conformant share rises between assessments; no node carrying a Tier 1 application has a support-end date inside eighteen months; standards are retired as well as added
EA-09	Architecture sees a design after a vendor has been chosen. No building blocks, so acceptance criteria are whatever the supplier proposed	`RPT29` exit criteria are met for the transition state in progress before the next one starts, and `059` holds an assessed verdict against every criterion at G2
EA-10	Architecture is a person, not a function. No role records, no authority levels, no cover. Work stops when that person is on leave	Skills are assessed against the roles and the gap is funded; the practice survives any one departure, demonstrated by a handover rather than asserted in an operating model

(b) The insurance architecture health dimensions

Eleven dimensions, weights totalling 100. These are the industry-specific ones, and each is explained properly in volume 1 — this instrument scores what volume 1 teaches, and does not re-teach it.

Ref	Dimension	Wt	Volume 1	Evidence
IH-01	Product change and configuration	11	ch 11, ch 15	`O18` (`Config or Code`, `Policies In Force`, `GWP (Rm)`), `RPT23 Process Flow`, `O29`
IH-02	Party and customer mastering	11	ch 9, ch 10	`O20` (party: `Mastering Style`, `Authoritative Source`), `O43`, `R21 APP-DEN`, `RPT05`, `RPT17`
IH-03	Policy and claims data history	11	ch 9, ch 15	`O20` (`Data Category`, `Retention Period`), `O21`, `O22`, `RPT04 CRUD Matrix`
IH-04	Delegated authority and the binder estate	9	ch 5, ch 15	`O19` (binder, UMA, outsourcing), `O10` (`Internal/External`), `R16 PRD-CTR`, `RPT06`
IH-05	Regulatory reporting and the IFRS 17 chain	13	ch 15, ch 6	`O63 Control Objective`, `R60 O63-REQ`, `O40 Source To Target Mapping`, `RPT15`, `RPT18`
IH-06	Collections and DebiCheck	7	ch 8, ch 6	`O20` (mandate), `O24`, `O16`, `RPT06`, `O04` (lapse, collection success)
IH-07	Distribution and intermediary systems	7	ch 5, ch 11	`O23` (portals), `O15` (`Channel Agnostic`), `R20 APP-BSV`, `RPT07`, `O11`, `O19`
IH-08	Reinsurance administration	6	ch 4, ch 6	`O19` (treaties), `O20` (cession, recovery), `R21 APP-DEN`, `RPT05`, `O63`
IH-09	Resilience against criticality	8	ch 13, ch 15	`O49 Resilience Tier` (`RTO`, `RPO`, `Test Cadence`, `Last Tested`), `O12`, `RPT26`
IH-10	Conduct and POPIA position	10	ch 15	`O20` (`Classification`, `Retention Basis`), `O37` (`Personal Information`), `O38` (`Lawful Basis`), `RPT16`, `RPT22`
IH-11	Automated decisions and model governance	7	ch 15, ch 11	`O23` (decisioning, scoring), `O32` (model documentation), `O63`, `O29` (model governance)

`IH-11` is an addition rather than one of the ten dimensions the instrument was commissioned with, and it is marked as added in the spreadsheet. It is there because POPIA section 71 and the conduct position on automated underwriting and pricing make it a live question in a South African insurer [1][4]. Drop it and renormalise the weights if you disagree; the weights column is yours.

The span, pole to pole

Ref	Level 1 — what you would see	Level 5 — what you would see
IH-01	Product definition is embedded in code on a fixed release train. Rate decision to live quote is a quarter or more, and nobody can state the last launch date without checking	Lead time from decision to live quote is a reported measure with a target, and a variant cannot be launched without a licence class and sub-class decision attached
IH-02	Each system holds its own party record under its own key. No common identifier; matching is manual and repeated per project	Parties are modelled by role — policyholder, life assured, beneficiary, third-party claimant, intermediary, operator — so basis, purpose and retention differ correctly by role
IH-03	Systems hold the position now. Cover at date of loss is inferred from the current policy version, and it has been wrong in a way that reached a complaint	The same question asked twice over the same period returns the same number, after a restatement, a model change or a departure, and the walk back is recorded rather than remembered
IH-04	Delegated business arrives as periodic spreadsheets in the intermediary's format. Between submissions the insurer does not know its own policy count on that book	Delegated and direct business answer the same conduct and prudential question at the same grain, and an exceedance of mandate is detected in the flow, not in a quarterly review
IH-05	Returns are assembled through manual spreadsheet steps with no lineage. The prior quarter cannot be reproduced because the assumption set was overwritten in place	The IFRS 17 group key is stamped on the source event at initial recognition and never recomputed; locked-in rates and coverage units are stored per group; every judgement names who or what made it
IH-06	Collections are submitted and failures retried without reference to mandate state. No component masters the mandate. Suspense cannot be aged at policy level	Collection failure reason codes are analysed as a leading cause of lapse in the mass market, and the retry and re-mandate strategy is a measured design decision
IH-07	Each channel has its own quotation path, customer record and commission calculation. The same risk prices differently depending on where it is asked	A new intermediary or embedded partner is onboarded by configuration against published interfaces, with mandate, remuneration, operator contract and data flows registered first
IH-08	The programme lives in a workbook held by one person. Cessions are calculated outside any system of record and recoveries are raised from memory of the structure	Any recovery traces to the treaty clause that supports it; cession reconciliation is a report rather than an exercise; facultative and non-proportional arrangements are modelled
IH-09	No criticality tiers. Recovery expectations differ depending on who is asked, and every outage is argued from first principles	Resilience investment follows the tier, the tier follows a criticality defined once and applied consistently, and a failed test raises a gap record rather than a retest
IH-10	Privacy and conduct exist as documents. Retention is a schedule nothing executes. Special personal information is not tagged, although underwriting is health-data processing by construction	Design assumptions are held as data and compared against experience at cohort grain; complaints and outcome codes are event-level; one external data flow can be switched off without stopping business
IH-11	Models and rules influencing customer outcomes run in production with no register, no named owner and no monitoring. Nobody can list them without asking the teams	Drift, decline rates and manual overrides are monitored by cohort and reported alongside the conduct position; a model change passes a gate; earlier decisions stay reconstructible on the version that made them

> **LEVEL 1 ON IH-10 IS THE DEFAULT, NOT THE EXCEPTION**
>
> Read that `IH-10` level-1 descriptor again. Underwriting is health-data processing and fraud investigation is criminal-behaviour and biometric processing, so an insurer that has not tagged special personal information separately is not behind on a project — it is running the prohibited categories through one gate. Since 6 March 2026 the POPIA health information regulations name insurers expressly, with their own handling, disposal and cross-border transfer obligations [4]. Where you score this at 1 or 2, say in `Evidence seen` which attributes you looked for and did not find.

How to run it

Step 1 — Ask for the week-one artefacts before you book a single conversation

Produce. A folder of what arrives, and a list of what did not.

You need. Nothing but an email, sent on your first or second day.

How.

1. Ask for the overnight batch schedule from the people who get paged, and the month-end close calendar. Together they are the true operating model, and they arrive faster than an application inventory.
2. Ask for the complaints register, three policy documents and three whole claim files including file notes.
3. Ask for the rate tables and wordings by product version, with in-force counts against each. What comes back, or does not, is the first real finding.
4. Ask for the approved baselines list with dates and approvers, and one repository change with its change request.
5. Write down what did not arrive, and when you asked. That list is evidence for `EA-02` and `IH-01`.

Signed by. Nobody. This is collection.

Done when. You have the batch schedule and the close calendar in your hands.

Goes wrong when. You start with the application inventory. It is eighteen months old, it names decommissioned systems and misses live ones, and waiting for it costs you a fortnight.

Step 2 — Interview in order: practice first, estate second

Produce. Eight conversations of forty-five minutes, notes in one notebook, no slides.

You need. Step 1. The interview guide on `07 Interview guide`, which holds forty-nine questions with the role to ask and what to ask to be shown [3].

How.

1. Week 1, the fast and truthful ones: Chief Enterprise Architect (governance, repository, capacity), Chief Data Officer (party authority), Chief Information Officer (how many inventories, and which is right), operations (the batch).
2. Weeks 2 and 3, the estate: pricing actuary, reserving actuary, head of claims, head of distribution, reinsurance manager, collections, Chief Risk Officer, Information Officer.
3. Week 4 and after, the ones needing the earlier answers: programme director on sequencing, finance and actuarial on the IFRS 17 group key, Chief Compliance Officer on design assumptions, Chief Underwriter on the three-way decision outcome.
4. Ask for the demonstration, not the answer. "Can you reconstruct this policy as at 14 March 2023?" is a request, and the ten minutes somebody spends attempting it teach you more than a workshop.
5. Close every conversation the same way — what would you fix if it were yours? — and find the person who has been there twenty years by asking three people who to call when something strange happens.

Signed by. Nobody yet.

Done when. Every dimension has at least one conversation behind it and every `Ask to be shown` entry is either in the folder or on the did-not-arrive list.

Goes wrong when. You accept answers. "We have a single customer view" and "the warehouse feeds everything" are both answers, and neither is evidence.

Step 3 — Score, and record the evidence in the same movement

Produce. Twenty-one dimension scores, each with `Evidence seen` and `Confidence` filled.

You need. Steps 1 and 2. The five descriptors per dimension.

How.

1. Score the practice dimensions in the room, immediately after each conversation, while the hesitation is still fresh. First impressions on governance and repository are usually right.
2. Score the estate dimensions afterwards, against the artefacts, because they turn on what a sheet reads rather than on what somebody said.
3. Write `Evidence seen` before you write the score. If you cannot fill the evidence cell, you do not yet have a score.
4. Where the evidence does not exist, score it low, set `Confidence` = Low, and write what you looked for and could not find. Absence of evidence is the finding, and a blank hides it.

5. Set `Target` to what you can defend as achievable inside the period, not what reads well. A health target above 3 in one period is rarely credible where the estate carries a legacy core.
6. Record who assessed it and the date, on every row.

Signed by. You, as the assessor. The signature that counts comes in step 6.

Done when. No dimension is blank, no score has an empty `Evidence seen`, and both weight columns still total 100.

Goes wrong when. You score what you recognise rather than what you can evidence. The descriptors are written as observations for exactly this reason: level 3 is a thing you saw, not a thing that is plainly intended.

Step 4 — Convert every 1 and 2 into a gap record

Produce. One `O29 Gap` row per weak dimension, with a severity you chose.

You need. Step 3. The `06 Findings` sheet, which pre-drafts twenty-one rows shaped for `O29 Gap` [1].

Where it lands.

> **`O29 Gap`**
>
> - `ID` — the next free number in the target sheet. The findings sheet deliberately numbers its drafts `GAP-NEW-01` onward so you cannot paste them without renumbering; pasting over an existing gap is the one mistake this sheet can cause.
> - `Domain` — Business, Data, Application, Technology or Realization. Practice dimensions are usually `Realization`; estate dimensions take the domain they sit in.
> - `Gap Type` — `New` for something absent, `Eliminate` where a current practice has to stop.
> - `Severity` — yours. The findings sheet suggests one arithmetically from the score; you are the one who has to defend it.
> - `Affected Object` — the sheet or object the gap bites on, such as `O05 Principle` or `O20 Data Entity (Party)`.
> - `Traces To` — the requirement the gap breaches. `Addressed By` stays blank until a work package exists; an invented work-package reference is worse than an empty cell.

How.

1. Delete every pre-drafted row where the score came out 3 or above.
2. Renumber the survivors from the next free `GAP-` number.
3. Set `Severity` from the score and the weight together, then adjust by hand where the weight says the arithmetic is wrong.
4. Fill `Traces To`. A gap that breaches no requirement is either a preference or a missing requirement, and you have to say which.
5. Attach the evidence named in the findings sheet's last column to the gap, not only to the score sheet.

Signed by. The Architecture Board (`GB-01`), which owns the gap register's severities.

Done when. Every dimension scoring 1 or 2 has a gap with a severity, an affected object and either a `Traces To` value or a recorded missing requirement.

Goes wrong when. The arithmetic sets the severity. A gap of 3 on a 4% dimension moves less than a gap of 1 on a 14% one, and only you can see that from the weights.

Step 5 — Raise one change request carrying the whole assessment

Produce. One row in `03 Change Requests` holding the scores, the new gaps and the severities that moved on existing gaps.

You need. Step 4.

How.

1. Raise one request, not one per dimension.
2. Put the two weighted means and the counts in `Decision`, then the named exceptions.
3. Write the practice weighted mean into `O04 Measure` with a `Target`, a `Frequency` of Annual and a date. Raise a second measure for the estate rather than averaging.
4. List the gaps raised and the severities changed, so the assessment and its consequences arrive together.
5. Reference the decision record that carries the reasoning.

Signed by. The Architecture Board, with the scores reported to the Executive Committee (`GB-04`).

Done when. The change request names every gap raised and every severity moved, and the measures exist with targets.

Goes wrong when. The assessment is run, filed and never referenced. Its purpose is to move severities and release a work package. If no severity changed, you scored yourself for nothing.

Step 6 — Get it challenged, then get it signed

Produce. A countersigned assessment with the challenges recorded.

You need. Step 5. One person outside the architecture function per assessment — the Chief Risk Officer for the practice, the Chief Operating Officer or a line executive for the estate.

How.

1. Walk them down the score sheet, reading `Evidence seen` aloud.
2. Where they disagree, write both numbers in the cell and move on. An averaged score satisfies nobody.
3. Change a score only where they produce evidence you had not seen, and record what they produced.
4. Have them sign the two weighted means and the gap list, not the narrative.
5. Report the two numbers separately to the Executive Committee, each with its band.

Signed by. The Architecture Board, countersigned by the challenger.

Done when. Both weighted means carry two signatures, and every changed score names the evidence that changed it.

Goes wrong when. You present it rather than having it challenged. A self-assessed score survives one question from a board member and then the whole instrument is discredited, including the parts that were right.

Turning a low score into a gap

The `06 Findings` sheet pre-drafts a row per dimension in `O29 Gap` shape, so the conversion is editing rather than authoring [1][5]. The shape, in sheet order: `ID`, `Name`, `Description`, `Domain`, `Gap Type`, `Severity`, `Affected Object`, `Traces To`, `Addressed By` — columns A to I.

Worked end to end. `IH-02 Party and customer mastering`, weight 11.

- **What you asked.** Who is system of record for a person, and does any transaction actually call it? The second clause is the one that matters — a master nothing calls is a reconciliation target, not an authority [3].
- **What you were shown.** `RPT05 App-Data Matrix`, the party row: several masters, `Status` = `CONTESTED`. A matching exercise had run once; its result was loaded and not maintained. No transaction resolves through the master at point of sale.
- **The score.** Level 2, `Contested`. Not level 3, because level 3 requires one authoritative source recorded in `O20` and agreed in `RPT05` as `Single`, with a golden-record definition and a survivorship rule in `O43`. `Evidence seen`: "`RPT05` party row, 14 Oct, several masters / CONTESTED; `O43` empty for party". `Confidence`: High.
- **The gap row.** `Name` = No single party authority. `Description` = several applications master the party entity with no common identifier, and transactions do not resolve through a designated master. `Domain` = Data. `Gap Type` = New. `Affected Object` = `O20 Data Entity (Party)`. `Traces To` = the requirement for a single enterprise customer identifier. `Addressed By` = blank, because no work package exists yet.
- **The severity.** Score 2 on an 11% dimension, and the entity is a Tier 1 entity that every other dimension depends on — `IH-03`, `IH-07` and `IH-10` all degrade from it. Critical. Had the same score landed on `EA-10 Skills and capacity` at 4%, High would have been defensible.

How severity is set. Three inputs, in this order. The score, which gives the arithmetic suggestion. The weight, which says how much of the number it carries. And whether other dimensions depend on it — a gap that is the cause of three other gaps outranks its own score. The findings sheet computes the first and shows you the second; the third is yours, and it is the reason the suggested severity is a suggestion.

> **DO NOT RAISE TWENTY-ONE GAPS**
>
> If every dimension scores 1 or 2 you have twenty-one gap records, a register nobody reads and a roadmap that moves nothing. Raise the gaps for the dimensions you intend to move this period — the two highest-weighted ones sitting at 2 — and record the rest as scored-and-not-yet-raised in `Evidence seen`. A plan that moves ten dimensions moves none.

Re-running it

The second run is harder than the first, for one reason: you are now marking your own homework. In year one the scores describe what somebody else built. In year two they describe what you did with it, and every dimension you chose not to move is a dimension you have to explain.

What changes:

- **The evidence requests are known.** Two days instead of ten, because you know which sheet reads what and who holds the artefact.
- **The comparison is the output.** Last year's score, this year's score, and the work package that moved it. A dimension that moved with no delivered work package moved because you scored it differently, and saying so is better than letting somebody else find it.
- **The weights come under pressure.** Re-weighting between runs destroys the comparison. Change the weights only at band 4 and above, and when you do, publish both means — old weights and new — for the year of the change.
- **The targets get tested.** A target you set last year and missed is a conversation. Have it before the board does, with the reason: unfunded, descoped, or wrong in the first place.

Who holds the pen the second time. Not you alone. The cleanest arrangement is that you score, a domain architect from outside your own domain scores the same dimensions independently, and the two sheets are compared before either is shown. Where that is not available, the challenger from

step 6 scores the five highest-weighted dimensions themselves rather than reviewing yours. Either way, the practice assessment should not be run by the person whose performance it describes two years running without someone else's number beside it.

Run it alongside the principles review in appendix G — the tier distribution check and the `RPT11 Principle Impact` read are the same morning's work as `EA-03`.

The printable material

SET 1 — THE CHIEF ENTERPRISE ARCHITECT AND THE BOARD CHAIR (EA-01, EA-02, EA-03, EA-10)

Which body may refuse a design, and when did it last do so? Ask for the last three sets of minutes and one recorded refusal.

What happens to a dispensation when it expires? Ask for the register with expiry dates and what happened on expiry.

Where is the repository, and when was the last baseline approved? Ask for the baselines list with dates and approvers.

Name a decision in the last year that a principle changed. Ask for the decision record naming the principle.

How many principles are at Tier 1, what breaks a tie between two of them, and which architecture roles are vacant?

SET 2 — THE DATA AND PRIVACY SIDE (EA-06, IH-02, IH-03, IH-10)

Who is the authoritative source for a person, and does any transaction call it? Ask for the app-data matrix party row and one transaction that resolves through the master.

Which retention obligations are executed by a process, and which by a policy document? Ask for one disposal job that ran.

How do we establish cover as at the date of loss, and has it ever been wrong? Ask for one claim validated on the version in force at the loss date.

Which attributes are special personal information, and what different gate applies to them?

Name every operator holding policyholder data, under what contract, in which country, on which transfer basis.

SET 3 — ACTUARIAL, FINANCE AND REINSURANCE (IH-01, IH-03, IH-05, IH-08)

Where is a rate change actually made, who deploys it, and how long from decision to live quote? Ask for the last three with dates.

Can any historical quote be reproduced exactly, on the rate version that priced it?

Is reserve movement history immutable and correctly dated — and do you trust it? Ask for a loss development triangle, gross and net, re-run. The pause before the answer is the finding.

Where is the IFRS 17 group key set, and can a 2023 assignment be reproduced today?

Which regulatory returns come out of the warehouse, and which out of a parallel manual process? Ask for lineage from one return line to the source field.

Where does the reinsurance programme live, and can you evidence one recovery to one treaty clause?

SET 4 — DISTRIBUTION, CLAIMS AND OPERATIONS (IH-04, IH-06, IH-07, IH-09, IH-11)

What share of the book is written or administered under delegated authority, and at what granularity does that data reach us? Ask for one bordereau and its reconciliation.

What share of collections are authenticated, and which component masters the mandate?

Can suspense be aged at policy level, and how old is the oldest item?

How many quotation paths are there, is premium calculated once, and is representative authorisation checked at the point of sale?

Which recovery objectives have been tested, and when? Ask for last-tested dates against the stated cadence.

What share of new business is decided with no human touch, and what happens when the engine cannot decide at all?

Form MA-1 — EA practice maturity score sheet

Feeds the `02 EA Maturity Score` sheet of `Insurance_EA_Maturity_Assessment.xlsx`, columns `Score 1-5`, `Target`, `Evidence seen — what you actually looked at` and `Confidence`. `Gap` and the weighted columns calculate; do not write in them on the spreadsheet. **Typed into Excel** `Assessed by` and `Date`, once per sheet rather than per row. **First pass** all ten rows, one copy. **Signed by** the Architecture Board, countersigned by a challenger outside the architecture function. A self-assessed score is worth nothing to a board.

Dimension	Score	Target	Gap	Evidence seen — sheet and reading	Conf.
EA-01 *example — delete*	2	3	1	`RPT31`, no refusal, no expiry enforced	High
EA-01 Architecture governance (14)					
EA-02 The architecture repository (10)					
EA-03 Principles (8)					
EA-04 Requirements traceability (12)					
EA-05 Business architecture (12)					
EA-06 Data governance (14)					
EA-07 Application portfolio (10)					
EA-08 Technology standards position (8)					
EA-09 Delivery engagement (8)					
EA-10 Skills and capacity (4)					

Weighted mean, and the band it falls in. Write both, and the date.

Form MA-2 — Insurance architecture health score sheet

Feeds the `04 Insurance Health Score` sheet, same four columns. **Kept separate from Form MA-1 deliberately** — a competent function can inherit a poor estate, and one number hides that. **First pass** all eleven rows, one copy. **Signed by** the Architecture Board, countersigned by a line executive or the Chief Operating Officer. Where you drop `IH-11`, renormalise the weights and say so on the form.

Dimension	Score	Target	Gap	Evidence seen — sheet and reading	Conf.
IH-02 *example — delete*	2	3	1	`RPT05` party row, several masters, CONTESTED	High
IH-01 Product change and configuration (11)					
IH-02 Party and customer mastering (11)					
IH-03 Policy and claims data history (11)					
IH-04 Delegated authority and the binder estate (9)					
IH-05 Regulatory reporting and the IFRS 17 chain (13)					
IH-06 Collections and DebiCheck (7)					
IH-07 Distribution and intermediary systems (7)					
IH-08 Reinsurance administration (6)					
IH-09 Resilience against criticality (8)					
IH-10 Conduct and POPIA position (10)					
IH-11 Automated decisions and model governance (7)					

Weighted mean, the band, and what you looked for and could not find.

Form MA-3 — Findings, shaped for O29 Gap

Feeds `O29 Gap` — `ID`, `Name`, `Domain`, `Gap Type`, `Severity`, `Affected Object` — with `Description`, `Traces To` and `Addressed By` on the second table. **Typed into Excel** `Description`, which is a sentence. **One row per dimension scoring 1 or 2**, and no more than the ones you intend to move this period. **Renumber every ID** from the next free `GAP-` number in the target sheet before you transcribe. **Signed by** the Architecture Board, which owns the severities.

ID	Name	Domain	Gap Type	Severity	Affected Object
GAP-22	*example — delete*	Data	New	Critical	`O20 Data Entity (Party)`

`Traces To` and `Addressed By`. Leave `Addressed By` blank until a work package exists — an invented reference is worse than an empty cell.

ID	Source dimension	Traces To — requirement breached	Addressed By	Evidence attached
GAP-22	IH-02	*example — delete*	—	`RPT05` party row

Where the spreadsheet adds something

`Insurance_EA_Maturity_Assessment.xlsx` carries what a printed page cannot. Eight sheets.

Sheet	What it adds beyond this appendix
`00 Start Here`	How to run it, who to involve, the scoring rule, the five bands and what to do at each
`01 EA Maturity Levels`	All ten practice dimensions at five levels each — the three middle descriptors this appendix leaves out
`02 EA Maturity Score`	Where you score, with the weights, `How to read the evidence` per dimension, and the computed weighted mean and target
`03 Insurance Health Levels`	All eleven estate dimensions at five levels each
`04 Insurance Health Score`	The estate score sheet, same shape
`05 Summary`	Both weighted means, the bands, the largest single gap read against its weight, and a radar-ready block
`06 Findings`	Twenty-one pre-drafted gap rows in `O29 Gap` shape, with a suggested severity and the evidence to attach
`07 Interview guide`	Forty-nine questions, with the role to ask, why you are asking, what a bad answer sounds like, and what to ask to be shown

Two columns are worth the download on their own. `How to read the evidence` on sheets `02` and `04` says what each report's cells mean when you open it — links per record below 1.2 means objects catalogued and barely linked; `Agrees` = No on the portfolio assessment means the story and the data have come apart; `Held indefinitely` against a stated retention obligation is an exposure with a section number attached [1]. And `What a bad answer sounds like` on `07` saves a conversation: "everything comes to the board", "we have a single customer view", "the broker holds the structure" are all answers, and each one is the finding [3].

The forty-nine questions are not reproduced here on purpose. Four sets earn a printed page because you will ask them in a room with a notebook; the rest are evidence requests, and an evidence request belongs in an email with a filename at the end of it.

THE CHAPTER IN TEN LINES

- Run it in your first month, then annually. The first run says what you walked into; the second says whether anything moved.
- Two scores, never averaged: is the architecture function any good, and is this insurer's architecture any good.
- Score the level whose observable descriptor you can point at, not the one that sounds closest.
- No score is recorded without a named sheet, a document or a person who showed you. A blank hides the finding.
- Where the evidence does not exist, score it low with Confidence = Low and say what you looked for.
- Ten practice dimensions weighted to 100; eleven estate dimensions weighted to 100 separately.
- The weights are yours, and may not be edited after you have seen the scores.
- Every dimension at 1 or 2 becomes a gap row, but raise only the ones you intend to move this period.
- Severity comes from the score, the weight, and whether other dimensions depend on it. Only the third is judgement.
- The second run is harder: you are marking your own homework, so somebody else holds a pen beside yours.

SOURCES FOR THIS CHAPTER

- **1.** Insurance_EA_Maturity_Assessment.xlsx, sheets 00 Start Here, 01 EA Maturity Levels, 02 EA Maturity Score, 03 Insurance Health Levels, 04 Insurance Health Score, 05 Summary and 06 Findings
- **2.** Meridian_TOGAF_Metamodel.xlsx, sheets O31 Deliverable and O34 Architecture Deliverable (DLV-04), O03 Objective (OBJ-12) and O04 Measure (MSR-13)
- **3.** Insurance_EA_Maturity_Assessment.xlsx, sheet 07 Interview guide
- **4.** Moonstone, "New POPIA regulations on health information now in force" (effective 6 March 2026; named parties include insurers); Webber Wentzel, "Artificial intelligence has POPIA implications" (section 71) https://www.moonstone.co.za/new-popia-regulations-on-health-information-now-in-force/
- **5.** Meridian_TOGAF_Metamodel.xlsx, sheets O29 Gap and 03 Change Requests
- **6.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method, Preliminary Phase https://pubs.opengroup.org/architecture/togaf10-doc/adm/